



P4₁₆ Language Specification

The P4 Language Consortium

Version v1.2.5, 2026-01-22

Contents

1. Scope	9
2. Terms, definitions, and symbols	9
3. Overview	10
3.1. Benefits of P4	12
3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1)	13
4. Architecture Model	14
4.1. Standard architectures	15
4.2. Data plane interfaces	15
4.3. Extern objects and functions	15
5. Example: A very simple switch	16
5.1. Very Simple Switch Architecture	17
5.2. Very Simple Switch Architecture Description	19
5.2.1. Arbiter block.	19
5.2.2. Parser runtime block	19
5.2.3. Demux block	20
5.2.4. Available extern blocks.	20
5.3. A complete Very Simple Switch program.	20
6. P4 language definition	24
6.1. Syntax and semantics	24
6.1.1. Grammar	25
6.1.2. Semantics and the P4 abstract machines	25
6.1.2.1. Pseudo-code snippets	25
6.1.2.2. Prose algorithms	25
6.1.2.2.1. Prose algorithm shorthands	26
6.2. Lexical constructs.	27
6.2.1. Identifiers	27
6.2.2. Literal constants	28
6.2.2.1. Boolean literals.	28
6.2.2.2. Integer literals.	28
6.2.2.3. String literals.	29
6.2.3. Comments	29
6.2.4. Optional trailing commas	30
6.3. Scoping	30
6.3.1. Name resolution	31
6.3.2. Name visibility	32
6.4. Stateful elements.	32
6.5. Call convention: copy-in/copy-out.	32
6.5.1. Justification.	35
6.6. Overloading.	36
6.7. Naming conventions	36
7. The P4 Abstract Machine	36
7.1. Preprocessing.	37
7.2. Type Checking.	38
7.2.1. P4 Intermediate Representation	38
7.2.2. Typing a P4 program	39
7.2.3. Typing context.	40
7.3. Instantiation	41
7.3.1. Control-plane names	42

7.3.1.1. Computing control-plane names	43
7.3.1.1.1. Value sets	43
7.3.1.1.2. Tables	43
7.3.1.1.3. Keys	43
7.3.1.1.4. Actions.	44
7.3.1.1.5. Instances.	44
7.3.1.2. Annotations controlling naming	46
7.3.1.3. Recommendations.	46
7.3.2. Instantiating a P4 program	46
7.3.3. Instantiation context	47
7.4. Dynamic evaluation	48
7.4.1. Evaluating a P4 program	49
7.4.2. Evaluation context	49
7.4.3. The parser abstract machine	50
7.4.4. The match-action pipeline abstract machine	50
7.4.5. Concurrency model	50
7.5. Compile-time known and local compile-time known values.	51
8. P4 types and values	53
8.1. Base types and values.	54
8.1.1. Well-formedness	55
8.1.2. The void type	55
8.1.2.1. Type checking	56
8.1.3. Booleans	56
8.1.3.1. Type checking	56
8.1.4. Errors	56
8.1.4.1. Type checking	57
8.1.5. Match kinds	57
8.1.5.1. Type checking	57
8.1.6. Strings	58
8.1.6.1. Type checking	58
8.1.7. Integer types and values (signed and unsigned)	58
8.1.7.1. Portability	59
8.1.7.2. Arbitrary-precision integers	59
8.1.7.2.1. Type checking.	60
8.1.7.3. Fixed-width unsigned integers (bit-strings)	60
8.1.7.3.1. Type checking.	60
8.1.7.4. Fixed-width signed integers	61
8.1.7.4.1. Type checking.	61
8.1.7.5. Dynamically-sized bit-strings	62
8.1.7.5.1. Type checking.	62
8.2. Named types.	62
8.2.1. Type definitions (type constructors)	64
8.2.2. Type resolution	65
8.2.3. Type specialization	65
8.2.4. Prefixed type names.	65
8.2.4.1. Type checking	66
8.2.5. Specialized types	66
8.2.5.1. Type checking	66
8.3. Data types and values	67
8.3.1. List types and values	67

8.3.1.1. Type checking	68
8.3.1.2. Well-formedness	68
8.3.2. Tuple types and values.	68
8.3.2.1. Type checking	68
8.3.2.2. Well-formedness	68
8.3.3. Header stack types and values	69
8.3.3.1. Type checking	69
8.3.3.2. Well-formedness	70
8.3.4. Array types and values.	70
8.3.5. Struct types and values	70
8.3.5.1. Type checking	71
8.3.5.2. Well-formedness	71
8.3.6. Header types and values	71
8.3.6.1. Type checking	73
8.3.6.2. Well-formedness	73
8.3.7. Header union types and values	74
8.3.7.1. Type checking	74
8.3.7.2. Well-formedness	74
8.3.8. Enum types and values	75
8.3.8.1. Enum type declaration without an underlying type	75
8.3.8.2. Enum type declaration with an underlying type	76
8.3.8.3. Type checking	76
8.3.8.4. Well-formedness	77
8.4. Object types and values	77
8.4.1. Extern object types	77
8.4.1.1. Type checking	78
8.4.1.2. Well-formedness	78
8.4.2. Parser object types	78
8.4.2.1. Type checking	79
8.4.2.2. Well-formedness	79
8.4.3. Control object types	79
8.4.3.1. Type checking	80
8.4.3.2. Well-formedness	80
8.4.4. Package object types	80
8.4.4.1. Type checking	80
8.4.4.2. Well-formedness	81
8.4.5. Table object types	81
8.4.5.1. Well-formedness	81
8.4.6. Object reference values	81
8.5. Alias types	82
8.5.1. <code>typedef</code> declaration	82
8.5.1.1. Type checking	82
8.5.1.2. Well-formedness	83
8.5.2. <code>type</code> declaration	83
8.5.2.1. Type checking	83
8.5.2.2. Well-formedness	83
8.6. Synthesized types and values	84
8.6.1. Default types and values	84
8.6.1.1. Well-formedness	85
8.6.2. Invalid header types and values	85

8.6.2.1. Well-formedness	86
8.6.3. Sequence types and values	86
8.6.3.1. Well-formedness	87
8.6.4. Record types and values	87
8.6.4.1. Well-formedness	88
8.6.5. Set types and values	89
8.6.5.1. Well-formedness	89
8.6.6. Table metadata types and values	89
8.6.6.1. Well-formedness	90
8.7. Unrolling	90
8.8. Subtyping	91
9. P4 callables	91
9.1. Parameters	93
9.1.1. Type checking	93
9.1.2. Well-formedness	94
9.2. Callable definitions (callable constructors)	95
9.2.1. Well-formedness	96
9.3. Actions	96
9.3.1. Well-formedness	97
9.4. Functions	98
9.4.1. User-defined functions	98
9.4.1.1. Well-formedness	99
9.4.2. Extern functions	99
9.4.2.1. Well-formedness	100
9.5. Methods	100
9.5.1. Extern methods	100
9.5.1.1. Well-formedness	101
9.5.2. Parser apply methods	102
9.5.2.1. Well-formedness	103
9.5.3. Control apply methods	103
9.5.3.1. Well-formedness	104
9.5.4. Table apply methods	104
9.5.4.1. Well-formedness	104
9.5.5. Built-in methods	104
10. P4 objects and constructors	105
10.1. Objects	105
10.1.1. Fully-qualified names	105
10.2. Constructors	105
10.2.1. Constructor parameters	105
10.2.1.1. Well-formedness	107
10.2.2. Constructor definitions	107
10.2.2.1. Well-formedness	108
10.3. Extern objects	108
10.3.1. Well-formedness	109
10.3.2. Instantiation	109
10.4. Parser objects	110
10.4.1. Well-formedness	112
10.4.2. Instantiation	112
10.5. Control objects	113
10.5.1. Well-formedness	114

10.5.2. Instantiation	114
10.6. Table objects	116
10.6.1. Well-formedness	117
10.7. Parser value set objects	117
10.8. Package objects	118
10.8.1. Well-formedness	119
10.8.2. Instantiation	119
11. Declarations	120
11.1. Type checking declarations	121
11.2. Compile-time evaluation of declarations	122
11.3. Runtime evaluation of declarations	122
11.4. Variable declarations	122
11.4.1. Type checking	123
11.4.2. Runtime evaluation	124
11.5. Constant declarations	125
11.5.1. Type checking	126
11.5.2. Compile-time evaluation	127
11.5.3. Runtime evaluation	127
11.6. Instantiations	128
11.6.1. Objects with abstract methods	129
11.6.1.1. Type checking	130
11.6.1.2. Compile-time evaluation	131
11.6.2. Type checking	132
11.6.3. Compile-time evaluation	134
11.7. Function declarations	136
11.7.1. Type checking	136
11.7.2. Compile-time evaluation	137
11.8. Action declarations	138
11.8.1. Type checking	138
11.8.2. Compile-time evaluation	139
11.9. Error declarations	140
11.9.1. Type checking	140
11.9.2. Compile-time evaluation	141
11.10. Match kind declarations	141
11.10.1. Type checking	142
11.10.2. Compile-time evaluation	142
11.11. Extern declarations	142
11.11.1. Extern function declarations	143
11.11.1.1. Type checking	143
11.11.1.2. Compile-time evaluation	143
11.11.2. Extern object declarations	144
11.11.2.1. Type checking	144
11.11.2.2. Compile-time evaluation	145
11.12. Parser declarations	146
11.12.1. Type checking	147
11.12.2. Compile-time evaluation	148
11.13. Control declarations	148
11.13.1. Type checking	149
11.13.2. Compile-time evaluation	150
11.14. Enum type declaration	150

11.14.1. Enum type declarations without an underlying type	151
11.14.1.1. Type checking.	151
11.14.1.2. Compile-time evaluation	151
11.14.2. Enum type declarations with an underlying type	152
11.14.2.1. Type checking.	152
11.14.2.2. Compile-time evaluation	153
11.15. Struct type declaration	154
11.15.1. Type checking	155
11.15.2. Compile-time evaluation	155
11.16. Header type declaration.	156
11.16.1. Type checking	156
11.16.2. Compile-time evaluation	157
11.17. Header union type declaration	157
11.17.1. Type checking	158
11.17.2. Compile-time evaluation	158
11.18. Typedef declaration	159
11.18.1. <code>typedef</code> declaration	159
11.18.1.1. Type checking.	159
11.18.1.2. Compile-time evaluation	160
11.18.2. New type declaration.	161
11.18.2.1. Type checking.	161
11.18.2.2. Compile-time evaluation	162
11.19. Parser type declaration	162
11.19.1. Type checking	162
11.19.2. Compile-time evaluation	163
11.20. Control type declaration.	163
11.20.1. Type checking	163
11.20.2. Compile-time evaluation	164
11.21. Package type declaration	164
11.21.1. Type checking	164
11.21.2. Compile-time evaluation	165
12. Statements	166
12.1. Empty statements	167
12.2. Assignment statements	168
12.2.1. L-values	169
12.3. Call statements	173
12.3.1. L-values	174
12.4. Direct application statements	176
12.5. Return statements.	179
12.6. Exit statements	180
12.7. Block statements	181
12.8. Conditional statements	184
12.9. For statements	187
12.10. Break statements	188
12.11. Continue statements	188
12.12. Switch statements.	188
12.12.1. Notes common to all switch statements	189
12.12.2. Switch statement on match-action table result.	189
12.12.3. Switch statement on numeric or enumerated type	191
13. Expressions	193

13.1. Literal expressions	195
13.1.1. Boolean literal expressions.....	195
13.1.2. Numeric literal expressions	196
13.1.3. String literal expressions.....	198
13.2. Reference expressions	199
13.3. Default expressions.....	199
13.4. Unary expressions.....	200
13.4.1. Negation operators	200
13.4.2. Bitwise complement operators	201
13.4.3. Plus and minus operators.....	201
13.5. Binary expressions	202
13.5.1. Plus, minus, and multiplication operators	203
13.5.2. Saturating plus and minus operators	203
13.5.3. Division and modulo operators	204
13.5.4. Shift left and right operators	204
13.5.5. Equality and inequality operators	205
13.5.6. Comparison operators.....	206
13.5.7. Bitwise and, xor, and or operators	206
13.5.8. Concatenation operators	207
13.5.9. Logical and, and or operator	207
13.6. Ternary expressions	209
13.7. Cast expressions	211
13.8. Data expressions.....	212
13.8.1. Invalid header expressions.....	212
13.8.2. Sequence expressions.....	213
13.8.3. Record expressions	215
13.9. Access expressions	218
13.9.1. Error access expressions.....	218
13.9.2. Member access expressions.....	219
13.9.2.1. Type accesses.....	219
13.9.2.2. Field accesses.....	221
13.9.3. Index access expressions	224
13.9.4. Bitslice access expressions	225
13.10. Call expressions.....	227
13.10.1. Callable call expressions	228
13.10.2. Constructor call expressions	231
13.10.3. Parenthesized calls.....	233
13.11. Parenthesized expressions	233
14. Casting	234
14.1. Explicit casting	234
14.2. Implicit casting	234
14.3. Coercion	234
15. Abstract Parser Machine	234
15.1. Parser local declarations	234
15.1.1. Constant declarations	235
15.1.2. Instantiations.....	235
15.1.3. Variable declarations	235
15.1.4. Parser value set declarations	235
15.2. Parser states.....	235
15.3. Parser statements.....	236

15.3.1. Constant declaration	236
15.3.2. Variable declaration	236
15.3.3. Empty statement	236
15.3.4. Assignment statement	236
15.3.5. Call statement	237
15.3.6. Direct application statement	237
15.3.7. Block statement	237
15.3.8. Conditional statement	237
15.4. Parser transition statements	237
15.4.1. Parser select expression	238
16. Abstract Control Machine	238
16.1. Control local declarations	238
16.1.1. Constant declarations	239
16.1.2. Instantiations	239
16.1.3. Variable declarations	239
16.1.4. Action declarations	239
16.1.5. Table declarations	239
16.2. Match-action tables	240
16.2.1. Table keys	240
16.2.2. Table actions	240
16.2.3. Table default actions	240
16.2.4. Table entries	240
16.2.5. Table custom properties	241
17. Call convention	241
17.1. Arguments and type arguments	241
17.1.1. Arguments	241
17.1.2. Type arguments	241
17.2. Overload resolution	242
17.3. Static checks for call typing	242
17.4. Call semantics	242
18. Annotations	242
Appendix A: Revision History	244
A.1. Summary of changes made in unreleased version	244
A.2. Summary of changes made in version 1.2.5, released October 11, 2024	244
A.3. Summary of changes made in version 1.2.4, released May 15, 2023	244
A.4. Summary of changes made in version 1.2.3, released July 11, 2022	245
A.5. Summary of changes made in version 1.2.2, released May 17, 2021	246
A.6. Summary of changes made in version 1.2.1, released June 11, 2020	246
A.7. Summary of changes made in version 1.2.0, released October 14, 2019	247
A.8. Summary of changes made in version 1.1.0, released November 26, 2017	247
A.9. Initial version 1.0.0, released May 17, 2017	248
Appendix B: P4 reserved keywords	248
Appendix C: P4 reserved annotations	249
Appendix D: P4 core library	249

Abstract

P4 is a language for programming the data plane of network devices. This document provides a precise definition of the P4₁₆ language, which is the 2016 revision of the P4 language (<http://p4.org>). The target audience for this document includes developers who want to write compilers, simulators, IDEs, and debuggers for P4 programs. This document may also be of interest to P4 programmers who are interested in understanding the syntax and semantics of the language at a deeper level.

1. Scope

This specification document defines the structure and interpretation of programs in the P4₁₆ language. It defines the syntax, semantic rules, and requirements for conformant implementations of the language.

It does not define:

- Mechanisms by which P4 programs are compiled, loaded, and executed on packet-processing systems,
- Mechanisms by which data are received by one packet-processing system and delivered to another system,
- Mechanisms by which the control plane manages the match-action tables and other stateful objects defined by P4 programs,
- The size or complexity of P4 programs,
- The minimal requirements of packet-processing systems that are capable of providing a conformant implementation.

It is understood that some implementations may be unable to implement the behavior defined here in all cases, or may provide options to eliminate some safety guarantees in exchange for better performance or handling larger programs. They should document where they deviate from this specification.

2. Terms, definitions, and symbols

Throughout this document, the following terms will be used:

- **Architecture:** A set of P4-programmable components and the data plane interfaces between them.
- **Control plane:** A class of algorithms and the corresponding input and output data that are concerned with the provisioning and configuration of the data plane.
- **Data plane:** A class of algorithms that describe transformations on packets by packet-processing systems.
- **Metadata:** Intermediate data generated during execution of a P4 program.
- **Packet:** A network packet is a formatted unit of data carried by a packet-switched network.
- **Packet header:** Formatted data at the beginning of a packet. A given packet may contain a sequence of packet headers representing different network protocols.
- **Packet payload:** Packet data that follows the packet headers.
- **Packet-processing system:** A data-processing system designed for processing network packets. In general, packet-processing systems implement control plane and data plane algorithms.
- **Target:** A packet-processing system capable of executing a P4 program.

All terms defined explicitly in this document should not be understood to refer implicitly to similar terms defined elsewhere. Conversely, any terms not defined explicitly in this document should be interpreted

according to generally recognizable sources---e.g., IETF RFCs.

3. Overview

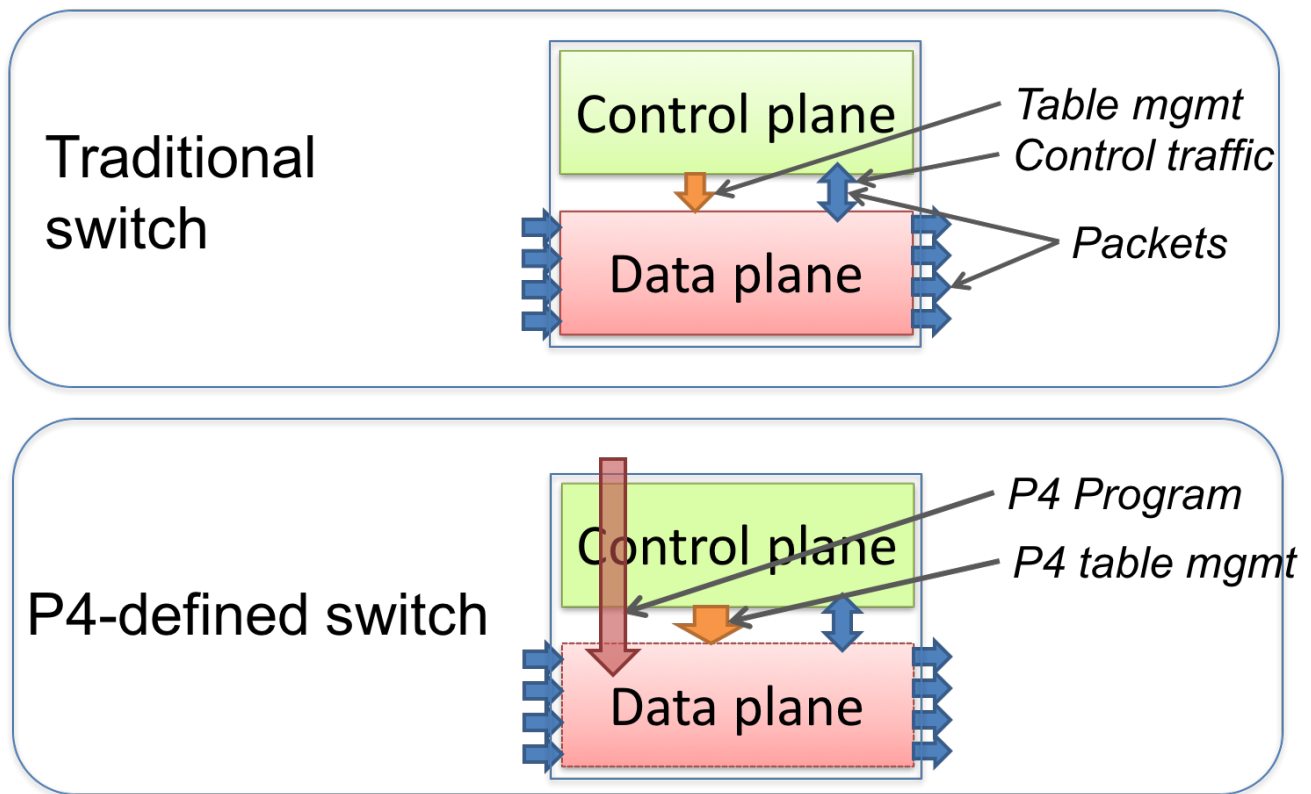


Figure 1. Traditional switches vs. programmable switches.

P4 is a language for expressing how packets are processed by the data plane of a programmable forwarding element such as a hardware or software switch, network interface card, router, or network appliance. The name P4 comes from the original paper that introduced the language, "Programming Protocol-independent Packet Processors," <https://arxiv.org/pdf/1312.1719.pdf>. While P4 was initially designed for programming switches, its scope has been broadened to cover a large variety of devices. In the rest of this document we use the generic term *target* for all such devices.

Many targets implement both a control plane and a data plane. P4 is designed to specify only the data plane functionality of the target. P4 programs also partially define the interface by which the control plane and the data-plane communicate, but P4 cannot be used to describe the control-plane functionality of the target. In the rest of this document, when we talk about P4 as "programming a target", we mean "programming the data plane of a target".

As a concrete example of a target, Figure 1 illustrates the difference between a traditional fixed-function switch and a P4-programmable switch. In a traditional switch the manufacturer defines the data-plane functionality. The control-plane controls the data plane by managing entries in tables (e.g. routing tables), configuring specialized objects (e.g. meters), and by processing control-packets (e.g. routing protocol packets) or asynchronous events, such as link state changes or learning notifications.

A P4-programmable switch differs from a traditional switch in two essential ways:

- The data plane functionality is not fixed in advance but is defined by a P4 program. The data plane is configured at initialization time to implement the functionality described by the P4 program (shown by the long red arrow) and has no built-in knowledge of existing network protocols.
- The control plane communicates with the data plane using the same channels as in a fixed-function device, but the set of tables and other objects in the data plane are no longer fixed, since they are defined by a P4 program. The P4 compiler generates the API that the control plane uses to

communicate with the data plane.

Hence, P4 can be said to be protocol independent, but it enables programmers to express a rich set of protocols and other data plane behaviors.

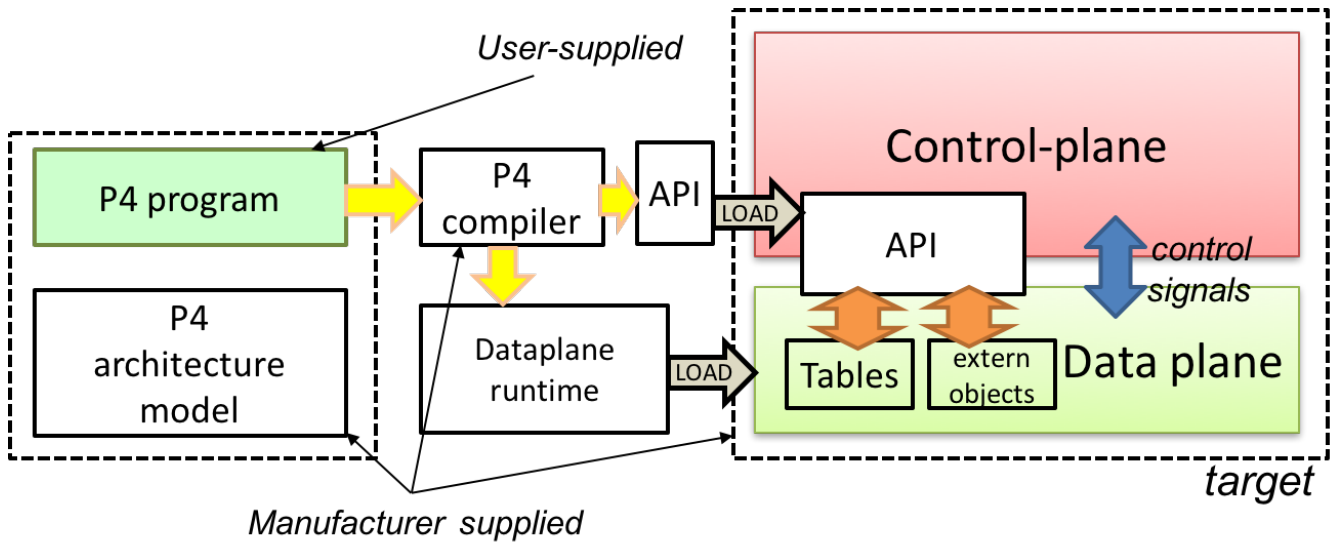


Figure 2. Programming a target with P4.

The core abstractions provided by the P4 language are:

- **Header types** describe the format (the set of fields and their sizes) of each header within a packet.
- **Parsers** describe the permitted sequences of headers within received packets, how to identify those header sequences, and the headers and fields to extract from packets.
- **Tables** associate user-defined keys with actions. P4 tables generalize traditional switch tables; they can be used to implement routing tables, flow lookup tables, access-control lists, and other user-defined table types, including complex multi-variable decisions.
- **Actions** are code fragments that describe how packet header fields and metadata are manipulated. Actions can include data, which is supplied by the control-plane at runtime.
- **Match-action units** perform the following sequence of operations:
 - Construct lookup keys from packet fields or computed metadata,
 - Perform table lookup using the constructed key, choosing an action (including the associated data) to execute, and
 - Finally, execute the selected action.
- **Control flow** expresses an imperative program that describes packet-processing on a target, including the data-dependent sequence of match-action unit invocations. Deparsing (packet reassembly) can also be performed using a control flow.
- **Extern objects** are architecture-specific constructs that can be manipulated by P4 programs through well-defined APIs, but whose internal behavior is hard-wired (e.g., checksum units) and hence not programmable using P4.
- **User-defined metadata**: user-defined data structures associated with each packet.
- **Intrinsic metadata**: metadata provided by the architecture associated with each packet---e.g., the input port where a packet has been received.

Figure 2 shows a typical tool workflow when programming a target using P4.

Target manufacturers provide the hardware or software implementation framework, an architecture definition, and a P4 compiler for that target. P4 programmers write programs for a specific architecture, which defines a set of P4-programmable components on the target as well as their external data plane

interfaces.

Compiling a set of P4 programs produces two artifacts:

- a data plane configuration that implements the forwarding logic described in the input program and
- an API for managing the state of the data plane objects from the control plane

P4 is a domain-specific language that is designed to be implementable on a large variety of targets including programmable network interface cards, FPGAs, software switches, and hardware ASICs. As such, the language is restricted to constructs that can be efficiently implemented on all of these platforms.

Assuming a fixed cost for table lookup operations and interactions with extern objects, all P4 programs (i.e., parsers and controls) execute a constant number of operations for each byte of an input packet received and analyzed. Although parsers may contain loops, provided some header is extracted on each cycle, the packet itself provides a bound on the total execution of the parser. In other words, under these assumptions, the computational complexity of a P4 program is linear in the total size of all headers, and never depends on the size of the state accumulated while processing data (e.g., the number of flows, or the total number of packets processed). These guarantees are necessary (but not sufficient) for enabling fast packet processing across a variety of targets.

P4 conformance of a target is defined as follows: if a specific target T supports only a subset of the P4 programming language, say $P4^T$, programs written in $P4^T$ executed on the target should provide the exact same behavior as is described in this document. Note that P4 conformant targets can provide arbitrary P4 language extensions and **extern** elements.

3.1. Benefits of P4

Compared to state-of-the-art packet-processing systems (e.g., based on writing microcode on top of custom hardware), P4 provides a number of significant advantages:

- **Flexibility:** P4 makes many packet-forwarding policies expressible as programs, in contrast to traditional switches, which expose fixed-function forwarding engines to their users.
- **Expressiveness:** P4 can express sophisticated, hardware-independent packet processing algorithms using solely general-purpose operations and table look-ups. Such programs are portable across hardware targets that implement the same architectures (assuming sufficient resources are available).
- **Resource mapping and management:** P4 programs describe storage resources abstractly (e.g., IPv4 source address); compilers map such user-defined fields to available hardware resources and manage low-level details such as allocation and scheduling.
- **Software engineering:** P4 programs provide important benefits such as type checking, information hiding, and software reuse.
- **Component libraries:** Component libraries supplied by manufacturers can be used to wrap hardware-specific functions into portable high-level P4 constructs.
- **Decoupling hardware and software evolution:** Target manufacturers may use abstract architectures to further decouple the evolution of low-level architectural details from high-level processing.
- **Debugging:** Manufacturers can provide software models of an architecture to aid in the development and debugging of P4 programs.

3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1)

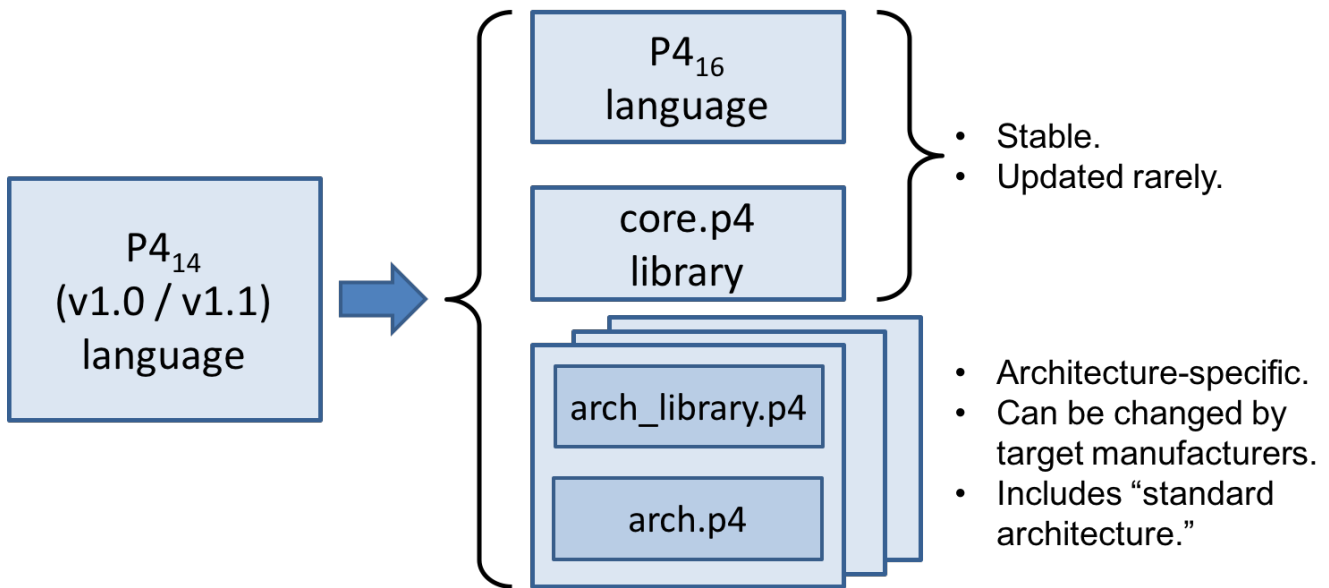


Figure 3. Evolution of the language between versions $P4_{14}$ (versions 1.0 and 1.1) and $P4_{16}$.

Compared to $P4_{14}$, the earlier version of the language, $P4_{16}$ makes a number of significant, backwards-incompatible changes to the syntax and semantics of the language. The evolution from the previous version ($P4_{14}$) to the current one ($P4_{16}$) is depicted in Figure 3. In particular, a large number of language features have been eliminated from the language and moved into libraries including counters, checksum units, meters, etc.

Hence, the language has been transformed from a complex language (more than 70 keywords) into a relatively small core language (less than 40 keywords, shown in Appendix B) accompanied by a library of fundamental constructs that are needed for writing most P4.

The v1.1 version of P4 introduced a language construct called `extern` that can be used to describe library elements. Many constructs defined in the v1.1 language specification will thus be transformed into such library elements (including constructs that have been eliminated from the language, such as counters and meters). Some of these `extern` objects are expected to be standardized, and they will be in the scope of a future document describing a standard library of P4 elements. In this document we provide several examples of `extern` constructs. $P4_{16}$ also introduces and repurposes some v1.1 language constructs for describing the programmable parts of an architecture. These language constructs are: `parser`, `state`, `control`, and `package`.

One important goal of the $P4_{16}$ language revision is to provide a **stable** language definition. In other words, we strive to ensure that all programs written in $P4_{16}$ will remain syntactically correct and behave identically when treated as programs for future versions of the language. Moreover, if some future version of the language requires breaking backwards compatibility, we will seek to provide an easy path for migrating $P4_{16}$ programs to the new version.

4. Architecture Model

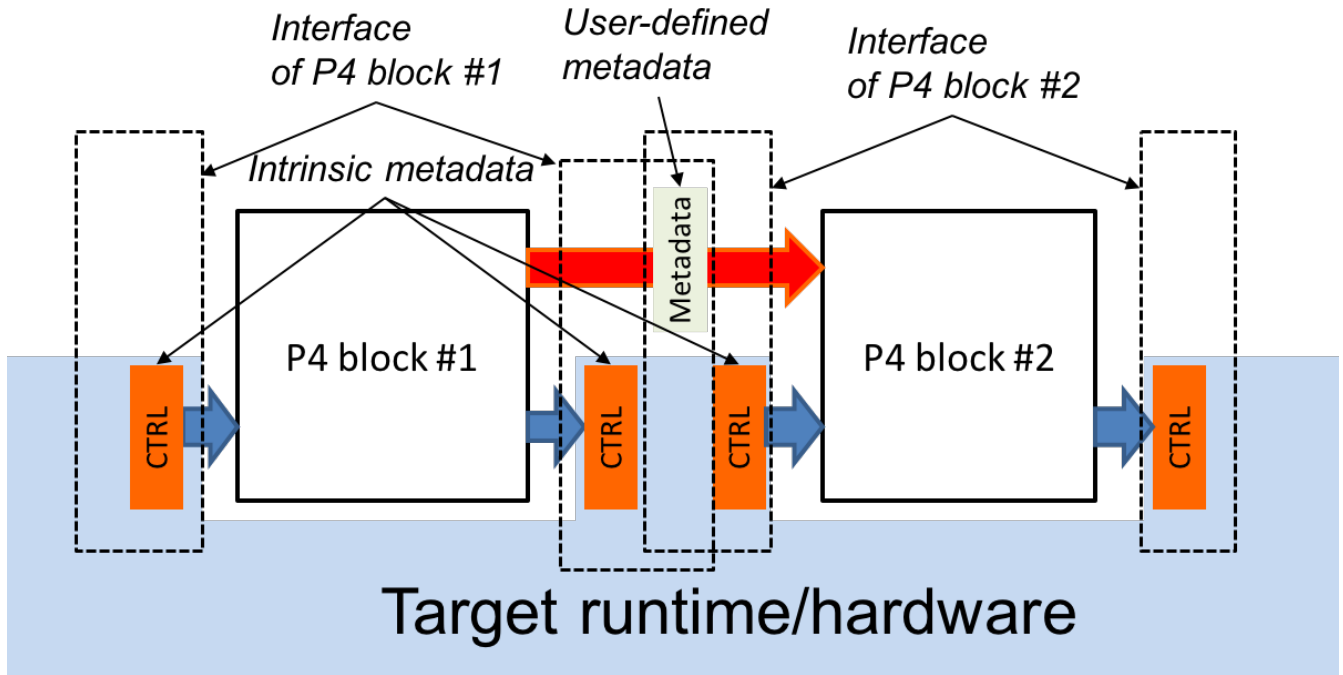


Figure 4. P4 program interfaces.

The *P4 architecture* identifies the P4-programmable blocks (e.g., parser, ingress control flow, egress control flow, deparser, etc.) and their data plane interfaces.

The P4 architecture can be thought of as a contract between the program and the target. Each manufacturer must therefore provide both a P4 compiler as well as an accompanying architecture definition for their target. (We expect that P4 compilers can share a common front-end that handles all architectures). The architecture definition does not have to expose the entire programmable surface of the data plane---a manufacturer may even choose to provide multiple definitions for the same hardware device, each with different capabilities (e.g., with or without multicast support).

Figure 4 illustrates the data plane interfaces between P4-programmable blocks. It shows a target that has two programmable blocks (#1 and #2). Each block is programmed through a separate fragment of P4 code. The target interfaces with the P4 program through a set of control registers or signals. Input controls provide information to P4 programs (e.g., the input port that a packet was received from), while output controls can be written to by P4 programs to influence the target behavior (e.g., the output port where a packet has to be directed). Control registers/signals are represented in P4 as *intrinsic metadata*. P4 programs can also store and manipulate data pertaining to each packet as *user-defined metadata*.

The behavior of a P4 program can be fully described in terms of transformations that map vectors of bits to vectors of bits. To actually process a packet, the architecture model interprets the bits that the P4 program writes to intrinsic metadata. For example, to cause a packet to be forwarded on a specific output port, a P4 program may need to write the index of an output port into a dedicated control register. Similarly, to cause a packet to be dropped, a P4 program may need to set a "drop" bit into another dedicated control register. Note that the details of how intrinsic metadata are interpreted is architecture-specific.

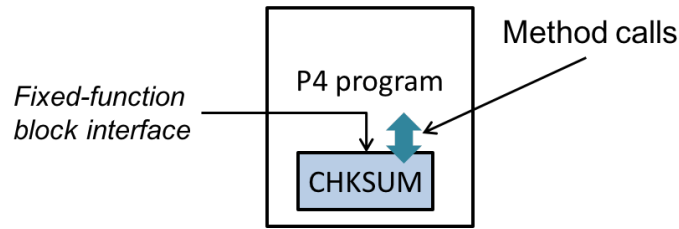


Figure 5. P4 program invoking the services of a fixed-function object.

P4 programs can invoke services implemented by extern objects and functions provided by the architecture. Figure 5 depicts a P4 program invoking the services of a built-in checksum computation unit on a target. The implementation of the checksum unit is not specified in P4, but its interface is. In general, the interface for an extern object describes each operation it provides, as well as their parameter and return types.

In general, P4 programs are not expected to be portable across different architectures. For example, executing a P4 program that broadcasts packets by writing into a custom control register will not function correctly on a target that does not have the control register. However, P4 programs written for a given architecture should be portable across all targets that faithfully implement the corresponding model, provided there are sufficient resources.

4.1. Standard architectures

We expect that the P4 community will evolve a small set of standard architecture models pertaining to specific verticals. Wide adoption of such standard architectures will promote portability of P4 programs across different targets. However, defining these standard architectures is outside of the scope of this document.

4.2. Data plane interfaces

To describe a functional block that can be programmed in P4, the architecture includes a type declaration that specifies the interfaces between the block and the other components in the architecture. For example, the architecture might contain a declaration such as the following:

```
control MatchActionPipe<H>(in bit<4> inputPort,
                           inout H parsedHeaders,
                           out bit<4> outputPort);
```

This type declaration describes a block named `MatchActionPipe` that can be programmed using a data-dependent sequence of match-action unit invocations and other imperative constructs (indicated by the `control` keyword). The interface between the `MatchActionPipe` block and the other components of the architecture can be read off from this declaration:

- The first parameter is a 4-bit value named `inputPort`. The direction `in` indicates that this parameter is an input that cannot be modified.
- The second parameter is an object of type `H` named `parsedHeaders`, where `H` is a type variable representing the headers that will be defined later by the P4 programmer. The direction `inout` indicates that this parameter is both an input and an output.
- The third parameter is a 4-bit value named `outputPort`. The direction `out` indicates that this parameter is an output whose value is undefined initially but can be modified.

4.3. Extern objects and functions

P4 programs can also interact with objects and functions provided by the architecture. Such objects are

described using the `extern` construct, which describes the interfaces that such objects expose to the data-plane.

An `extern` object describes a set of methods that are implemented by an object, but not the implementation of these methods (i.e., it is similar to an abstract class in an object-oriented language). For example, the following construct could be used to describe the operations offered by an incremental checksum unit:

```
extern Checksum16 {
    Checksum16();           // constructor
    void clear();           // prepare unit for computation
    void update<T>(in T data); // add data to checksum
    void remove<T>(in T data); // remove data from existing checksum
    bit<16> get();          // get the checksum for the data added since last clear
}
```

5. Example: A very simple switch

As an example to illustrate the features of architectures, consider implementing a very simple switch in P4. We will first describe the architecture of the switch and then write a complete P4 program that specifies the data plane behavior of the switch. This example demonstrates many important features of the P4 programming language.

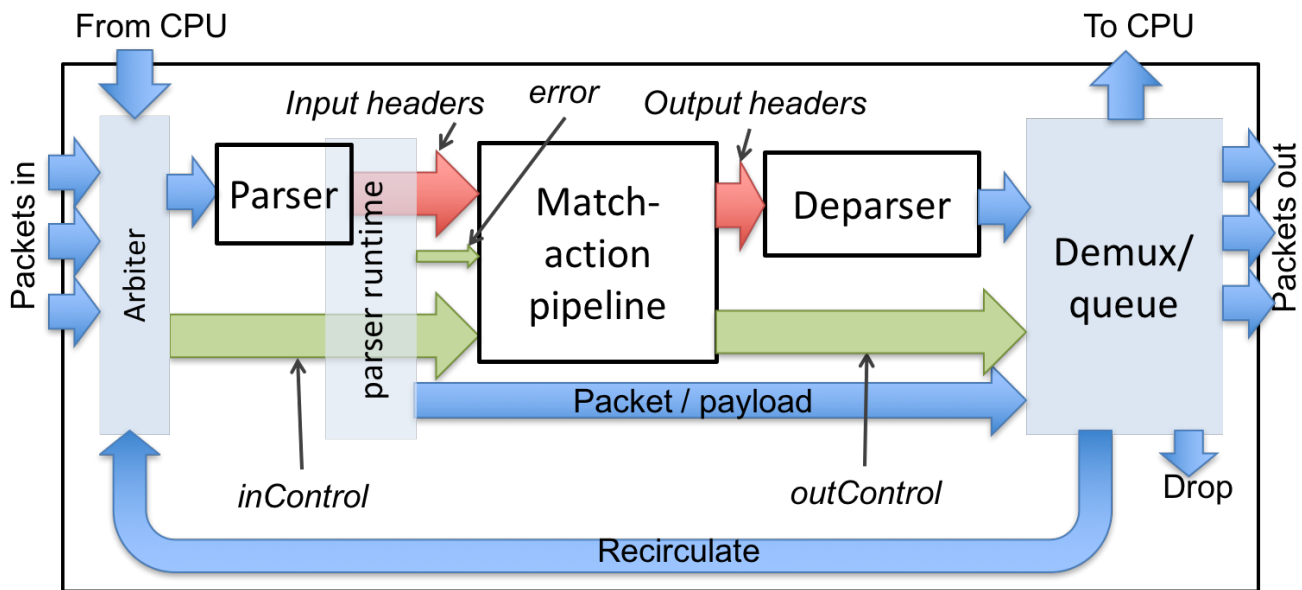


Figure 6. The Very Simple Switch (VSS) architecture.

We call our architecture the "Very Simple Switch" (VSS). [Figure 6](#) is a diagram of this architecture. There is nothing inherently special about VSS---it is just a pedagogical example that illustrates how programmable switches can be described and programmed in P4. VSS has a number of fixed-function blocks (shown in cyan in our example), whose behavior is described in [Section 5.2](#). The white blocks are programmable using P4.

VSS receives packets through one of 8 input Ethernet ports, through a recirculation channel, or from a port connected directly to the CPU. VSS has one single parser, feeding into a single match-action pipeline, which feeds into a single deparser. After exiting the deparser, packets are emitted through one of 8 output Ethernet ports or one of 3 "special" ports:

- Packets sent to the "CPU port" are sent to the control plane
- Packets sent to the "Drop port" are discarded

- Packets sent to the "Recirculate port" are re-injected in the switch through a special input port

The white blocks in the figure are programmable, and the user must provide a corresponding P4 program to specify the behavior of each such block. The red arrows indicate the flow of user-defined data. The cyan blocks are fixed-function components. The green arrows are data plane interfaces used to convey information between the fixed-function blocks and the programmable blocks---exposed in the P4 program as intrinsic metadata.

5.1. Very Simple Switch Architecture

The following P4 program provides a declaration of VSS in P4, as it would be provided by the VSS manufacturer. The declaration contains several type declarations, constants, and finally declarations for the three programmable blocks; the code uses syntax highlighting. The programmable blocks are described by their types; the implementation of these blocks has to be provided by the switch programmer.

```
// File "very_simple_switch_model.p4"
// Very Simple Switch P4 declaration
// core library needed for packet_in and packet_out definitions
# include <core.p4>
/* Various constants and structure declarations */
/* ports are represented using 4-bit values */
typedef bit<4> PortId;
/* only 8 ports are "real" */
const PortId REAL_PORT_COUNT = 4w8; // 4w8 is the number 8 in 4 bits
/* metadata accompanying an input packet */
struct InControl {
    PortId inputPort;
}
/* special input port values */
const PortId RECIRCULATE_IN_PORT = 0xD;
const PortId CPU_IN_PORT = 0xE;
/* metadata that must be computed for outgoing packets */
struct OutControl {
    PortId outputPort;
}
/* special output port values for outgoing packet */
const PortId DROP_PORT = 0xF;
const PortId CPU_OUT_PORT = 0xE;
const PortId RECIRCULATE_OUT_PORT = 0xD;
/* Prototypes for all programmable blocks */
/**
 * Programmable parser.
 * @param <H> type of headers; defined by user
 * @param b input packet
 * @param parsedHeaders headers constructed by parser
 */
parser Parser<H>(packet_in b,
                 out H parsedHeaders);
/**
 * Match-action pipeline
 * @param <H> type of input and output headers
 * @param headers headers received from the parser and sent to the deparser
 * @param parseError error that may have surfaced during parsing
 * @param inCtrl information from architecture, accompanying input packet
 * @param outCtrl information for architecture, accompanying output packet
 */
control Pipe<H>(inout H headers,
                in error parseError, // parser error
                in InControl inCtrl, // input port
                out OutControl outCtrl); // output port
/**
 * VSS deparser.
 * @param <H> type of headers; defined by user
```

```

* @param b output packet
* @param outputHeaders headers for output packet
*/
control Deparser<H>(inout H outputHeaders,
                    packet_out b);
/**
* Top-level package declaration - must be instantiated by user.
* The arguments to the package indicate blocks that
* must be instantiated by the user.
* @param <H> user-defined type of the headers processed.
*/
package VSS<H>(Parser<H> p,
               Pipe<H> map,
               Deparser<H> d);
// Architecture-specific objects that can be instantiated
// Checksum unit
extern Checksum16 {
    Checksum16(); // constructor
    void clear(); // prepare unit for computation
    void update<T>(in T data); // add data to checksum
    void remove<T>(in T data); // remove data from existing checksum
    bit<16> get(); // get the checksum for the data added since last clear
}

```

Let us describe some of these elements:

- The included file `core.p4` is described in more detail in [Appendix D](#). It defines some standard data-types and error codes.
- `bit<4>` is the type of bit-strings with 4 bits.
- The syntax `4w0xF` indicates the value 15 represented using 4 bits. An alternative notation is `4w15`. In many circumstances the width modifier can be omitted, writing just `15`.
- `error` is a built-in P4 type for holding error codes
- Next follows the declaration of a parser:

```

parser Parser<H>(packet_in b, out H parsedHeaders);

```

This declaration describes the interface for a parser, but not yet its implementation, which will be provided by the programmer. The parser reads its input from a `packet_in`, which is a pre-defined P4 extern object that represents an incoming packet, declared in the `core.p4` library. The parser writes its output (the `out` keyword) into the `parsedHeaders` argument. The type of this argument is `H`, yet unknown---it will also be provided by the programmer.

- The declaration

```

control Pipe<H>(inout H headers,
                in error parseError,
                in InControl inCtrl,
                out OutControl outCtrl);

```

describes the interface of a Match-Action pipeline named `Pipe`.

The pipeline receives three inputs: the headers `headers`, a parser error `parseError`, and the `inCtrl` control data. [Figure 6](#) indicates the different sources of these pieces of information. The pipeline writes its outputs into `outCtrl`, and it must update in place the headers to be consumed by the deparser.

- The top-level package is called `VSS`; in order to program a VSS, the user will have to instantiate a package of this type (shown in the next section). The top-level package declaration also depends on a

type variable `H`:

```
package VSS<H>
```

A type variable indicates a type yet unknown that must be provided by the user at a later time. In this case `H` is the type of the set of headers that the user program will be processing; the parser will produce the parsed representation of these headers, and the match-action pipeline will update the input headers in place to produce the output headers.

- The `package VSS` declaration has three complex parameters, of types `Parser`, `Pipe`, and `Deparser` respectively; which are exactly the declarations we have just described. In order to program the target one has to supply values for these parameters.
- In this program the `inCtrl` and `outCtrl` structures represent control registers. The content of the headers structure is stored in general-purpose registers.
- The `extern Checksum16` declaration describes an extern object whose services can be invoked to compute checksums.

5.2. Very Simple Switch Architecture Description

In order to fully understand VSS's behavior and write meaningful P4 programs for it, and for implementing a control plane, we also need a full behavioral description of the fixed-function blocks. This section can be seen as a simple example illustrating all the details that have to be handled when writing an architecture description. The P4 language is not intended to cover the description of all such functional blocks---the language can only describe the interfaces between programmable blocks and the architecture. For the current program, this interface is given by the `Parser`, `Pipe`, and `Deparser` declarations. In practice we expect that the complete description of the architecture will be provided as an executable program and/or diagrams and text; in this document we will provide informal descriptions in English.

5.2.1. Arbiter block

The input arbiter block performs the following functions:

- It receives packets from one of the physical input Ethernet ports, from the control plane, or from the input recirculation port.
- For packets received from Ethernet ports, the block computes the Ethernet trailer checksum and verifies it. If the checksum does not match, the packet is discarded. If the checksum does match, it is removed from the packet payload.
- Receiving a packet involves running an arbitration algorithm if multiple packets are available.
- If the arbiter block is busy processing a previous packet and no queue space is available, input ports may drop arriving packets, without indicating the fact that the packets were dropped in any way.
- After receiving a packet, the arbiter block sets the `inCtrl.inputPort` value that is an input to the match-action pipeline with the identity of the input port where the packet originated. Physical Ethernet ports are numbered 0 to 7, while the input recirculation port has a number 13 and the CPU port has the number 14.

5.2.2. Parser runtime block

The parser runtime block works in concert with the parser. It provides an error code to the match-action pipeline, based on the parser actions, and it provides information about the packet payload (e.g., the size of the remaining payload data) to the demux block. As soon as a packet's processing is completed by the parser, the match-action pipeline is invoked with the associated metadata as inputs (packet headers and user-defined metadata).

5.2.3. Demux block

The core functionality of the demux block is to receive the headers for the outgoing packet from the deparser and the packet payload from the parser, to assemble them into a new packet and to send the result to the correct output port. The output port is specified by the value of `outCtrl.outputPort`, which is set by the match-action pipeline.

- Sending the packet to the drop port causes the packet to disappear.
- Sending the packet to an output Ethernet port numbered between 0 and 7 causes it to be emitted on the corresponding physical interface. The packet may be placed in a queue if the output interface is already busy emitting another packet. When the packet is emitted, the physical interface computes a correct Ethernet checksum trailer and appends it to the packet.
- Sending a packet to the output CPU port causes the packet to be transferred to the control plane. In this case, the packet that is sent to the CPU is the **original input packet**, and not the packet received from the deparser---the latter packet is discarded.
- Sending the packet to the output recirculation port causes it to appear at the input recirculation port. Recirculation is useful when packet processing cannot be completed in a single pass.
- If the `outputPort` has an illegal value (e.g., 9), the packet is dropped.
- Finally, if the demux unit is busy processing a previous packet and there is no capacity to queue the packet coming from the deparser, **the demux unit may drop the packet**, irrespective of the output port indicated.

Please note that some of the behaviors of the demux block may be unexpected---we have highlighted them in bold. We are not specifying here several important behaviors related to queue size, arbitration, and timing, which also influence the packet processing.

The arrow shown from the parser runtime to the demux block represents an additional information flow from the parser to the demux: the packet being processed as well as the offset within the packet where parsing ended (i.e., the start of the packet payload).

5.2.4. Available extern blocks

The VSS architecture provides an incremental checksum extern block, called `Checksum16`. The checksum unit has a constructor and four methods:

- `clear()`: prepares the unit for a new computation
- `update<T>(in T data)`: add some data to be checksummed. The data must be either a bit-string, a header-typed value, or a `struct` containing such values. The fields in the header/struct are concatenated in the order they appear in the type declaration.
- `get()`: returns the 16-bit one's complement checksum. When this function is invoked the checksum must have received an integral number of bytes of data.
- `remove<T>(in T data)`: assuming that `data` was used for computing the current checksum, `data` is removed from the checksum.

5.3. A complete Very Simple Switch program

Here we provide a complete P4 program that implements basic forwarding for IPv4 packets on the VSS architecture. This program does not utilize all of the features provided by the architecture---e.g., recirculation---but it does use preprocessor `#include` directives (see [\[sec-preprocessor\]](#)).

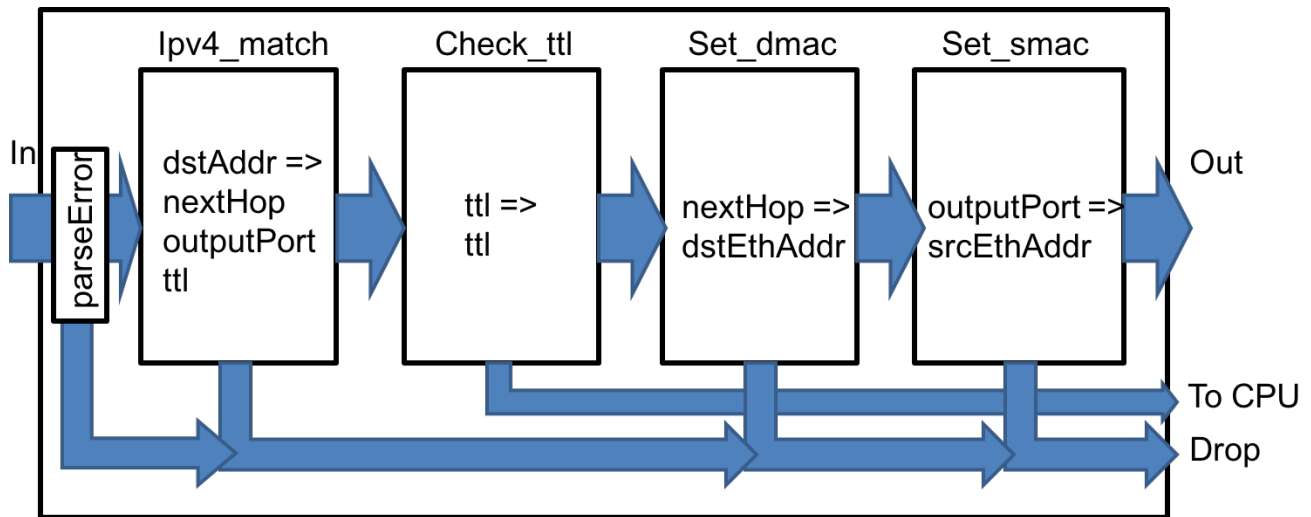


Figure 7. Diagram of the match-action pipeline expressed by the VSS P4 program.

The parser attempts to recognize an Ethernet header followed by an IPv4 header. If either of these headers are missing, parsing terminates with an error. Otherwise it extracts the information from these headers into a `Parsed_packet` structure. The match-action pipeline is shown in Figure 7; it comprises four match-action units (represented by the P4 `table` keyword):

- If any parser error has occurred, the packet is dropped (i.e., by assigning `outputPort` to `DROP_PORT`)
- The first table uses the IPv4 destination address to determine the `outputPort` and the IPv4 address of the next hop. If this lookup fails, the packet is dropped. The table also decrements the IPv4 `ttl` value.
- The second table checks the `ttl` value: if the `ttl` becomes 0, the packet is sent to the control plane through the CPU port.
- The third table uses the IPv4 address of the next hop (which was computed by the first table) to determine the Ethernet address of the next hop.
- Finally, the last table uses the `outputPort` to identify the source Ethernet address of the current switch, which is set in the outgoing packet.

The deparser constructs the outgoing packet by reassembling the Ethernet and IPv4 headers as computed by the pipeline.

```

// Include P4 core library
# include <core.p4>

// Include very simple switch architecture declarations
# include "very_simple_switch_model.p4"

// This program processes packets comprising an Ethernet and an IPv4
// header, and it forwards packets using the destination IP address

typedef bit<48>  EthernetAddress;
typedef bit<32>  IPv4Address;

// Standard Ethernet header
header Ethernet_h {
    EthernetAddress dstAddr;
    EthernetAddress srcAddr;
    bit<16> etherType;
}

// IPv4 header (without options)
header IPv4_h {
    bit<4> version;
    bit<4> ihl;
    bit<8> diffserv;

```

```

    bit<16>    totalLen;
    bit<16>    identification;
    bit<3>     flags;
    bit<13>    fragOffset;
    bit<8>     ttl;
    bit<8>     protocol;
    bit<16>    hdrChecksum;
    IPv4Address srcAddr;
    IPv4Address dstAddr;
}

// Structure of parsed headers
struct Parsed_packet {
    Ethernet_h ethernet;
    IPv4_h      ip;
}

// Parser section

// User-defined errors that may be signaled during parsing
error {
    IPv4OptionsNotSupported,
    IPv4IncorrectVersion,
    IPv4ChecksumError
}

parser TopParser(packet_in b, out Parsed_packet p) {
    Checksum16() ck; // instantiate checksum unit

    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            0x0800: parse_ipv4;
            // no default rule: all other packets rejected
        }
    }

    state parse_ipv4 {
        b.extract(p.ip);
        verify(p.ip.version == 4w4, error.IPv4IncorrectVersion);
        verify(p.ip.ihl == 4w5, error.IPv4OptionsNotSupported);
        ck.clear();
        ck.update(p.ip);
        // Verify that packet checksum is zero
        verify(ck.get() == 16w0, error.IPv4ChecksumError);
        transition accept;
    }
}

// Match-action pipeline section

control TopPipe(inout Parsed_packet headers,
    in error parseError, // parser error
    in InControl inCtrl, // input port
    out OutControl outCtrl) {
    IPv4Address nextHop; // local variable

    /**
     * Indicates that a packet is dropped by setting the
     * output port to the DROP_PORT
     */
    action Drop_action() {
        outCtrl.outputPort = DROP_PORT;
    }

    /**
     * Set the next hop and the output port.
     * Decrements ipv4 ttl field.
     * @param ipv4_dest ipv4 address of next hop

```

```

* @param port output port
*/
action Set_nhop(IPv4Address ipv4_dest, PortId port) {
    nextHop = ipv4_dest;
    headers.ip.ttl = headers.ip.ttl - 1;
    outCtrl.outputPort = port;
}

/**
* Computes address of next IPv4 hop and output port
* based on the IPv4 destination of the current packet.
* Decrements packet IPv4 TTL.
* @param nextHop IPv4 address of next hop
*/
table ipv4_match {
    key = { headers.ip.dstAddr: lpm; } // longest-prefix match
    actions = {
        Drop_action;
        Set_nhop;
    }
    size = 1024;
    default_action = Drop_action;
}

/**
* Send the packet to the CPU port
*/
action Send_to_cpu() {
    outCtrl.outputPort = CPU_OUT_PORT;
}

/**
* Check packet TTL and send to CPU if expired.
*/
table check_ttl {
    key = { headers.ip.ttl: exact; }
    actions = { Send_to_cpu; NoAction; }
    const default_action = NoAction; // defined in core.p4
}

/**
* Set the destination MAC address of the packet
* @param dmac destination MAC address.
*/
action Set_dmac(EthernetAddress dmac) {
    headers.ethernet.dstAddr = dmac;
}

/**
* Set the destination Ethernet address of the packet
* based on the next hop IP address.
* @param nextHop IPv4 address of next hop.
*/
table dmac {
    key = { nextHop: exact; }
    actions = {
        Drop_action;
        Set_dmac;
    }
    size = 1024;
    default_action = Drop_action;
}

/**
* Set the source MAC address.
* @param smac: source MAC address to use
*/
action Set_smac(EthernetAddress smac) {
    headers.ethernet.srcAddr = smac;
}

```

```

    }

    /**
     * Set the source mac address based on the output port.
     */
    table smac {
        key = { outCtrl.outputPort: exact; }
        actions = {
            Drop_action;
            Set_smac;
        }
        size = 16;
        default_action = Drop_action;
    }

    apply {
        if (parseError != error.NoError) {
            Drop_action(); // invoke drop directly
            return;
        }

        ipv4_match.apply(); // Match result will go into nextHop
        if (outCtrl.outputPort == DROP_PORT) return;

        check_ttl.apply();
        if (outCtrl.outputPort == CPU_OUT_PORT) return;

        dmac.apply();
        if (outCtrl.outputPort == DROP_PORT) return;

        smac.apply();
    }
}

// deparser section
control TopDeparser(inout Parsed_packet p, packet_out b) {
    Checksum16() ck;
    apply {
        b.emit(p.ethernet);
        if (p.ip.isValid()) {
            ck.clear(); // prepare checksum unit
            p.ip.hdrChecksum = 16w0; // clear checksum
            ck.update(p.ip); // compute new checksum.
            p.ip.hdrChecksum = ck.get();
        }
        b.emit(p.ip);
    }
}

// Instantiate the top-level VSS package
VSS(TopParser(),
    TopPipe(),
    TopDeparser()) main;

```

6. P4 language definition

This section provides a high-level overview of the P4 programming language, focusing on its syntax, semantics, lexical constructs, scoping rules, calling conventions, and naming conventions.

[Chapter 7](#) introduces the P4 abstract machine, which serves as the model for describing the behavior of P4 programs. [Chapter 8](#) through [Chapter 16](#) define the P4 syntax and semantics in detail.

NOTE | Added a high-level introduction here. This can be improved later.

6.1. Syntax and semantics

6.1.1. Grammar

The complete grammar of P4₁₆ is given in [\[sec-grammar\]](#), using Yacc/Bison grammar description language. This text is based on the same grammar. We adopt several standard conventions when we provide excerpts from the grammar:

- UPPERCASE symbols denote terminals in the grammar.
- Excerpts from the grammar are given in BNF notation as follows:

```
p4program
: /* empty */
| p4program declaration
| p4program ;
;
```

6.1.2. Semantics and the P4 abstract machines

We describe the semantics of P4 in terms of abstract machines executing traditional imperative code.

P4 compilers are free to reorganize the code they generate in any way as long as the externally visible behaviors of the P4 programs are preserved as described by this specification where externally visible behavior is defined as:

- The input/output behavior of all P4 blocks, and
- The state maintained by extern blocks.

Throughout this specification, the P4 semantics is described using a combination of:

- P4 code snippets
- Pseudo-code snippets
- Prose algorithms

6.1.2.1. Pseudo-code snippets

Pseudo-code (mostly used for describing the semantics of various P4 constructs) are shown with fixed-size fonts as in the following example:

```
ParserModel.verify(bool condition, error err) {
    if (condition == false) {
        ParserModel.parserError = err;
        goto reject;
    }
}
```

6.1.2.2. Prose algorithms

The semantics of P4 constructs are also described using prose algorithms. For example, below is the signature of an algorithm used to describe type checking of P4 expressions.

[Expr_ok](#):

- Typing `expression`, under cursor `cursor` and typing context `typingContext`:
- Results in `typedExpressionIR`.

Detailed discussion of what `cursor`, `typingContext`, and `typedExpressionIR` mean is given in [Section 7.2](#). For now, it suffices to know that `Expr_ok` takes a `cursor`, a `typingContext`, and a `typedExpressionIR` as inputs and results in a `typedExpressionIR`.

An algorithm is defined in terms of case-analysis, where each case corresponds to a variant of a P4 construct. For example, the following algorithm defines how to type check boolean literal expressions.

Typing `booleanLiteral`, under cursor `p` and typing context `TC`:

1. If `booleanLiteral` is `TRUE`:
 - a. Let `expressionNoteIR` be `type BOOL and compile-time known-ness LCTK`.
 - b. Result in `TRUE annotated with expressionNoteIR`.
2. Else:
 - a. Let `expressionNoteIR` be `type BOOL and compile-time known-ness LCTK`.
 - b. Result in `FALSE annotated with expressionNoteIR`.

And the following algorithm defines type checking of integer literal expressions.

Typing `integerLiteral`, under cursor `p` and typing context `TC`:

1. If let `D i` be `integerLiteral`:
 - a. Let `expressionNoteIR` be `type INT and compile-time known-ness LCTK`.
 - b. Result in `D i annotated with expressionNoteIR`.
2. Else if let `n S i` be `integerLiteral`:
 - a. Let `expressionNoteIR` be `type INT < n > and compile-time known-ness LCTK`.
 - b. Result in `n S i annotated with expressionNoteIR`.
3. Else:
 - a. Let `n W i` be `integerLiteral`.
 - b. Let `expressionNoteIR` be `type BIT < n > and compile-time known-ness LCTK`.
 - c. Result in `n W i annotated with expressionNoteIR`.

And so on for other variants of P4 expressions. These prose algorithms are presented throughout this specification.

6.1.2.2.1. Prose algorithm shorthands

To make the prose algorithms concise, we often use the shorthand `!`. Some algorithms result in an optional result to indicate possible failure. The following example algorithm computes modulo of two natural numbers (including zero).

`na mod nb`

1. If n_b is not equal to 0:
 - a. Return $n_a \setminus n_b$.
2. If n_b is equal to 0:
 - a. Return $\cdot$.

Because the divisor may be zero, the result is optional. Now consider the case when the algorithm is called.

$n \bmod 42$

1. Let n_{result} be $! n \bmod 42$.
2. Return n_{result} .

Here, $!$ indicates unwrapping of the optional result. This shortened form is equivalent to first getting the optional result, checking that it is not `None`, and then unwrapping it.

6.2. Lexical constructs

All P4 keywords use only ASCII characters. All P4 identifiers must use only ASCII characters. P4 compilers should handle correctly strings containing 8-bit characters in comments and string literals. P4 is case-sensitive. Whitespace characters, including newlines are treated as token separators. Indentation is free-form; however, P4 has C-like block constructs, and all our examples use C-style indentation. Tab characters are treated as spaces.

The lexer recognizes the following kinds of terminals:

- **IDENTIFIER**: start with a letter or underscore, and contain letters, digits and underscores
- **TYPE_IDENTIFIER**: identifier that denotes a type name
- **INTEGER**: integer literals
- **DONTCARE**: a single underscore
- Keywords such as **RETURN**. By convention, each keyword terminal corresponds to a language keyword with the same spelling but using lowercase. For example, the **RETURN** terminal corresponds to the `return` keyword.

6.2.1. Identifiers

P4 identifiers may contain only letters, numbers, and the underscore character `_`, and must start with a letter or underscore. The special identifier consisting of a single underscore `_` is reserved to indicate a "don't care" value; its type may vary depending on the context. Certain keywords (e.g., `apply`) can be used as identifiers if the context makes it unambiguous.

```

identifier
: `ID text
;

nonTypeName
: identifier
| APPLY
| KEY
| ACTIONS
| STATE

```

```

| ENTRIES
| TYPE
| PRIORITY
;

typeIdentifier
: `TID text
;

typeName = typeIdentifier

name
: nonTypeName
| typeName
| LIST
;

```

6.2.2. Literal constants

6.2.2.1. Boolean literals

```

booleanLiteral
: TRUE
| FALSE
;

```

There are two Boolean literal constants: `true` and `false`.

6.2.2.2. Integer literals

```

integerLiteral
: D int
| nat W int
| nat S int
;

```

Integer literals are non-negative arbitrary-precision integers. By default, literals are represented in base 10. The following prefixes must be employed to specify the base explicitly:

- `0x` or `0X` indicates base 16 (hexadecimal)
- `0o` or `0O` indicates base 8 (octal)
- `0d` or `0D` indicates base 10 (decimal)
- `0b` or `0B` indicates base 2

The width of a numeric literal in bits can be specified by an unsigned number prefix consisting of a number of bits and a signedness indicator:

- `w` indicates unsigned numbers
- `s` indicates signed numbers

Note that a leading zero by itself does not indicate an octal (base 8) constant. The underscore character is considered a digit within number literals but is ignored when computing the value of the parsed number. This allows long constant numbers to be more easily read by grouping digits together. The underscore cannot be used in the width specification or as the first character of an integer literal. No comments or whitespaces are allowed within a literal. Here are some examples of numeric literals:

```

32w255      // a 32-bit unsigned number with value 255
32w0d255    // same value as above
32w0xFF     // same value as above
32s0xFF     // a 32-bit signed number with value 255
8w0b10101010 // an 8-bit unsigned number with value 0xAA
8w0b_1010_1010 // same value as above
8w170      // same value as above
8s0b1010_1010 // an 8-bit signed number with value -86
16w0377     // 16-bit unsigned number with value 377 (not 255!)
16w0o377    // 16-bit unsigned number with value 255 (base 8)

```

6.2.2.3. String literals

```

stringLiteral
: " text "
;

```

String literals are specified as an arbitrary sequence of 8-bit characters, enclosed within double quote characters " (ASCII code 34). Strings start with a double quote character and extend to the first double quote sign which is not immediately preceded by an odd number of backslash characters (ASCII code 92). P4 does not make any validity checks on strings (i.e., it does not check that strings represent legal UTF-8 encodings).

Since P4 does not allow strings to exist at runtime, string literals are generally passed unchanged through the P4 compiler to other third-party tools or compiler-backends. The compiler can, however, perform compile-time concatenation (constant-folding) of concatenation expressions into single literal. When such concatenation is performed, the binary representation of the string literals (excluding the quotes) is concatenated in the order they appear in the source code. There are no escape sequences that would be treated specially when strings are concatenated.

The backends and other tools can define their own handling of escape sequences (e.g., how to specify Unicode characters, or handle unprintable ASCII characters).

Here are 3 examples of string literals:

```

"simple string"
"string \" with \" embedded \" quotes"
"string with embedded
line terminator"

```

Here is an example of concatenation expression and an equivalent string literal:

```

"one string \" with a quote inside;" ++ (" " ++ "another string")
// can be constant folded to
"one string \" with a quote inside; another string"

```

6.2.3. Comments

P4 supports several kinds of comments:

- Single-line comments, introduced by `//` and spanning to the end of line,
- Multi-line comments, enclosed between `/*` and `*/`
- Nested multi-line comments are not supported.
- Javadoc-style comments, starting with `/**` and ending with `*/`

Use of Javadoc-style comments is strongly encouraged for the tables and actions that are used to synthesize the interface with the control-plane.

P4 treats comments as token separators and no comments are allowed within a token---e.g. `bi/**/t` is parsed as two tokens, `bi` and `t`, and not as a single token `bit`.

6.2.4. Optional trailing commas

The P4 grammar allows several kinds of comma-separated lists to end in an optional comma.

```
trailingCommaOpt
: /* empty */
| ,
;
```

For example, the following declarations are both legal, and have the same meaning:

```
enum E {
    a, b, c
}

enum E {
    a, b, c,
}
```

This is particularly useful in combination with preprocessor directives:

```
enum E {
#ifdef SUPPORT_A
    a,
#endif
    b,
    c,
}
```

6.3. Scoping

Scopes are used to organize P4 programs into nested namespaces. There are three kinds of scopes in P4, from outermost to innermost: the top-level scope (global namespace), the scope within a `parser`, `control`, or `extern` declaration (block namespace), and the scope within a block statement (local namespace). Local namespaces can be nested within other local namespaces.

```
// top-level (global) scope
const bit<8> GLOBAL = 255;

control c() {
    // block scope within a control
    bit<8> block = GLOBAL;

    apply {
        // local scope
        bit<8> local = block;
    }
}

bit<8> f() {
    // local scope
```

```

bit<8> local = GLOBAL;
{
    // nested local scope
    bit<8> local_nested = local;
}
return local;
}

```

Cursors are used to track scopes while defining the semantic rules of P4.

```

cursor
: GLOBAL
| BLOCK
| LOCAL
;

```

NOTE

The division of global/block/local scopes and `cursor` is newly introduced. Maybe add more explanation here later. Also introduce a metavariable for `cursor`.

6.3.1. Name resolution

P4 namespaces are organized in a hierarchical fashion.

```

prefixedNonTypeName
: nonTypeName
| `ID . nonTypeName
;

prefixedTypeName
: typeName
| `TID . typeName
;

```

Identifiers prefixed with a dot are always resolved in the top-level namespace.

```

const bit<32> x = 2;
control c() {
    int<32> x = 0;
    apply {
        x = x + (int<32>).x; // x is the int<32> local variable,
                           // .x is the top-level bit<32> variable
    }
}

```

References to resolve an identifier are attempted inside-out, starting with the current scope and proceeding to all lexically enclosing scopes. The compiler may provide a warning if multiple resolutions are possible for the same name (name shadowing).

```

const bit<4> x = 1;
control c() {
    const bit<8> x = 8; // x declaration shadows global x
    const bit<4> y = .x; // reference to top-level x
    const bit<8> z = x; // reference to c's local x
    apply {}
}

```

The order of declarations is important; with the exception of parser states, all uses of a symbol must

follow the symbol's declaration. (This is a departure from P4₁₄, which allows declarations in any order. This requirement significantly simplifies the implementation of compilers for P4, allowing compilers to use additional information about declared identifiers to resolve ambiguities.)

6.3.2. Name visibility

Identifiers defined in the top-level namespace are globally visible. Declarations within a `parser` or `control` are private and cannot be referred to from outside of the enclosing `parser` or `control`.

6.4. Stateful elements

Most P4 constructs are stateless: given some inputs they produce a result that solely depends on these inputs. There are only two stateful constructs that may retain information across packets:

- `tables`: Tables are read-only for the data plane, but their entries can be modified by the control-plane,
- `extern` objects: many objects have state that can be read and written by the control plane and data plane. All constructs from the P4₁₄ language version that encapsulate state (e.g., counters, meters, registers) are represented using `extern` objects in P4₁₆.

In P4 all stateful elements must be explicitly allocated at compilation-time through the process called "instantiation".

In addition, `parsers`, `control` blocks, and `packages` may contain stateful element instantiations. Thus, they are also treated as stateful elements, even if they appear to contain no state, and must be instantiated before they can be used. However, although they are stateful, `tables` do not need to be instantiated explicitly---declaring a `table` also creates an instance of it. This convention is designed to support the common case, since most tables are used just once. To have finer-grained control over when a `table` is instantiated, a programmer can declare it within a `control`.

Recall the example in [Section 5.3](#): `TopParser`, `TopPipe`, `TopDeparser`, `Checksum16`, and `Switch` are types. There are two instances of `Checksum16`, one in `TopParser` and one in `TopDeparser`, both called `ck`. The `TopParser`, `TopDeparser`, `TopPipe`, and `Switch` are instantiated at the end of the program, in the declaration of the `main` instance object, which is an instance of the `Switch` type (a `package`).

6.5. Call convention: copy-in/copy-out

P4 provides multiple constructs for writing modular programs: `extern` methods, `parsers`, `controls`, `actions`. All these constructs behave similarly to procedures in standard general-purpose programming languages:

- They have named and typed parameters.
- They introduce a new local scope for parameters and local variables.
- They allow arguments to be passed by binding them to their parameters.

```
parameterList
: /* empty */
| nonEmptyParameterList
;

nonEmptyParameterList
: parameter
| nonEmptyParameterList , parameter
;

parameter
```

```

: annotationList direction type name initializerOpt
;

direction
: /* empty */
| IN
| OUT
| INOUT
;

```

Invocations are executed using copy-in/copy-out semantics.

Each parameter may be labeled with a direction:

- **in** parameters are read-only. It is an error to use an **in** parameter on the left-hand side of an assignment or to pass it to a callee as a non-**in** argument. **in** parameters are initialized by copying the value of the corresponding argument when the invocation is executed.
- **out** parameters are, with a few exceptions listed below, uninitialized and are treated as l-values (See [\[sec-lvalues\]](#)) within the body of the method or function. An argument passed as an **out** parameter must be an l-value; after the execution of the call, the value of the parameter is copied to the corresponding storage location for that l-value.
- **inout** parameters behave like a combination of **in** and **out** parameters simultaneously: On entry the value of the arguments is copied to the parameters. On return the value of the parameters is copied back to the arguments. In consequence, an argument passed as an **inout** parameter must be an l-value.
- The meaning of parameters with no direction depends upon the kind of entity the parameter is for:
 - For anything other than an action, e.g. a control, parser, or function, a directionless parameter means that the value supplied as an argument in a call must be a compile-time known value (see [Section 7.5](#)).
 - For an action, a directionless parameter indicates that it is "action data". See [\[sec-actions\]](#) for the meaning of action data, but its meaning includes the following possibilities:
 - The parameter's value is provided in the P4 program. In this case, the parameter behaves as if the direction were **in**. Such an argument expression need not be a compile-time known value.
 - The parameter's value is provided by the control plane software when an entry is added to a table that uses that action. See [\[sec-actions\]](#).
 - A directionless parameter of an extern object type is passed by reference.

Direction **out** parameters are always initialized at the beginning of execution of the portion of the program that has the **out** parameters, e.g. **control**, **parser**, **action**, function, etc. This initialization is not performed for parameters with any direction that is not **out**.

- If a direction **out** parameter is of type **header** or **header_union**, it is set to "invalid".
- If a direction **out** parameter is of type header stack, all elements of the header stack are set to "invalid", and its **nextIndex** field is initialized to 0 (see [\[sec-expr-hs\]](#)).
- If a direction **out** parameter is a compound type, e.g. a struct or tuple, other than one of the types listed above, then apply these rules recursively to its members.
- If a direction **out** parameter has any other type, e.g. **bit<W>**, an implementation need not initialize it to any predictable value.

For example, if a direction **out** parameter has type **s2_t** named **p**:

```

header h1_t {
    bit<8> f1;
}

```

```

    bit<8> f2;
}
struct s1_t {
    h1_t h1a;
    bit<3> a;
    bit<7> b;
}
struct s2_t {
    h1_t h1b;
    s1_t s1;
    bit<5> c;
}

```

then at the beginning of execution of the part of the program that has the `out` parameter `p`, it must be initialized so that `p.h1b` and `p.s1.h1a` are invalid. No other parts of `p` are required to be initialized.

Arguments are evaluated from left to right prior to the invocation of the function itself. The order of evaluation is important when the expression supplied for an argument can have side-effects. Consider the following example:

```

extern void f(inout bit x, in bit y);
extern bit g(inout bit z);
bit a;
f(a, g(a));

```

Note that the evaluation of `g` may mutate its argument `a`, so the compiler has to ensure that the value passed to `f` for its first parameter is not changed by the evaluation of the second argument. The semantics for evaluating a function call is given by the following algorithm (implementations can be different as long as they provide the same result):

1. Arguments are evaluated from left to right as they appear in the function call expression.
2. If a parameter has a default value and no corresponding argument is supplied, the default value is used as an argument.
3. For each `out` and `inout` argument the corresponding l-value is saved (so it cannot be changed by the evaluation of the following arguments). This is important if the argument contains indexing operations into a header stack.
4. The value of each argument is saved into a temporary.
5. The function is invoked with the temporaries as arguments. We are guaranteed that the temporaries that are passed as arguments are never aliased to each other, so this "generated" function call can be implemented using call-by-reference if supported by the architecture.
6. On function return, the temporaries that correspond to `out` or `inout` arguments are copied in order from left to right into the l-values saved in Step 3.

According to this algorithm, the previous function call is equivalent to the following sequence of statements:

```

bit tmp1 = a;    // evaluate a; save result
bit tmp2 = g(a); // evaluate g(a); save result; modifies a
f(tmp1, tmp2);   // evaluate f; modifies tmp1
a = tmp1;        // copy inout result back into a

```

To see why Step 3 in the above algorithm is important, consider the following example:

```

header H { bit z; }
H[2] s;

```

```
f(s[a].z, g(a));
```

The evaluation of this call is equivalent to the following sequence of statements:

```
bit tmp1 = a;           // save the value of a
bit tmp2 = s[tmp1].z;    // evaluate first argument
bit tmp3 = g(a);         // evaluate second argument; modifies a
f(tmp2, tmp3);           // evaluate f; modifies tmp2
s[tmp1].z = tmp2;        // copy inout result back; dest is not s[a].z
```

When used as arguments, **extern** objects can only be passed as directionless parameters---e.g., see the packet argument in the very simple switch example.

NOTE Put a forward reference to [Chapter 17](#).

6.5.1. Justification

The main reason for using copy-in/copy-out semantics (instead of the more common call-by-reference semantics) is for controlling the side-effects of **extern** functions and methods. **extern** methods and functions are the main mechanism by which a P4 program communicates with its environment. With copy-in/copy-out semantics **extern** functions cannot hold references to P4 program objects; this enables the compiler to limit the side-effects that **extern** functions may have on the P4 program both in space (they can only affect **out** parameters) and in time (side-effects can only occur at function call time).

In general, **extern** functions are arbitrarily powerful: they can store information in global storage, spawn separate threads, "collude" with each other to share information --- but they cannot access any variable in a P4 program. With copy-in/copy-out semantics the compiler can still reason about P4 programs that invoke **extern** functions.

There are additional benefits of using copy-in copy-out semantics:

- It enables P4 to be compiled for architectures that do not support references (e.g., where all data is allocated to named registers. Such architectures may require indices into header stacks that appear in a program to be compile-time known values.)
- It simplifies some compiler analyses, since function parameters can never alias to each other within the function body.

Following is a summary of the constraints imposed by the parameter directions:

- When used as arguments, **extern** objects can only be passed as directionless parameters.
- All constructor parameters are evaluated at compilation-time, and in consequence they must all be directionless (they cannot be **in**, **out**, or **inout**); this applies to **package**, **control**, **parser**, and **extern** objects. Expressions for these parameters must be supplied at compile-time, and they must evaluate to compile-time known values. See [\[sec-parameterization\]](#) for further details.
- For actions all directionless parameters must be at the end of the parameter list. When an action appears in a **table**'s **actions** list, only the parameters with a direction must be bound. See [\[sec-actions\]](#) for further details.
- Actions can also be explicitly invoked using function call syntax, either from a control block or from another action. In this case, values for all action parameters must be supplied explicitly, including values for the directionless parameters. The directionless parameters in this case behave like **in** parameters. See [\[sec-invoke-actions\]](#) for further details.
- Default expressions are only allowed for **in** or directionless parameters, and the expressions supplied as defaults must be compile-time known values.

- If parameters with default values do not appear at the end of the list of parameters, invocations that use the default values must use named arguments, as in the following example:

```
extern void f(in bit a, in bit<3> b = 2, in bit<5> c);

void g() {
  f(a = 1, b = 2, c = 3); // ok
  f(a = 1, c = 3);       // ok, equivalent to the previous call, b uses default value
  f(1, 2, 3);            // ok, equivalent to the previous call
  // f(1, 3); // illegal, since the parameter b is not the last in the list
}
```

6.6. Overloading

Functions, methods, and constructors are the only P4 constructs that support overloading: there can exist multiple methods with the same name in the same scope. When overloading is used, the compiler must be able to disambiguate at compile-time which method, function, or constructor is being called, either by the number of arguments or by the names of the arguments, when calls are specifying argument names. Argument type information is not used in disambiguating calls.

Notice that overloading of abstract methods, parsers, controls, or packages is not allowed:

```
parser p(packet_in p, out bit<32> value) {
  ...
}

// The following will cause an error about a duplicate declaration
//parser p(packet_in p, out Headers headers) {
//  ...
//}
```

6.7. Naming conventions

P4 provides a rich assortment of types. Base types include bit-strings, numbers, and errors. There are also built-in types for representing constructs such as parsers, pipelines, actions, and tables. Users can construct new types based on these: structures, enumerations, headers, header stacks, header unions, etc.

In this document we adopt the following conventions:

- Built-in types are written with lowercase characters---e.g., `int<20>`,
- User-defined types are capitalized---e.g., `IPv4Address`,
- Type variables are always uppercase---e.g., `parser P<H, IH>()`,
- Variables are uncapitalized--- e.g., `ipv4header`,
- Constants are written with uppercase characters---e.g., `CPU_PORT`, and
- Errors and enumerations are written in camel-case--- e.g. `PacketTooShort`.

7. The P4 Abstract Machine

The evaluation of a P4 program is done in four stages:

- **preprocessing**: at compile time, the P4 program is preprocessed to handle directives such as `#include`

and `#define`.

- **type checking:** at compile time, the P4 program is type checked according to the P4 type system.
- **instantiation:** at compile time, all stateful blocks in the P4 program are instantiated.
- **dynamic evaluation:** at runtime each P4 functional block is executed to completion, in isolation, when it receives control from the architecture

The preprocessing stage is described in [\[sec-abstract-machine-preprocessing\]](#).

The type system is described in [\[sec-abstract-machine-type-checking\]](#). In addition to checking the types, the type system also converts an input P4 program into an intermediate representation (P4_{IR}). P4_{IR} is an extension of the surface syntax of P4, e.g., the types of all expressions are made explicit, implicit type casts are inserted, and omitted type arguments are inferred. P4_{IR} is described in [\[sec-abstract-machine-p4-ir\]](#).

The instantiation stage is described in [\[sec-abstract-machine-instantiation\]](#). It traverses the type checked P4_{IR} program, and recursively instantiates all stateful blocks. The result of the instantiation stage is a global store that holds all instantiated stateful blocks.

The dynamic evaluation stage is described in [\[sec-abstract-machine-dynamic-evaluation\]](#). It describes how each P4 block, represented as stateful objects within the global store, is executed to completion. These blocks are orchestrated by the architecture to compose the overall packet processing pipeline.

NOTE

Added an overview of the P4 abstract machine. The contents are either added or gathered from various sections of the P4 Language Specification.

7.1. Preprocessing

To aid the composition of programs from multiple source files, P4 compilers should support the following subset of the C preprocessor functionality:

- `#define` for defining macros (without arguments)
- `#undef`
- `#if #else #endif #ifdef #ifndef #elif`
- `#include`

The preprocessor should also remove the sequence backslash newline (ASCII codes 92, 10) to facilitate splitting content across multiple lines when convenient for formatting.

Additional C preprocessor capabilities may be supported, but are not guaranteed--e.g., macros with arguments. Similar to C, `#include` can specify a file name either within double quotes or within `<>`.

```
# include <system_file>
# include "user_file"
```

The difference between the two forms is the order in which the preprocessor searches for header files when the path is incompletely specified.

P4 compilers should correctly handle `#line` directives that may be generated during preprocessing. This functionality allows P4 programs to be built from multiple source files, potentially produced by different programmers at different times:

- the P4 core library, defined in this document,

- the architecture, defining data plane interfaces and extern blocks,
- user-defined libraries of useful components (e.g. standard protocol header definitions), and
- the P4 programs that specify the behavior of each programmable block.

7.2. Type Checking

P4₁₆ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct derived types.

7.2.1. P4 Intermediate Representation

P4_{IR} is an intermediate representation for P4 programs. P4_{IR} extends the surface syntax of P4₁₆ with additional information obtained during type checking. They include:

- All expressions are annotated with their types.
- Implicit casts are made explicit.
- Omitted type arguments are inferred and made explicit.

P4_{IR} is designed to stay as close as possible to the surface syntax of P4₁₆. This makes it easy to relate P4_{IR} programs to their source P4₁₆ programs.

Take expressions as an example.

```
expression
: literalExpression
| referenceExpression
| defaultExpression
| unaryExpression
| binaryExpression
| ternaryExpression
| castExpression
| dataExpression
| accessExpression
| callExpression
| parenthesizedExpression
;

expressionIR
: literalExpressionIR
| referenceExpressionIR
| defaultExpressionIR
| unaryExpressionIR
| binaryExpressionIR
| ternaryExpressionIR
| castExpressionIR
| dataExpressionIR
| accessExpressionIR
| callExpressionIR
| parenthesizedExpressionIR
;

expressionNoteIR
: `( typeIR ctk )
;

typedExpressionIR
: expressionIR # expressionNoteIR
;
```

`typedExpressionIR` is an intermediate representation of `expression` that is annotated with type information, `expressionNodeIR`. Nevertheless, the structure of the grammar mostly stays the same.

```
binaryExpression
: expression binop expression
;

binaryExpressionIR
: typedExpressionIR binop typedExpressionIR
;
```

`binaryExpressionIR` is an intermediate representation of `binaryExpression`. The only difference is that the operands are now of type `typedExpressionIR`.

NOTE | Added explanation for `P4IR`. More explanation needed?

7.2.2. Typing a P4 program

A `P416` program is a sequence of declarations:

```
p4program
: /* empty */
| p4program declaration
| p4program ;
;
```

For a well-typed `P416` program, `p4program`, a type checker produces a `P4IR` program, `p4programIR`:

```
p4programIR
: declarationIR* ;
;
```

`Program_ok` type checks a `p4program` and produces a `p4programIR`.

Program_ok:

- Typing `p4program`:
- Results in the typed P4 program `p4programIR`.

Below is a prose algorithm that implements `Program_ok`. Details of what each meta-variable means are provided in the subsequent sections.

Typing `p4program`:

1. Let `declaration*` be the list of declarations in `p4program`.
2. Let `TC0` be an empty typing context.
3. Let the updated typing context `TC1` and `declarationIR*` be
 - the result of typing `declaration*` under typing context `TC0`.
4. Result in the typed P4 program `declarationIR*`.

7.2.3. Typing context

```
globalTypingLayer = {CONSTRUCTOR constructorTypeDefEnv, TYPE typeDefEnv, CALLABLE
callableTypeDefEnv, FRAME typeFrame}

blockTypingLayer = {KIND blockKind, TYPE typeDefEnv, CALLABLE callableTypeDefEnv, FRAME
typeFrame}

localTypingLayer = {KIND localKind, TYPE typeDefEnv, FRAMES typeFrame*}

typingContext = {GLOBAL globalTypingLayer, BLOCK blockTypingLayer, LOCAL
localTypingLayer}
```

A typing context is used to keep track of the types of all named declarations in scope at a given program point. It consists of three layers:

- A global typing layer, which contains all declarations at the top level of a $P4_{16}$ program.
- A block typing layer, which contains all declarations in the current block scope.
- A local typing layer, which contains all declarations in the current local scope.

Each layer contains mappings from names to types.

```
varTypeIR
: direction typeIR ctk value?
;

typeFrame = map<id, varTypeIR>
```

`typeFrame` is a map from variable names to their types, `varTypeIR`. In addition to the type of the variable, `varTypeIR` also contains the direction of the variable (`direction`), whether it is a compile-time constant (`ctk`), and whether it is associated with a compile-time known value (`value?`).

The compile-time known-ness is discussed in [Section 7.5](#).

Notice that `localTypingLayer` contains multiple `typeFrames`. This is because a local scope can be nested with block statements.

```
typeDefIR
: nameTypeDefIR
| aliasTypeDefIR
| dataTypeDefIR
| objectTypeDefIR
;

typeDefEnv = map<typeId, typeDefIR>
```

`typeDefEnv` is a map from type names to their type definitions, `typeDefIR`. `typeDefIR` is a type constructor, derived from user-defined type declarations.

```
callableTypeDefIR
: actionTypeDefIR
| functionTypeDefIR
| methodTypeDefIR
;

callableTypeDefEnv = map<callableId, callableTypeDefIR>
```

Functions, methods, and actions can be defined in P_{416} . These are collectively called callables. `callableTypeDefEnv` is a map from callable names to their type definitions, `callableTypeDefIR`.

```
constructorTypeDefIR
  : CONSTRUCTOR `< typeParameterIR* , typeParameterIR* > `( constructorParameterIR* ) :
  typeIR
  ;

constructorTypeDefEnv = map<constructorId, constructorTypeDefIR>
```

Stateful objects can be defined in P_{416} , which are instantiated using constructors. `constructorTypeDefEnv` is a map from constructor names to their type definitions, `constructorTypeDefIR`.

NOTE | Added explanation for the typing context. Is this the right place?

7.3. Instantiation

Instantiation of a program proceeds in order of declarations, starting in the top-level namespace:

- All declarations (e.g., `parsers`, `controls`, types, constants) evaluate to themselves.
- Each `table` evaluates to a table instance.
- Constructor invocations evaluate to stateful objects of the corresponding type. For this purpose, all constructor arguments are evaluated recursively and bound to the constructor parameters. Constructor arguments must be compile-time known values. The order of evaluation of the constructor arguments should be unimportant --- all evaluation orders should produce the same results.
- Instantiations evaluate to named stateful objects.
- The instantiation of a `parser` or `control` block recursively evaluates all stateful instantiations declared in the block.
- The result of the program's evaluation is the value of the top-level `main` variable.

Note that all stateful values are instantiated at compilation time.

As an example, consider the following program fragment:

```
// architecture declaration
parser P(/* parameters omitted */);
control C(/* parameters omitted */);
control D(/* parameters omitted */);

package Switch(P prs, C ctrl, D dep);

extern Checksum16 { /* body omitted */}

// user code
Checksum16() ck16; // checksum unit instance

parser TopParser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}
control Pipe(/* parameters omitted */) { /* body omitted */}
control TopDeparser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}

Switch(TopParser(ck16),
      Pipe(),
      TopDeparser(ck16)) main;
```

The evaluation of this program proceeds as follows:

1. The declarations of `P`, `C`, `D`, `Switch`, and `Checksum16` all evaluate to themselves.
2. The `Checksum16() ck16` instantiation is evaluated and it produces an object named `ck16` with type `Checksum16`.
3. The declarations for `TopParser`, `Pipe`, and `TopDeparser` evaluate as themselves.
4. The `main` variable instantiation is evaluated:
 - a. The arguments to the constructor are evaluated recursively.
 - b. `TopParser(ck16)` is a constructor invocation.
 - i. Its argument is evaluated recursively; it evaluates to the `ck16` object.
 - c. The constructor itself is evaluated, leading to the instantiation of an object of type `TopParser`.
 - d. Similarly, `Pipe()` and `TopDeparser(ck16)` are evaluated as constructor calls.
 - e. All the arguments of the `Switch` package constructor have been evaluated (they are an instance of `TopParser`, an instance of `Pipe`, and an instance of `TopDeparser`). Their signatures are matched with the `Switch` declaration.
 - f. Finally, the `Switch` constructor can be evaluated. The result is an instance of the `Switch` package (that contains a `TopParser` named `prs` the first parameter of the `Switch`; a `Pipe` named `ctrl`; and a `TopDeparser` named `dep`).
5. The result of the program evaluation is the value of the `main` variable, which is the above instance of the `Switch` package.

Figure 8 shows the result of the evaluation in a graphical form. The result is always a graph of instances. There is only one instance of `Checksum16`, called `ck16`, shared between the `TopParser` and `TopDeparser`. Whether this is possible is architecture-dependent. Specific target compilers may require distinct checksum units to be used in distinct blocks.

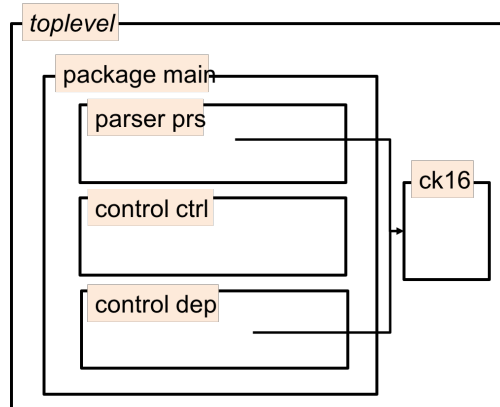


Figure 8. Evaluation result.

7.3.1. Control-plane names

Every controllable entity exposed in a P4 program must be assigned a unique, fully-qualified name, which the control plane may use to interact with that entity. The following entities are controllable.

- value sets
- tables
- keys
- actions
- extern instances

A fully qualified name consists of the local name of a controllable entity prepended with the fully

qualified name of its enclosing namespace. Hence, the following program constructs, which enclose controllable entities, must themselves have unique, fully-qualified names.

- control instances
- parser instances

Evaluation may create multiple instances from one type, each of which must have a unique, fully-qualified name.

7.3.1.1. Computing control-plane names

The fully-qualified name of a construct is derived by concatenating the fully-qualified name of its enclosing construct with its local name. Constructs with no enclosing namespace, i.e. those defined at the global scope, have the same local and fully-qualified names. The local names of controllable entities and enclosing constructs are derived from the syntax of a P4 program as follows.

7.3.1.1.1. Value sets

For each `value_set` construct, its syntactic name becomes the local name of the value set. For example:

```
struct vsk_t {
    @match(ternary)
    bit<16> port;
}
value_set<vsk_t>(4) pvs;
```

This `value_set`'s local name is `pvs`.

NOTE | `value_set` is not properly modeled in P4-SpecTec yet.

7.3.1.1.2. Tables

For each `table` construct, its syntactic name becomes the local name of the table. For example:

```
control c(/* parameters omitted */)() {
    table t { /* body omitted */ }
}
```

This table's local name is `t`.

7.3.1.1.3. Keys

Syntactically, table keys are expressions. For simple expressions, the local key name can be generated from the expression itself; the algorithm by which a compiler derives control-plane names for complex key expressions is target-dependent.

The spec suggests, but does not mandate, the following algorithm for generating names for some kinds of key expressions:

Kind	Example	Name
The <code>isValid()</code> method.	<code>h.isValid()</code>	"h.isValid()"
Array accesses.	<code>header_stack[1]</code>	"header_stack[1]"
Constants.	<code>1</code>	"1"

Field projections.	<code>data.f1</code>	<code>"data.f1"</code>
Slices.	<code>f1[3:0]</code>	<code>"f1[3:0]"</code>
Masks.	<code>h.src & 0xFFFF</code>	<code>"h.src & 0xFFFF"</code>

In the following example, the previous algorithm would derive for table `t` two keys with names `data.f1` and `hdrs[3].f2`.

```
table t {
  keys = {
    data.f1 : exact;
    hdrs[3].f2 : exact;
  }
  actions = { /* body omitted */ }
}
```

If a compiler cannot generate a name for a key it **requires** the key expression to be annotated with a `@name` annotation ([\[sec-control-plane-api-annotations\]](#)), as in the following example:

```
table t {
  keys = {
    data.f1 + 1 : exact @name("f1_mask");
  }
  actions = { /* body omitted */ }
}
```

Here, the `@name("f1_mask")` annotation assigns the local name `"f1_mask"` to this key.

7.3.1.1.4. Actions

For each `action` construct, its syntactic name is the local name of the action. For example:

```
control c(/* parameters omitted */)( ) {
  action a(...) { /* body omitted */ }
}
```

This action's local name is `a`.

7.3.1.1.5. Instances

The local names of `extern`, `parser`, and `control` instances are derived based on how the instance is used. If the instance is bound to a name, that name becomes its local control plane name. For example, if `control C` is declared as,

```
control C(/* parameters omitted */)( ) { /* body omitted */ }
```

and instantiated as,

```
C() c_inst;
```

then the local name of the instance is `c_inst`.

Alternatively, if the instance is created as an actual argument, then its local name is the name of the formal parameter to which it will be bound. For example, if `extern E` and `control C` are declared as,

```
extern E { /* body omitted */ }
control C(/* parameters omitted */)(E e_in) { /* body omitted */ }
```

and instantiated as,

```
C(E()) c_inst;
```

then the local name of the extern instance is `e_in`.

If the construct being instantiated is passed as an argument to a package, the instance name is derived from the user-supplied type definition when possible. In the following example, the local name of the instance of `MyC` is `c`, and the local name of the `extern` is `e2`, not `e1`.

```
extern E { /* body omitted */ }
control ArchC(E e1);
package Arch(ArchC c);

control MyC(E e2)() { /* body omitted */ }
Arch(MyC()) main;
```

Note that in this example, the architecture will supply an instance of the extern when it applies the instance of `MyC` passed to the `Arch` package. The fully-qualified name of that instance is `main.c.e2`.

Next, consider a larger example that demonstrates name generation when there are multiple instances.

```
control Callee() {
  table t { /* body omitted */ }
  apply { t.apply(); }
}
control Caller() {
  Callee() c1;
  Callee() c2;
  apply {
    c1.apply();
    c2.apply();
  }
}
control Simple();
package Top(Simple s);
Top(Caller()) main;
```

The compile-time evaluation of this program produces the structure in [Figure 9](#). Notice that there are two instances of the `table t`. These instances must both be exposed to the control plane. To name an object in this hierarchy, one uses a path composed of the names of containing instances. In this case, the two tables have names `s.c1.t` and `s.c2.t`, where `s` is the name of the argument to the package instantiation, which is derived from the name of its corresponding formal parameter.

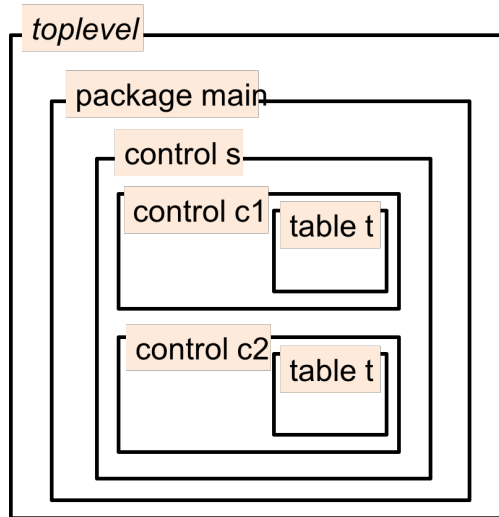


Figure 9. Evaluating a program that has several instantiations of the same component.

7.3.1.2. Annotations controlling naming

Control plane-related annotations ([\[sec-control-plane-api-annotations\]](#)) can alter the names exposed to the control plane in the following ways.

- The `@hidden` annotation hides a controllable entity from the control plane. This is the only case in which a controllable entity is not required to have a unique, fully-qualified name.
- The `@name` annotation may be used to change the local name of a controllable entity.

Programs that yield the same fully-qualified name for two different controllable entities are invalid.

NOTE Annotations are beyond the scope of P4-SpecTec at this time.

7.3.1.3. Recommendations

The control plane may refer to a controllable entity by a postfix of its fully qualified name when it is unambiguous in the context in which it is used. Consider the following example.

```
control c(/* parameters omitted */)( ) {
  action a (/* parameters omitted */) { /* body omitted */ }
  table t {
    keys = { /* body omitted */ }
    actions = { a; } }
}
c() c_inst;
```

Control plane software may refer to action `c_inst.a` as `a` when inserting rules into table `c_inst.t`, because it is clear from the definition of the table which action `a` refers to.

Not all unambiguous postfix shortcuts are recommended. For instance, consider the last example in [Section 7.3.1.1.5](#). One might be tempted to refer to `s.c1` simply as `c1`, as no other instance named `c1` appears in the program. However, this leads to a brittle program since future modifications can never introduce an instance named `c1`, or include libraries of P4 code that contain instances with that name.

7.3.2. Instantiating a P4 program

```
objectId = nameIR*
```

```

object
: externObject
| parserObject
| controlObject
| packageObject
| tableObject
| valueSetObject
;

```

The stateful elements of a P4 program are represented as **objects**. These objects are identified by their fully-qualified names (**objectId**).

```
store = map<objectId, object>
```

A $P4_{16}$ program is first type checked to yield a $P4_{IR}$ program, and is then instantiated to create a **store**, which maps **objectIds** to **objects**.

Program_inst:

- Loading **p4program**:
- Results in the global instantiation layer **globalInstLayer** and the store **store**.

Loading p4program:

1. Let the typed P4 program **declarationIR***; be
 - the result of **typing p4program**.
2. Let IC_0 be **an empty instantiation context**.
3. Let STO_0 be **an empty store**.
4. Let the updated instantiation context IC_1 and the store STO_1 be
 - the result of **loading declarationIR***, **under instantiation context IC_0** , and **store STO_0** .
5. Result in the global instantiation layer $IC_1.GLOBAL$ and the store STO_1 .

7.3.3. Instantiation context

```

globalInstLayer = {CONSTRUCTOR constructorDefEnv, TYPE typeDefEnv, CALLABLE
callableDefEnv, FRAME frame}

blockInstLayer = {TYPE theta, CALLABLE callableDefEnv, STATE stateEnv, FRAME frame}

localInstLayer = {TYPE theta, FRAMES frame*}

instContext = {PATH objectId, GLOBAL globalInstLayer, BLOCK blockInstLayer, LOCAL
localInstLayer}

```

An instantiation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global instantiation layer, which contains all declarations at the top level of a P4 program.
- A block instantiation layer, which contains all declarations in the current block scope.

- A local instantiation layer, which contains all declarations in the current local scope.

```
value
: baseValue
| dataValue
| objectReferenceValue
| synthesizedValue
;

frame = map<id, value>
```

`frame` is a map from variable names to their compile-time known values, `value`. The compile-time knownness is discussed in [Section 7.5](#).

```
typeDefIR
: nameTypeDefIR
| aliasTypeDefIR
| dataTypeDefIR
| objectTypeDefIR
;

typeDefEnv = map<typeId, typeDefIR>
```

`typeDefEnv` is a map from type names to their type definitions, `typeDefIR`. `typeDefIR` is a type constructor, derived from user-defined type declarations.

```
callableDef
: actionDef
| functionDef
| methodDef
;

callableDefEnv = map<callableId, callableDef>
```

`callableDefEnv` is a map from callable names to their definitions, `callableDef`.

```
constructorDef
: externObjectConstructorDef
| parserObjectConstructorDef
| controlObjectConstructorDef
| packageObjectConstructorDef
;

constructorDefEnv = map<callableId, constructorDef>
```

Stateful objects are instantiated using constructors. `constructorDefEnv` is a map from constructor names to their definitions, `constructorDef`.

NOTE | Added explanation for the typing context. Is this the right place?

7.4. Dynamic evaluation

The dynamic evaluation of a P4 program is orchestrated by the architecture model. Each architecture model needs to specify the order and the conditions under which the various P4 component programs are dynamically executed. For example, in the Simple Switch example from [Section 5.1](#) the execution flow goes `Parser` `Pipe` `Deparser`.

Once a P4 execution block is invoked its execution proceeds until termination according to the semantics defined in this document. P4 execution blocks are represented as objects in the global store, which is statically allocated during instantiation.

NOTE Edited contents from Section 18.4.

7.4.1. Evaluating a P4 program

P4 program evaluation proceeds by invoking the objects in the global store.

A call to a programmable block invokes the corresponding abstract machine:

Call_eval:

- Calling `callee < typeArgumentListIR > ( argumentListIR )`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

A call to an extern function invokes the corresponding extern function:

ExternFunctionCall_eval:

- Calling an extern function `nameIR ( nameIR* )`, under evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

A call to an extern method invokes the corresponding extern method:

ExternMethodCall_eval:

- Calling an extern method `nameIR ( nameIR* )` on object `objectId`, under evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

7.4.2. Evaluation context

```
globalEvalLayer = globalInstLayer
blockEvalLayer = blockInstLayer
localEvalLayer = localInstLayer
evalContext = {GLOBAL globalEvalLayer, BLOCK blockEvalLayer, LOCAL localEvalLayer}
```

An evaluation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global evaluation layer, which contains all declarations at the top level of a P4 program.
- A block evaluation layer, which contains all declarations in the current block scope.
- A local evaluation layer, which contains all declarations in the current local scope.

The layers are the same as in the instantiation context, except that the `frame` is a map from variable names to their current runtime values.

The following sections describe the abstract machines that define the dynamic semantics of the programmable blocks in a P4 target.

NOTE | Added explanation for the evaluation context. Is this the right place?

7.4.3. The parser abstract machine

The semantics of a P4 parser can be formulated in terms of an abstract machine that manipulates a `ParserModel` data structure. This section describes this abstract machine in pseudo-code.

A parser starts execution in the `start` state and ends execution when one of the `reject` or `accept` states has been reached.

```
ParserModel {
    error parseError;
    onPacketArrival(packet p) {
        ParserModel.parseError = error.NoError;
        goto start;
    }
}
```

An architecture must specify the behavior when the `accept` and `reject` states are reached. For example, an architecture may specify that all packets reaching the `reject` state are dropped without further processing. Alternatively, it may specify that such packets are passed to the next block after the parser, with intrinsic metadata indicating that the parser reached the `reject` state, along with the error recorded.

7.4.4. The match-action pipeline abstract machine

We can describe the computational model of a match-action pipeline, embodied by a control block: the body of the control block is executed, similarly to the execution of a traditional imperative program:

- At runtime, statements within a block are executed in the order they appear in the control block.
- Execution of the `return` statement causes the immediate termination of the execution of the current `control` block and a return to the caller.
- Execution of the `exit` statement causes the immediate termination of the execution of the current `control` block and of all the enclosing caller `control` blocks.
- Applying a `table` executes the corresponding match-action unit.

7.4.5. Concurrency model

A typical packet processing system needs to execute multiple simultaneous logical "threads." At the very least there is a thread executing the control plane, which can modify the contents of the tables. Architecture specifications should describe in detail the interactions between the control-plane and the data-plane. The data plane can exchange information with the control plane through `extern` function and method calls. Moreover, high-throughput packet-processing systems may be processing multiple packets simultaneously, e.g., in a pipelined fashion, or concurrently parsing a first packet while performing match-action operations on a second packet. This section specifies the semantics of P4 programs with respect to such concurrent executions.

Each top-level `parser` or `control` block is executed as a separate thread when invoked by the architecture. All the parameters of the block and all local variables are thread-local---i.e., each thread has a private copy of

these resources. This applies to the `packet_in` and `packet_out` parameters of parsers and deparsers.

As long as a P4 block uses only thread-local storage (e.g., metadata, packet headers, local variables), its behavior in the presence of concurrency is identical with the behavior in isolation, since any interleaving of statements from different threads must produce the same output.

In contrast, `extern` blocks instantiated by a P4 program are global, shared across all threads. If `extern` blocks mediate access to state (e.g., counters, registers)---i.e., the methods of the `extern` block read and write state, these stateful operations are subject to data races. P4 mandates that execution of a method call on an `extern` instance is atomic.

To allow users to express atomic execution of larger code blocks, P4 provides an `@atomic` annotation, which can be applied to block statements, parser states, control blocks, or whole parsers.

Consider the following example:

```
extern Register { /* body omitted */ }
control Ingress() {
  Register() r;
  table flowlet { /* read state of r in an action */ }
  table new_flowlet { /* write state of r in an action */ }
  apply {
    @atomic {
      flowlet.apply();
      if (ingress_metadata.flow_ipg > FLOWLET_INACTIVE_TIMEOUT)
        new_flowlet.apply();
    }
  }
}
```

This program accesses an `extern` object `r` of type `Register` in actions invoked from tables `flowlet` (reading) and `new_flowlet` (writing). Without the `@atomic` annotation these two operations would not execute atomically: a second packet may read the state of `r` before the first packet had a chance to update it.

Note that even within an `action` definition, if the action does something like reading a register, modifying it, and writing it back, in a way that only the modified value should be visible to the next packet, then, to guarantee correct execution in all cases, that portion of the action definition should be enclosed within a block annotated with `@atomic`.

A compiler backend must reject a program containing `@atomic` blocks if it cannot implement the atomic execution of the instruction sequence. In such cases, the compiler should provide reasonable diagnostics.

The P4 semantics described throughout this document assumes a sequential execution model. Thus, it does not describe the interactions between multiple threads accessing shared state. The purpose of this document is to clearly define the semantics of P4 constructs in isolation. The exact concurrency model, including the interactions between multiple threads, is target-dependent and beyond the scope of this document.

NOTE | Added disclaimer about concurrency model.

7.5. Compile-time known and local compile-time known values

Certain expressions in a P4 program have the property that their value can be determined at compile time. Moreover, for some of these expressions, their value can be determined only using information in the current scope. We call these *compile-time known values* and *local compile-time known values* respectively.

The following are local compile-time known values:

- Integer literals, Boolean literals, and string literals.
- Identifiers declared as constants using the `const` keyword.
- Identifiers declared in an `error`, `enum`, or `match_kind` declaration.
- The `default` identifier.
- The `size` field of a value with type header stack.
- The `_` identifier when used as a `select` expression label.
- The expression `{#}` representing an invalid header or header union value.
- Instances constructed by instance declarations and constructor invocations.
- Identifiers that represent declared types, actions, functions, tables, parsers, controls, or packages.
- Sequence expression where all components are local compile-time known values.
- Structure-valued expressions, where all fields are local compile-time known values.
- Expressions evaluating to a list type, where all elements are local compile-time known values.
- Legal casts applied to local compile-time known values.
- Indexing a local compile-time known stack or tuple value with a local compile-time known index.
- Accessing a field of a local compile-time known struct, header, or header union value.
- The following expressions `(, `', `|', |-, *, /, %, !, &, |, ^, &&, ||, <<`, `>>, ~, >, <, ==, !=, , >=, ++, [:, ?:)` when their operands are all local compile-time known values.
- Expressions of the form `e.minSizeInBits()`, `e.minSizeInBytes()`, `e.maxSizeInBits()` and `e.maxSizeInBytes()` where the type of `e` is not generic.

NOTE

It is questionable whether instances constructed by instance declarations and constructor invocations should be considered local compile-time known values. A related spec issue is made here: <https://github.com/p4lang/p4-spec/issues/1309>.

The following are compile-time known values:

- All local compile-time known values.
- Constructor parameters (i.e., the declared parameters for a `parser`, `control`, etc.)
- Tuple expression where all components are compile-time known values.
- Expressions evaluating to a list type, where all elements are compile-time known values.
- Structure-valued expressions, where all fields are compile-time known values.
- Expressions evaluating to a list type, where all elements are compile-time known values.
- Legal casts applied to compile-time known values.
- Indexing a compile-time known stack or tuple value with a compile-time known index.
- Accessing a field of a compile-time known struct, header, or header union value.
- The following expressions `(, `', `|', |-, *, /, %, !, &, |, ^, &&, ||, <<`, `>>, ~, >, <, ==, !=, , >=, ++, [:, ?:)` when their operands are all compile-time known values.
- Expressions of the form `e.minSizeInBits()`, `e.minSizeInBytes()`, `e.maxSizeInBits()` and `e.maxSizeInBytes()` where the type of `e` is generic.

Intuitively, the main difference between *compile-time known values* and *local compile-time known values* is that the former also contains constructor parameters. The distinction is important when it comes to defining the meaning of features like types. For example, in the type `bit<e>`, the expression `e`

must be a local compile-time known value. Suppose instead that `e` were a constructor parameter—i.e., merely a compile-time known value. In this situation, it would be impossible to resolve `bit<e>` to a concrete type using purely local information—we would have to wait until the constructor was instantiated and the value of `e` known.

Local compile-time known values can be identified in the type checking phase. Compile-time known values can be identified after type checking, during the instantiation phase. Once the constructor parameters are bound to actual arguments and type parameters are bound to actual types, compile-time known values can be evaluated to concrete values.

`ctk` is a marker used in $P4_{IR}$ to indicate whether an expression is local compile-time known, compile-time known, or neither.

```
ctk
: LCTK
| CTK
| DYN
;
```

`typedExpressionIRs` in $P4_{IR}$, produced by typing `expressions` in $P4_{16}$, annotated with their type `typeIR` and a `ctk` marker indicating the compile-time known status.

```
expressionNoteIR
: `( typeIR ctk )
;

typedExpressionIR
: expressionIR # expressionNoteIR
;
```

8. P4 types and values

$P4_{16}$ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct derived types.

The syntax for P4 types is as follows:

```
type
: baseType
| namedType
| headerStackType
| listType
| tupleType
;

typeOrVoid
: type
| VOID
| identifier
;
```

These types are represented in $P4_{IR}$ as follows:

```
typeIR
: baseTypeIR
| nameTypeIR
```

```

| aliasTypeIR
| dataTypeIR
| objectTypeIR
| synthesizedTypeIR
;

```

`typeIR` is constructed from type checking the surface syntax of P4 types (`type`) with the following relation:

`Type_ok`:

- Typing `typeOrVoid`, under cursor `cursor` and typing context `typingContext`:
- Results in `typeIR` and a set of fresh type variables `typeId*`.

The P4 type system also imposes well-formedness constraints on types, which enforce additional restrictions on the structure of types, such as type nesting.

`Type_wf`:

- `typeIR` is a well-formed type, with bound type variables `B`

The types represent the kinds of values that can be used in P4 programs. The runtime representation of P4 values is as follows:

```

value
: baseValue
| dataValue
| objectReferenceValue
| synthesizedValue
;

```

8.1. Base types and values

P4 supports the following built-in base types:

```

baseType
: BOOL
| ERROR
| MATCH_KIND
| STRING
| INT
| INT `< int >
| INT `< `( expression ) >
| BIT
| BIT `< int >
| BIT `< `( expression ) >
| VARBIT `< int >
| VARBIT `< `( expression ) >
;

```

- `bool`, which represents Boolean values
- The `error` type, which is used to convey errors in a target-independent, compiler-managed way.
- The `match_kind` type, which is used for describing the implementation of table lookups,

- The `string` type, which can be used with compile-time known values of type `string`.
- `int`, which represents arbitrary-sized integer values
- Fixed-width signed integers represented using two's complement `int<>`
- Bit-strings of fixed width, denoted by `bit<>`
- Bit-strings of dynamically-computed width with a fixed maximum width `varbit<>`

In addition, P4 supports void types:

```
typeOrVoid
: type
| VOID
| identifier
;
```

- The `void` type, which has no values and can be used only in a few restricted circumstances.

In $P4_{IR}$, base types are represented as:

```
baseTypeIR
: voidTypeIR
| boolTypeIR
| errorTypeIR
| matchKindTypeIR
| stringTypeIR
| integerTypeIR
;
```

The base values corresponding to the base types are:

```
baseValue
: boolValue
| errorValue
| matchKindValue
| stringValue
| integerValue
;
```

8.1.1. Well-formedness

The well-formedness rules for base types are as follows:

`baseTypeIR` is a well-formed type, with bound type variables `B` when:

1. The relation holds.

8.1.2. The void type

```
voidTypeIR
: VOID
;
```

The void type is written `void`. It contains no values. It is not included in the production rule `baseType` as it

can only appear in few restricted places in P4 programs.

There is no value of type `void`.

8.1.2.1. Type checking

Typing `VOID`, under cursor `p` and typing context `TC`:

1. Result in `VOID` and a set of fresh type variables `·`.

8.1.3. Booleans

```
boolTypeIR
: BOOL
;

boolValue
: `B bool
;
```

The Boolean type `bool` contains just two values, `false` and `true`. Boolean values are not integers or bit-strings.

8.1.3.1. Type checking

Typing `BOOL`, under cursor `p` and typing context `TC`:

1. Result in `BOOL` and a set of fresh type variables `·`.

8.1.4. Errors

```
errorTypeIR
: ERROR
;

errorValue
: ERROR . nameIR
;
```

The error type contains opaque distinct values that can be used to signal errors. It is written as `error`. New elements of the error type are defined with the following syntax:

```
errorDeclaration
: ERROR `{ nameList }
;
```

All elements of the `error` type are inserted into the `error` namespace, irrespective of the place where an error is defined. `error` is similar to an enumeration (`enum`) type in other languages. A program can contain multiple `error` declarations, which the compiler will merge together. It is an error to declare the same identifier multiple times.

For example, the following declaration creates two elements of the `error` type (these errors are declared in the P4 core library):

```
error { ParseError, PacketTooShort }
```

The underlying representation of errors is target-dependent.

Error declaration is described in detail in [Section 11.9](#).

8.1.4.1. Type checking

Typing `ERROR`, under cursor `p` and typing context `TC`:

1. Result in `ERROR` and a set of fresh type variables `.`

8.1.5. Match kinds

```
matchKindTypeIR
  : MATCH_KIND
  ;

matchKindValue
  : MATCH_KIND . nameIR
  ;
```

The `match_kind` type is very similar to the `error` type and is used to declare a set of distinct names that may be used in a table's key property (described in [\[sec-table-props\]](#)). All identifiers are inserted into the top-level namespace. It is an error to declare the same `match_kind` identifier multiple times.

```
matchKindDeclaration
  : MATCH_KIND `{ nameList trailingCommaOpt }
  ;
```

The P4 core library contains the following `match_kind` declaration:

```
match_kind {
  exact,
  ternary,
  lpm
}
```

Architectures may support additional `match_kinds`. The declaration of new `match_kinds` can only occur within model description files; P4 programmers cannot declare new match kinds.

Match kind declaration is described in detail in [Section 11.10](#).

8.1.5.1. Type checking

Typing `MATCH_KIND`, under cursor `p` and typing context `TC`:

1. Result in `MATCH_KIND` and a set of fresh type variables `.`

8.1.6. Strings

```
stringTypeIR
  : STRING
  ;

stringValue = stringLiteral
```

The type `string` represents strings. The values of type `string` are either string literals, or concatenations of multiple `string`-typed expressions.

One cannot declare variables with a `string` type. Parameters with type `string` can be only directionless (see Section [Chapter 17](#)). P4 does not support string manipulation in the dataplane; the `string` type is only allowed for describing compile-time known values (i.e., string literals, as discussed in Section [\[sec-string-literals\]](#)). Even so, the string type is useful, for example, in giving the type signature for extern functions such as the following:

```
extern void log(string message);
```

As another example, the following annotation indicates that the specified name should be used for a given table in the generated control-plane API:

```
@name("acl") table t1 { /* body omitted */ }
```

8.1.6.1. Type checking

Typing `STRING`, under cursor `p` and typing context `TC`:

1. Result in `STRING` and a set of fresh type variables `·`.

8.1.7. Integer types and values (signed and unsigned)

```
integerTypeIR
  : INT
  | INT `< nat >
  | BIT `< nat >
  | VARBIT `< nat >
  ;

integerLiteral
  : D int
  | nat W int
  | nat S int
  ;

integerValue
  : integerLiteral
  | nat . nat V int
  ;
```

P4 supports arbitrary-size integer values. The typing rules for the integer types are chosen according to the following principles:

- **Inspired by C:** Typing of integers is modeled after the well-defined parts of C, expanded to cope with

arbitrary fixed-width integers. In particular, the type of the result of an expression only depends on the expression operands, and not on how the result of the expression is consumed.

- **No undefined behaviors:** P4 attempts to avoid many of C's behaviors, which include the size of an integer (`int`), the results produced on overflow, and the results produced for some input combinations (e.g., shifts with negative amounts, overflows on signed numbers, etc.). P4 computations on integer types have no undefined behaviors.
- **Least surprise:** The P4 typing rules are chosen to behave as closely as possible to traditional well-behaved C programs.
- **Forbid rather than surprise:** Rather than provide surprising or undefined results (e.g., in C comparisons between signed and unsigned integers), we have chosen to forbid expressions with ambiguous interpretations. For example, P4 does not allow binary operations that combine signed and unsigned integers.

The priority of arithmetic operations is identical to C---e.g., multiplication binds tighter than addition.

8.1.7.1. Portability

No P4 target can support all possible types and operations. For example, the type `bit<23132312>` is legal in P4, but it is highly unlikely to be supported on any target in practice. Hence, each target can impose restrictions on the types it can support. Such restrictions may include:

- The maximum width supported
- Alignment and padding constraints (e.g., arithmetic may only be supported on widths which are an integral number of bytes).
- Constraints on some operands (e.g., some architectures may only support multiplications by small values, or shifts with small values).

The documentation supplied with a target should clearly specify restrictions, and target-specific compilers should provide clear error messages when such restrictions are encountered. An architecture may reject a well-typed P4 program and still be conformant to the P4 spec. However, if an architecture accepts a P4 program as valid, the runtime program behavior should match this specification.

8.1.7.2. Arbitrary-precision integers

The arbitrary-precision data type describes integers with an unlimited precision. This type is written as `int`.

This type is reserved for integer literals and expressions that involve only literals. No P4 runtime value can have an `int` type; at compile time the compiler will convert all `int` values that have a runtime component to fixed-width types, according to the rules described below.

The following example shows three constant definitions whose values are arbitrary-precision integers.

```
const int a = 5;
const int b = 2 * a;
const int c = b - a + 3;
```

Parameters with type `int` are not supported for actions. Parameters with type `int` for other callable entities of a program, e.g. controls, parsers, or functions, must be directionless, indicating that all calls must provide a compile-time known value as an argument for such a parameter. See [Chapter 17](#) for more details on directionless parameters.

8.1.7.2.1. Type checking

Typing `INT`, under cursor `p` and typing context `TC`:

1. Result in `INT` and a set of fresh type variables $\cdot$.

8.1.7.3. Fixed-width unsigned integers (bit-strings)

An unsigned integer (which we also call a "bit-string") has an arbitrary width, expressed in bits. A bit-string of width `W` is declared as: `bit<W>`. `W` must be an expression that evaluates to a local compile-time known value (see [Section 7.5](#)) that is a non-negative integer. When using an expression for the size, the expression must be parenthesized. Bitstrings with width 0 are allowed; they have no actual bits, and can only have the value 0. See [\[sec-uninitialized-values-and-writing-invalid-headers\]](#) for additional details. Note that `bit<W>` type refers to both cases of `bit<W>` and `bit<(expression)>` where the width is a local compile-time known value.

```
const bit<32> x = 10;    // 32-bit constant with value 10.
const bit<(x + 2)> y = 15; // 12-bit constant with value 15.
                        // expression for width must use ()
```

Bits within a bit-string are numbered from 0 to `W-1`. Bit 0 is the least significant, and bit `W-1` is the most significant.

For example, the type `bit<128>` denotes the type of bit-string values with 128 bits numbered from 0 to 127, where bit 127 is the most significant.

The type `bit` is a shorthand for `bit<1>`.

P4 architectures may impose additional constraints on bit types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64-bit values).

8.1.7.3.1. Type checking

Typing `baseType`, under cursor `p` and typing context `TC`:

1. If `baseType` is `BIT`:
 - a. Result in `BIT < 1 >` and a set of fresh type variables $\cdot$.
2. Else if let `BIT < int >` be `baseType`:
 - a. Check that `int` has type `nat`.
 - b. Let `n` be `int`.
 - c. Result in `BIT < n >` and a set of fresh type variables $\cdot$.
3. Else if let `BIT < ( expression ) >` be `baseType`:
 - a. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
 - b. Check that the compile-time known-ness of `typedExpressionIR` is `LCTK`.
 - c. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIR`, under cursor `p` and typing

context TC.

- d. Check that `value` has type `integerValue`.
- e. Let `integerValue` be `value`.
- f. Let `int` be the integer representation of `integerValue`.
- g. Check that `int` has type `nat`.
- h. Let `n` be `int`.
- i. Result in `BIT < n >` and a set of fresh type variables $\cdot$.

8.1.7.4. Fixed-width signed integers

Signed integers are represented using two's complement. An integer with `W` bits is declared as: `int<W>`. `W` must be an expression that evaluates to a local compile-time known (see Section 7.5) value that is a non-negative integer. Note that `int<W>` type refers to both cases of `int<W>` and `int<(expression)>` where the width is a local compile-time known value.

Bits within an integer are numbered from 0 to `W-1`. Bit 0 is the least significant, and bit `W-1` is the sign bit.

For example, the type `int<64>` describes the type of integers represented using exactly 64 bits with bits numbered from 0 to 63, where bit 63 is the most significant (sign) bit.

P4 architectures may impose additional constraints on signed types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64-bit values).

A signed integer with width 1 can only have two legal values: 0 and -1.

8.1.7.4.1. Type checking

Typing `baseType`, under cursor `p` and typing context TC:

1. If let `INT < int >` be `baseType`:
 - a. Check that `int` has type `nat`.
 - b. Let `n` be `int`.
 - c. Result in `INT < n >` and a set of fresh type variables $\cdot$.
2. Else if let `INT < ( expression ) >` be `baseType`:
 - a. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `p` and typing context TC.
 - b. Check that the compile-time known-ness of `typedExpressionIR` is LCTK.
 - c. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIR`, under cursor `p` and typing context TC.
 - d. Check that `value` has type `integerValue`.
 - e. Let `integerValue` be `value`.
 - f. Let `int` be the integer representation of `integerValue`.
 - g. Check that `int` has type `nat`.

- h. Let n be int .
- i. Result in $\text{INT} < n >$ and a set of fresh type variables $\cdot$.

8.1.7.5. Dynamically-sized bit-strings

Some network protocols use fields whose size is only known at runtime (e.g., IPv4 options). To support restricted manipulations of such values, P4 provides a special bit-string type whose size is set at runtime, called a **varbit**.

The type $\text{varbit} < W >$ denotes a bit-string with a width of at most W bits, where W is a local compile-time known value (see Section 7.5) that is a non-negative integer. For example, the type $\text{varbit} < 120 >$ denotes the type of bit-string values that may have between 0 and 120 bits. Most operations that are applicable to fixed-size bit-strings (unsigned numbers) **cannot** be performed on dynamically sized bit-strings. Note that $\text{varbit} < W >$ type refers to both cases of $\text{varbit} < W >$ and $\text{varbit} < (\text{expression}) >$ where the width is a local compile-time known value.

P4 architectures may impose additional constraints on varbit types: for example, they may limit the maximum size, or they may require varbit values to always contain an integer number of bytes at runtime.

8.1.7.5.1. Type checking

Typing baseType , under cursor p and typing context TC :

1. If let $\text{VARBIT} < \text{int} >$ be baseType :
 - a. Check that int has type nat .
 - b. Let n be int .
 - c. Result in $\text{VARBIT} < n >$ and a set of fresh type variables $\cdot$.
2. Else if let $\text{VARBIT} < (\text{expression}) >$ be baseType :
 - a. Let typedExpressionIR be
 - the result of typing expression , under cursor p and typing context TC .
 - b. Check that the compile-time known-ness of typedExpressionIR is LCTK .
 - c. Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
 - d. Check that value has type integerValue .
 - e. Let integerValue be value .
 - f. Let int be the integer representation of integerValue .
 - g. Check that int has type nat .
 - h. Let n be int .
 - i. Result in $\text{VARBIT} < n >$ and a set of fresh type variables $\cdot$.

8.2. Named types

```
prefixedTypeName
: typeName
```

```

    | `TID . typeName
    ;

specializedType
: prefixedTypeName `< typeArgumentList >
;

namedType
: prefixedTypeName
| specializedType
;

```

Named types are types that reference another type by name. User-defined types can be introduced with their names. For example, the following declares a new struct type `S`:

```

struct S {
    bit<32> x;
    bit<32> y;
}

```

This type can then be referenced by its name, as can be seen in the parameter type of the following example:

```

void f(S s) {}

```

Here, `S` is a named type (`typeName`) that refers to the struct type defined earlier.

Types such as struct, header, and header union types can be generic. In order to use such a generic type it must be specialized with appropriate type arguments. For example,

```

// generic structure type
struct S<T> {
    T field;
    bool valid;
}

struct G<T> {
    S<T> s;
}

// specialize S by replacing 'T' with 'bit<32>'
const S<bit<32>> s = { field = 32w0, valid = false };
// Specialize G by replacing 'T' with 'bit<32>'
const G<bit<32>> g = { s = { field = 0, valid = false } };

// generic header type
header H<T> {
    T field;
}

// Specialize H by replacing 'T' with 'bit<8>'
const H<bit<8>> h = { field = 1 };
// Header stack produced from a specialization of a generic header type
H<bit<8>>[10] stack;

// Generic header union
header_union HU<T> {
    H<bit<32>> h32;
    H<bit<8>> h8;
    H<T> ht;
}

```

```
// Header union with a type obtained by specializing a generic header union type
HU<bit> hu;
```

8.2.1. Type definitions (type constructors)

```
typeDeclaration
: derivedTypeDeclaration
| typedefDeclaration
| parserTypeDeclaration
| controlTypeDeclaration
| packageTypeDeclaration
;
```

`typeDeclaration` introduces a user-defined type definition. These can be referenced by name. Generic type definitions can be specialized with type arguments to produce a type instance.

The internal representation of type definitions is as follows:

```
nameTypeDefIR = nameTypeIR

aliasTypeDefIR = aliasTypeIR

dataTypeDefIR
: structTypeDefIR
| headerTypeDefIR
| headerUnionTypeDefIR
| enumTypeDefIR
;

objectTypeDefIR
: externObjectTypeDefIR
| parserObjectTypeDefIR
| controlObjectTypeDefIR
| packageObjectTypeDefIR
| tableObjectTypeDefIR
;

typeDefIR
: nameTypeDefIR
| aliasTypeDefIR
| dataTypeDefIR
| objectTypeDefIR
;
```

For example, `structTypeDeclaration` introduces a struct type definition, `structTypeDefIR`:

```
structTypeDefIR
: STRUCT typeId `< typeParameterIR* > `{ fieldTypeIR* }
;

structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;
```

Some type definitions like struct and header type definitions can be polymorphic, meaning they can have type variables as parameters. On the other hand, type definitions such as enum and type alias definitions can only be monomorphic. Below table summarizes which kinds of type definitions can be polymorphic:

NOTE | Add a table for `$is_monomorphic_typeDefIR` here.

NOTE Add explainer on explicit and implicit type parameters for type definitions here.

8.2.2. Type resolution

Notice that P4 has struct types, but the surface syntax does not have an explicit construct for struct types. Instead, struct types are always referenced by name. To easily handle named types in $P4_{IR}$, the internal representation of types contain representations for the referred types.

```
structTypeIR
: STRUCT typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;
```

For example, a struct type is represented as `structTypeIR` in $P4_{IR}$. Thus, after type checking, a named type in $P4_{16}$ is resolved to its referred type in $P4_{IR}$. For named types referring to monomorphic type definitions, the resolved type is simply the type defined by the type definition.

```
enum E_t { A, B, C; }
void f(E_t e) {}
```

In the above example, the named type `E_t` is resolved to the enum type `enumTypeIR`.

```
struct S<T> { T t; }
void f(S<bit<32>> s) {}
```

In the above example, the named type `S<bit<32>>` is resolved to a `structTypeIR`, where it specializes a `structTypeDefIR` with the type argument `bit<32>`. Type specialization is discussed in detail in the next section.

8.2.3. Type specialization

`typeDefIRbase` specialized by `typeArgumentIR*`

1. Let $(typeParameterIR_{expl}^*, typeParameterIR_{impl}^*)$ be the type parameters of `typeDefIRbase`.
2. Let `typeParameterIR*` be `typeParameterIRexpl*` concatenated with `typeParameterIRimpl*`.
3. Check that the length of `typeParameterIR*` is equal to the length of `typeArgumentIR*`.
4. Let `theta` be $\{ (typeParameterIR : typeArgumentIR)^* \}$.
5. Let `typeIRbase` be the underlying type of `typeDefIRbase`.
6. Let `typeIRspec` be `typeIRbase` substituted by `theta`.
7. Return `typeIRspec`.

8.2.4. Prefixed type names

```
prefixedTypeName
: typeName
| `TID . typeName
;
```

A `prefixedTypeName` may refer to either a monomorphic type definition or a polymorphic type definition

without type arguments. The names can be prefixed by a dot (.) to refer to the top-level namespace.

On type checking, the referenced type definition is looked up by name. If the referenced type definition is monomorphic, then the resolved type is simply the type defined by the type definition. If the referenced type definition is polymorphic (but without type arguments), then the resolved type is a specialized type with no type arguments.

8.2.4.1. Type checking

Typing `prefixedTypeName`, under cursor `p` and typing context `TC`:

1. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
2. Let `typeDefIR'` be ! the type definition of `prefixedNameIR` in the `p` layer of `TC`.
3. If `typeDefIR'` is monomorphic:
 - a. Let `typeIR` be the underlying type of `typeDefIR'`.
 - b. Result in `typeIR` and a set of fresh type variables `.`.
4. Else:
 - a. Let `typeIR` be `typeDefIR'` specialized by `.`.
 - b. Result in `typeIR` and a set of fresh type variables `.`.

8.2.5. Specialized types

A generic type may be specialized by specifying arguments for its type variables. In cases where the compiler can infer type arguments, type specialization is not necessary. When a type is specialized, all its type variables must be bound.

NOTE

P4-SpecTec currently does **not** infer type arguments for type specialization, where it only performs inference for function and method calls.

```
specializedType
: prefixedTypeName `< typeArgumentList >
;
```

For example, the following extern declaration describes a generic block of registers, where the type of the elements stored in each register is an arbitrary `T`.

```
extern Register<T> {
    Register(bit<32> size);
    T read(bit<32> index);
    void write(bit<32> index, T value);
}
```

The type `T` has to be specified when instantiating a set of registers, by specializing the `Register` type:

```
Register<bit<32>>(128) registerBank;
```

The instantiation of `registerBank` is made using the `Register` type specialized with the `bit<32>` bound to the `T` type argument.

8.2.5.1. Type checking

Typing `prefixedTypeName < typeArgumentList >`, under cursor `p` and typing context `TC`:

1. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
2. Let `typeDefIR'` be ! the type definition of `prefixedNameIR` in the `p` layer of `TC`.
3. Check that `typeDefIR'` is polymorphic.
4. Let `typeIRarg*` and a set of fresh type variables `typeIdfresh*` be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context `TC`.
5. Let `typeIR` be `typeDefIR'` specialized by `typeIRarg*`.
6. Result in `typeIR` and a set of fresh type variables `typeIdfresh*`.

8.3. Data types and values

```
dataTypeIR
: listTypeIR
| tupleTypeIR
| headerStackTypeIR
| structTypeIR
| headerTypeIR
| headerUnionTypeIR
| enumTypeIR
;

dataValue
: listValue
| tupleValue
| headerStackValue
| structValue
| headerValue
| headerUnionValue
| enumValue
;
```

Data types represent the types that aggregate other types, including lists, tuples, header stacks, structs, headers, header unions, and enums.

8.3.1. List types and values

A list holds zero or more values, where every element must have the same type. The type of a list where all elements have type `T` is written as

```
list<T>
```

```
listType
: LIST `< typeArgument >
;

listTypeIR
: LIST `< typeIR >
;

listValue
```

```

: LIST `[ value* ]
;

```

8.3.1.1. Type checking

Typing LIST < typeArgument >, under cursor p and typing context TC:

1. Let typeIR_{arg} and a set of fresh type variables typeId_{fresh}^{*} be
 - the result of typing typeArgument, under cursor p and typing context TC.
2. Result in LIST < typeIR_{arg} > and a set of fresh type variables typeId_{fresh}^{*}.

8.3.1.2. Well-formedness

LIST < typeIR' > is a well-formed type, with bound type variables B when:

1. Check that typeIR' can be nested in a list type.
2. Check that typeIR' is a well-formed type, with bound type variables B.
3. The relation holds.

8.3.2. Tuple types and values

A tuple is similar to a struct, in that it holds multiple values. The type of tuples with n component types T1, ..., Tn is written as

```

tuple<T1, /* more fields omitted */, Tn>

```

```

tupleType
: TUPLE `< typeArgumentList >
;

tupleTypeIR
: TUPLE `< typeIR* >
;

tupleValue
: TUPLE `( value* )
;

```

8.3.2.1. Type checking

Typing TUPLE < typeArgumentList >, under cursor p and typing context TC:

1. Let typeIR_{arg}^{*} and a set of fresh type variables typeId_{fresh}^{*} be
 - the result of typing typeArgumentList, under cursor p and typing context TC.
2. Result in TUPLE < typeIR_{arg}^{*} > and a set of fresh type variables typeId_{fresh}^{*}.

8.3.2.2. Well-formedness

`TUPLE < typeIR* >` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be nested in a tuple type, for all `typeIR'` in `typeIR*`.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
3. The relation holds.

8.3.3. Header stack types and values

A header stack represents an array of headers or header unions. A header stack type is defined as:

```
headerStackType
: namedType `[ expression ]
;

headerStackTypeIR
: typeIR `[ nat ]
;
```

where `namedType` refers to a header or header union type. For a header stack `hs[n]`, the term `n` is the maximum defined size, and must be a local compile-time known value that is a positive integer. Nested header stacks are not supported.

```
headerStackValue
: HEADER_STACK `[ value* `( nat ; nat ) ]
;
```

At runtime a stack contains `n` values with type `namedType`, only some of which may be valid.

For example, the following declarations,

```
header Mpls_h {
  bit<20> label;
  bit<3> tc;
  bit bos;
  bit<8> ttl;
}
Mpls_h[10] mpls;
```

introduce a header stack called `mpls` containing ten entries, each of type `Mpls_h`.

8.3.3.1. Type checking

Typing `namedType [ expressionsize ]`, under cursor `p` and typing context `TC`:

1. Let `typeIRbase` and a set of fresh type variables `typeIdfresh*` be
 - the result of typing `namedType`, under cursor `p` and typing context `TC`.
2. Let `typedExpressionIRsize` be
 - the result of typing `expressionsize`, under cursor `p` and typing context `TC`.

3. Check that **the compile-time known-ness of $\text{typedExpressionIR}_{\text{size}}$ is LCTK**.
4. Let **value** be
 - the result of **local compile-time evaluation of $\text{typedExpressionIR}_{\text{size}}$, under cursor p and typing context TC** .
5. Check that **value** has type **integerValue**.
6. Let **integerValue_{size}** be **value**.
7. Let **int** be **the integer representation of integerValue_{size}**.
8. Check that **int** has type **nat**.
9. Let **n_{size}** be **int**.
10. Check that **n_{size}** is greater than 0.
11. Result in **$\text{typeIR}_{\text{base}} [n_{\text{size}}]$** and a set of fresh type variables **$\text{typeID}_{\text{fresh}}^*$** .

8.3.3.2. Well-formedness

$\text{typeIR}' [\text{nat}]$ is a well-formed type, with bound type variables B when:

1. Check that **typeIR' can be nested in a header stack type**.
2. Check that **typeIR' is a well-formed type, with bound type variables B** .
3. The relation holds.

8.3.4. Array types and values

NOTE | This feature will be added to P4 v1.2.5.6. It is not modeled in P4-SpecTec yet.

8.3.5. Struct types and values

P4 **struct** types are defined with the following syntax:

```
typeField
: annotationList type name ;
;

structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;

```

This declaration introduces a new struct type definition with the specified name in the current scope. Field names have to be distinct. An empty struct (with no fields) is legal.

```
structTypeDefIR
: STRUCT typeId `< typeParameterIR* > `{ fieldTypeIR* }
;

```

For example, the structure **Parsed_headers** below contains the headers recognized by a simple parser:

```
header Tcp_h { /* fields omitted */ }
```

```

header Udp_h { /* fields omitted */ }
struct Parsed_headers {
    Ethernet_h ethernet;
    Ip_h      ip;
    Tcp_h     tcp;
    Udp_h     udp;
}

```

Struct types can be used by referring to their names. Generic struct types can be used by specializing their type parameters with type arguments. Internally, struct types and values are represented as follows:

```

fieldTypeIR
: typeIR nameIR ;
;

structTypeIR
: STRUCT typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;

fieldValue
: value nameIR ;
;

structValue
: STRUCT typeId `{ fieldValue* }
;

```

8.3.5.1. Type checking

Struct types are introduced by `struct` declarations. Its typing rule is explained in [Section 11.15](#). Struct types are then used by referencing their names. Thus struct types can be obtained from named types, as illustrated in [Section 8.2](#).

8.3.5.2. Well-formedness

`STRUCT typeId < typeArgumentIR* > { (typeIRfield nameIRfield ;)* }` is a well-formed type, with bound type variables B when:

1. Check that the elements of $\text{nameIR}_{\text{field}}^*$ are distinct.
2. Check that $\text{typeIR}_{\text{field}}$ can be nested in a struct type, for all $\text{typeIR}_{\text{field}}$ in $\text{typeIR}_{\text{field}}^*$.
3. Check that $\text{typeIR}_{\text{field}}$ is a well-formed type, with bound type variables B , for all $\text{typeIR}_{\text{field}}$ in $\text{typeIR}_{\text{field}}^*$.
4. The relation holds.

8.3.6. Header types and values

The declaration of a `header` type is given by the following syntax:

```

typeField
: annotationList type name ;
;

headerTypeDeclaration
: annotationList HEADER name typeParameterListOpt `{ typeFieldList }
;

```

where each `type` is restricted to a bit-string type (fixed or variable), a fixed-width signed integer type, a boolean type, or a struct that itself contains other struct fields, nested arbitrarily, as long as all of the "leaf" types are `bit<W>`, `int<W>`, a serializable `enum`, or a `bool`. If a `bool` is used inside a P4 header, all implementations encode the `bool` as a one bit long field, with the value 1 representing `true` and 0 representing `false`. Field names have to be distinct.

A header declaration introduces a new identifier in the current scope; the type definition can be referred to using this identifier.

```
headerTypeDefIR
: HEADER typeId `< typeParameterIR* > `{ fieldTypeIR* }
;
```

A header is similar to a `struct` in C, containing all the specified fields. However, in addition, a header also contains a hidden Boolean "validity" field. When the "validity" bit is `true` we say that the "header is valid". When a local variable with a header type is declared, its "validity" bit is automatically set to `false`. The "validity" bit can be manipulated by using the header methods `isValid()`, `setValid()`, and `setInvalid()`.

Header types may be empty:

```
header Empty_h { }
```

Note that an empty header still contains a validity bit.

When a struct is inside of a header, the order of the fields for the purposes of `extract` and `emit` calls is the order of the fields as defined in the source code. An example of a header including a struct is included below.

```
struct ipv6_addr {
    bit<32> Addr0;
    bit<32> Addr1;
    bit<32> Addr2;
    bit<32> Addr3;
}

header ipv6_t {
    bit<4>    version;
    bit<8>    trafficClass;
    bit<20>   flowLabel;
    bit<16>   payloadLen;
    bit<8>    nextHdr;
    bit<8>    hopLimit;
    ipv6_addr src;
    ipv6_addr dst;
}
```

Headers that do not contain any `varbit` field are "fixed size." Headers containing `varbit` fields have "variable size." The size (in bits) of a fixed-size header is simply the sum of the sizes of all component fields (without counting the validity bit). There is no padding or alignment of the header fields. Targets may impose additional constraints on header types---e.g., restricting headers to sizes that are an integer number of bytes.

For example, the following declaration describes a typical Ethernet header:

```
header Ethernet_h {
    bit<48> dstAddr;
    bit<48> srcAddr;
    bit<16> etherType;
```

```
}
```

The following variable declaration uses the newly introduced type `Ethernet_h`:

```
Ethernet_h ethernetHeader;
```

P4's parser language provides an `extract` method that can be used to "fill in" the fields of a header from a network packet, as described in [\[sec-packet-data-extraction\]](#). The successful execution of an `extract` operation also sets the validity bit of the extracted header to `true`.

Here is an example of an IPv4 header with variable-sized options:

```
header IPv4_h {
  bit<4>    version;
  bit<4>    ihl;
  bit<8>    diffserv;
  bit<16>   totallen;
  bit<16>   identification;
  bit<3>    flags;
  bit<13>   fragOffset;
  bit<8>    ttl;
  bit<8>    protocol;
  bit<16>   hdrChecksum;
  bit<32>   srcAddr;
  bit<32>   dstAddr;
  varbit<320> options;
}
```

As demonstrated by a code example in [\[sec-packet-extract-two\]](#), another way to support headers that contain variable-length fields is to define two headers—one fixed length, one containing a `varbit` field—and extract each part in separate parsing steps.

Notice that the names `isValid`, `setValid`, `minSizeInBits`, etc. are all valid header field names.

Internally, header types and values are represented as follows:

```
headerTypeIR
: HEADER typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;

headerValue
: HEADER typeId `{ bool ; fieldValue* }
;
```

8.3.6.1. Type checking

Header types are introduced by `header` declarations. Its typing rule is explained in [Section 11.16](#). Header types are then used by referencing their names. Thus header types can be obtained from named types, as illustrated in [Section 8.2](#).

8.3.6.2. Well-formedness

`HEADER typeId < typeArgumentIR* > { (typeIRfield nameIRfield ;)* }` is a well-formed type, with bound type variables `B` when:

1. Check that the elements of `nameIRfield*` are distinct.

2. Check that `typeIRfield` can be used in a header type, for all `typeIRfield` in `typeIRfield*`.
3. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
4. The relation holds.

8.3.7. Header union types and values

A header union represents an alternative containing at most one of several different headers. Header unions can be used to represent "options" in protocols like TCP and IP. They also provide hints to P4 compilers that only one alternative will be present, allowing them to conserve storage resources.

A header union is defined as:

```
typeField
: annotationList type name ;
;

headerUnionTypeDeclaration
: annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList }
;
```

This declaration introduces a new type definition with the specified name in the current scope.

```
headerUnionTypeDefIR
: HEADER_UNION typeId `< typeParameterIR* > `{ fieldTypeIR* }
;
```

Each element of the list of fields used to declare a header union must be of header type. An empty list of fields is legal. Field names have to be distinct.

As an example, the type `Ip_h` below represents the union of an IPv4 and IPv6 headers:

```
header_union IP_h {
  IPv4_h v4;
  IPv6_h v6;
}
```

A header union is not considered a type with fixed length.

Internally, header union types and values are represented as follows:

```
headerUnionTypeIR
: HEADER_UNION typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;

headerUnionValue
: HEADER_UNION typeId `{ fieldValue* }
;
```

8.3.7.1. Type checking

Header union types are introduced by `header_union` declarations. Its typing rule is explained in [Section 11.17](#). Header union types are then used by referencing their names. Thus header union types can be obtained from named types, as illustrated in [Section 8.2](#).

8.3.7.2. Well-formedness

`HEADER_UNION typeId < typeArgumentIR* > { (typeIRfield nameIRfield ;)* }` is a well-formed type, with bound type variables `B` when:

1. Check that the elements of `nameIRfield*` are distinct.
2. Check that `typeIRfield` can be used in a header union type, for all `typeIRfield` in `typeIRfield*`.
3. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
4. The relation holds.

8.3.8. Enum types and values

The declaration of an `enum` type is given by the following syntax:

```
namedExpression
: name = expression
;

namedExpressionList
: namedExpression
| namedExpressionList , namedExpression
;

enumTypeDeclaration
: annotationList ENUM name `{ nameList trailingCommaOpt }
| annotationList ENUM type name `{ namedExpressionList trailingCommaOpt }
;
```

Enums can be declared in two different ways, depending on whether or not an underlying representation type is provided. The two forms are described below.

Enums cannot be parameterized over type parameters, thus is always monomorphic.

```
enumTypeDefIR = enumTypeIR
```

Internally, enum types and values are represented as follows:

```
valueFieldIR
: nameIR = value ;
;

enumTypeIR
: ENUM typeId `{ nameIR* }
| ENUM typeId `< typeIR > `{ valueFieldIR* }
;

enumValue
: typeId . nameIR
| typeId . nameIR . value
;
```

8.3.8.1. Enum type declaration without an underlying type

For example, the declaration

```
enum Suits { Clubs, Diamonds, Hearths, Spades }
```

introduces a new enumeration type, which contains four elements---e.g., `Suits.Clubs`. An `enum` declaration introduces a new identifier in the current scope for naming the created type along with its distinct elements. The underlying representation of the `Suits` enum is not specified, so their "size" in bits is not specified (it is target-specific).

8.3.8.2. Enum type declaration with an underlying type

It is also possible to specify an `enum` with an underlying representation. These are sometimes called serializable `enums`, because headers are allowed to have fields with such `enum` types. This requires the programmer provide both the fixed-width unsigned (or signed) integer type and an associated integer value for each symbolic entry in the enumeration.

For example, the declaration

```
enum bit<16> EtherType {  
    VLAN      = 0x8100,  
    QINQ      = 0x9100,  
    MPLS      = 0x8847,  
    IPV4      = 0x0800,  
    IPV6      = 0x86dd  
}
```

introduces a new enumeration type, which contains five elements---e.g., `EtherType.IPV4`. This `enum` declaration specifies the fixed-width unsigned integer representation for each entry in the `enum` and provides an underlying type: `bit<16>`. This kind of `enum` declaration can be thought of as declaring a new `bit<16>` type, where variables or fields of this type are expected to be unsigned 16-bit integer values, and the mapping of symbolic to numeric values defined by the `enum` are also defined as a part of this declaration. In this way, an `enum` with an underlying type can be thought of as being a type derived from the underlying type carrying equality, assignment, and casts to/from the underlying type.

Compiler implementations are expected to raise an error if the fixed-width integer representation for an enumeration entry falls outside the representation range of the underlying type.

For example, the declaration

```
enum bit<8> FailingExample {  
    first      = 1,  
    second     = 2,  
    third      = 3,  
    unrepresentable = 300  
}
```

would raise an error because `300`, the value associated with `FailingExample.unrepresentable` cannot be represented as a `bit<8>` value.

The initializer `expression` must be a local compile-time known value.

Annotations, represented by the non-terminal `annotationList`, are described in [\[sec-annotations\]](#).

8.3.8.3. Type checking

Enum types are introduced by `enum` declarations. Its typing rule is explained in [Section 11.14](#). Enum types are then used by referencing their names. Thus enum types can be obtained from named types, as illustrated in [Section 8.2](#).

8.3.8.4. Well-formedness

`enumTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `ENUM _ { nameIRfield* }` be `enumTypeIR`:
 - a. Check that the elements of `nameIRfield*` are distinct.
 - b. The relation holds.
2. Else:
 - a. Let `ENUM _ < typeIR' > { (nameIRfield = _ ; )* }` be `enumTypeIR`.
 - b. Check that the elements of `nameIRfield*` are distinct.
 - c. Check that `typeIR'` can be used in a serializable enum type.
 - d. Check that `typeIR'` is a well-formed type, with bound type variables `B`.
 - e. The relation holds.

8.4. Object types and values

```
objectTypeIR
: externObjectTypeIR
| parserObjectTypeIR
| controlObjectTypeIR
| packageObjectTypeIR
| tableObjectTypeIR
;

objectReferenceValue
: REF objectId
;
```

Stateful objects are represented using `objectTypeIR`. At runtime, these objects are passed along directionless parameters as references (`objectReferenceValue`).

8.4.1. Extern object types

An extern object declaration declares an object and all methods that can be invoked to perform computations and to alter the state of the object. Extern declarations may only appear as allowed by the architecture model and may be specific to a target.

```
externConstructorPrototype
: annotationList typeIdIdentifier `( parameterList ) ;
;

externMethodPrototype
: annotationList functionPrototype ;
| annotationList ABSTRACT functionPrototype ;
;

externConstructorOrMethodPrototype
: externConstructorPrototype
| externMethodPrototype
;

externObjectDeclaration
: annotationList EXTERN nonTypeName typeParameterListOpt `{
```

```
externConstructorOrMethodPrototypeList }
;
```

For example, the P4 core library introduces two extern objects `packet_in` and `packet_out` used for manipulating packets (see [\[sec-packet-data-extraction\]](#) and [\[sec-deparse\]](#)). Here is an example showing how the methods of these objects can be invoked on a packet:

```
extern packet_out {
    void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
    apply {
        b.emit(h.ipv4);           // write ipv4 header into output packet
    }                             // by calling emit method
}
```

An extern object declaration introduces a new extern object type definition.

```
externObjectTypeDefIR
: EXTERN typeId `< typeParameterIR* , typeParameterIR* > externMethodTypeDefEnv
;
```

Extern object types can be referred to using their name. Internally, extern object types are represented as:

```
externObjectTypeIR
: EXTERN typeId `< typeArgumentIR* > externMethodTypeDefEnv
;
```

8.4.1.1. Type checking

Extern object types are introduced by `extern` declarations. Its typing rule is explained in [\[sem-extern-object-declaration\]](#). Extern object types are then used by referencing their names. Thus extern object types can be obtained from named types, as illustrated in [Section 8.2](#).

8.4.1.2. Well-formedness

`EXTERN typeId < typeArgumentIR* > { (callableId : externMethodTypeDefIR)* }` is a well-formed type, with bound type variables `B` when:

1. Check that `externMethodTypeDefIR` is a well-formed callable type definition, with bound type variables `B`, for all `externMethodTypeDefIR` in `externMethodTypeDefIR*`.
2. The relation holds.

8.4.2. Parser object types

A parser type declaration describes the signature of a parser. A parser should have at least one argument of type `packet_in`, representing the received packet that is processed.

```
parserTypeDeclaration
: annotationList PARSE name typeParameterListOpt `( parameterList ) ;
;
```

For example, the following is a type declaration of a parser type named **P** that is parameterized on a type variable **H**. That parser receives as input a **packet_in** value **b** and produces two values:

- A value with a user-defined type **H**
- A value with a predefined type **Counters**

```
struct Counters { /* Fields omitted */ }  
parser P<H>(packet_in b,  
            out H packetHeaders,  
            out Counters counters);
```

A parser type declaration introduces a new parser object type definition.

```
parserObjectTypeDefIR  
: PARSEr typeId `< typeParameterIR* , typeParameterIR* > `( parameterIR* )  
;
```

Parser types can be referred to using their name and type arguments. Internally, parser types are represented as:

```
parserObjectTypeIR  
: PARSEr typeId `< typeArgumentIR* > `( parameterIR* )  
;
```

8.4.2.1. Type checking

Parser object types are introduced by parser type declarations. Its typing rule is explained in [Section 11.19](#). Parser object types are then used by referencing their names. Thus parser object types can be obtained from named types, as illustrated in [Section 8.2](#).

8.4.2.2. Well-formedness

PARSEr typeId < typeArgumentIR* > (parameterIR*) is a well-formed type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR***.
2. The relation holds.

8.4.3. Control object types

A control type declaration describes the signature of a control block.

```
controlTypeDeclaration  
: annotationList CONTROL name typeParameterListOpt `( parameterList ) ;  
;
```

Control type declarations are similar to parser type declarations.

A control type declaration introduces a new control object type definition.

```
controlObjectTypeDefIR
: CONTROL typeId `< typeParameterIR* , typeParameterIR* > `( parameterIR* )
;
```

Control types can be referred to using their name and type arguments. Internally, control types are represented as:

```
controlObjectTypeIR
: CONTROL typeId `< typeArgumentIR* > `( parameterIR* )
;
```

8.4.3.1. Type checking

Control object types are introduced by control type declarations. Its typing rule is explained in [Section 11.20](#). Control object types are then used by referencing their names. Thus control object types can be obtained from named types, as illustrated in [Section 8.2](#).

8.4.3.2. Well-formedness

CONTROL typeId < typeArgumentIR* > (parameterIR*) is a well-formed type, with bound type variables B when:

1. Check that parameterIR is a well-formed parameter type, with bound type variables B, for all parameterIR in parameterIR*.
2. The relation holds.

8.4.4. Package object types

A package type describes the signature of a package.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

This introduces a new package object type definition.

```
packageObjectTypeDefIR
: PACKAGE typeId `< typeParameterIR* , typeParameterIR* > `{ typeIR* }
;
```

Package types can be referred to using their name and type arguments. Internally, package types are represented as:

```
packageObjectTypeIR
: PACKAGE typeId `< typeArgumentIR* > `{ typeIR* }
;
```

8.4.4.1. Type checking

Package object types are introduced by **package** declarations. Its typing rule is explained in [Section 11.21](#).

Package object types are then used by referencing their names. Thus package object types can be obtained from named types, as illustrated in [Section 8.2](#).

8.4.4.2. Well-formedness

$\text{PACKAGE typeId} < \text{typeArgumentIR}^* > \{ \text{typeIR}^* \}$ is a well-formed type, with bound type variables B when:

1. Check that typeIR' is a well-formed type, with bound type variables B , for all typeIR' in typeIR^* .
2. The relation holds.

8.4.5. Table object types

A **table** declaration introduces a table instance and a table type definition.

```
tableDeclaration
  : annotationList TABLE name `{ tablePropertyList }
  ;

tableObjectTypeDefIR = tableObjectTypeIR
```

Internally, table types are represented as:

```
tableObjectTypeIR
  : TABLE typeId `{ tableMetadataStructTypeIR }
  ;
```

Table metadata types are introduced in [Section 8.6.6](#).

8.4.5.1. Well-formedness

$\text{TABLE typeId} \{ \text{tableMetadataStructTypeIR} \}$ is a well-formed type, with bound type variables B when:

1. Check that $\text{tableMetadataStructTypeIR}$ is a well-formed type, with bound type variables B .
2. The relation holds.

8.4.6. Object reference values

P4 objects are stateful, thus their values are references to the actual objects allocated in the global store.

```
objectId = nameIR*

objectReferenceValue
  : REF objectId
  ;
```

Objects are referred to by their fully-qualified names from the top-level scope.

8.5. Alias types

A **typedef** or **type** declaration can be used to give an alias to a type.

```
typedef
  : type
  | derivedTypeDeclaration
  ;

typedefDeclaration
  : annotationList TYPEDEF typedef name ;
  | annotationList TYPE type name ;
  ;
```

Internally, the alias is represented as:

```
aliasTypeIR
  : typedefTypeIR
  | newTypeIR
  ;
```

8.5.1. **typedef** declaration

```
typedef bit<32> u32;
typedef struct Point { int<32> x; int<32> y; } Pt;
typedef Empty_h[32] HeaderStack;
```

The two types are treated as synonyms, and all operations that can be executed using the original type can be also executed using the newly created type.

If **typedef** is used with a generic type the type must be specialized with the suitable number of type arguments:

```
struct S<T> {
  T field;
}

// typedef S X; -- illegal: S does not have type arguments
typedef S<bit<32>> X; // -- legal
```

Internally, the alias is represented as:

```
typedefTypeIR
  : TYPEDEF typeId typeIR
  ;
```

8.5.1.1. Type checking

typedefs are introduced by **typedef** declarations. Its typing rule is explained in [Section 11.18.1](#). **typedefs** are then used by referencing their names. Thus **typedef** types can be obtained from named types, as illustrated in [\[sec-def-namedType\]](#).

8.5.1.2. Well-formedness

`TYPEDEF typeId typeIR'` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be used in a typedef.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`.
3. The relation holds.

8.5.2. type declaration

Similarly to `typedef`, the keyword `type` can be used to introduce a new type.

```
type bit<32> U32;  
U32 x = (U32)0;
```

While similar to `typedef`, the `type` keyword introduces a new type which is not a synonym with the original type: values of the original type and the newly introduced type cannot be mixed in expressions.

Currently the types that can be created by the `type` keyword are restricted to one of: `bit<>`, `int<>`, `bool`, or types defined using `type` from such types.

One important use of such types is in describing P4 values that need to be exchanged with the control plane through communication channels (e.g., through the control-plane API or through network packets sent to the control plane). For example, a P4 architecture may define a type for the switch ports:

```
type bit<9> PortId_t;
```

This declaration will prevent `PortId_t` values from being used in arithmetic expressions without casts. Moreover, this declaration may enable special manipulation of such values by software that lies outside of the datapath (e.g., a target-specific toolchain could include software that automatically converts values of type `PortId_t` to a different representation when exchanged with the control-plane software).

Internally, the alias is represented as:

```
newTypeIR  
: TYPE typeId typeIR  
;
```

8.5.2.1. Type checking

types are introduced by `type` declarations. Its typing rule is explained in [Section 11.18.2](#). types are then used by referencing their names. Thus `type` types can be obtained from named types, as illustrated in [\[sec-def-namedType\]](#).

8.5.2.2. Well-formedness

`TYPE typeId typeIR'` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be used in a new type.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`.

3. The relation holds.

8.6. Synthesized types and values

For the purposes of type-checking the P4 compiler can synthesize some type representations which cannot be directly expressed by users.

These types and values are represented as:

```
synthesizedTypeIR
: defaultTypeIR
| invalidHeaderTypeIR
| sequenceTypeIR
| recordTypeIR
| setTypeIR
| tableMetadataTypeIR
;

synthesizedValue
: defaultValue
| invalidHeaderValue
| sequenceValue
| recordValue
| setValue
| tableMetadataValue
;
```

8.6.1. Default types and values

A left-value can be initialized automatically with a default value of the suitable type using:

```
defaultExpression
: ...
;
```

```
struct S {
    bit<32> b32;
    bool b;
}

enum int<8> NO {
    one = 1,
    zero = 0,
    two = 2
}

enum N1 {
    A, B, C, F
}

S s0 = ...; // initialize s0 with the default value { 0, false }
NO n0 = ...; // initialize n0 with the default value 0
N1 n1 = ...; // initialize n1 with the default value N1.A
```

Internally, the type and value of ... itself are represented as:

```
defaultTypeIR
```

```

: DEFAULT
;

defaultValue
: DEFAULT
;

```

NOTE

The default type and value is new. Because every expressions must have a type and value, we need to represent those of ...

8.6.1.1. Well-formedness

DEFAULT is a well-formed type, with bound type variables **B** when:

1. The relation holds.

8.6.2. Invalid header types and values

```

dataExpression
: {#}
| `{ dataElementExpression trailingCommaOpt }
;

```

The expression `{#}` represents an invalid header of some type, but it can be any header or header union type. A P4 compiler may require an explicit cast on this expression in cases where it cannot determine the particular header or header union type from the context.

For example:

```

header H { bit<32> x; bit<32> y; }
H h;
h = {#}; // This makes the header h become invalid

```

Note that the `#` character cannot be misinterpreted as a preprocessor directive, since it cannot be the first character on a line when it occurs in the single lexical token `{#}`, which may not have whitespace or any other characters between those shown.

Internally, the type and value of `{#}` itself are represented as:

```

invalidHeaderTypeIR
: HEADER_INVALID
;

invalidHeaderValue
: {#}
;

```

NOTE

The invalid header type and value is new. Because every expressions must have a type and value, we need to represent those of `{#}`.

8.6.2.1. Well-formedness

`HEADER_INVALID` is a well-formed type, with bound type variables `B` when:

1. The relation holds.

8.6.3. Sequence types and values

A sequence expression is written using curly braces, with each element separated by a comma:

```
sequenceElementExpression = expressionList

dataElementExpression
: sequenceElementExpression
| recordElementExpression
;

dataExpression
: {#}
| `{ dataElementExpression trailingCommaOpt }
;
```

Sequence expressions can be assigned to expressions of type `tuple`, `struct` or `header`, and can also be passed as arguments to methods. Sequences may be nested.

For instance, sequences can be assigned to other tuples with the same type:

```
tuple<bit<32>, bool> x = { 10, false };
```

The following program fragment uses a sequence expression to pass several header fields simultaneously to a learning provider:

```
extern LearningProvider<T> {
    LearningProvider();
    void learn(in T data);
}

LearningProvider<tuple<bit<48>, bit<32>>>() lp;

lp.learn( { hdr.ethernet.srcAddr, hdr.ipv4.src } );
```

A sequence may be used to initialize a structure if the tuple has the same number of elements as fields in the structure. The effect of such an initializer is to assign the n^{th} element of the tuple to the n^{th} field in the structure:

```
struct S {
    bit<32> a;
    bit<32> b;
}

const S x = { 10, 20 }; // a = 10, b = 20
```

Sequence expressions that have `...` as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default value (see [\[sec-initializing-with-default-values\]](#)).

The type of a sequence expression is a sequence type. Its value is a sequence value.

```
sequenceTypeIR
: SEQ `< typeIR* >
| SEQ `< typeIR* , ... >
;

sequenceValue
: SEQ `( value* )
| SEQ `( value* , ... )
;
```

NOTE

The sequence type is newly introduced. In the current P4 language specification, sequence expressions are referred to as tuple expressions, and (somewhat confusingly and implicitly) has tuple type. However, this is problematic since tuples have limited casting rules that cannot be applied to several initialization scenarios. Thus, we introduce sequence types to represent the expression sequence within curly braces, apart from tuple types.

8.6.3.1. Well-formedness

`sequenceTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `SEQ < typeIR* >` be `sequenceTypeIR`:
 - a. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
 - b. The relation holds.
2. Else:
 - a. Let `SEQ < typeIR* , ... >` be `sequenceTypeIR`.
 - b. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
 - c. The relation holds.

8.6.4. Record types and values

One can write record expressions that evaluate to a structure or header. The syntax of these expressions is given by:

```
recordElementExpression
: name = expression
| name = expression , ...
| name = expression , namedExpressionList
| name = expression , namedExpressionList , ...
;

dataElementExpression
: sequenceElementExpression
| recordElementExpression
;

dataExpression
: {#}
| `{ dataElementExpression trailingCommaOpt }
;
```

The following example shows a structure-valued expression used in an equality comparison expression:

```
struct S {
    bit<32> a;
    bit<32> b;
}

void f() {
    S s;

    // Compare s with a record expression
    bool b = s == (S) { a = 1, b = 2 };
}
```

Record expressions can be used in the right-hand side of assignments, in comparisons, in field selection expressions, and as arguments to functions, method or actions. Record expressions are not left values.

Record expressions that do not have ... as their last element must provide a value for every member of the struct or header type to which it evaluates, by mentioning each field name exactly once.

Record expressions that have ... as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default values (see [\[sec-initializing-with-default-values\]](#)).

The order of the fields of the `struct` or `header` type does not need to match the order of the values of the structure-valued expression.

It is a compile-time error if a field name appears more than once in the same structure-valued expression.

Internally, the type and value of a record expression are represented as:

```
recordTypeIR
: RECORD `{ fieldTypeIR* }
| RECORD `{ fieldTypeIR* , ... }
;

recordValue
: RECORD `{ fieldValue* }
| RECORD `{ fieldValue* , ... }
;
```

NOTE

The record type and value is new. Because every expression must have a type and value, we need to represent those of `{ recordElementExpression optTrailingComma }`.

8.6.4.1. Well-formedness

`recordTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `RECORD { (typeIRfield nameIRfield ;)* }` be `recordTypeIR`:
 - a. Check that the elements of `nameIRfield*` are distinct.
 - b. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
 - c. The relation holds.
2. Else:

- Let `RECORD { (typeIRfield nameIRfield ;)* , ... }` be `recordTypeIR`.
- Check that the elements of `nameIRfield*` are distinct.
- Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
- The relation holds.

8.6.5. Set types and values

```
setTypeIR
: SET `< typeIR >
;
```

The set type describes *sets* of values of some type `T`. Set types can only appear in restricted contexts in P4 programs. For example, the range expression `8w5 .. 8w8` describes a set containing the 8-bit numbers 5, 6, 7, and 8, so its type is `set<bit<8>>`. This expression can be used as a label in a `select` expression (see [\[sec-select\]](#)), matching any value in this range. Set types cannot be named or declared by P4 programmers, they are only synthesized by the compiler internally and used for type-checking.

Set values are represented as:

```
setValue
: SET `{ value }
| SET `{ value* `( nat ) }
| SET `{ value &&& value }
| SET `{ value .. value }
| SET `{ ... }
;
```

8.6.5.1. Well-formedness

`SET < typeIR' >` is a well-formed type, with bound type variables `B` when:

- Check that `typeIR'` can be used in a set type.
- Check that `typeIR'` is a well-formed type, with bound type variables `B`.
- The relation holds.

8.6.6. Table metadata types and values

A `table` can be invoked by calling its `apply` method. Calling an `apply` method on a table instance returns a value with a table metadata `struct` type with three fields. This structure is synthesized by the compiler automatically. For each `table T`, the compiler synthesizes a table metadata `enum` and a `struct`, shown in pseudo-P4:

```
enum action_list(T) {
    // one field for each action in the actions list of table T
}
struct apply_result(T) {
    bool hit;
    bool miss;
    action_list(T) action_run;
```

```
}
```

The evaluation of the `apply` method sets the `hit` field to `true` and the field `miss` to `false` if a match is found in the lookup-table; if a match is not found `hit` is set to `false` and `miss` to `true`. These bits can be used to drive the execution of the control-flow in the control block that invoked the table:

```
if (ipv4_match.apply().hit) {  
    // there was a hit  
} else {  
    // there was a miss  
}  
  
if (ipv4_host.apply().miss) {  
    ipv4_lpm.apply(); // Look up the route only if host table missed  
}
```

The `action_run` field indicates which kind of action was executed (irrespective of whether it was a hit or a miss). It can be used in a switch statement:

```
switch (dmac.apply().action_run) {  
    Drop_action: { return; }  
}
```

Internally, the type and value of table metadata are represented as:

```
tableMetadataTypeIR  
: tableMetadataEnumTypeIR  
| tableMetadataStructTypeIR  
;  
  
tableMetadataValue  
: tableMetadataEnumValue  
| tableMetadataStructValue  
;
```

8.6.6.1. Well-formedness

`typeIR` is a well-formed type, with bound type variables `B` when:

1. If let `TABLE_ENUM _ { nameIR* }` be `typeIR`:
 - a. Check that the elements of `nameIR*` are distinct.
 - b. The relation holds.
2. Else if let `TABLE_STRUCT _ { HIT _; MISS _; ACTION_RUN tableMetadataEnumTypeIR; }` be `typeIR`:
 - a. Check that `tableMetadataEnumTypeIR` is a well-formed type, with bound type variables `B`.
 - b. The relation holds.

8.7. Unrolling

```
typedefTypeIR  
: TYPEDEF typeId typeIR
```

;

`typedefs` introduce indirection in types.

```
struct S {  
    bit<8> t;  
    int<8> u;  
}  
  
typedef S A;
```

The type `A` is an alias of the struct type `S`. Nevertheless, the underlying type of `A` is a `structTypeIR` with two fields, `t` and `u`, of types `int<8>` and `bit<8>` respectively. It is useful to be able to work with the underlying type of `A`, when dealing with type checking, type equivalence, and other type-based analyses.

Unrolling is the process of recursively expanding all `typedefs` until a type is fully expanded to its underlying type.

`typeIR` with `typedefs` unrolled

1. If let `TYPEDEF _ typeIR'` be `typeIR`:
 - a. Return `unroll_typeIR(typeIR')`.
2. Otherwise:
 - a. Return `typeIR`.

8.8. Subtyping

`typedExpressionIR` implicitly cast to `typeIRto`

1. Let `type typeIR` and `compile-time known-ness ctk` be the note of `typedExpressionIR`.
2. If `typeIR` and `typeIRto` are the same type:
 - a. Return `typedExpressionIR`.
3. Else:
 - a. Check that `typeIR` can be implicitly cast to `typeIRto`.
 - b. Let `typedExpressionIRcast` be (`typeIRto`) `typedExpressionIR` annotated with type `typeIRto` and `compile-time known-ness ctk`.
 - c. Return `typedExpressionIRcast`.

9. P4 callables

`P416` includes four entities that can be invoked: actions, functions, methods, and object constructors. Object constructors are explained in [\[sec-def-object\]](#). The other three callable entities are collectively referred to as *callables*.

Callable types are created by the P4 compiler internally to represent the types of actions, functions, and methods during type-checking. We also call the type of a callable its signature. Libraries can contain

callable declarations.

For example, consider the following declarations:

```
extern void random(in bit<5> logRange, out bit<32> value);  
bit<32> add(in bit<32> left, in bit<32> right) {  
    return left + right;  
}
```

These declarations describe two objects:

- `random`, which has an extern function type, representing the following information:
 - the result type is `void`
 - the function has two inputs
 - the first formal parameter has direction `in`, type `bit<5>`, and name `logRange`
 - the second formal parameter has direction `out`, type `bit<32>`, and name `value`
- `add`, also has a function type, representing the following information:
 - the result type is `bit<32>`
 - the function has two inputs
 - both inputs have direction `in` and type `bit<32>`

Specifically, during type checking, callable declarations introduce a type constructor for the callable type:

```
callableTypeDefIR  
: actionTypeDefIR  
| functionTypeDefIR  
| methodTypeDefIR  
;
```

These callable type definitions are used to construct callable types via specialization:

```
callableTypeIR  
: actionTypeIR  
| functionTypeIR  
| methodTypeIR  
;
```

The well-formedness rules for callable types are checked by:

`CallableType_wf`:

- `callableTypeIR` is a well-formed callable type, with bound type variables `B`

The runtime representation of a callable is given by:

```
callableDef  
: actionDef  
| functionDef  
| methodDef
```

;

9.1. Parameters

```
direction
: /* empty */
| IN
| OUT
| INOUT
;

parameter
: annotationList direction type name initializerOpt
;
```

Callables can have parameters. Each parameter has a direction, a type, a name, and an optional default value.

A parameter is internally represented as:

```
parameterIR
: annotationList direction typeIR id constantInitializerOptIR
;
```

The high-level overview of parameters with respect to the copy in/copy out calling convention is described in [Section 6.5](#). Below are the formal typing and well-formedness rules for parameters that implement the restrictions described there.

9.1.1. Type checking

Parameter_ok:

- Typing `parameter`, under cursor `cursor` and typing context `typingContext`:
- Results in the parameter `parameterIR` and a set of fresh type variables `typeId*`.

Typing a parameter produces a `parameterIR` if it is well-typed. A parameter is well-typed when its type is well-formed and its default value (if any) has the same type as the parameter type.

Typing `annotationList direction type name initializerOpt`, under cursor `p` and typing context `TC`:

1. If `initializerOpt` is `EMPTY`:
 - a. Let `typeIR` and a set of fresh type variables `typeIdfresh*` be
 - the result of **typing** `type`, under cursor `p` and typing context `TC`.
 - b. Let `bound` be **bound type variables from the `p` layer of `TC`**.
 - c. Let `B` be **the union of the sets `bound` and `{ typeIdfresh* }`**.
 - d. Check that `typeIR` is a well-formed type, with bound type variables `B`.
 - e. Let `nameIR` be **the name of `name`**.
 - f. Result in the parameter `annotationList direction typeIR nameIR` and a set of fresh type variables `typeIdfresh*`.

2. Else:

- a. Let $\text{expression}_{\text{init}}$ be `initializerOpt`.
- b. Let typeIR and a set of fresh type variables $\text{typeId}_{\text{fresh}}^*$ be
 - the result of `typing type`, under cursor p and typing context TC .
- c. Let bound be `bound type variables` from the p layer of TC .
- d. Let B be `the union of the sets bound and  $\{\text{typeId}_{\text{fresh}}^*\}$` .
- e. Check that typeIR is a well-formed type, with bound type variables B .
- f. Let $\text{typedExpressionIR}_{\text{init}}$ be
 - the result of `typing expressioninit`, under cursor p and typing context TC .
- g. Check that `the compile-time known-ness of  $\text{typedExpressionIR}_{\text{init}}$` is LCTK.
- h. Let $\text{typedExpressionIR}_{\text{init_cast}}$ be `! typedExpressionIRinit implicitly cast to typeIR`.
- i. Let nameIR be `the name of name`.
- j. Let $\text{value}_{\text{init}}$ be
 - the result of `local compile-time evaluation of  $\text{typedExpressionIR}_{\text{init\_cast}}$` under cursor p and typing context TC .
- k. Result in the parameter `annotationList direction typeIR nameIR = valueinit` and a set of fresh type variables $\text{typeId}_{\text{fresh}}^*$.

9.1.2. Well-formedness

`ParameterType_wf`:

- `annotationList direction typeIR id constantInitializerIR2` is a well-formed parameter type, with bound type variables B

A parameter type is well-formed when its type is well-formed. Also, the type should not be a synthesized type. Types that can only be local compile-time known or compile-time known (e.g., `int` type) must be directionless. A default value can be specified for `in` or directionless parameters only.

`annotationList direction typeIR id constantInitializerIR2` is a well-formed parameter type, with bound type variables B when:

1. If `constantInitializerIR2` is none:
 - a. Check that typeIR is a well-formed type, with bound type variables B .
 - b. Let $\text{typeIR}_{\text{unroll}}$ be typeIR with `typeddefs` unrolled.
 - c. Check that `~ is_synthesizedTypeIR( $\text{typeIR}_{\text{unroll}}$ )`.
 - d. Check that `is_stringTypeIR( $\text{typeIR}_{\text{unroll}}$ )  $\vee$  is_arbitraryIntTypeIR( $\text{typeIR}_{\text{unroll}}$ )  $\vee$  is_objectTypeIR( $\text{typeIR}_{\text{unroll}}$ )` implies `direction = `EMPTY`.
 - e. The relation holds.
2. Else:
 - a. Check that typeIR is a well-formed type, with bound type variables B .
 - b. Let $\text{typeIR}_{\text{unroll}}$ be typeIR with `typeddefs` unrolled.

- c. Check that $\sim \text{is_synthesizedTypeIR}(\text{typeIR}_{\text{unroll}})$.
- d. Check that $\text{direction} = \text{IN}$ or $\text{direction} = \text{'EMPTY'}$.
- e. Check that $\text{is_stringTypeIR}(\text{typeIR}_{\text{unroll}}) \vee \text{is_arbitraryIntTypeIR}(\text{typeIR}_{\text{unroll}}) \vee \text{is_objectTypeIR}(\text{typeIR}_{\text{unroll}})$ implies $\text{direction} = \text{'EMPTY'}$.
- f. The relation holds.

9.2. Callable definitions (callable constructors)

Similar to how type declarations introduce type definitions (see [Section 8.2.1](#)), callable declarations introduce callable definitions. These are constructors for callable types, that are parameterized by type parameters (if any).

```
actionTypeDefIR = actionTypeIR

functionTypeDefIR
: definedFunctionTypeDefIR
| externFunctionTypeDefIR
;

methodTypeDefIR
: externMethodTypeDefIR
| parserApplyMethodTypeDefIR
| controlApplyMethodTypeDefIR
| tableApplyMethodTypeDefIR
;

callableTypeDefIR
: actionTypeDefIR
| functionTypeDefIR
| methodTypeDefIR
;
```

When specialized with concrete type arguments, callable type definitions yield callable types:

```
actionTypeIR
: ACTION `( parameterIR* )
;

functionTypeIR
: definedFunctionTypeIR
| externFunctionTypeIR
;

methodTypeIR
: builtinMethodTypeIR
| externMethodTypeIR
| parserApplyMethodTypeIR
| controlApplyMethodTypeIR
| tableApplyMethodTypeIR
;

callableTypeIR
: actionTypeIR
| functionTypeIR
| methodTypeIR
;
```

For example, a function declaration introduces a function type definition:

```
T identity<T>(in T t) { return t; }
```

`identity` is a generic function type definition that falls into:

```
definedFunctionTypeDefIR
: FUNCTION `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
;
```

Notice that `definedFunctionTypeDefIR` does *not* include the function body. Because callable type definitions are used during type checking, the body of the callable is not part of the type definition. After specialization with proper type argument(s), it yields a function type:

```
definedFunctionTypeIR
: FUNCTION `( parameterIR* ) : typeIR
;
```

9.2.1. Well-formedness

`#{relation: CallableTypeDef_wf}`

`#{rulegroup: CallableTypeDef_wf} #{ruleprose: CallableTypeDef_wf}`

9.3. Actions

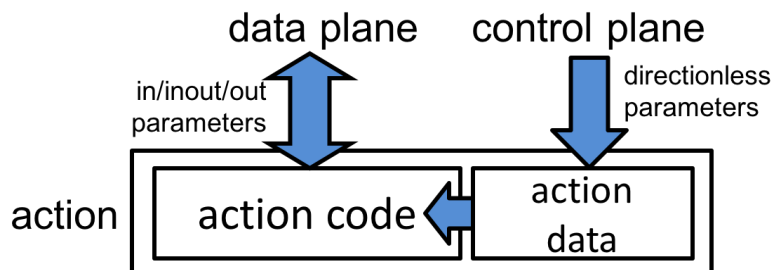


Figure 10. Actions contain code and data. The code is in the P4 program, while the data is provided in the table entries, typically populated by the control plane. Other parameters are bound by the data plane.

Actions are code fragments that can read and write the data being processed. Actions may contain data values that can be written by the control plane and read by the data plane. Actions are the main construct by which the control plane can dynamically influence the behavior of the data plane. [Figure 10](#) shows the abstract model of an `action`.

```
actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;
```

Syntactically actions resemble functions with no return value. Actions may be declared within a control block; in this case they can only be used within instances of that control block.

The following example shows an action declaration:

```
action Forward_a(out bit<9> outputPort, bit<9> port) {
    outputPort = port;
```

```
}
```

Action parameters may not have `extern` types. Action parameters that have no direction (e.g., `port` in the previous example) indicate "action data." All such parameters must appear at the end of the parameter list. When used in a match-action table (see [\[sec-table-action-list\]](#)), these parameters will be provided by the table entries (e.g., as specified by the control plane, the `default_action` table property, or the `entries` table property).

The body of an action consists of a sequence of statements and declarations. No `table`, `control`, or `parser` applications can appear within actions.

Some targets may impose additional restrictions on action bodies---e.g., only allowing straight-line code, with no conditional statements or expressions.

Actions can be executed in two ways:

- Implicitly: by tables during match-action processing.
- Explicitly: either from a `control` block or from another `action`. In either case, the values for all action parameters must be supplied explicitly, including values for the directionless parameters. In this case, the directionless parameters behave like `in` parameters.

Actions are introduced by `action` declarations. Its typing and instantiation rules are explained in [Section 11.8](#). It introduces an action type definition:

```
actionTypeDefIR = actionTypeIR
```

Because actions cannot be generic, the action type is the same as the action type definition:

```
actionTypeIR
: ACTION `( parameterIR* )
;
```

The runtime representation of an action is as follows:

```
actionDef
: ACTION nameIR `( parameterListIR ) blockStatementIR
;
```

9.3.1. Well-formedness

`ACTION ( parameterIR* )` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
2. Let `direction*` be the list of `direction` and `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `direction` and `typeIR` be the direction and the type of `parameterIR`.for each `parameterIR` in `parameterIR*`
3. Check that `directionless` parameters in `direction*` only appear at the end.
4. Check that `typeIR` can be used in an action type, for all `typeIR` in `typeIR*`.

5. The relation holds.

9.4. Functions

Two kinds of functions are supported in P4: user-defined functions and extern functions. The former are functions whose implementation is provided in P4 source code, while the latter are functions whose implementation is provided externally by the target architecture.

The internal representation of functions is as follows:

```
functionTypeDefIR
: definedFunctionTypeDefIR
| externFunctionTypeDefIR
;

functionTypeIR
: definedFunctionTypeIR
| externFunctionTypeIR
;

functionDef
: definedFunctionDef
| externFunctionDef
;
```

9.4.1. User-defined functions

Functions can only be declared at the top level and all parameters must have a direction. P4 functions are modeled after functions as found in most other programming languages, but the language does not permit recursive functions.

```
functionPrototype
: typeOrVoid name typeParameterListOpt `( parameterList )
;

functionDeclaration
: annotationList functionPrototype blockStatement
;
```

Here is an example of a function that returns the maximum of two 32-bit values:

```
bit<32> max(in bit<32> left, in bit<32> right) {
    return (left > right) ? left : right;
}
```

A function returns a value using the `return` statement. A function with a return type of `void` can simply use the `return` statement with no arguments. A function with a non-void return type must return a value of the suitable type on all possible execution paths.

The typing and instantiation rules for `function` declarations is explained in [Section 11.7](#). This introduces a function type definition, which can be specialized to a function type.

```
definedFunctionTypeDefIR
: FUNCTION `( typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
;
```

```

definedFunctionTypeIR
  : FUNCTION `( parameterIR* ) : typeIR
  ;

```

Also, the runtime representation of a defined function is as follows:

```

definedFunctionDef
  : FUNCTION nameIR `< typeParameterListIR > `( parameterListIR ) blockStatementIR
  ;

```

9.4.1.1. Well-formedness

$\text{FUNCTION (parameterIR*) : typeIR}_{\text{ret}}$ is a well-formed callable type, with bound type variables B when:

1. Check that parameterIR is a well-formed parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
2. Let typeIR^* be the list of typeIR , obtained by repeating:
 - Let typeIR be the type of parameterIR .
 for each parameterIR in parameterIR^*
3. Check that typeIR can be used in a defined function type, for all typeIR in typeIR^* .
4. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
5. The relation holds.

9.4.2. Extern functions

An extern function declaration describes the name and type signature of the function, but not its implementation.

```

externDeclaration
  : externFunctionDeclaration
  | externObjectDeclaration
  ;

```

extern declarations introduce extern function type definitions, which can be specialized to extern function types.

```

externFunctionTypeDefIR
  : EXTERN_FUNCTION `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
  ;

externFunctionTypeIR
  : EXTERN_FUNCTION `( parameterIR* ) : typeIR
  ;

```

Also, the runtime representation of an extern function is as follows:

```

externFunctionDef
  : EXTERN_FUNCTION nameIR `< typeParameterListIR > `( parameterListIR )

```

;

See [Section 11.11.1](#) for the typing and instantiation rules for **extern** function declarations.

9.4.2.1. Well-formedness

EXTERN_FUNCTION (parameterIR^*) : $\text{typeIR}_{\text{ret}}$ is a well-formed callable type, with bound type variables B when:

1. Check that parameterIR is a well-formed parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
2. Let typeIR^* be the list of typeIR , obtained by repeating:
 - Let typeIR be the type of parameterIR .for each parameterIR in parameterIR^*
3. Check that typeIR can be used in an extern function type, for all typeIR in typeIR^* .
4. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
5. The relation holds.

9.5. Methods

Objects in P4 can have methods associated with them. Specifically, **extern** objects have methods declared in their definitions. **parser**, **control**, and **table** objects have built-in **apply** methods.

Further, P4 allows built-in methods for certain types, such as **isValid()** and **setValid()** methods for header types.

The internal representation of methods is defined as follows:

```
methodTypeDefIR
: externMethodTypeDefIR
| parserApplyMethodTypeDefIR
| controlApplyMethodTypeDefIR
| tableApplyMethodTypeDefIR
;

methodTypeIR
: builtinMethodTypeIR
| externMethodTypeIR
| parserApplyMethodTypeIR
| controlApplyMethodTypeIR
| tableApplyMethodTypeIR
;

methodDef
: externMethodDef
| parserApplyMethodDef
| controlApplyMethodDef
| tableApplyMethodDef
;
```

9.5.1. Extern methods

An extern object declaration declares an object and all methods that can be invoked to perform computations and to alter the state of the object.

For example, the P4 core library introduces two extern objects `packet_in` and `packet_out` used for manipulating packets (see [\[sec-packet-data-extraction\]](#) and [\[sec-deparse\]](#)). Here is an example showing how the methods of these objects can be invoked on a packet:

```
extern packet_out {
    void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
    apply {
        b.emit(h.ipv4);          // write ipv4 header into output packet
    }                          // by calling emit method
}
```

Internally, extern methods are represented as follows:

```
externMethodTypeDefIR
: EXTERN_METHOD `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
| EXTERN_METHOD ABSTRACT `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) :
typeIR
;

externMethodTypeIR
: EXTERN_METHOD `( parameterIR* ) : typeIR
| EXTERN_METHOD ABSTRACT `( parameterIR* ) : typeIR
;

externMethodDef
: EXTERN_METHOD nameIR `< typeParameterListIR > `( parameterListIR ) blockStatementIR?
| EXTERN_METHOD ABSTRACT nameIR `< typeParameterListIR > `( parameterListIR )
;
```

Typical extern object methods are implemented by the target architecture. P4 programmers can only call such methods.

Some types of extern objects may provide methods that can be implemented by the P4 programmers. Such methods are described with the `abstract` keyword prior to the method definition. Here is an example:

```
extern Balancer {
    Balancer();
    // get the number of active flows
    bit<32> getFlowCount();
    // return port index used for load-balancing
    // @param address: IPv4 source address of flow
    abstract bit<4> on_new_flow(in bit<32> address);
}
```

When such an object is instantiated the user has to supply an implementation of all the `abstract` methods (see [\[sec-instantiating-abstract-methods\]](#)).

9.5.1.1. Well-formedness

`externMethodTypeIR` is a well-formed callable type, with bound type variables `B` when:

1. If let `EXTERN_METHOD ( parameterIR* ) : typeIRret` be `externMethodTypeIR`:

- a. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
 - b. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.
 for each `parameterIR` in `parameterIR*`
 - c. Check that `typeIR` can be used in an extern method type, for all `typeIR` in `typeIR*`.
 - d. Check that `typeIRret` is a well-formed return type, with bound type variables `B`.
 - e. The relation holds.
2. Else:
- a. Let `EXTERN_METHOD ABSTRACT ( parameterIR* ) : typeIRret` be `externMethodTypeIR`.
 - b. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
 - c. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.
 for each `parameterIR` in `parameterIR*`
 - d. Check that `typeIR` can be used in an extern method type, for all `typeIR` in `typeIR*`.
 - e. Check that `typeIRret` is a well-formed return type, with bound type variables `B`.
 - f. The relation holds.

9.5.2. Parser apply methods

P4 allows parsers to invoke the services of other parsers, similar to subroutines. To invoke the services of another parser, the sub-parser must be first instantiated; the services of an instance are invoked by calling it using its `apply` method.

The following example shows a sub-parser invocation:

```
parser callee(packet_in packet, out IPv4 ipv4) { /* body omitted */ }
parser caller(packet_in packet, out Headers h) {
  callee() subparser; // instance of callee
  state subroutine {
    subparser.apply(packet, h.ipv4); // invoke sub-parser
    transition accept; // accept if sub-parser ends in accept state
  }
}
```

A parser apply method type definition is implicitly introduced by a `parser` declaration. Its typing rule is explained in [\[sec-sem-Decl_ok-parserDeclaration\]](#). A parser declaration may not be generic. Thus, the parser apply method type definition does not have type parameters, and is same as the parser apply method type. A parser apply method is represented internally as follows:

```
parserApplyMethodTypeDefIR = parserApplyMethodTypeIR

parserApplyMethodTypeIR
: PARSE_APPLY `( parameterIR* )
;
```

```

parserApplyMethodDef
: PARSE_APPLY `( parameterListIR ) `{ parserLocalDeclarationListIR ; stateEnv }
;

```

9.5.2.1. Well-formedness

`PARSE_APPLY ( parameterIR* )` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
2. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.
for each `parameterIR` in `parameterIR*`
3. Check that `typeIR` can be used in a parser apply method type, for all `typeIR` in `typeIR*`.
4. The relation holds.

9.5.3. Control apply methods

P4 allows controls to invoke the services of other controls, similar to subroutines. To invoke the services of another control, it must be first instantiated; the services of an instance are invoked by calling it using its `apply` method.

The following example shows a control invocation:

```

control Callee(inout IPv4 ipv4) { /* body omitted */ }
control Caller(inout Headers h) {
  Callee() instance; // instance of callee
  apply {
    instance.apply(h.ipv4); // invoke control
  }
}

```

A control apply method type definition is implicitly introduced by a `control` declaration. Its typing rule is explained in [\[sec-sem-Decl_ok-controlDeclaration\]](#). A control declaration may not be generic. Thus, the control apply method type definition does not have type parameters, and is same as the control apply method type. A control apply method is represented internally as follows:

```

controlApplyMethodTypeDefIR = controlApplyMethodTypeIR

controlApplyMethodTypeIR
: CONTROL_APPLY `( parameterIR* )
;

controlApplyMethodDef
: CONTROL_APPLY `( parameterListIR ) `{ controlLocalDeclarationListIR ; actionDefEnv ;
controlBodyIR }
;

```

9.5.3.1. Well-formedness

`CONTROL_APPLY ( parameterIR* )` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
2. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.for each `parameterIR` in `parameterIR*`
3. Check that `typeIR` can be used in a control apply method type, for all `typeIR` in `typeIR*`.
4. The relation holds.

9.5.4. Table apply methods

A `table` can be invoked by calling its `apply` method. Calling an apply method on a table instance returns a synthesized table metadata structure, explained in [\[sec-def-tableMetadataTypeIR\]](#).

A table apply method definition is introduced by a `table` declaration. Its typing rule is explained in [\[sec-sem-ControlLocalDecl_ok-tableDeclaration\]](#). As with parsers and controls, a table declaration may not be generic. Thus, the table apply method type definition does not have type parameters, and is same as the table apply method type. A table apply method is represented internally as follows:

```
tableApplyMethodTypeDefIR = tableApplyMethodTypeIR

tableApplyMethodTypeIR
  : TABLE_APPLY : tableMetadataStructTypeIR
  ;

tableApplyMethodDef
  : TABLE_APPLY `{ tablePropertyListIR }
  ;
```

9.5.4.1. Well-formedness

`TABLE_APPLY : tableMetadataStructTypeIR` is a well-formed callable type, with bound type variables `B` when:

1. The relation holds.

9.5.5. Built-in methods

Built-in methods are methods associated with certain types defined by the P4 language. Thus, they do not have method type definitions, and only have method types. The internal representations are:

```
builtinMethodTypeIR
  : BUILTIN_METHOD `( parameterIR* ) : typeIR
  ;
```

10. P4 objects and constructors

10.1. Objects

`tables`, `value_sets`, and `extern` objects are stateful entities in P4 programs. These objects should be instantiated statically prior to program execution. `parsers`, `controls`, and `packages` may contain stateful objects, thus they must also be instantiated.

The runtime representation of these objects is defined as follows:

```
object
  : externObject
  | parserObject
  | controlObject
  | packageObject
  | tableObject
  | valueSetObject
  ;
```

The instantiation phase (described in [Section 7.3](#)) creates objects in the source P4 program, and allocates them in the global store. At runtime (described in [Section 7.4](#)), these objects interact, as defined by the target architecture, to process packets.

10.1.1. Fully-qualified names

Instantiation may create multiple instances of a type, each of which must have a unique, fully-qualified name. Thus, objects are identified by their full-qualified names (`objectId`). [Section 7.3.1](#) describes how a fully-qualified name is constructed.

```
objectId = nameIR*
```

10.2. Constructors

Objects can be instantiated in three ways:

- Using instantiations, described in [Section 11.6](#).
- Using constructor call expressions, described in [Section 13.10.2](#).
- Using direct application statements, described in [Section 12.4](#).

Instantiations and constructor call expressions invoke constructors, which creates an object as a result. Direct application statements are syntactic sugar for instantiations of `parser` and `control` objects.

Constructor invocation is similar to callable invocation; constructors can also be overloaded, and called using named arguments.

10.2.1. Constructor parameters

In order to support libraries of useful P4 components, `externs`, `parsers`, `controls`, and `packages` can additionally be parameterized through the use of constructor parameters.

Consider the parser declaration syntax:

```
constructorParameterListOpt
: /* empty */
| `( parameterList )
;

parserDeclaration
: annotationList PARSE name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
;
```

From this grammar fragment we infer that a **parser** declaration may have two sets of parameters:

- The runtime parser parameters (**parameterList**)
- Optional parser constructor parameters (**constructorParameterListOpt**)

Because constructors are evaluated entirely at compilation time, all constructor arguments must be expressions that can be evaluated at compilation time. In consequence, constructor parameters must be directionless (i.e., they cannot be **in**, **out**, or **inout**).

Consider the following example:

```
parser GenericParser(packet_in b, out Packet_header p)
    (bool udpSupport) { // constructor parameters
    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            16w0x0800: ipv4;
        }
    }
    state ipv4 {
        b.extract(p.ipv4);
        transition select(p.ipv4.protocol) {
            6: tcp;
            17: tryudp;
        }
    }
    state tryudp {
        transition select(udpSupport) {
            false: accept;
            true : udp;
        }
    }
    state udp {
        // body omitted
    }
}
```

When instantiating the **GenericParser** it is necessary to supply a value for the **udpSupport** parameter, as in the following example:

```
// topParser is a GenericParser where udpSupport = false
GenericParser(false) topParser;
```

Internally, constructor parameters are represented as:

```
parameterIR
: annotationList direction typeIR id constantInitializerOptIR
```

```

;
constructorParameterIR = parameterIR

```

10.2.1.1. Well-formedness

ConstructorParameterType_wf:

- **constructorParameterIR** is a well-formed constructor parameter type, with bound type variables **B**

constructorParameterIR is a well-formed constructor parameter type, with bound type variables **B** when:

1. Let **direction** be the direction of **constructorParameterIR**.
2. Check that **direction** is `EMPTY`.
3. Check that **constructorParameterIR** is a well-formed parameter type, with bound type variables **B**.
4. The relation holds.

10.2.2. Constructor definitions

Similar to how type declarations introduce type definitions and callable declarations introduce callable definitions (see [Section 9.2](#)), constructor declarations introduce constructor definitions. These are parameterized by type parameters, and produces a constructor type when specialized with specific type arguments.

The runtime representation of constructors is defined as follows:

```

constructorDef
  : externObjectConstructorDef
  | parserObjectConstructorDef
  | controlObjectConstructorDef
  | packageObjectConstructorDef
  ;

```

When type arguments and constructor arguments are supplied to a constructor definition, an object of the constructed type is created.

Similar to how callable types are internally used during type checking to represent callables, constructor types internally represent the types of constructors.

```

constructorTypeIR
  : CONSTRUCTOR `( constructorParameterIR* ) : typeIR
  ;

constructorTypeDefIR
  : CONSTRUCTOR `< typeParameterIR* , typeParameterIR* > `( constructorParameterIR* ) :
typeIR
  ;

```

10.2.2.1. Well-formedness

ConstructorTypeDef_wf:

- $\text{CONSTRUCTOR} < \text{typeParameterIR}_{\text{expl}}^*, \text{typeParameterIR}_{\text{impl}}^* > (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type definition, with bound type variables B

$\text{CONSTRUCTOR} < \text{typeParameterIR}_{\text{expl}}^*, \text{typeParameterIR}_{\text{impl}}^* > (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type definition, with bound type variables B when:

1. Check that $\text{typeIR}_{\text{object}}$ can be constructed.
2. Let typeParameterIR^* be $\text{typeParameterIR}_{\text{expl}}^*$ concatenated with $\text{typeParameterIR}_{\text{impl}}^*$.
3. Check that the elements of typeParameterIR^* are distinct.
4. Let B_{inner} be the union of the sets B and $\{ \text{typeParameterIR}^* \}$.
5. Check that $\text{CONSTRUCTOR} (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type, with bound type variables B_{inner} .
6. The relation holds.

10.3. Extern objects

An **extern** object declaration introduces an extern object type and its associated methods. Extern object declarations can also optionally declare constructor methods; these must have the same name as the enclosing **extern** type, no type parameters, and no return type.

```
externConstructorPrototype
  : annotationList typeIdentifier `( parameterList ) ;
  ;

externMethodPrototype
  : annotationList functionPrototype ;
  | annotationList ABSTRACT functionPrototype ;
  ;

externObjectDeclaration
  : annotationList EXTERN nonTypeName typeParameterListOpt `{
  externConstructorOrMethodPrototypeList }
  ;
```

Internally, an extern object constructor is represented as:

```
externMethodPrototypeIR
  : annotationList functionPrototypeIR ;
  | annotationList ABSTRACT functionPrototypeIR ;
  ;

externObjectConstructorDef
  : EXTERN nameIR `< typeParameterListIR > `( constructorParameterListIR ) `{
  externMethodPrototypeListIR }
  ;
```

Extern objects are then instantiated by constructor invocations to yield:

```
externObject
: EXTERN typeId `< theta > `{ externState frame externMethodDefEnv }
;
```

The only operation on **extern** objects is invoking a method of an extern object instance using a method call expression.

Controls, parsers, packages, and extern constructors can have parameters of type **extern** only if they are directionless parameters. Extern object instances can thus be used as arguments in the construction of such objects.

No other operations are available on extern objects, e.g., equality comparison is not defined.

10.3.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
3. Let **typeIR** be **typeIR_{object}** with **typedefs** unrolled.
4. Check that **typeIR** has type **externObjectTypeIR**.
5. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
 for each **parameterIR** in **parameterIR^{*}**
6. Check that **typeIR'** can be used in an extern constructor type, for all **typeIR'** in **typeIR^{*}**.
7. The relation holds.

10.3.2. Instantiation

Loading **externObjectConstructorDef** < typeArgumentIR^{*} > (argumentIR^{*}) with default parameters **id_{default}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **EXTERN** **nameIR** < typeParameterIR^{*} > (constructorParameterIR^{*}) { externMethodPrototypeIR^{*} } be **externObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.
4. Let **theta** be { (typeParameterIR : typeArgumentIR^{*}) }.
5. Let **IC_{callee_1}** be **IC_{callee_0}** with **BLOCK.TYPE** set to **theta**.
6. Let (constructorParameterIR_{aligned}^{*}, argumentIR_{aligned}^{*}, constructorParameterIR_{default}^{*}, value_{default}^{*}) be the result of **aligning** **constructorParameterIR^{*}** with **argumentIR^{*}** where default parameters are **id_{default}^{*}**.
7. Let **id_{aligned}^{*}** be the list of **id_{aligned}**, obtained by repeating:

- Let $id_{aligned}$ be the name of $constructorParameterIR_{aligned}$.
- for each $constructorParameterIR_{aligned}$ in $constructorParameterIR_{aligned}^*$
- Let the updated store STO_1 and the instantiation context IC_{callee_2} be
 - the result of **copy-in from p layer of IC with argumentIR_{aligned}^{*} to p_{callee} layer of IC_{callee_1} with id_{aligned}^{*}, under store STO_0 .**
 - Let $id_{cparam_default}^*$ be the list of $id_{cparam_default}$, obtained by repeating:
 - Let $id_{cparam_default}$ be the name of $constructorParameterIR_{default}$.

for each $constructorParameterIR_{default}$ in $constructorParameterIR_{default}^*$
 - Let IC_{callee_3} be IC_{callee_2} where each of $id_{cparam_default}^*$ to each of $value_{default}^*$ are added to the p_{callee} layer.
 - Let the updated instantiation context IC_{callee_4} be
 - the result of **compile-time evaluation of externMethodPrototypeIR^{*}, under instantiation context IC_{callee_3} .**
 - Let $\{ (callableId : callableDef)^* \}$ be $IC_{callee_4}.BLOCK.CALLABLE$.
 - Check that $callableDef$ has type $externMethodDef$, for all $callableDef$ in $callableDef^*$.
 - Let $externMethodDef^*$ be the list of $externMethodDef$, obtained by repeating:
 - Let $externMethodDef$ be $callableDef$.

for each $callableDef$ in $callableDef^*$
 - Let id_{cparam}^* be the list of id_{cparam} , obtained by repeating:
 - Let id_{cparam} be the name of $constructorParameterIR$.

for each $constructorParameterIR$ in $constructorParameterIR^*$
 - Let $value^{?*}$ be the list of $value^?$, obtained by repeating:
 - Let $value^?$ be id_{cparam} in map $IC_{callee_4}.BLOCK.FRAME$.

for each id_{cparam} in id_{cparam}^*
 - Check that $value^?$ is defined, for all $value^?$ in $value^{?*}$.
 - Let $value_{cparam}^*$ be the list of $value_{cparam}$, obtained by repeating:
 - Let $value_{cparam}$ be $value^?$.

for each $value^?$ in $value^{?*}$
 - Let $externState$ be $init_externState(nameIR, typeArgumentIR^*, value_{cparam}^*)$.
 - Let $object_{extern}$ be $EXTERN\ nameIR < IC_{callee_4}.BLOCK.TYPE > \{ externState\ IC_{callee_4}.BLOCK.FRAME\ \{ (callableId : externMethodDef)^* \} \}$.
 - Result in
 - the updated store STO_1 ,
 - and the constructed $object_{extern}$.

10.4. Parser objects

A parser declaration comprises a name, a list of parameters, an optional list of constructor parameters, local elements, and parser states (as well as optional annotations).

```
parserLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| valueSetDeclaration
;

parserState
: annotationList STATE name `{ parserStatementList transitionStatement }
;

parserDeclaration
: annotationList PARSE name typeParameterListOpt `( parameterList )
constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
;
```

Although the syntax for a parser declaration allows for type parameters, the current version of the P4 language does not support generic parser declarations. Note that this restriction applies only to parser declarations; parser type declarations may still be generic. For example, the following parser declaration is illegal:

```
parser P<H>(inout H data); // generic parser type declaration is legal
parser P<H>(inout H data) { /* body omitted */ } // generic parser declaration is illegal
```

At least one state, named `start`, must be present in any `parser`. A parser may not define two states with the same name. It is also illegal for a parser to give explicit definitions for the `accept` and `reject` states—those states are logically distinct from the states defined by the programmer.

Preceding the parser states, a `parser` may also contain a list of local elements. These can be constants, variables, or instantiations of objects that may be used within the parser. Such objects may be instantiations of `extern` objects, or other `parsers` that may be invoked as subroutines. However, it is illegal to instantiate a `control` block within a `parser`.

The states and local elements are all in the same namespace, thus the following example will produce an error:

```
// erroneous example
parser p() {
    bit<4> t;
    state start {
        t = 1;
        transition t;
    }
    state t { // error: name t is duplicated
        transition accept;
    }
}
```

For an example containing a complete declaration of a parser see [Section 5.3](#).

A `parser` declaration introduces a parser object constructor, represented as:

```
parserObjectConstructorDef
```

```

: PARSE ( parameterListIR ) ( constructorParameterListIR ) {
  parserLocalDeclarationListIR parserStateListIR }
;

```

Parser objects are then instantiated by constructor invocations to yield:

```

parserObject
: PARSE ( parameterListIR ) { frame parserLocalDeclarationListIR stateEnv }
;

```

10.4.1. Well-formedness

$\text{CONSTRUCTOR (parameterIR }^* \text{) : typeIR}_{\text{object}}$ is a well-formed constructor type, with bound type variables B when:

1. Check that parameterIR is a well-formed constructor parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
2. Check that $\text{typeIR}_{\text{object}}$ is a well-formed type, with bound type variables B .
3. Let typeIR be $\text{typeIR}_{\text{object}}$ with typedefs unrolled.
4. Check that typeIR has type $\text{parserObjectTypeIR}$.
5. Let typeIR^* be the list of typeIR' , obtained by repeating:
 - Let typeIR' be the type of parameterIR .
 for each parameterIR in parameterIR^*
6. Check that typeIR' can be used in a parser constructor type, for all typeIR' in typeIR^* .
7. The relation holds.

10.4.2. Instantiation

Loading $\text{parserObjectConstructorDef} \langle \cdot \rangle (\text{argumentIR}^*)$ with default parameters $\text{id}_{\text{default}}^*$, under cursor p , instantiation context IC , and store STO_0 :

1. Let $\text{PARSER (parameterIR}^* \text{) (constructorParameterIR}^* \text{) \{ parserLocalDeclarationIR}^* \text{ parserStateIR}^* \}}$ be $\text{parserObjectConstructorDef}$.
2. Let p_{callee} be BLOCK.
3. Let IC_{callee_0} be an instantiation context same as IC up to the GLOBAL layer.
4. Let $(\text{constructorParameterIR}_{\text{aligned}}^*, \text{argumentIR}_{\text{aligned}}^*, \text{constructorParameterIR}_{\text{default}}^*, \text{value}_{\text{default}}^*)$ be the result of aligning $\text{constructorParameterIR}^*$ with argumentIR^* where default parameters are $\text{id}_{\text{default}}^*$.
5. Let $\text{id}_{\text{aligned}}^*$ be the list of $\text{id}_{\text{aligned}}$, obtained by repeating:
 - Let $\text{id}_{\text{aligned}}$ be the name of $\text{constructorParameterIR}_{\text{aligned}}$.
 for each $\text{constructorParameterIR}_{\text{aligned}}$ in $\text{constructorParameterIR}_{\text{aligned}}^*$
6. Let the updated store STO_1 and the instantiation context IC_{callee_1} be
 - the result of copy-in from p layer of IC with $\text{argumentIR}_{\text{aligned}}^*$ to p_{callee} layer of IC_{callee_0} with $\text{id}_{\text{aligned}}^*$,

under store STO_0 .

7. Let $id_{cparam_default}^*$ be the list of $id_{cparam_default}$, obtained by repeating:

- Let $id_{cparam_default}$ be the name of $constructorParameterIR_{default}$.

for each $constructorParameterIR_{default}$ in $constructorParameterIR_{default}^*$

8. Let IC_{callee_2} be IC_{callee_1} where each of $id_{cparam_default}^*$ to each of $value_{default}^*$ are added to the p_{callee} layer.

9. Let the updated instantiation context IC_{local} , the store STO_2 , and $parserLocalDeclarationIR_{inst}^*$ be

- the result of compile-time evaluation of $parserLocalDeclarationIR^*$, under instantiation context IC_{callee_2} and store STO_1 .

10. Let the updated instantiation context IC_{state} and the store STO_3 be

- the result of compile-time evaluation of $parserStateIR^*$, under instantiation context IC_{local} and store STO_2 .

11. Let $object_{parser}$ be $PARSER (parameterIR^*) \{ IC_{callee_2}.BLOCK.FRAME parserLocalDeclarationIR_{inst}^* IC_{state}.BLOCK.STATE \}$.

12. Result in

- the updated store STO_3 ,
- and the constructed $object_{parser}$.

10.5. Control objects

P4 parsers are responsible for extracting bits from a packet into headers. These headers (and other metadata) can be manipulated and transformed within **control** blocks. The body of a control block resembles a traditional imperative program. Within the body of a control block, match-action units can be invoked to perform data transformations. Match-action units are represented in P4 by constructs called **tables**.

Syntactically, a **control** block is declared with a name, parameters, optional type parameters, and a sequence of declarations of constants, variables, **actions**, **tables**, and other instantiations:

```
controlLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| actionDeclaration
| tableDeclaration
;

controlBody = blockStatement

controlDeclaration
: annotationList CONTROL name typeParameterListOpt `( parameterList )
constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody }
;
```

It is illegal to instantiate a **parser** within a **control** block. Similar to parser declarations, control declarations may not be generic—e.g., the following declaration is illegal:

```
control C<H>(inout H data); // generic control type declaration is legal
control C<H>(inout H data) { /* body omitted */ } // generic control declaration is
```

P4 does not support exceptional control-flow within a **control** block. The only statement which has a non-local effect on control flow is **exit**, which causes execution of the enclosing control block to immediately terminate. That is, there is no equivalent of the **verify** statement or the **reject** state from parsers. Hence, all error handling must be performed explicitly by the programmer.

A **control** declaration introduces a control object constructor, represented as:

```
controlObjectConstructorDef
  : CONTROL `( parameterListIR ) `( constructorParameterListIR ) `{
  controlLocalDeclarationListIR APPLY controlBodyIR }
  ;
```

Control objects are then instantiated by constructor invocations to yield:

```
controlObject
  : CONTROL `( parameterListIR ) `{ frame controlLocalDeclarationListIR actionDefEnv
  controlBodyIR }
  ;
```

10.5.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
3. Let **typeIR** be **typeIR_{object}** with **typedefs unrolled**.
4. Check that **typeIR** has type **controlObjectTypeIR**.
5. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
 for each **parameterIR** in **parameterIR^{*}**
6. Check that **typeIR'** can be used in a control constructor type, for all **typeIR'** in **typeIR^{*}**.
7. The relation holds.

10.5.2. Instantiation

Loading **controlObjectConstructorDef** < · > (**argumentIR^{*}**) with default parameters **id_{default}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **CONTROL (parameterIR^{*}) (constructorParameterIR^{*}) { controlLocalDeclarationIR^{*} APPLY controlBodyIR }** be **controlObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.

4. Let $(\text{constructorParameterIR}_{\text{aligned}}^*, \text{argumentIR}_{\text{aligned}}^*, \text{constructorParameterIR}_{\text{default}}^*, \text{value}_{\text{default}}^*)$ be the result of aligning $\text{constructorParameterIR}^*$ with argumentIR^* where default parameters are $\text{id}_{\text{default}}^*$.
5. Let $\text{id}_{\text{aligned}}^*$ be the list of $\text{id}_{\text{aligned}}$, obtained by repeating:
 - Let $\text{id}_{\text{aligned}}$ be the name of $\text{constructorParameterIR}_{\text{aligned}}$.
 for each $\text{constructorParameterIR}_{\text{aligned}}$ in $\text{constructorParameterIR}_{\text{aligned}}^*$
6. Let the updated store STO_1 and the instantiation context $\text{IC}_{\text{callee}_1}$ be
 - the result of copy-in from p layer of IC with $\text{argumentIR}_{\text{aligned}}^*$ to p_{callee} layer of $\text{IC}_{\text{callee}_0}$ with $\text{id}_{\text{aligned}}^*$, under store STO_0 .
7. Let $\text{id}_{\text{cparam_default}}^*$ be the list of $\text{id}_{\text{cparam_default}}$, obtained by repeating:
 - Let $\text{id}_{\text{cparam_default}}$ be the name of $\text{constructorParameterIR}_{\text{default}}$.
 for each $\text{constructorParameterIR}_{\text{default}}$ in $\text{constructorParameterIR}_{\text{default}}^*$
8. Let $\text{IC}_{\text{callee}_2}$ be $\text{IC}_{\text{callee}_1}$ where each of $\text{id}_{\text{cparam_default}}^*$ to each of $\text{value}_{\text{default}}^*$ are added to the p_{callee} layer.
9. Let the updated instantiation context $\text{IC}_{\text{local}_0}$, the store STO_2 , and $\text{controlLocalDeclarationIR}^*$ be
 - the result of compile-time evaluation of $\text{controlLocalDeclarationIR}^*$, under instantiation context $\text{IC}_{\text{callee}_2}$ and store STO_1 .
10. Let the updated instantiation context $\text{IC}_{\text{local}_1}$, the store STO_3 , and $\text{controlBodyIR}'$ be
 - the result of compile-time evaluation of controlBodyIR , under instantiation context $\text{IC}_{\text{local}_0}$, and store STO_2 .
11. Let $\{ (\text{callableId} : \text{callableDef})^* \}$ be $\text{IC}_{\text{local}_1}.\text{BLOCK.CALLABLE}$.
12. Check that callableDef has type actionDef , for all callableDef in callableDef^* .
13. Let actionDef^* be the list of actionDef , obtained by repeating:
 - Let actionDef be callableDef .
 for each callableDef in callableDef^*
14. Let $\text{object}_{\text{control}}$ be $\text{CONTROL} (\text{parameterIR}^*) \{ \text{IC}_{\text{callee}_2}.\text{BLOCK.FRAME } \text{controlLocalDeclarationIR}^* \{ (\text{callableId} : \text{actionDef})^* \} \text{controlBodyIR}' \}$.
15. Result in
 - the updated store STO_3 ,
 - and the constructed $\text{object}_{\text{control}}$.

10.6. Table objects

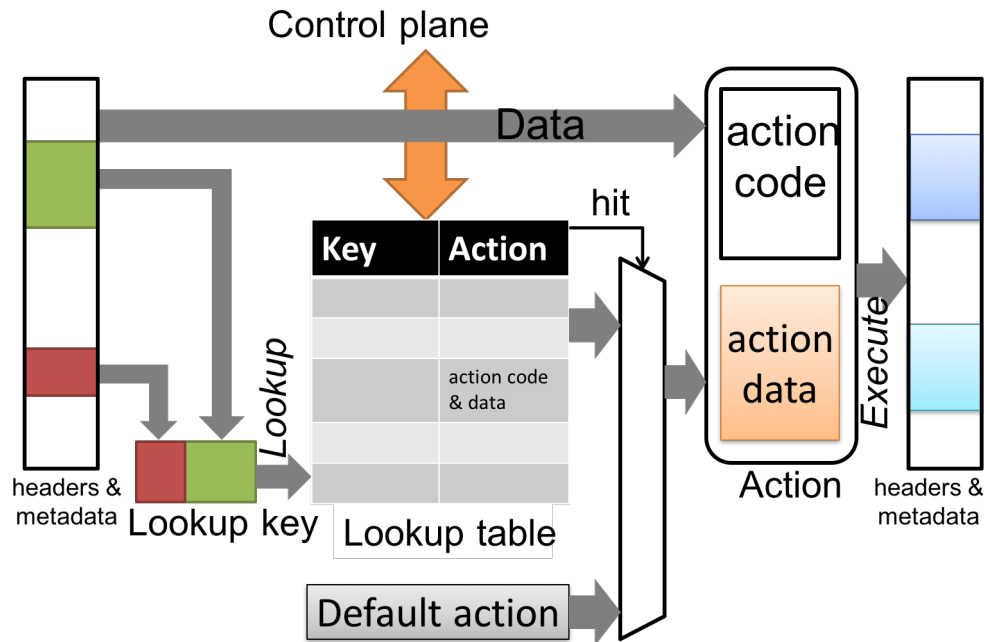


Figure 11. Match-Action Unit Dataflow.

A **table** describes a match-action unit. The structure of a match-action unit is shown in Figure 11. Processing a packet using a match-action table executes the following steps:

- Key construction.
- Key lookup in a lookup table (the "match" step). The result of key lookup is an "action".
- Action execution (the "action step") over the input data, resulting in mutations of the data.

A **table** declaration introduces a table instance. To obtain multiple instances of a table, it must be declared within a control block that is itself instantiated multiple times.

The look-up table is a finite map whose contents are manipulated asynchronously (read/write) by the target control plane, through a separate control-plane API (see Figure 11). Note that the term "table" is overloaded: it can refer to the P4 **table** objects that appear in P4 programs, as well as the internal look-up tables used in targets. We will use the term "match-action unit" when necessary to disambiguate.

Syntactically, a table is defined in terms of a set of key-value properties. Some of these properties are "standard" properties, but the set of properties can be extended by target-specific compilers as needed. Note that duplicated properties are invalid and the compiler should reject them.

```
tableProperty
: KEY = `{ tableKeyList }
| ACTIONS = `{ tableActionList }
| annotationList constOpt ENTRIES = `{ tableEntryList }
| annotationList constOpt tableCustomName initializer ;
;

tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;
;
```

The standard table properties include:

- **key**: An expression that describes how the key used for look-up is computed.

- **actions**: A list of all actions that may be found in the table.

In addition, the tables may optionally define the following properties,

- **default_action**: an action to execute when the lookup in the lookup table fails to find a match for the key used.
- **size**: an integer specifying the desired size of the table.
- **entries**: entries that are initially added to a table when the P4 program is loaded, some or all of which may be unchangeable by the control plane software.
- **largest_priority_wins** - Only useful for some tables with the **entries** property. See [\[sec-entries\]](#) for details.
- **priority_delta** - Only useful for some tables with the **entries** property. See [\[sec-entries\]](#) for details.

The compiler must set the **default_action** to **NoAction** (and also insert it into the list of **actions**) for tables that do not define the **default_action** property. Hence, all tables can be thought of as having a **default_action** property, either implicitly or explicitly.

In addition, tables may contain architecture-specific properties (see [\[sec-additional-table-properties\]](#)).

A property marked as **const** cannot be changed dynamically by the control plane. The **key**, **actions**, and **size** properties cannot be modified so the **const** keyword is not needed for these.

A **table** declaration creates a table object, thus table constructors are not defined in P4_{IR}. Table objects are defined as:

```
tablePropertyIR
: tableKeysPropertyIR
| tableActionsPropertyIR
| tableDefaultActionPropertyIR
| tableEntriesPropertyIR
| tableCustomPropertyIR
;

tableObjectProperty = {KEYS tableKeysPropertyIR, ACTIONS tableActionsPropertyIR,
DEFAULT_ACTION tableDefaultActionPropertyIR, ENTRIES tableEntriesPropertyIR, CUSTOMS
tableCustomPropertyIR*}

tableObject
: TABLE typeId `{ frame tableObjectProperty }
;
```

10.6.1. Well-formedness

CONSTRUCTOR (·) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
2. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
3. Check that **typeIR** has type **tableObjectTypeIR**.
4. The relation holds.

10.7. Parser value set objects

In some cases, the values that determine the transition from one parser state to another need to be

determined at runtime. MPLS is one example where the value of the MPLS label field is used to determine what headers follow the MPLS tag and this mapping may change dynamically at runtime. To support this functionality, P4 supports the notion of a Parser Value Set. This is a named set of values with a runtime API to add and remove values from the set.

Value sets are declared locally within a parser. They should be declared before being referenced in parser `keysetExpression` and can be used as a label in a select expression.

The syntax for declaring value sets is:

```
valueSetType
: baseType
| tupleType
| prefixedTypeName
;

valueSetDeclaration
: annotationList VALUE_SET `< valueSetType > `( expression ) name ;
;
```

Parser Value Sets support a `size` argument to provide hints to the compiler to reserve hardware resources to implement the value set. Thus, the `size` argument must be a compile-time known value. For example, this parser value set:

```
value_set<bit<16>>(4) pvs;
```

creates a `value_set` of size 4 with entries of type `bit<16>`.

The semantics of the `size` argument is similar to the `size` property of a table. If a value set has a `size` argument with value `N`, it is recommended that a compiler should choose a data plane implementation that is capable of storing `N` value set entries. See "Size property of P4 tables and parser value sets" ^[1] for further discussion on the implementation of parser value set size.

The value set is populated by the control plane by methods specified in the P4Runtime specification. ^[2]

Similar to `tables`, a `value_set` declaration creates a new parser value set object. Thus, no constructor is defined for them. The object is represented as:

```
valueSetObject
: VALUE_SET `{ value* `( nat ) }
;
```

10.8. Package objects

A package type declaration introduces a package object constructor.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

All parameters of a package are evaluated at compilation time, and in consequence they must all be directionless (they cannot be `in`, `out`, or `inout`). Otherwise package types are very similar to parser type declarations. Packages can only be instantiated; there are no runtime behaviors associated with them.

Package object constructors are defined as follows:

```
packageObjectConstructorDef
: PACKAGE `< typeParameterListIR > `( constructorParameterListIR )
;
```

Package objects are then instantiated to yield:

```
packageObject
: PACKAGE `< theta > `{ frame }
;
```

10.8.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
3. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
4. Check that **typeIR** has type **packageObjectTypeIR**.
5. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
for each **parameterIR** in **parameterIR^{*}**
6. Check that **typeIR'** can be used in a package constructor type, for all **typeIR'** in **typeIR^{*}**.
7. The relation holds.

10.8.2. Instantiation

Loading **packageObjectConstructorDef** < typeArgumentIR^{*} > (argumentIR^{*}) with default parameters **id_{default}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **PACKAGE** < typeParameterIR^{*} > (constructorParameterIR^{*}) be **packageObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.
4. Let **IC_{callee_1}** be **IC_{callee_0}** with **BLOCK.TYPE** set to { (typeParameterIR : typeArgumentIR^{*}) }.
5. Let (constructorParameterIR_{aligned}^{*}, argumentIR_{aligned}^{*}, constructorParameterIR_{default}^{*}, value_{default}^{*}) be the result of aligning constructorParameterIR^{*} with argumentIR^{*} where default parameters are **id_{default}^{*}**.
6. Let **id_{aligned}^{*}** be the list of **id_{aligned}**, obtained by repeating:
 - Let **id_{aligned}** be the name of **constructorParameterIR_{aligned}**.
for each **constructorParameterIR_{aligned}** in **constructorParameterIR_{aligned}^{*}**
7. Let the updated store **STO₁** and the instantiation context **IC_{callee_2}** be

- the result of **copy-in** from **p** layer of **IC** with argument $R_{aligned}^*$ to p_{callee} layer of IC_{callee_1} with $id_{aligned}^*$, under store STO_0 .
- Let $id_{cparam_default}^*$ be the list of $id_{cparam_default}$, obtained by repeating:
 - Let $id_{cparam_default}$ be the name of $constructorParameterIR_{default}$.
 for each $constructorParameterIR_{default}$ in $constructorParameterIR_{default}^*$
 - Let IC_{callee_3} be IC_{callee_2} where each of $id_{cparam_default}^*$ to each of $value_{default}^*$ are added to the p_{callee} layer.
 - Let $object_{package}$ be $PACKAGE < IC_{callee_3}.BLOCK.TYPE > \{ IC_{callee_3}.BLOCK.FRAME \}$.
 - Result in
 - the updated store STO_1 ,
 - and the constructed $object_{package}$.

[1] <https://github.com/p4lang/p4-spec/blob/master/p4-16/spec/docs/p4-table-and-parser-value-set-sizes.md>

[2] The P4Runtime API is defined as a Google Protocol Buffer **.proto** file and an accompanying English specification document here: <https://github.com/p4lang/p4runtime>

11. Declarations

The syntax of top-level declarations is defined as follows:

```
declaration
  : constantDeclaration
  | instantiation
  | functionDeclaration
  | actionDeclaration
  | errorDeclaration
  | matchKindDeclaration
  | externDeclaration
  | parserDeclaration
  | controlDeclaration
  | typeDeclaration
  ;

typeDeclaration
  : derivedTypeDeclaration
  | typedefDeclaration
  | parserTypeDeclaration
  | controlTypeDeclaration
  | packageTypeDeclaration
  ;
```

As described in [Chapter 7](#), declarations are type checked, instantiated at compile time, and evaluated at runtime.

11.1. Type checking declarations

[Decl_ok](#):

- Typing [declaration](#) under typing context [typingContext](#):
- Results in the updated typing context [typingContext](#) and [declarationIR](#).

After type checking, declarations are represented in $P4_{IR}$ as follows:

```
declarationIR
  : constantDeclarationIR
  | instantiationIR
  | functionDeclarationIR
  | actionDeclarationIR
  | errorDeclarationIR
  | matchKindDeclarationIR
  | externDeclarationIR
  | parserDeclarationIR
  | controlDeclarationIR
  | typeDeclarationIR
  ;

typeDeclarationIR
  : derivedTypeDeclarationIR
  | typedefDeclarationIR
  | parserTypeDeclarationIR
  | controlTypeDeclarationIR
  | packageTypeDeclarationIR
  ;
```

11.2. Compile-time evaluation of declarations

`Decl_inst`:

- Loading `declarationIR`, under instantiation context `instContext`, and store `store`:
- Results in the updated instantiation context `instContext` and the store `store`.

Instantiation statically allocates instantiated objects in the global `store`. [Section 11.6](#) describes how objects are instantiated from a declaration.

11.3. Runtime evaluation of declarations

All declarations except constant and variable declarations are evaluated at compile-time. See [Section 11.4](#) for the runtime evaluation of variable declarations and [Section 11.5](#) for the runtime evaluation of constant declarations.

The subsequent sections describe each kind of declaration in detail.

11.4. Variable declarations

Local variables are declared with a type, a name, and an optional initializer (as well as an optional annotation):

```
initializer
  : = expression
  ;

initializerOpt
  : /* empty */
  | initializer
  ;

variableDeclaration
  : annotationList type name initializerOpt ;
  ;
```

Variable declarations without an initializer are uninitialized (except for headers and other header-related types, which are initialized to invalid in the same way as described for direction `out` parameters in [Chapter 17](#)). The language places few restrictions on the types of the variables: most P4 types that can be written explicitly can be used (e.g., base types, `struct`, `header`, header stack, `tuple`). However, it is impossible to declare variables with type `int`, or with types that are only synthesized by the compiler (e.g., `set`). In addition, variables of type `parser`, `control`, `package`, or `extern` types must be declared using instantiations (see [Section 11.6](#)).

After typing, variable declarations are represented in $P4_{IR}$ as:

```
initializerIR
  : = typedExpressionIR
  ;

variableDeclarationIR
  : annotationList typeIR nameIR initializerOptIR ;
  ;
```

Reading the value of a variable that has not been initialized yields an undefined result. The compiler should attempt to detect and emit a warning in such situations.

NOTE | P4-SpecTec implements zero-initialization for local variables.

Variables declarations can appear in the following locations within a P4 program:

- In a block statement,
- In a **parser** state,
- In an **action** body,
- In a **control** block's **apply** sub-block,
- In the list of local declarations in a **parser**, and
- In the list of local declarations in a **control**.

Note that variable declarations *cannot* appear at the top level of a P4 program.

Variables have local scope, and behave like stack-allocated variables in languages such as C. The value of a variable is never preserved from one invocation of its enclosing block to the next. In particular, variables cannot be used to maintain state between different network packets.

11.4.1. Type checking

Variable declarations are type checked using the following relation:

VarDecl_ok:

- Typing **variableDeclaration**, under cursor **cursor** and typing context **typingContext**:
- Results in the updated typing context **typingContext** and **variableDeclarationIR**.

It is defined by the following rules:

Typing **annotationList type name initializerOpt ;**, under cursor **p** and typing context **TC₀**:

1. If **initializerOpt** is **EMPTY**:
 - a. Let **typeIR** and a set of fresh type variables **typed*** be
 - the result of **typing type**, under cursor **p** and typing context **TC₀**.
 - b. Check that **typed*** is an empty list.
 - c. Let **bound** be **bound type variables from the p layer of TC₀**.
 - d. Check that **typeIR** is a well-formed type, with bound type variables **bound**.
 - e. Check that **typeIR** supports assignment.
 - f. Let **nameIR** be **the name of name**.
 - g. Let **TC₁** be **TC₀ where nameIR to INOUT typeIR DYN · is added to the p layer**.
 - h. Let **variableDeclarationIR** be **annotationList typeIR nameIR · ;**.
 - i. Result in the updated typing context **TC₁** and **variableDeclarationIR**.
2. Else:

- a. Let $\text{expression}_{\text{init}}$ be `initializerOpt`.
- b. Let typeIR and a set of fresh type variables typeId^* be
 - the result of `typing type`, under cursor p and typing context TC_0 .
- c. Check that typeId^* is an empty list.
- d. Let bound be `bound type variables from the  $p$  layer of  $\text{TC}_0$` .
- e. Check that typeIR is a well-formed type, with bound type variables bound .
- f. Check that typeIR supports assignment.
- g. Let $\text{typedExpressionIR}_{\text{init}}$ be
 - the result of `typing  $\text{expression}_{\text{init}}$` , under cursor p and typing context TC_0 .
- h. Let $\text{typedExpressionIR}_{\text{init_cast}}$ be `! typedExpressionIRinit implicitly cast to  $\text{typeIR}$` .
- i. Let nameIR be `the name of  $\text{name}$` .
- j. Let TC_1 be TC_0 `where  $\text{nameIR}$  to INOUT  $\text{typeIR}$  DYN · is added to the  $p$  layer`.
- k. Let $\text{variableDeclarationIR}$ be `annotationList  $\text{typeIR}$   $\text{nameIR}$  =  $\text{typedExpressionIR}_{\text{init\_cast}}$  ;`.
- l. Result in the updated typing context TC_1 and $\text{variableDeclarationIR}$.

11.4.2. Runtime evaluation

Variable declarations are evaluated at runtime using the following relation:

VarDecl_{eval}:

- Evaluating $\text{variableDeclarationIR}$, under cursor cursor , evaluation context evalContext , and store store :
- Results in
 - the updated evaluation context evalContext , the store store ,
 - and the result declarationResult .

The result of evaluating a variable declaration is:

```

continueEmptyResult
: /* empty */
;

exitResult
: EXIT
;

abortResult
: exitResult
| rejectTransitionResult
;

declarationResult
: continueEmptyResult
| abortResult
;

```

See [Section 12.6](#) for how exitResult may occur. The relation is defined by the following rule:

Evaluating `annotationList typeIR nameIR initializerIR2 ;`, under cursor `p`, evaluation context `EC0`, and store `store`:

1. If `initializerIR2` is none:
 - a. Let `typeIRsubst` be `typeIR` substituted by bound type variables in `EC0` from the BLOCK layer.
 - b. Let `valueinit` be the default value for type `typeIRsubst`.
 - c. Let `EC1` be `EC0` where `nameIR` to `valueinit` is added to the `p` layer.
 - d. Result in
 - the updated evaluation context `EC1`, the store `store`,
 - and the result ``EMPTY`.
2. Else:
 - a. Let `typedExpressionIRinit` be `initializerIR2`.
 - b. Let the updated evaluation context `EC1`, store `STO1` and expression result `expressionResult` be
 - the result of evaluating `typedExpressionIRinit`, under cursor `p`, evaluation context `EC0`, and store `store`.
 - c. If let `abortResult` be `expressionResult`:
 - i. Result in
 - the updated evaluation context `EC1`, the store `STO1`,
 - and the result `abortResult`.
 - d. Else:
 - i. Let `valueinit` be `expressionResult`.
 - ii. Let `EC2` be `EC1` where `nameIR` to `valueinit` is added to the `p` layer.
 - iii. Result in
 - the updated evaluation context `EC2`, the store `STO1`,
 - and the result ``EMPTY`.

NOTE

Currently in P4-SpecTec, instantiation of a variable declaration is implemented as a no-op. But, instantiation *may* occur within the right-hand side, initializer expression. Ideally, revisit this.

11.5. Constant declarations

Constant values are defined with the syntax:

```
initializer
: = expression
;

constantDeclaration
: annotationList CONST type name initializer ;
;
```

The `initializer` expression must be a local compile-time known value. In P4_{IR}, constant declarations have the syntax:

```

constantInitializerIR
  : = value
  ;

constantDeclarationIR
  : annotationList CONST typeIR nameIR constantInitializerIR ;
  ;

```

Notice that `constantInitializerIR` is reduced to a `value` in $P4_{IR}$, where `value` is computed from `initializer` via local compile-time evaluation.

A constant declaration introduces a constant whose value has the specified type. The following are all legal constant declarations:

```

const bit<32> COUNTER = 32w0x0;
struct Version {
  bit<32> major;
  bit<32> minor;
}
const Version version = { 32w0, 32w0 };

```

11.5.1. Type checking

Constant declarations are type checked with the relation:

`ConstDecl_ok`:

- Typing `constantDeclaration`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `constantDeclarationIR`.

It is defined by the following rule:

Typing `annotationList CONST type name = expressionvalue ;`, under cursor `p` and typing context TC_0 :

1. Let `typeIR` and a set of fresh type variables `typed*` be
 - the result of **typing type**, under cursor `p` and typing context TC_0 .
2. Check that `typed*` is an empty list.
3. Let `bound` be **bound type variables from the `p` layer of TC_0** .
4. Check that `typeIR` is a well-formed type, with bound type variables `bound`.
5. Let `typedExpressionIRvalue` be
 - the result of **typing expression_{value}**, under cursor `p` and typing context TC_0 .
6. Check that **the compile-time known-ness of `typedExpressionIRvalue` is LCTK**.
7. Let `typedExpressionIRvalue_cast` be **! typedExpressionIR_{value} implicitly cast to typeIR**.
8. Let `value` be
 - the result of **local compile-time evaluation of `typedExpressionIRvalue_cast`**, under cursor `p` and typing context TC_0 .
9. Let `nameIR` be **the name of name**.

10. Let TC_1 be TC_0 where $nameIR$ to $\text{'EMPTY typeIR LCTK value is added to the p layer'}$.
11. Let $constantDeclarationIR$ be $annotationList\ CONST\ typeIR\ nameIR = value ;$.
12. Result in the updated typing context TC_1 and $constantDeclarationIR$.

When used as a top-level declaration, constant declarations are typed with:

Typing $constantDeclaration$ under typing context TC_0 :

1. Let the updated typing context TC_1 and $constantDeclarationIR$ be
 - the result of **typing** $constantDeclaration$, under cursor $GLOBAL$ and typing context TC_0 .
2. Result in the updated typing context TC_1 and $constantDeclarationIR$.

11.5.2. Compile-time evaluation

Constant declarations are evaluated during instantiation phase with:

ConstDecl_inst:

- Loading $constantDeclarationIR$, under cursor $cursor$, and instantiation context $instContext$:
- Results in the updated instantiation context $instContext$ and $constantDeclarationIR$.

It is defined by the following rule:

Loading $annotationList\ CONST\ typeIR\ nameIR = value ;$, under cursor p , and instantiation context IC_0 :

1. Let IC_1 be IC_0 where $nameIR$ to $value$ is added to the p layer.
2. Let $constantDeclarationIR$ be $annotationList\ CONST\ typeIR\ nameIR = value ;$.
3. Result in the updated instantiation context IC_1 and $constantDeclarationIR$.

When used as a top-level declaration, constant declarations are compile-time evaluated with:

Loading $constantDeclarationIR$, under instantiation context IC_0 , and store STO :

1. Let the updated instantiation context IC_1 and $constantDeclarationIR'$ be
 - the result of **loading** $constantDeclarationIR$, under cursor $GLOBAL$, and instantiation context IC_0 .
2. Check that $constantDeclarationIR'$ is equal to $constantDeclarationIR$.
3. Result in the updated instantiation context IC_1 and the store STO .

11.5.3. Runtime evaluation

Constant declarations are evaluated at runtime using the following relation:

ConstDecl_eval:

- Evaluating `constantDeclarationIR`, under cursor `cursor`, and evaluation context `evalContext`:
- Results in the updated evaluation context `evalContext`.

Evaluating `annotationList` `CONST typeIR nameIR = value ;`, under cursor `p`, and evaluation context `EC0`:

1. Let `EC1` be `EC0` where `nameIR` to `value` is added to the `p` layer.
2. Result in the updated evaluation context `EC1`.

11.6. Instantiations

Instantiations are similar to variable declarations, but are reserved for the types with constructors (`extern` objects, `control` blocks, `parsers`, and `packages`):

```
instantiation
: annotationList type `( argumentList ) name ;
| annotationList type `( argumentList ) name objectInitializer ;
;
```

An instantiation is written as a constructor invocation followed by a name. In `P4IR`, instantiations have the syntax:

```
instantiationIR
: annotationList typeIR prefixedNameIR `< typeArgumentListIR > `( argumentListIR )
nameIR objectInitializerOptIR ;
;
```

Instantiations are always executed at compilation time (Section [Section 7.3](#)). The effect is to allocate an object with the specified name, and to bind it to the result of the constructor invocation. Note that instantiation arguments can be specified by name.

For example, a hypothetical bank of counter objects can be instantiated as follows:

```
// from target library
enum CounterType {
    Packets,
    Bytes,
    Both
}
extern Counter {
    Counter(bit<32> size, CounterType type);
    void increment(in bit<32> index);
}
// user program
control c(/* parameters omitted */) {
    Counter(32w1024, CounterType.Both) ctr; // instantiation
    apply { /* body omitted */ }
}
```

A `P4` program may not instantiate controls and parsers at the top-level. This restriction is designed to ensure that most state resides in the architecture itself, or is local to a `parser` or `control`. For example, the following program is not valid:

```
// Program
```

```
control c(/* parameters omitted */) { /* body omitted */ }
c() c1; // illegal top-level instantiation
```

because control `c1` is instantiated at the top-level. Note that top-level declarations of constants and instantiations of extern objects are permitted.

11.6.1. Objects with abstract methods

When instantiating an extern type that has `abstract` methods users have to supply implementations for all such methods. This is done using object initializers:

```
objectDeclaration
: functionDeclaration
| instantiation
;

objectDeclarationList
: /* empty */
| objectDeclarationList objectDeclaration
;

objectInitializer
: = `{ objectDeclarationList }
;
```

In $P4_{IR}$, object initializers have the syntax:

```
objectDeclarationIR
: functionDeclarationIR
| instantiationIR
;

objectDeclarationListIR = objectDeclarationIR*

objectInitializerIR
: = `{ objectDeclarationListIR }
;
```

Abstract method implementations must use the same number of parameters, and the same parameter directions as those declared in the extern object declaration. Note that overloading of abstract methods is not allowed, thus an implementation could be matched to a declaration only with the method name.

Abstract method implementations can only use the supplied arguments or refer to values that are in the same initializer block or the top-level scope. When calling another method of the same instance the `this` keyword is used to indicate the current object instance:

```
// Instantiate a balancer
Balancer() b = { // provide an implementation for the abstract methods
    bit<4> on_new_flow(in bit<32> address) {
        // uses the address and the number of flows to balance the load
        bit<32> count = this.getFlowCount(); // call method of the same instance
        return (address + count)[3:0];
    }
}
```

Abstract methods may be invoked by users explicitly, or they may be invoked by the target architecture. The architectural description has to specify when the abstract methods are invoked and what the meaning of their arguments and return values is; target architectures may impose additional constraints

on abstract methods.

11.6.1.1. Type checking

An object declaration is type checked with the relation:

ObjectDecl_ok:

- Typing `objectDeclaration` under cursor `cursor` and typing context `typingContext` with `typeFrame` and `externMethodTypeDefEnv`:
- Results in `typeFrame` and `externMethodTypeDefEnv` of the object declaration and `objectDeclarationIR`.

It is defined by the following rules:

Typing `objectDeclaration` under cursor `p` and typing context `TC0` with `typeFrame` and `externMethodTypeDefEnv`:

1. If let `instantiation` be `objectDeclaration`:
 - a. Let the updated typing context `TC1` and `instantiationIR` be
 - the result of **typing** `instantiation`, under cursor `p` and typing context `TC0`.
 - b. Let `nameIR` be the name of `instantiationIR`.
 - c. Let `varTypeIR'` be **! the type of variable** `nameIR` in the `p` layer of `TC1`.
 - d. Let `typeFrameinit` be **the map** `typeFrame` with the value for `nameIR` is updated to `varTypeIR'`.
 - e. Result in `typeFrameinit` and `externMethodTypeDefEnv` of the object declaration and `instantiationIR`.
2. Else:
 - a. Let `annotationList` `typeOrVoid` `name` `typeParameterListOpt` (`parameterList`) `blockStatement` be `objectDeclaration`.
 - b. Let `TC1` be `TC0` with `BLOCK.KIND` set to `EXTERN`.
 - c. Let `TC2` be `TC1` with `BLOCK.FRAME` set to `typeFrame`.
 - d. Let the updated typing context `TC3` and type parameters `typeIdexpl*` be
 - the result of **typing** `typeParameterListOpt`, under cursor `LOCAL` and typing context `TC2`.
 - e. Let `typeIRret` and a set of fresh type variables `typeId*` be
 - the result of **typing** `typeOrVoid`, under cursor `LOCAL` and typing context `TC3`.
 - f. Check that `typeId*` is an empty list.
 - g. Let the parameter list `parameterIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of **typing** `parameterList`, under cursor `LOCAL` and typing context `TC3`.
 - h. Let `TC4` be `parameterIR*` added to the `LOCAL` layer of `TC3`.
 - i. Let `TC5` be `TC4` with `LOCAL.KIND` set to `EXTERN_METHOD : typeIRret`.
 - j. Let the typing context `_`, abstract control flow `f`, and `blockStatementIR` be
 - the result of **typing** `blockStatement`, under typing context `TC5`, abstract control flow `CONT`, and loop context `NOLOOP`.
 - k. Check that `f = RET` or `typeIRret = VOID`.

- l. Let `callableId` be the callable identifier for `name ( parameterList )`.
- m. Let `externMethodTypeDefIR` be `EXTERN_METHOD < typeIdexpl*, typeIdimpl* > ( parameterIR* ) : typeIdret*`.
- n. Let `bound` be bound type variables from the `p` layer of `TC0`.
- o. Check that `externMethodTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
- p. Let `externMethodTypeDefEnvinit` be the map `externMethodTypeDefEnv` with the value for `callableId` is updated to `externMethodTypeDefIR`.
- q. Let `nameIR` be the name of `name`.
- r. Let `functionDeclarationIR` be `annotationList typeIdret nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* ) blockStatementIR`.
- s. Result in `typeFrame` and `externMethodTypeDefEnvinit` of the object declaration and `functionDeclarationIR`.

A list of object declarations is type checked with the relation:

ObjectDeclList_ok:

- Typing `objectDeclarationList` under cursor `cursor` and typing context `typingContext`:
- Results in `typeFrame` and `externMethodTypeDefEnv` of the object declaration block and `objectDeclarationListIR`.

11.6.1.2. Compile-time evaluation

At compile-time, an object declaration is loaded with the relation:

ObjectDecl_inst:

- Loading `objectDeclarationIR` under instantiation context `instContext` and store `store`:
- Results in the updated instantiation context `instContext` and store `store`.

It is defined by the following rules:

Loading `objectDeclarationIR` under instantiation context `IC0` and store `store`:

1. If let `functionDeclarationIR` be `objectDeclarationIR`:
 - a. Let the updated instantiation context `IC1` be
 - the result of loading `functionDeclarationIR`, under cursor `BLOCK`, and instantiation context `IC0`.
 - b. Result in the updated instantiation context `IC1` and store `store`.
2. Else:
 - a. Let `instantiationIR` be `objectDeclarationIR`.
 - b. Let the updated instantiation context `IC1`, store `STO1`, and the reference to the object `_` be
 - the result of loading `instantiationIR`, under cursor `GLOBAL`, instantiation context `IC0`, and store `store`.

c. Result in the updated instantiation context IC_1 and store STO_1 .

A list of object declarations is loaded with the relation:

ObjectDecls_inst:

- Loading `objectDeclarationListIR` under instantiation context `instContext` and store `store`:
- Results in the updated instantiation context `instContext` and store `store`.

11.6.2. Type checking

When used as a top-level declaration, instantiations are typed with:

Typing instantiation under typing context TC_0 :

1. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of **typing instantiation, under cursor GLOBAL and typing context TC_0** .
2. Result in the updated typing context TC_1 and `instantiationIR`.

Instantiations are type checked with the relation:

InstDecl_ok:

- Typing `instantiation`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `instantiationIR`.

Typing instantiation, under cursor p and typing context TC_0 :

1. If `let annotationList type ( argumentList ) name ; be instantiation`:
 - a. If `let prefixedTypeName be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of **typing annotationList prefixedTypeName < `EMPTY > (argumentList) name ; under cursor p and typing context TC_0** .
 - ii. Result in the updated typing context TC_1 and `instantiationIR`.
 - b. Else if `let prefixedTypeName < typeArgumentList > be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of **typing annotationList prefixedTypeName < typeArgumentList > (argumentList) name ; under cursor p and typing context TC_0** .
 - ii. Result in the updated typing context TC_1 and `instantiationIR`.
2. Else:
 - a. Let `annotationList type ( argumentList ) name = { objectDeclarationList } ; be instantiation`.
 - b. If `let prefixedTypeName be type`:

- i. Let the updated typing context TC_1 and instantiationIR be
 - the result of **typing** annotationList prefixedTypeName < `EMPTY` > (argumentList) name = { objectDeclarationList } ; under cursor p and typing context TC_0 .
- ii. Result in the updated typing context TC_1 and instantiationIR.
- c. Else if let prefixedTypeName < typeArgumentList > be type:
 - i. Let the updated typing context TC_1 and instantiationIR be
 - the result of **typing** annotationList prefixedTypeName < typeArgumentList > (argumentList) name = { objectDeclarationList } ; under cursor p and typing context TC_0 .
 - ii. Result in the updated typing context TC_1 and instantiationIR.

There are two sub-relations, depending on whether object initializers were provided or not.

When object initializers are absent

InstDecl_non_objectInitializer_ok:

- Typing annotationList prefixedTypeName < typeArgumentList > (argumentList) name ; under cursor cursor and typing context typingContext:
- Results in the updated typing context typingContext and instantiationIR.

Typing annotationList prefixedTypeName < typeArgumentList > (argumentList) name ; under cursor p and typing context TC_0 :

1. Let typeArgumentIR^{*} and a set of fresh type variables typeId_{impl}^{*} be
 - the result of **typing** typeArgumentList, under cursor p and typing context TC_0 .
2. Let argumentIR^{*} be
 - the result of **typing** argumentList, under cursor p and typing context TC_0 .
3. Let prefixedNameIR be the prefixed name of prefixedTypeName.
4. Let constructorTypeIR with fresh type variables typeId_{inserted}^{*} and default parameter ids id_{default}^{*} be
 - the result of **type checking constructor** prefixedNameIR < typeArgumentIR^{*} > (argumentIR^{*}), under cursor p and typing context TC_0 .
5. Let typeId_{infer}^{*} be typeId_{impl}^{*} concatenated with typeId_{inserted}^{*}.
6. Let constructed type typeIR_{object} with inferred typeArgumentIR_{inferred}^{*} and cast-inserted argumentIR_{cast}^{*} be
 - the result of **typing the instantiation** constructorTypeIR < typeArgumentIR^{*} > (argumentIR^{*}) with omitted type arguments typeId_{infer}^{*} and omitted arguments id_{default}^{*}, under cursor p, typing context TC_0 , and NAMED instantiation site context.
7. Let nameIR be the name of name.
8. Let TC_1 be TC_0 where nameIR to `EMPTY` typeIR_{object} CTK · is added to the p layer.
9. Let instantiationIR be annotationList typeIR_{object} prefixedNameIR < typeArgumentIR_{inferred}^{*} > (argumentIR_{cast}^{*}) nameIR ;.
10. Result in the updated typing context TC_1 and instantiationIR.

When object initializers are present

InstDecl_objectInitializer_ok:

- Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name = { objectDeclarationList }` ; under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `instantiationIR`.

Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name = { objectDeclarationList }` ; under cursor `p` and typing context `TC0`:

1. Let `typeArgumentIR*` and a set of fresh type variables `typedimpl*` be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context `TC0`.
2. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC0`.
3. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
4. Let `constructorTypeIR` with fresh type variables `typedinserted*` and default parameter ids `iddefault*` be
 - the result of type checking constructor `prefixedNameIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC0`.
5. Let `typedinfer*` be `typedimpl*` concatenated with `typedinserted*`.
6. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of typing the instantiation `constructorTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typedinfer*` and omitted arguments `iddefault*`, under cursor `p`, typing context `TC0`, and NAMED instantiation site context.
7. Let `typeIR` be `typeIRobject` with typedefs unrolled.
8. Check that `typeIR` has type `externObjectTypeIR`.
9. Let `EXTERN typedextern < typeIRarg* > externMethodTypeDefEnvextern` be `typeIR`.
10. Let `TC1` be `TC0` where "this" to `EMPTY typeIRobject CTK` · is added to the LOCAL layer.
11. Let `typeFrameinit` and `externMethodTypeDefEnvinit` of the object declaration block and `objectDeclarationIR*` be
 - the result of typing `objectDeclarationList` under cursor `p` and typing context `TC1`.
12. Let `externMethodTypeDefEnvinit_subst` be substitution of abstract methods in `externMethodTypeDefEnvextern` by implementations in `externMethodTypeDefEnvinit`.
13. Let `typeIRobject_init` be `EXTERN typedextern < typeIRarg* > externMethodTypeDefEnvinit_subst`.
14. Check that `typeIRobject_init` is an extern object type without abstract methods.
15. Let `nameIR` be the name of `name`.
16. Let `TC2` be `TC0` where `nameIR` to `EMPTY typeIRobject_init CTK` · is added to the p layer.
17. Let `instantiationIR` be `annotationList typeIRobject_init prefixedNameIR < typeArgumentIRinferred* > ( argumentIRcast* )` `nameIR = { objectDeclarationIR* } ;`.
18. Result in the updated typing context `TC2` and `instantiationIR`.

11.6.3. Compile-time evaluation

When used as a top-level declaration, instantiations are evaluated with:

Loading *instantiationIR*, under instantiation context IC_0 , and store STO :

1. Let the updated instantiation context IC_1 , store STO_1 , and the reference to the object $_$ be
 - the result of **loading** *instantiationIR*, under cursor GLOBAL, instantiation context IC_0 , and store STO .
2. Result in the updated instantiation context IC_1 and the store STO_1 .

Instantiations are evaluated at compile-time with the relation:

InstDecl_inst:

- Loading *instantiationIR*, under cursor *cursor*, instantiation context *instContext*, and store *store*:
- Results in
 - the updated instantiation context *instContext*, store *store*,
 - and the reference to the object *constantDeclarationIR*.

Loading *annotationList* *typeIR* *prefixedNameIR* < *typeArgumentListIR* > (*argumentListIR*) *nameIR* *objectInitializerOptIR* ;, under cursor *p*, instantiation context IC_0 , and store STO_0 :

1. Let the constructor *constructorDef* < *typeArgumentListIR_{inst}* > with default parameters *id_{default}** be
 - the result of **finding constructor for** *prefixedNameIR* < *typeArgumentListIR* > (*argumentListIR*) under cursor *p*, and instantiation context IC_0 .
2. Let IC_{inner} be IC_0 with *nameIR* added to the path.
3. Let the updated store STO_1 , and the constructed object be
 - the result of **loading** *constructorDef* < *typeArgumentListIR_{inst}* > (*argumentListIR*) with default parameters *id_{default}**, under cursor *p*, instantiation context IC_{inner} , and store STO_0 .
4. If let EXTERN *typed* < *theta* > { *externState* *frame* *externMethodDefEnv* } be object:
 - a. Let IC_{decl} be an instantiation context same as IC_0 up to the GLOBAL layer.
 - b. Let *objectDeclarationListIR* be the list of object declarations in *objectInitializerOptIR*.
 - c. Let the updated instantiation context IC_{decl_post} and store STO_2 be
 - the result of **loading** *objectDeclarationListIR* under instantiation context IC_{decl} and store STO_1 .
 - d. Let *frame_{merged}* be the merged frame of *frame* and IC_{decl_post} .BLOCK.FRAME.
 - e. Let *externMethodDefEnv_{merged}* be the merged extern method definition environment of *externMethodDefEnv* and IC_{decl_post} .BLOCK.CALLABLE.
 - f. Let *object_{merged}* be EXTERN *typed* < *theta* > { *externState* *frame_{merged}* *externMethodDefEnv_{merged}* }.
 - g. Let *objectId* be IC_0 .PATH concatenated with [*nameIR*].
 - h. Let STO_3 be STO_2 where *objectId* to *object_{merged}* is added.
 - i. Let IC_1 be IC_0 where *nameIR* to REF *objectId* is added to the *p* layer.

- j. Let `constantDeclarationIR` be ``EMPTY CONST typeIR nameIR = REF objectId ;`.
- k. Result in
 - the updated instantiation context `IC1`, store `STO3`,
 - and the reference to the object `constantDeclarationIR`.
- 5. If `~is_externObject(object)`:
 - a. Let `objectId` be `IC0.PATH` concatenated with `[nameIR]`.
 - b. Let `STO2` be `STO1` where `objectId` to object is added.
 - c. Let `IC1` be `IC0` where `nameIR` to `REF objectId` is added to the `p` layer.
 - d. Let `constantDeclarationIR` be ``EMPTY CONST typeIR nameIR = REF objectId ;`.
 - e. Result in
 - the updated instantiation context `IC1`, store `STO2`,
 - and the reference to the object `constantDeclarationIR`.

11.7. Function declarations

Function declarations introduce user-defined functions into a P4 program.

```
functionPrototype
: typeOrVoid name typeParameterListOpt `( parameterList )
;

functionDeclaration
: annotationList functionPrototype blockStatement
;
```

Section 9.4.1 describes the semantics of user-defined functions in P4. After type checking, function declarations are represented in P4_{IR} as:

```
functionPrototypeIR
: typeIR nameIR `< typeParameterListIR , typeParameterListIR > `( parameterListIR )
;

functionDeclarationIR
: annotationList functionPrototypeIR blockStatementIR
;
```

11.7.1. Type checking

Function declarations are type checked with the relation:

`FuncDecl_ok`:

- Typing `functionDeclaration`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `functionDeclarationIR`.

It is defined by the following rule:

Typing annotationList typeOrVoid name typeParameterListOpt (parameterList) blockStatement, under cursor p and typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed_{expl}^*$ be
 - the result of **typing** typeParameterListOpt, under cursor LOCAL and typing context TC_0 .
2. Let $typed_{ret}$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** typeOrVoid, under cursor LOCAL and typing context TC_1 .
3. Check that $typed^*$ is an empty list.
4. Let the parameter list $parameterIR^*$ and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** parameterList, under cursor LOCAL and typing context TC_1 .
5. Let TC_2 be $parameterIR^*$ added to the LOCAL layer of TC_1 .
6. Let TC_3 be TC_2 with LOCAL.KIND set to FUNCTION : $typed_{ret}$.
7. Let the typing context $_$, abstract control flow f , and blockStatementIR be
 - the result of **typing** blockStatement, under typing context TC_3 , abstract control flow CONT, and loop context NOLOOP.
8. Check that $f = RET$ or $typed_{ret} = VOID$.
9. Let callableId be the callable identifier for name (parameterList).
10. Let definedFunctionTypeDefIR be FUNCTION < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$) : $typed_{ret}$.
11. Let bound be bound type variables from the p layer of TC_0 .
12. Check that definedFunctionTypeDefIR is a well-formed callable type definition, with bound type variables bound.
13. Let TC_4 be TC_0 where callableId to definedFunctionTypeDefIR is added as an overloaded callable to the p layer.
14. Let nameIR be the name of name.
15. Let functionDeclarationIR be annotationList $typed_{ret}$ nameIR < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$) blockStatementIR.
16. Result in the updated typing context TC_4 and functionDeclarationIR.

When used as a top-level declaration, function declarations are typed with:

Typing functionDeclaration under typing context TC_0 :

1. Let the updated typing context TC_1 and functionDeclarationIR be
 - the result of **typing** functionDeclaration, under cursor GLOBAL and typing context TC_0 .
2. Result in the updated typing context TC_1 and functionDeclarationIR.

11.7.2. Compile-time evaluation

At compile-time, function declarations are loaded into the context with the relation:

FuncDecl_inst:

- Loading `functionDeclarationIR`, under cursor `cursor`, and instantiation context `instContext`:
- Results in the updated instantiation context `instContext`.

It is defined by the following rule:

Loading `annotationList typeIR nameIR < typeParameterListIRexpl , typeParameterListIRimpl > ( parameterListIR ) blockStatementIR`, under cursor `p`, and instantiation context `IC0`:

1. Let `callableId` be the callable identifier for `nameIR ( parameterListIR )`.
2. Let `typeParameterListIR` be `typeParameterListIRexpl` concatenated with `typeParameterListIRimpl`.
3. Let `definedFunctionDef` be `FUNCTION nameIR < typeParameterListIR > ( parameterListIR ) blockStatementIR`.
4. Let `IC1` be `IC0` where `callableId` to `definedFunctionDef` is added as an overloaded callable to the `p` layer.
5. Result in the updated instantiation context `IC1`.

When used as a top-level declaration, function declarations are loaded with:

Loading `functionDeclarationIR`, under instantiation context `IC0`, and store `STO`:

1. Let the updated instantiation context `IC1` be
 - the result of **loading** `functionDeclarationIR`, under cursor `GLOBAL`, and instantiation context `IC0`.
2. Result in the updated instantiation context `IC1` and the store `STO`.

11.8. Action declarations

`action` declarations introduce actions:

```
actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;
```

Section 9.3 describes the semantics of actions. After type checking, action declarations are represented in `P4IR` as:

```
actionDeclarationIR
: annotationList ACTION nameIR `( parameterListIR ) blockStatementIR
;
```

11.8.1. Type checking

Action declarations are type checked with the relation:

ActionDecl_ok:

- Typing `actionDeclaration`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `actionDeclarationIR`.

It is defined by the following rule:

Typing annotationList ACTION name (parameterList) blockStatement, under cursor p and typing context TC_0 :

1. Let TC_1 be TC_0 with **LOCAL.KIND** set to **ACTION**.
2. Let the parameter list $parameterIR^*$ and a set of fresh type variables $typeId^*$ be
 - the result of **typing** parameterList, under cursor **LOCAL** and typing context TC_1 .
3. Check that $typeId^*$ is an empty list.
4. Let TC_2 be $parameterIR^*$ added to the **LOCAL** layer of TC_1 .
5. Let the typing context $_$, abstract control flow $_$, and $blockStatementIR$ be
 - the result of **typing** blockStatement, under typing context TC_2 , abstract control flow **CONT**, and loop context **NOLOOP**.
6. Let $callableId$ be the callable identifier for name (parameterList).
7. Let $actionTypeDefIR$ be **ACTION** ($parameterIR^*$).
8. Let $bound$ be bound type variables from the p layer of TC_0 .
9. Check that $actionTypeDefIR$ is a well-formed callable type definition, with bound type variables $bound$.
10. Let TC_3 be TC_0 where $callableId$ to $actionTypeDefIR$ is added as a non-overloaded callable to the p layer.
11. Let $nameIR$ be the name of name.
12. Let $actionDeclarationIR$ be annotationList ACTION nameIR ($parameterIR^*$) $blockStatementIR$.
13. Result in the updated typing context TC_3 and $actionDeclarationIR$.

When used as a top-level declaration, action declarations are typed with:

Typing actionDeclaration under typing context TC_0 :

1. Let the updated typing context TC_1 and $actionDeclarationIR$ be
 - the result of **typing** actionDeclaration, under cursor **GLOBAL** and typing context TC_0 .
2. Result in the updated typing context TC_1 and $actionDeclarationIR$.

11.8.2. Compile-time evaluation

At compile-time, function declarations are loaded into the context with the relation:

ActionDecl_inst:

- Loading $actionDeclarationIR$, under cursor **cursor**, and instantiation context $instContext$:
- Results in the updated instantiation context $instContext$.

It is defined by the following rule:

Loading `annotationList ACTION nameIR ( parameterListIR ) blockStatementIR`, under cursor `p`, and instantiation context `IC0`:

1. Let `callableId` be the callable identifier for `nameIR ( parameterListIR )`.
2. Let `actionDef` be `ACTION nameIR ( parameterListIR ) blockStatementIR`.
3. Let `IC1` be `IC0` where `callableId` to `actionDef` is added as a non-overloaded callable to the `p` layer.
4. Result in the updated instantiation context `IC1`.

When used as a top-level declaration, action declarations are loaded with:

Loading `actionDeclarationIR`, under instantiation context `IC0`, and store `STO`:

1. Let the updated instantiation context `IC1` be
 - the result of loading `actionDeclarationIR`, under cursor `GLOBAL`, and instantiation context `IC0`.
2. Result in the updated instantiation context `IC1` and the store `STO`.

11.9. Error declarations

`error` declarations introduce error elements.

```
errorDeclaration
: ERROR `{ nameList }
;
```

See [Section 8.1.4](#) for more details on error elements. After type checking, an `error` declaration is represented as:

```
errorDeclarationIR
: ERROR `{ nameListIR }
;
```

11.9.1. Type checking

Typing `ERROR { nameList }` under typing context `TC0`:

1. Let `name*` be the list of names in `nameList`.
2. Let `nameIR*` be the list of `nameIR`, obtained by repeating:
 - Let `nameIR` be the name of `name`.for each `name` in `name*`
3. Check that the elements of `nameIR*` are distinct.
4. Let `nameIRerror*` be the list of `nameIRerror`, obtained by repeating:
 - Let `nameIRerror` be "error." concatenated with `nameIR`.

for each nameIR in nameIR^*

5. Let $\text{value}_{\text{error}}^*$ be the list of $\text{value}_{\text{error}}$, obtained by repeating:

- Let $\text{value}_{\text{error}}$ be $\text{ERROR} . \text{nameIR}$.

for each nameIR in nameIR^*

6. Let varTypeIR^* be the list of varTypeIR , obtained by repeating:

- Let varTypeIR be $\text{EMPTY ERROR LCTK value}_{\text{error}}$.

for each $\text{value}_{\text{error}}$ in $\text{value}_{\text{error}}^*$

7. Let TC_1 be TC_0 where each of $\text{nameIR}_{\text{error}}^*$ to each of varTypeIR^* are added to the GLOBAL layer.

8. Result in the updated typing context TC_1 and $\text{ERROR} \{ \text{nameIR}^* \}$.

11.9.2. Compile-time evaluation

At compile-time, error declarations are loaded into the context by:

Loading $\text{ERROR} \{ \text{nameIR}^* \}$, under instantiation context IC_0 , and store STO :

1. Let $\text{nameIR}_{\text{error}}^*$ be the list of $\text{nameIR}_{\text{error}}$, obtained by repeating:

- Let $\text{nameIR}_{\text{error}}$ be the concatenation of $["\text{error}.", \text{nameIR}]$.

for each nameIR in nameIR^*

2. Let $\text{value}_{\text{error}}^*$ be the list of $\text{value}_{\text{error}}$, obtained by repeating:

- Let $\text{value}_{\text{error}}$ be $\text{ERROR} . \text{nameIR}$.

for each nameIR in nameIR^*

3. Let IC_1 be IC_0 where each of $\text{nameIR}_{\text{error}}^*$ to each of $\text{value}_{\text{error}}^*$ are added to the GLOBAL layer.

4. Result in the updated instantiation context IC_1 and the store STO .

11.10. Match kind declarations

Similar to `error` declarations, `match_kind` declarations introduce match kinds.

```
matchKindDeclaration
: MATCH_KIND `{ nameList trailingCommaOpt }
;
```

See [Section 8.1.5](#) for more details on match kinds. After type checking, an `match_kind` declaration is represented as:

```
matchKindDeclarationIR
: MATCH_KIND `{ nameListIR }
```

;

11.10.1. Type checking

Typing `MATCH_KIND { nameList trailingCommaOpt }` under typing context TC_0 :

1. Let $name^*$ be the list of names in `nameList`.
2. Let $nameIR^*$ be the list of `nameIR`, obtained by repeating:
 - Let `nameIR` be the name of `name`.for each `name` in $name^*$
3. Check that the elements of $nameIR^*$ are distinct.
4. Let $value_{match_kind}^*$ be the list of `valuematch_kind`, obtained by repeating:
 - Let `valuematch_kind` be `MATCH_KIND . nameIR`.for each `nameIR` in $nameIR^*$
5. Let $varTypeIR^*$ be the list of `varTypeIR`, obtained by repeating:
 - Let `varTypeIR` be `EMPTY MATCH_KIND LCTK valuematch_kind`.for each `valuematch_kind` in $value_{match_kind}^*$
6. Let TC_1 be TC_0 where each of $nameIR^*$ to each of $varTypeIR^*$ are added to the GLOBAL layer.
7. Result in the updated typing context TC_1 and `MATCH_KIND { nameIR* }`.

11.10.2. Compile-time evaluation

At compile-time, match kind declarations are loaded into the context by:

Loading `MATCH_KIND { nameIR* }`, under instantiation context IC_0 , and store STO :

1. Let $value_{match_kind}^*$ be the list of `valuematch_kind`, obtained by repeating:
 - Let `valuematch_kind` be `MATCH_KIND . nameIR`.for each `nameIR` in $nameIR^*$
2. Let IC_1 be IC_0 where each of $nameIR^*$ to each of $value_{match_kind}^*$ are added to the GLOBAL layer.
3. Result in the updated instantiation context IC_1 and the store STO .

11.11. Extern declarations

P4 supports extern object declarations and extern function declarations using the following syntax.

`externDeclaration`

```

: externFunctionDeclaration
| externObjectDeclaration
;

```

11.11.1. Extern function declarations

An extern function declaration introduces an extern function.

```

externFunctionDeclaration
: annotationList EXTERN functionPrototype ;
;

```

After type checking, an extern function declaration is represented in $P4_{IR}$ as:

```

externFunctionDeclarationIR
: annotationList EXTERN functionPrototypeIR ;
;

```

See [Section 9.4.2](#) for how extern functions are internally represented.

11.11.1.1. Type checking

Typing annotationList EXTERN typeOrVoid name typeParameterListOpt (parameterList) ; under typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed_{expl}^*$ be
 - the result of **typing** typeParameterListOpt, under cursor LOCAL and typing context TC_0 .
2. Let $typeIR_{ret}$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** typeOrVoid, under cursor LOCAL and typing context TC_1 .
3. Check that $typed^*$ is an empty list.
4. Let TC_2 be TC_1 with LOCAL.KIND set to EXTERN_FUNCTION : $typeIR_{ret}$.
5. Let the parameter list $parameterIR^*$ and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** parameterList, under cursor LOCAL and typing context TC_2 .
6. Let callableId be the callable identifier for name (parameterList).
7. Let externFunctionTypeDefIR be EXTERN_FUNCTION < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$) : $typeIR_{ret}$.
8. Let bound be bound type variables from the GLOBAL layer of TC_0 .
9. Check that externFunctionTypeDefIR is a well-formed callable type definition, with bound type variables bound.
10. Let TC_4 be TC_0 where callableId to externFunctionTypeDefIR is added as an overloaded callable to the GLOBAL layer.
11. Let nameIR be the name of name.
12. Let externFunctionDeclarationIR be annotationList EXTERN $typeIR_{ret}$ nameIR < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$);.
13. Result in the updated typing context TC_4 and externFunctionDeclarationIR.

11.11.1.2. Compile-time evaluation

At compile-time, extern function declarations are loaded into the context by:

Loading annotationList EXTERN typeIR nameIR < typeParameterListIR_{expl} , typeParameterListIR_{impl} > (parameterListIR)
;, under instantiation context IC₀, and store STO:

1. Let callableId be the callable identifier for nameIR (parameterListIR).
2. Let typeParameterListIR be typeParameterListIR_{expl} concatenated with typeParameterListIR_{impl}.
3. Let externFunctionDef be EXTERN_FUNCTION nameIR < typeParameterListIR > (parameterListIR).
4. Let IC₁ be IC₀ where callableId to externFunctionDef is added as an overloaded callable to the GLOBAL layer.
5. Result in the updated instantiation context IC₁ and the store STO.

11.11.2. Extern object declarations

An extern object declaration introduces an extern object.

```
externObjectDeclaration
  : annotationList EXTERN nonTypeName typeParameterListOpt `{
  externConstructorOrMethodPrototypeList }
  ;
```

After type checking, an extern object declaration is represented in P4_{IR} as:

```
externObjectDeclarationIR
  : annotationList EXTERN nameIR `< typeParameterListIR , > `{
  externConstructorPrototypeListIR externMethodPrototypeListIR }
  ;
```

See [Section 10.3](#) for how extern objects are internally represented.

11.11.2.1. Type checking

Typing annotationList EXTERN nonTypeName typeParameterListOpt { externConstructorOrMethodPrototypeList }
under typing context TC₀:

1. Let (externConstructorPrototype*, externMethodPrototype*) be externConstructorOrMethodPrototypeList split into constructors and methods.
2. Let TC₁ be TC₀ with BLOCK.KIND set to EXTERN.
3. Let the updated typing context TC₂ and type parameters typeId_{expl}* be
 - the result of typing typeParameterListOpt, under cursor BLOCK and typing context TC₁.
4. Let nameIR be the name of nonTypeName.
5. Let the updated typing context TC₃ and the typed method prototypes externMethodPrototypeIR* be
 - the result of typing externMethodPrototype*, under typing context TC₂ for an extern object nameIR.
6. Let { (callableId : callableTypeDefIR)* } be TC₃.BLOCK.CALLABLE.

7. Check that `callableTypeDefIR` has type `externMethodTypeDefIR`, for all `callableTypeDefIR` in `callableTypeDefIR*`.
8. Let `externMethodTypeDefIR*` be the list of `externMethodTypeDefIR`, obtained by repeating:
 - Let `externMethodTypeDefIR` be `callableTypeDefIR`.
 for each `callableTypeDefIR` in `callableTypeDefIR*`
9. Let `typeDefIRextern` be `EXTERN nameIR < typeIdexpl*, · > { (callableId : externMethodTypeDefIR)* }`.
10. Let `TC4` be `TC0` where `nameIR` to `typeDefIRextern` is added to the GLOBAL layer.
11. Let `TC5` be `TC4` with `BLOCK.KIND` set to `EXTERN`.
12. Let `TC6` be type parameters `typeIdexpl*` added to the BLOCK layer of `TC5`.
13. Let the updated typing context `TC7` and the typed constructor prototypes `externConstructorPrototypeIR*` be
 - the result of typing `externConstructorPrototype*`, under typing context `TC6` for an extern object `nameIR`.
14. Let `TC8` be `TC4` with `GLOBAL.CONSTRUCTOR` set to `TC7.GLOBAL.CONSTRUCTOR`.
15. Let `externObjectDeclarationIR` be annotationList `EXTERN nameIR < typeIdexpl*, > { externConstructorPrototypeIR* externMethodPrototypeIR* }`.
16. Result in the updated typing context `TC8` and `externObjectDeclarationIR`.

11.11.2.2. Compile-time evaluation

At compile-time, extern function declarations are loaded into the context by:

Loading annotationList `EXTERN nameIR < typeParameterListIR, > { externConstructorPrototypeIR* externMethodPrototypeIR* }`, under instantiation context `IC0`, and store `STO`:

1. If `externConstructorPrototypeIR*` is equal to `:`:
 - a. Let `callableIdmethod*` be the list of `callableIdmethod`, obtained by repeating:
 - Let `callableIdmethod` be the callable id of `externMethodPrototypeIR`.
 for each `externMethodPrototypeIR` in `externMethodPrototypeIR*`
 - b. Let `externMethodTypeDefIR*` be the list of `externMethodTypeDefIR`, obtained by repeating:
 - Let `externMethodTypeDefIR` be the callable type definition of `externMethodPrototypeIR`.
 for each `externMethodPrototypeIR` in `externMethodPrototypeIR*`
 - c. Let `externMethodTypeDefEnv` be `{ (callableIdmethod : externMethodTypeDefIR)* }`.
 - d. Let `externObjectTypeDefIR` be `EXTERN nameIR < typeParameterListIR, · > externMethodTypeDefEnv`.
 - e. Let `IC1` be `IC0` where `nameIR` to `externObjectTypeDefIR` is added to the GLOBAL layer.
 - f. Let `callableIdconstructor` be the callable identifier for `nameIR (·)`.
 - g. Let `constructorDef` be `EXTERN nameIR < typeParameterListIR > (·) { externMethodPrototypeIR* }`.
 - h. Let `IC2` be where `callableIdconstructor` to `constructorDef` is added to `IC1`.

i. Result in the updated instantiation context IC_2 and the store STO .

2. Else:

a. Let $callableId_{method}^*$ be the list of $callableId_{method}$, obtained by repeating:

- Let $callableId_{method}$ be the callable id of `externMethodPrototypeIR`.

for each $externMethodPrototypeIR$ in $externMethodPrototypeIR^*$

b. Let $externMethodTypeDefIR^*$ be the list of $externMethodTypeDefIR$, obtained by repeating:

- Let $externMethodTypeDefIR$ be the callable type definition of `externMethodPrototypeIR`.

for each $externMethodPrototypeIR$ in $externMethodPrototypeIR^*$

c. Let $externMethodTypeDefEnv$ be $\{ (callableId_{method} : externMethodTypeDefIR)^* \}$.

d. Let $externObjectTypeDefIR$ be `EXTERN nameIR < typeParameterListIR , · > externMethodTypeDefEnv`.

e. Let IC_1 be IC_0 where `nameIR` to $externObjectTypeDefIR$ is added to the GLOBAL layer.

f. Let $callableId_{constructor}^*$ be the list of $callableId_{constructor}$, obtained by repeating:

- Let $callableId_{constructor}$ be the constructor id of `externConstructorPrototypeIR`.

for each $externConstructorPrototypeIR$ in $externConstructorPrototypeIR^*$

g. Let $constructorDef^*$ be the list of $constructorDef$, obtained by repeating:

- Let $constructorDef$ be the constructor definition of `externConstructorPrototypeIR` with type parameters `typeParameterListIR` and methods `externMethodPrototypeIR`.

for each $externConstructorPrototypeIR$ in $externConstructorPrototypeIR^*$

h. Let IC_2 be where each of $callableId_{constructor}^*$ to each of $constructorDef^*$ are added to IC_1 .

i. Result in the updated instantiation context IC_2 and the store STO .

11.12. Parser declarations

A `parser` declaration introduces a programmable parser:

```
parserLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | valueSetDeclaration
  ;

parserState
  : annotationList STATE name `{ parserStatementList transitionStatement }
  ;

parserDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
  ;
```

After type checking, a parser declaration is represented in $P4_{IR}$ as:

```

parserLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
  | variableDeclarationIR
  | valueSetDeclarationIR
  ;

parserStateIR
  : annotationList STATE nameIR `{ parserStatementListIR transitionStatementIR }
  ;

parserDeclarationIR
  : annotationList PARSER nameIR `{ typeParameterListIR > `( parameterListIR ) `(
  constructorParameterListIR ) `{ parserLocalDeclarationListIR parserStateListIR }
  ;

```

When instantiated, an instance of a parser declaration is created:

```

parserObject
  : PARSER `( parameterListIR ) `{ frame parserLocalDeclarationListIR stateEnv }
  ;

```

See [Section 10.4](#) for details about parser objects.

11.12.1. Type checking

Typing `annotationList PARSER name `EMPTY (parameterList) constructorParameterListOpt { parserLocalDeclarationList parserStateList }` **under typing context** TC_0 :

1. Let B be **bound type variables from the GLOBAL layer of** TC_0 .
2. Let TC_1 be TC_0 with `BLOCK.KIND` set to `PARSER`.
3. Let the constructor parameter list `constructorParameterIR*` and a set of fresh type variables `typed*` be
 - the result of **typing** `constructorParameterListOpt` **under typing context** TC_1 .
4. Check that `typed*` is an empty list.
5. Let TC_2 be `constructorParameterIR*` **added to the BLOCK layer of** TC_1 .
6. Let the parameter list `parameterIR*` and a set of fresh type variables `typed*` be
 - the result of **typing** `parameterList`, **under cursor** `BLOCK` **and typing context** TC_2 .
7. Check that `typed*` is an empty list.
8. Let TC_3 be `parameterIR*` **added to the BLOCK layer of** TC_2 .
9. Let the updated typing context TC_4 and the typed parser local declarations `parserLocalDeclarationIR*` be
 - the result of **typing** `parserLocalDeclarationList`, **under typing context** TC_3 .
10. Let TC_5 be TC_4 with `LOCAL.KIND` set to `PARSER_STATE`.
11. Let the typed parser states `parserStateIR*` be
 - the result of **typing** `parserStateList`, **under typing context** TC_5 .
12. Let `callableTypeDefIR` be `PARSER_APPLY ( parameterIR* )`.
13. Check that `callableTypeDefIR` is a well-formed callable type definition, with bound type variables

B.

14. Let `nameIR` be the name of `name`.
15. Let `constructorId` be the constructor identifier for `name ( constructorParameterListOpt )`.
16. Let `typeIRparser` be `PARSER nameIR < · > ( parameterIR* )`.
17. Let `constructorTypeDefIR` be `CONSTRUCTOR < · , · > ( constructorParameterIR* ) : typeIRparser`.
18. Check that `constructorTypeDefIR` is a well-formed constructor type definition, with bound type variables `B`.
19. Let `TC6` be where `constructorId` to `constructorTypeDefIR` is added to `TC0`.
20. Let `parserDeclarationIR` be `annotationList PARSER nameIR < · > ( parameterIR* ) ( constructorParameterIR* ) { parserLocalDeclarationIR* parserStateIR* }`.
21. Result in the updated typing context `TC6` and `parserDeclarationIR`.

11.12.2. Compile-time evaluation

At compile-time, parser declarations are loaded into the context as parser object constructors.

Loading `annotationList PARSER nameIR < · > ( parameterListIR ) ( constructorParameterListIR ) { parserLocalDeclarationListIR parserStateListIR }`, under instantiation context `IC0`, and store `STO`:

1. Let `callableId` be the callable identifier for `nameIR ( constructorParameterListIR )`.
2. Let `constructorDef` be `PARSER ( parameterListIR ) ( constructorParameterListIR ) { parserLocalDeclarationListIR parserStateListIR }`.
3. Let `IC1` be where `callableId` to `constructorDef` is added to `IC0`.
4. Result in the updated instantiation context `IC1` and the store `STO`.

11.13. Control declarations

A `control` declaration introduces a control block:

```
controlLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | actionDeclaration
  | tableDeclaration
  ;

controlBody = blockStatement

controlDeclaration
  : annotationList CONTROL name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody }
  ;
```

After type checking, a control declaration is represented in `P4IR` as:

```
controlLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
```

```

| variableDeclarationIR
| actionDeclarationIR
| tableDeclarationIR
;

controlBodyIR = blockStatementIR

controlDeclarationIR
: annotationList CONTROL nameIR `< typeParameterListIR > `( parameterListIR ) `(
constructorParameterListIR ) `{ controlLocalDeclarationListIR APPLY controlBodyIR }
;

```

When instantiated, an instance of a control declaration is created:

```

controlObject
: CONTROL `( parameterListIR ) `{ frame controlLocalDeclarationListIR actionDefEnv
controlBodyIR }
;

```

See [Section 10.5](#) for details about control objects.

11.13.1. Type checking

Typing annotationList CONTROL name `EMPTY (parameterList) constructorParameterListOpt { controlLocalDeclarationList APPLY controlBody } **under typing context** TC_0 :

1. Let B be **bound type variables from the GLOBAL layer of** TC_0 .
2. Let TC_1 be TC_0 with **BLOCK.KIND** set to **CONTROL**.
3. Let the constructor parameter list $constructorParameterIR^*$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** $constructorParameterListOpt$ **under typing context** TC_1 .
4. Check that $typed^*$ is an empty list.
5. Let TC_2 be $constructorParameterIR^*$ **added to the BLOCK layer of** TC_1 .
6. Let the parameter list $parameterIR^*$ and a set of fresh type variables $typed'^*$ be
 - the result of **typing** $parameterList$, **under cursor** **BLOCK** **and typing context** TC_2 .
7. Check that $typed'^*$ is an empty list.
8. Let TC_3 be $parameterIR^*$ **added to the BLOCK layer of** TC_2 .
9. Let the updated typing context TC_4 and the typed control local declarations $controlLocalDeclarationIR^*$ be
 - the result of **typing** $controlLocalDeclarationList$, **under typing context** TC_3 .
10. Let TC_5 be TC_4 with **LOCAL.KIND** set to **CONTROL_APPLY_METHOD**.
11. Let the typing context $_$, abstract control flow $_$, and $controlBodyIR$ be
 - the result of **typing** $controlBody$, **under typing context** TC_5 , **abstract control flow** **CONT**, **and loop context** **NOLOOP**.
12. Let $callableTypeDefIR$ be **CONTROL_APPLY** ($parameterIR^*$).
13. Check that $callableTypeDefIR$ is a well-formed callable type definition, with bound type variables B .
14. Let $nameIR$ be **the name of** $name$.

15. Let `constructorId` be the constructor identifier for `name ( constructorParameterListOpt )`.
16. Let `typeIRcontrol` be `CONTROL nameIR < · > ( parameterIR* )`.
17. Let `constructorTypeDefIR` be `CONSTRUCTOR < · , · > ( constructorParameterIR* ) : typeIRcontrol`.
18. Check that `constructorTypeDefIR` is a well-formed constructor type definition, with bound type variables `B`.
19. Let `TC6` be where `constructorId` to `constructorTypeDefIR` is added to `TC0`.
20. Let `controlDeclarationIR` be `annotationList CONTROL nameIR < · > ( parameterIR* ) ( constructorParameterIR* ) { controlLocalDeclarationIR* APPLY controlBodyIR }`.
21. Result in the updated typing context `TC6` and `controlDeclarationIR`.

11.13.2. Compile-time evaluation

At compile-time, control declarations are loaded into the context as control object constructors.

Loading `annotationList CONTROL nameIR < · > ( parameterListIR ) ( constructorParameterListIR ) { controlLocalDeclarationListIR APPLY controlBodyIR }`, under instantiation context `IC0`, and store `STO`:

1. Let `callableId` be the callable identifier for `nameIR ( constructorParameterListIR )`.
2. Let `constructorDef` be `CONTROL ( parameterListIR ) ( constructorParameterListIR ) { controlLocalDeclarationListIR APPLY controlBodyIR }`.
3. Let `IC1` be where `callableId` to `constructorDef` is added to `IC0`.
4. Result in the updated instantiation context `IC1` and the store `STO`.

11.14. Enum type declaration

An enumeration type is defined using the following syntax:

```
enumTypeDeclaration
: annotationList ENUM name `{ nameList trailingCommaOpt }
| annotationList ENUM type name `{ namedExpressionList trailingCommaOpt }
;

namedExpression
: name = expression
;

namedExpressionList
: namedExpression
| namedExpressionList , namedExpression
;
```

After type checking, enum type declarations are represented as follows:

```
enumTypeDeclarationIR
: annotationList ENUM nameIR `{ nameListIR }
| annotationList ENUM typeIR nameIR `{ namedValueListIR }
;

namedValueIR
: nameIR = value
```

```
;
namedValueListIR = namedValueIR*
```

See [Section 8.3.8](#) for more details on enums.

11.14.1. Enum type declarations without an underlying type

11.14.1.1. Type checking

Typing annotationList ENUM name { nameList_{field} trailingCommaOpt } under typing context TC₀:

1. Let nameIR be the name of name.
2. Let name_{field}^{*} be the list of names in nameList_{field}.
3. Let nameIR_{field}^{*} be the list of nameIR_{field}, obtained by repeating:
 - Let nameIR_{field} be the name of name_{field}.for each name_{field} in name_{field}^{*}
4. Let enumTypeDefIR be ENUM nameIR { nameIR_{field}^{*} }.
5. Let bound be bound type variables from the GLOBAL layer of TC₀.
6. Check that enumTypeDefIR is a well-formed type definition, with bound type variables bound.
7. Let TC₁ be TC₀ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
8. Let id_{field}^{*} be the list of id_{field}, obtained by repeating:
 - Let id_{field} be nameIR concatenated with "." concatenated with nameIR_{field}.for each nameIR_{field} in nameIR_{field}^{*}
9. Let value_{field}^{*} be the list of value_{field}, obtained by repeating:
 - Let value_{field} be nameIR . nameIR_{field}.for each nameIR_{field} in nameIR_{field}^{*}
10. Let varTypeIR^{*} be the list of varTypeIR, obtained by repeating:
 - Let varTypeIR be `EMPTY enumTypeDefIR LCTK value_{field}.for each value_{field} in value_{field}^{*}
11. Let TC₂ be TC₁ where each of id_{field}^{*} to each of varTypeIR^{*} are added to the GLOBAL layer.
12. Let enumTypeDeclarationIR be annotationList ENUM nameIR { nameIR_{field}^{*} }.
13. Result in the updated typing context TC₂ and enumTypeDeclarationIR.

11.14.1.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading annotationList ENUM nameIR { nameIR_{field}^{*} }, under instantiation context IC₀, and store STO:

1. Let nameIR_{enum_field}^{*} be the list of nameIR_{enum_field}, obtained by repeating:
 - Let nameIR_{enum_field} be nameIR concatenated with "." concatenated with nameIR_{field}.
 for each nameIR_{field} in nameIR_{field}^{*}
2. Let value_{enum_field}^{*} be the list of value_{enum_field}, obtained by repeating:
 - Let value_{enum_field} be nameIR . nameIR_{field}.
 for each nameIR_{field} in nameIR_{field}^{*}
3. Let IC₁ be IC₀ where each of nameIR_{enum_field}^{*} to each of value_{enum_field}^{*} are added to the GLOBAL layer.
4. Let enumTypeDefIR be ENUM nameIR { nameIR_{field}^{*} }.
5. Let IC₂ be IC₁ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
6. Result in the updated instantiation context IC₂ and the store STO.

11.14.2. Enum type declarations with an underlying type

11.14.2.1. Type checking

Typing annotationList ENUM type name { namedExpressionList_{field} trailingCommaOpt } under typing context TC₀:

1. Let B be bound type variables from the GLOBAL layer of TC₀.
2. Let typeIR and a set of fresh type variables typeld^{*} be
 - the result of typing type, under cursor GLOBAL and typing context TC₀.
3. Check that typeld^{*} is an empty list.
4. Check that typeIR is a well-formed type, with bound type variables B.
5. Let nameIR be the name of name.
6. Let the updated typing context TC₁ and field values namedValueIR_{field}^{*} be
 - the result of typing namedExpressionList_{field} under typing context TC₀ with enum name nameIR and underlying typeIR.
7. Let nameIR_{field}^{*} be the list of nameIR_{field} and value_{field}^{*} be the list of value_{field}, obtained by repeating:
 - Let nameIR_{field} = value_{field} be namedValueIR_{field}.
 for each namedValueIR_{field} in namedValueIR_{field}^{*}
8. Let id_{field}^{*} be the list of id_{field}, obtained by repeating:
 - Let id_{field} be nameIR concatenated with "." concatenated with nameIR_{field}.
 for each nameIR_{field} in nameIR_{field}^{*}
9. Let enumTypeDefIR be ENUM nameIR < typeIR > { (nameIR_{field} = value_{field} ;) }^{*}.
10. Let varTypeIR^{*} be the list of varTypeIR, obtained by repeating:

- Let varTypeIR be `EMPTY enumTypeDefIR LCTK valuefield`.

for each $\text{value}_{\text{field}}$ in $\text{value}_{\text{field}}^*$

11. Let TC_2 be TC_0 where each of $\text{id}_{\text{field}}^*$ to each of varTypeIR^* are added to the GLOBAL layer.
12. Check that enumTypeDefIR is a well-formed type definition, with bound type variables B.
13. Let TC_3 be TC_2 where nameIR to enumTypeDefIR is added to the GLOBAL layer.
14. Let $\text{enumTypeDeclarationIR}$ be `annotationList ENUM typeIR nameIR { namedValueIRfield }*`.
15. Result in the updated typing context TC_3 and $\text{enumTypeDeclarationIR}$.

An enum field is type-checked as follows:

Enum_serializable_field_ok:

- Typing namedExpression under typing context typingContext with enum name nameIR and underlying typeIR :
- Results in the updated typing context typingContext , and field value namedValueIR .

Typing $\text{name} = \text{expression}$ under typing context TC_0 with enum name $\text{nameIR}_{\text{enum}}$ and underlying typeIR :

1. Let typedExpressionIR be
 - the result of typing expression , under cursor BLOCK and typing context TC_0 .
2. Let $\text{typedExpressionIR}_{\text{cast}}$ be `! typedExpressionIR implicitly cast to typeIR`.
3. Check that the compile-time known-ness of $\text{typedExpressionIR}_{\text{cast}}$ is LCTK.
4. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{cast}}$ under cursor BLOCK and typing context TC_0 .
5. Let nameIR be the name of name .
6. Let varTypeIR be `EMPTY typeIR LCTK value`.
7. Let TC_1 be TC_0 where nameIR to varTypeIR is added to the BLOCK layer.
8. Result in the updated typing context TC_1 , and field value $\text{nameIR} = \text{value}$.

A list of enum fields is type-checked with the following relation:

Enum_serializable_fieldList_ok:

- Typing $\text{namedExpressionList}$ under typing context typingContext with enum name nameIR and underlying typeIR :
- Results in the updated typing context typingContext and field values namedValueIR^* .

11.14.2.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading annotationList ENUM typeIR nameIR { (nameIR_{field} = value_{field})^{*} }, under instantiation context IC₀, and store STO:

1. Let nameIR_{serenum_field}^{*} be the list of nameIR_{serenum_field}, obtained by repeating:
 - Let nameIR_{serenum_field} be nameIR concatenated with "." concatenated with nameIR_{field}.
 for each nameIR_{field} in nameIR_{field}^{*}
2. Let value_{serenum_field}^{*} be the list of value_{serenum_field}, obtained by repeating:
 - Let value_{serenum_field} be nameIR . nameIR_{field} . value_{field}.
 for each nameIR_{field} in nameIR_{field}^{*} and value_{field} in value_{field}^{*}
3. Let IC₁ be IC₀ where each of nameIR_{serenum_field}^{*} to each of value_{serenum_field}^{*} are added to the GLOBAL layer.
4. Let enumTypeDefIR be ENUM nameIR < typeIR > { (nameIR_{field} = value_{field} ;)^{*} }.
5. Let IC₂ be IC₁ where nameIR to enumTypeDefIR is added to the GLOBAL layer.
6. Result in the updated instantiation context IC₂ and the store STO.

11.15. Struct type declaration

A struct type is defined with the following syntax:

```
structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;

typeField
: annotationList type name ;
;

typeFieldList
: /* empty */
| typeFieldList typeField
;
;
```

After type checking, a struct type declaration is represented in P4_{IR} as:

```
structTypeDeclarationIR
: annotationList STRUCT nameIR `< typeParameterListIR > `{ typeFieldListIR }
;

typeFieldIR
: annotationList typeIR nameIR ;
;

typeFieldListIR = typeFieldIR*
```

See [Section 8.3.5](#) for more information about struct types.

11.15.1. Type checking

Typing annotationList STRUCT name typeParameterListOpt { typeFieldList } under typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed^*$ be
 - the result of **typing** typeParameterListOpt, under cursor BLOCK and typing context TC_0 .
2. Let $(annotationList_{field} type_{field} name_{field} ;)^*$ be the list of type fields in typeFieldList.
3. Let $typeIR_{field}^*$ be the list of $typeIR_{field}$ and $typed^{**}$ be the list of $typed^*$, obtained by repeating:
 - Let $typeIR_{field}$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** $type_{field}$, under cursor BLOCK and typing context TC_1 .for each $type_{field}$ in $type_{field}^*$
4. Check that $typed^*$ is an empty list, for all $typed^*$ in $typed^{**}$.
5. Let $nameIR$ be the name of name.
6. Let $nameIR_{field}^*$ be the list of $nameIR_{field}$, obtained by repeating:
 - Let $nameIR_{field}$ be the name of $name_{field}$.for each $name_{field}$ in $name_{field}^*$
7. Let structTypeDefIR be STRUCT nameIR < $typed^*$ > { $(typeIR_{field} nameIR_{field} ;)^*$ }.
8. Let bound be bound type variables from the GLOBAL layer of TC_0 .
9. Check that structTypeDefIR is a well-formed type definition, with bound type variables bound.
10. Let TC_2 be TC_0 where nameIR to structTypeDefIR is added to the GLOBAL layer.
11. Let structTypeDeclarationIR be annotationList STRUCT nameIR < $typed^*$ > { $(annotationList_{field} typeIR_{field} nameIR_{field} ;)^*$ }.
12. Result in the updated typing context TC_2 and structTypeDeclarationIR.

11.15.2. Compile-time evaluation

At compile-time, struct type declarations are loaded into the context by:

Loading annotationList STRUCT nameIR < typeParameterListIR > { typeFieldIR* }, under instantiation context IC_0 , and store STO:

1. Let $annotationList_{field}^*$ be the list of $annotationList_{field}$, $nameIR_{field}^*$ be the list of $nameIR_{field}$, and $typeIR_{field}^*$ be the list of $typeIR_{field}$, obtained by repeating:
 - Let $annotationList_{field} typeIR_{field} nameIR_{field} ;$ be typeFieldIR.for each typeFieldIR in typeFieldIR*
2. Let structTypeDefIR be STRUCT nameIR < typeParameterListIR > { $(typeIR_{field} nameIR_{field} ;)^*$ }.
3. Let IC_1 be IC_0 where nameIR to structTypeDefIR is added to the GLOBAL layer.
4. Result in the updated instantiation context IC_1 and the store STO.

11.16. Header type declaration

A header type is defined with the following syntax:

```
headerTypeDeclaration
  : annotationList HEADER name typeParameterListOpt `{ typeFieldList }
  ;

typeField
  : annotationList type name ;
  ;

typeFieldList
  : /* empty */
  | typeFieldList typeField
  ;
```

After type checking, a header type declaration is represented in $P4_{IR}$ as:

```
headerTypeDeclarationIR
  : annotationList HEADER nameIR `< typeParameterListIR > `{ typeFieldListIR }
  ;

typeFieldIR
  : annotationList typeIR nameIR ;
  ;

typeFieldListIR = typeFieldIR*
```

See [Section 8.3.6](#) for more information about header types.

11.16.1. Type checking

Typing `annotationList HEADER name typeParameterListOpt { typeFieldList }` **under typing context** TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed^*$ be
 - the result of **typing** `typeParameterListOpt`, **under cursor** `BLOCK` **and typing context** TC_0 .
2. Let $(annotationList_i type_i name_i;)^*$ be the list of type fields in `typeFieldList`.
3. Let $typedR_i^*$ be the list of $typedR_i$ and $typed^{**}$ be the list of $typed^*$, obtained by repeating:

- Let $typedR_i$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** $type_i$, **under cursor** `BLOCK` **and typing context** TC_1 .

for each $type_i$ in $type_i^*$

4. Check that $typed^*$ is an empty list, for all $typed^*$ in $typed^{**}$.
5. Let $nameIR$ be the name of `name`.
6. Let $nameIR_i^*$ be the list of $nameIR_i$, obtained by repeating:

- Let $nameIR_i$ be the name of $name_i$.

for each $name_i$ in $name_i^*$

7. Let `headerTypeDefIR` be `HEADER nameIR < typeId* > { (typeIRf nameIRf ;)* }`.
8. Let `bound` be `bound type variables from the GLOBAL layer of TC0`.
9. Check that `headerTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
10. Let `TC2` be `TC0 where nameIR to headerTypeDefIR is added to the GLOBAL layer`.
11. Let `headerTypeDeclarationIR` be `annotationList HEADER nameIR < typeId* > { (annotationListf typeIRf nameIRf ;)* }`.
12. Result in the updated typing context `TC2` and `headerTypeDeclarationIR`.

11.16.2. Compile-time evaluation

At compile-time, header type declarations are loaded into the context by:

Loading `annotationList HEADER nameIR < typeParameterListIR > { typeFieldIR* }`, under instantiation context `IC0`, and store `STO`:

1. Let `annotationListfield*` be the list of `annotationListfield`, `nameIRfield*` be the list of `nameIRfield`, and `typeIRfield*` be the list of `typeIRfield`, obtained by repeating:
 - Let `annotationListfield typeIRfield nameIRfield ;` be `typeFieldIR`.
 for each `typeFieldIR` in `typeFieldIR*`
2. Let `headerTypeDefIR` be `HEADER nameIR < typeParameterListIR > { (typeIRfield nameIRfield ;)* }`.
3. Let `IC1` be `IC0 where nameIR to headerTypeDefIR is added to the GLOBAL layer`.
4. Result in the updated instantiation context `IC1` and the store `STO`.

11.17. Header union type declaration

A header union type is defined with the following syntax:

```
headerUnionTypeDeclaration
: annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList }
;

typeField
: annotationList type name ;
;

typeFieldList
: /* empty */
| typeFieldList typeField
;
;
```

After type checking, a header union type declaration is represented in `P4IR` as:

```
headerUnionTypeDeclarationIR
: annotationList HEADER_UNION nameIR `< typeParameterListIR > `{ typeFieldListIR }
;

typeFieldIR
: annotationList typeIR nameIR ;
;
;
```

```
typeFieldListIR = typeFieldIR*
```

See [Section 8.3.7](#) for more information about header union types.

11.17.1. Type checking

Typing `annotationList HEADER_UNION name typeParameterListOpt { typeFieldList }` under typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typeId^*$ be
 - the result of **typing** `typeParameterListOpt`, under cursor `BLOCK` and typing context TC_0 .
2. Let $(annotationList_f type_f name_f ;)^*$ be the list of type fields in `typeFieldList`.
3. Let $typeIR_f^*$ be the list of $typeIR_f$ and $typeId^{**}$ be the list of $typeId^*$, obtained by repeating:
 - Let $typeIR_f$ and a set of fresh type variables $typeId^*$ be
 - the result of **typing** `type_f`, under cursor `BLOCK` and typing context TC_1 .
 for each $type_f$ in $type_f^*$
4. Check that $typeId^*$ is an empty list, for all $typeId^*$ in $typeId^{**}$.
5. Let $nameIR$ be the name of `name`.
6. Let $nameIR_f^*$ be the list of $nameIR_f$, obtained by repeating:
 - Let $nameIR_f$ be the name of `name_f`.
 for each $name_f$ in $name_f^*$
7. Let `headerUnionTypeDefIR` be `HEADER_UNION nameIR < typeId* > { (typeIR_f nameIR_f ;)^* }`.
8. Let `bound` be bound type variables from the GLOBAL layer of TC_0 .
9. Check that `headerUnionTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
10. Let TC_2 be TC_0 where `nameIR` to `headerUnionTypeDefIR` is added to the GLOBAL layer.
11. Let `headerUnionTypeDeclarationIR` be `annotationList HEADER_UNION nameIR < typeId* > { (annotationList_f typeIR_f nameIR_f ;)^* }`.
12. Result in the updated typing context TC_2 and `headerUnionTypeDeclarationIR`.

11.17.2. Compile-time evaluation

At compile-time, header union type declarations are loaded into the context by:

Loading `annotationList HEADER_UNION nameIR < typeParameterListIR > { typeFieldIR* }`, under instantiation context IC_0 , and store `STO`:

1. Let $annotationList_{field}^*$ be the list of $annotationList_{field}$, $nameIR_{field}^*$ be the list of $nameIR_{field}$, and $typeIR_{field}^*$ be the list of $typeIR_{field}$, obtained by repeating:
 - Let `annotationList_{field} typeIR_{field} nameIR_{field} ;` be `typeFieldIR`.

for each typeFieldIR in typeFieldIR^*

2. Let $\text{headerUnionTypeDefIR}$ be $\text{HEADER_UNION nameIR} < \text{typeParameterListIR} > \{ (\text{typeIR}_{\text{field}} \text{ nameIR}_{\text{field}} ;) \}^*$.
3. Let IC_1 be IC_0 where nameIR to $\text{headerUnionTypeDefIR}$ is added to the GLOBAL layer.
4. Result in the updated instantiation context IC_1 and the store STO .

11.18. Typedef declaration

A **typedef** or **type** declaration introduces an alias for an existing type.

```
typedef
: type
| derivedTypeDeclaration
;

typedefDeclaration
: annotationList TYPEDEF typedef name ;
| annotationList TYPE type name ;
;
```

In P4_{IR} , **typedef** and **type** declarations are represented as:

```
typedefIR
: typeIR
| derivedTypeDeclarationIR
;

typedefDeclarationIR
: annotationList TYPEDEF typedefIR nameIR ;
| annotationList TYPE typeIR nameIR ;
;
```

11.18.1. typedef declaration

See [Section 8.5.1](#) for more details about **typedef** aliases.

11.18.1.1. Type checking

Typing $\text{annotationList TYPEDEF typedef name ;}$ under typing context TC_0 :

1. If let **type** be **typedef**:
 - a. Let B be bound type variables from the GLOBAL layer of TC_0 .
 - b. Let typeIR and a set of fresh type variables typeId^* be
 - the result of typing **type**, under cursor GLOBAL and typing context TC_0 .
 - c. Check that typeId^* is an empty list.
 - d. Check that typeIR is a well-formed type, with bound type variables B .
 - e. Let nameIR be the name of **name**.
 - f. Let $\text{typeDefIR}_{\text{typedef}}$ be $\text{TYPEDEF nameIR typeIR}$.
 - g. Check that $\text{typeDefIR}_{\text{typedef}}$ is a well-formed type definition, with bound type variables B .

- h. Let TC_1 be TC_0 where $nameIR$ to $typeDefIR_{typedef}$ is added to the GLOBAL layer.
 - i. Let $typedefDeclarationIR$ be $annotationList\ TYPEDEF\ typeIR\ nameIR ;$.
 - j. Result in the updated typing context TC_1 and $typedefDeclarationIR$.
2. Else:
- a. Let $derivedTypeDeclaration$ be $typedef$.
 - b. Let the updated typing context TC_1 and $declarationIR$ be
 - the result of **typing** $derivedTypeDeclaration$ **under typing context** TC_0 .
 - c. Check that $declarationIR$ has type $derivedTypeDeclarationIR$.
 - d. Let $derivedTypeDeclarationIR$ be $declarationIR$.
 - e. Let $set<typed>$ be the domain of the map $TC_1.GLOBAL.TYPE$.
 - f. Let $set<typed>'$ be the domain of the map $TC_0.GLOBAL.TYPE$.
 - g. Let $\{ typed^* \}$ be the difference of the sets $set<typed>$ and $set<typed>'$.
 - h. Check that $typed^*$ is a list of length 1.
 - i. Let $[typed']$ be $typed^*$.
 - j. Let $typeDefIR'$ be ! the type definition of $typed'$ in the GLOBAL layer of TC_1 .
 - k. If $typeDefIR'$ is monomorphic:
 - i. Let $typeIR$ be the underlying type of $typeDefIR'$.
 - ii. Let $nameIR$ be the name of $name$.
 - iii. Let $typedefTypeIR$ be $TYPEDEF\ nameIR\ typeIR$.
 - iv. Let $bound$ be bound type variables from the GLOBAL layer of TC_0 .
 - v. Check that $typedefTypeIR$ is a well-formed type definition, with bound type variables $bound$.
 - vi. Let $typedefDeclarationIR$ be $annotationList\ TYPEDEF\ derivedTypeDeclarationIR\ nameIR ;$.
 - vii. Result in the updated typing context TC_1 and $typedefDeclarationIR$.
 - l. Else:
 - i. Check that $(, \cdot)$ is equal to the type parameters of $typeDefIR'$.
 - ii. Let $nameIR$ be the name of $name$.
 - iii. Let $typeIR$ be $typeDefIR'$ specialized by $\cdot$.
 - iv. Let $typedefTypeIR$ be $TYPEDEF\ nameIR\ typeIR$.
 - v. Let $bound$ be bound type variables from the GLOBAL layer of TC_0 .
 - vi. Check that $typedefTypeIR$ is a well-formed type definition, with bound type variables $bound$.
 - vii. Let $typedefDeclarationIR$ be $annotationList\ TYPEDEF\ derivedTypeDeclarationIR\ nameIR ;$.
 - viii. Result in the updated typing context TC_1 and $typedefDeclarationIR$.

11.18.1.2. Compile-time evaluation

At compile-time, **typedefs** are loaded into the context with:

Loading `annotationList TYPEDEF typedefIR nameIR ;`, under instantiation context IC_0 , and store STO :

1. If let `typeIR` be `typedefIR`:
 - a. Let `aliasTypeDefIR` be `TYPEDEF nameIR typeIR`.
 - b. Let IC_1 be IC_0 where `nameIR` to `aliasTypeDefIR` is added to the GLOBAL layer.
 - c. Result in the updated instantiation context IC_1 and the store STO .
2. Else:
 - a. Let `derivedTypeDeclarationIR` be `typedefIR`.
 - b. Let the updated instantiation context IC_{local} and the store $_$ be
 - the result of **loading** `derivedTypeDeclarationIR`, under instantiation context IC_0 , and store an empty store.
 - c. Let `set<typeId>` be the domain of the map $IC_{local}.GLOBAL.TYPE$.
 - d. Let `set<typeId>'` be the domain of the map $IC_0.GLOBAL.TYPE$.
 - e. Let `{ typeId* }` be the difference of the sets `set<typeId>` and `set<typeId>'`.
 - f. Check that `typeId*` is a list of length 1.
 - g. Let `[typeIdnewtype]` be `typeId*`.
 - h. Let `typeDefIR'` be `! find_typeDef_i(GLOBAL, IClocal, typeIdnewtype)`.
 - i. If `typeDefIR'` is monomorphic:
 - i. Let `typeIR` be the underlying type of `typeDefIR'`.
 - ii. Let `aliasTypeDefIR` be `TYPEDEF nameIR typeIR`.
 - iii. Let IC_1 be IC_0 where `nameIR` to `aliasTypeDefIR` is added to the GLOBAL layer.
 - iv. Result in the updated instantiation context IC_1 and the store STO .
 - j. Else:
 - i. Check that `(, .)` is equal to the type parameters of `typeDefIR'`.
 - ii. Let `typeIR` be `typeDefIR'` specialized by `.`
 - iii. Let `typedefTypeIR` be `TYPEDEF nameIR typeIR`.
 - iv. Let IC_1 be IC_0 where `nameIR` to `typedefTypeIR` is added to the GLOBAL layer.
 - v. Result in the updated instantiation context IC_1 and the store STO .

11.18.2. New type declaration

See [Section 8.5.2](#) for more details about new type aliases.

11.18.2.1. Type checking

Typing `annotationList TYPE type name ;` under typing context TC_0 :

1. Let B be bound type variables from the GLOBAL layer of TC_0 .
2. Let `typeIR` and a set of fresh type variables `typeId*` be
 - the result of **typing** `type`, under cursor GLOBAL and typing context TC_0 .

3. Check that `typel*` is an empty list.
4. Check that `typeIR` is a well-formed type, with bound type variables `B`.
5. Let `nameIR` be the name of `name`.
6. Let `newTypeIR` be `TYPE nameIR typeIR`.
7. Check that `newTypeIR` is a well-formed type definition, with bound type variables `B`.
8. Let `TC1` be `TC0` where `nameIR` to `newTypeIR` is added to the GLOBAL layer.
9. Let `typedefDeclarationIR` be `annotationList TYPE typeIR nameIR ;`.
10. Result in the updated typing context `TC1` and `typedefDeclarationIR`.

11.18.2.2. Compile-time evaluation

At compile-time, `types` are loaded into the context with:

Loading `annotationList TYPE typeIR nameIR ;`, under instantiation context `IC0`, and store `STO`:

1. Let `aliasTypeDefIR` be `TYPE nameIR typeIR`.
2. Let `IC1` be `IC0` where `nameIR` to `aliasTypeDefIR` is added to the GLOBAL layer.
3. Result in the updated instantiation context `IC1` and the store `STO`.

11.19. Parser type declaration

A parser type declaration introduces a new parser type.

```
parserTypeDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList ) ;
  ;
```

After type checking, a parser type declaration has the following form:

```
parserTypeDeclarationIR
  : annotationList PARSE nameIR `< typeParameterListIR , typeParameterListIR > `(
  parameterListIR ) ;
  ;
```

See [Section 8.4.2](#) for more details on parser object types.

11.19.1. Type checking

Typing `annotationList PARSE name typeParameterListOpt ( parameterList ) ;` under typing context `TC0`:

1. Let `TC1` be `TC0` with `BLOCK.KIND` set to `PARSER`.
2. Let the updated typing context `TC2` and type parameters `typelexpl*` be
 - the result of typing `typeParameterListOpt`, under cursor `BLOCK` and typing context `TC1`.
3. Let the parameter list `parameterIR*` and a set of fresh type variables `typelimpl*` be

- the result of **typing** `parameterList`, **under cursor** `BLOCK` and **typing context** `TC2`.
4. Let `nameIR` be **the name of** `name`.
 5. Let `parserObjectTypeDefIR` be `PARSER nameIR < typeldexpl*, typeldimpl* > ( parameterIR* )`.
 6. Let `bound` be **bound type variables from the** `GLOBAL` **layer of** `TC0`.
 7. Check that `parserObjectTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
 8. Let `TC3` be `TC0` **where** `nameIR` **to** `parserObjectTypeDefIR` **is added to the** `GLOBAL` **layer**.
 9. Let `parserTypeDeclarationIR` be `annotationList PARSER nameIR < typeldexpl*, typeldimpl* > ( parameterIR* ) ;`.
 10. Result in the updated typing context `TC3` and `parserTypeDeclarationIR`.

11.19.2. Compile-time evaluation

At compile-time, a parser type declaration is loaded into the context with:

Loading `annotationList PARSER nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* ) ;`, **under instantiation context** `IC0`, **and store** `STO`:

1. Let `parserObjectTypeDefIR` be `PARSER nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* )`.
2. Let `IC1` be `IC0` **where** `nameIR` **to** `parserObjectTypeDefIR` **is added to the** `GLOBAL` **layer**.
3. Result in the updated instantiation context `IC1` and the store `STO`.

11.20. Control type declaration

A control type declaration introduces a new control type.

```
controlTypeDeclaration
: annotationList CONTROL name typeParameterListOpt `( parameterList ) ;
;
```

After type checking, a control type declaration has the following form:

```
controlTypeDeclarationIR
: annotationList CONTROL nameIR `< typeParameterListIR , typeParameterListIR > `(
parameterListIR ) ;
;
```

See [Section 8.4.3](#) for more details on control object types.

11.20.1. Type checking

Typing `annotationList CONTROL name typeParameterListOpt ( parameterList ) ;` **under typing context** `TC0`:

1. Let `TC1` be `TC0` with `BLOCK.KIND` set to `CONTROL`.
2. Let the updated typing context `TC2` and type parameters `typeldexpl*` be

- the result of **typing** `typeParameterListOpt`, **under cursor** `BLOCK` and **typing context** `TC1`.
- Let the parameter list `parameterIR*` and a set of fresh type variables `typedimpl*` be
 - the result of **typing** `parameterList`, **under cursor** `BLOCK` and **typing context** `TC2`.
 - Let `nameIR` be **the name of** `name`.
 - Let `controlObjectTypeDefIR` be `CONTROL nameIR < typedexpl*, typedimpl* > ( parameterIR* )`.
 - Let `bound` be **bound type variables from the** `GLOBAL` **layer of** `TC0`.
 - Check that `controlObjectTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
 - Let `TC3` be `TC0` **where** `nameIR` **to** `controlObjectTypeDefIR` **is added to the** `GLOBAL` **layer**.
 - Let `controlTypeDeclarationIR` be `annotationList CONTROL nameIR < typedexpl*, typedimpl* > ( parameterIR* ) ;`.
 - Result in the updated typing context `TC3` and `controlTypeDeclarationIR`.

11.20.2. Compile-time evaluation

At compile-time, a control type declaration is loaded into the context with:

Loading `annotationList CONTROL nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* ) ;`, **under instantiation context** `IC0`, **and store** `STO`:

- Let `controlObjectTypeDefIR` be `CONTROL nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* )`.
- Let `IC1` be `IC0` **where** `nameIR` **to** `controlObjectTypeDefIR` **is added to the** `GLOBAL` **layer**.
- Result in the updated instantiation context `IC1` and the store `STO`.

11.21. Package type declaration

A package type declaration introduces a new package type.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

After type checking, a package type declaration has the following form:

```
packageTypeDeclarationIR
: annotationList PACKAGE nameIR `< typeParameterListIR , typeParameterListIR > `(
parameterListIR ) ;
;
```

See [Section 8.4.4](#) for more details on package object types.

11.21.1. Type checking

Typing `annotationList PACKAGE name typeParameterListOpt ( parameterList ) ;` **under typing context** `TC0`:

1. Let B be **bound type variables from the GLOBAL layer of TC_0** .
2. Let TC_1 be TC_0 with **BLOCK.KIND** set to **PACKAGE**.
3. Let the updated typing context TC_2 and type parameters $typed_{expl}^*$ be
 - the result of **typing** $typeParameterListOpt$, **under cursor** **BLOCK** and **typing context** TC_1 .
4. Let the constructor parameter list $constructorParameterIR^*$ and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** ($parameterList$) **under typing context** TC_2 .
5. Let $nameIR$ be **the name of name**.
6. Let $typeIR_{package_inner}^*$ be the list of $typeIR_{package_inner}$, obtained by repeating:
 - Let $typeIR_{package_inner}$ be the type of $constructorParameterIR$.
 for each $constructorParameterIR$ in $constructorParameterIR^*$
7. Let $packageObjectTypeDefIR$ be **PACKAGE** $nameIR$ < $typed_{expl}^*$, $typed_{impl}^*$ > { $typeIR_{package_inner}^*$ }.
8. Check that $packageObjectTypeDefIR$ is a **well-formed type definition, with bound type variables B** .
9. Let TC_3 be TC_0 **where** $nameIR$ **to** $packageObjectTypeDefIR$ **is added to the GLOBAL layer**.
10. Let $constructorId$ be **the constructor identifier for** $name$ (($parameterList$)).
11. Let $typeIR_{package}$ be **the underlying type of** $packageObjectTypeDefIR$.
12. Let $constructorTypeDefIR$ be **CONSTRUCTOR** < $typed_{expl}^*$, $typed_{impl}^*$ > ($constructorParameterIR^*$) : $typeIR_{package}$.
13. Check that $constructorTypeDefIR$ is a **well-formed constructor type definition, with bound type variables B** .
14. Let TC_4 be **where** $constructorId$ **to** $constructorTypeDefIR$ **is added to** TC_3 .
15. Let $packageTypeDeclarationIR$ be **annotationList** **PACKAGE** $nameIR$ < $typed_{expl}^*$, $typed_{impl}^*$ > ($constructorParameterIR^*$);.
16. Result in the updated typing context TC_4 and $packageTypeDeclarationIR$.

11.21.2. Compile-time evaluation

At compile-time, a package type declaration is loaded into the context with:

Loading $annotationList$ **PACKAGE** $nameIR$ < $typeParameterListIR$, $typeParameterListIR_{inferred}$ > ($parameterIR^*$) ;, **under instantiation context** IC_0 , **and store** STO :

1. Let $typeIR^*$ be the list of $typeIR$, obtained by repeating:
 - Let $typeIR$ be the type of $parameterIR$.
 for each $parameterIR$ in $parameterIR^*$
2. Let $packageObjectTypeDefIR$ be **PACKAGE** $nameIR$ < $typeParameterListIR$, $typeParameterListIR_{inferred}$ > { $typeIR^*$ }.
3. Let IC_1 be IC_0 **where** $nameIR$ **to** $packageObjectTypeDefIR$ **is added to the GLOBAL layer**.
4. Let $callableId$ be **the callable identifier for** $nameIR$ ($parameterIR^*$).
5. Let $constructorDef$ be **PACKAGE** < $typeParameterListIR$ ++ $typeParameterListIR_{inferred}$ > ($parameterIR^*$).
6. Let IC_2 be **where** $callableId$ **to** $constructorDef$ **is added to** IC_1 .

7. Result in the updated instantiation context IC_2 and the store STO .

12. Statements

```
statement
: emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| returnStatement
| exitStatement
| blockStatement
| conditionalStatement
| forStatement
| breakStatement
| continueStatement
| switchStatement
;

statementIR
: emptyStatementIR
| assignmentStatementIR
| callStatementIR
| directApplicationStatementIR
| returnStatementIR
| exitStatementIR
| blockStatementIR
| conditionalStatementIR
| forStatementIR
| breakStatementIR
| continueStatementIR
| switchStatementIR
;
```

Type checking

```
flow
: CONT
| RET
;
```

Stmt_ok:

- Typing `statement`, under cursor `cursor`, typing context `typingContext`, abstract control flow `flow`, and loop context `loopctxt`:
- Results in
 - the typing context `typingContext`,
 - the abstract control flow `flow`,
 - and the typed statement `statementIR`.

Compile-time evaluation

Stmt_inst:

- Compile-time evaluation of `statementIR`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in
 - the instantiation context `instContext`,
 - the store `store`,
 - and `statementIR`.

Dynamic evaluation

`Stmt_eval`:

- The compile-time evaluation of `statementIR`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`,
 - the store `store`,
 - and the instantiated statement `statementResult`.

12.1. Empty statements

```
emptyStatement
: ;
;

emptyStatementIR = emptyStatement
```

The empty statement, written `;` is a no-op.

Type checking

Typing `;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `;`.

Compile-time evaluation

Compile-time evaluation of `emptyStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in
 - the instantiation context `IC`,

- the store `STO`,
- and `emptyStatementIR`.

12.2. Assignment statements

```

assignop
: =
| +=
| -=
| |+|=
| |-|=
| *=
| /=
| %=
| <<=
| >>=
| &=
| ^=
| |=
;

assignmentStatement
: lvalue assignop expression ;
;

assignmentStatementIR
: typedLvalueIR assignop typedExpressionIR ;
;

```

An assignment, written with the `=` sign, first evaluates its left sub-expression to an l-value, then evaluates its right sub-expression to a value, and finally copies the value into the l-value. Derived types (e.g. `structs`) are copied recursively, and all components of `headers` are copied, including "validity" bits. Assignment is not defined for `extern` values.

An assignment may also be written with a binary arithmetic or bit manipulation operator immediately before the `=` sign. This performs the binary operator on the old value of the left sub-expression and the right sub-expression and assigns the result to the l-value. Thus an assignment like `A += B` is equivalent to `A = A + B`, except that `A` is only evaluated once. This means that any side-effects within this operand (eg, a function call inside an array index or slice expression) only occur once. This is not valid for comparison operators, logical operators, concat, range, or mask operators.

Type checking

Typing `lvalue assignop expression ;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If `assignop` is `=`:
 - a. Let `typedLvalueIR` be
 - the result of typing `lvalue`, under cursor `p` and typing context `TC`.
 - b. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
 - c. Let `typeIRl` be the type of `lvalue` `typedLvalueIR`.
 - d. Let `typedExpressionIRcast` be `!` `typedExpressionIR` implicitly cast to `typeIRl`.

e. Result in

- the typing context TC ,
- the abstract control flow f ,
- and the typed statement $\text{typedLvalueIR} = \text{typedExpressionIR}_{\text{cast}} ;$.

2. Else:

a. Let binop ' be ! the binary operator corresponding to assignop .

b. Let typedLvalueIR be

- the result of typing lvalue , under cursor p and typing context TC .

c. Let $\text{expression}_{\text{lvalue}}$ be $\text{lvalue_to_expression}(\text{lvalue})$.

d. Let $\text{expression}_{\text{binary}}$ be $\text{expression}_{\text{lvalue}} \text{ binop' expression}$.

e. Let $\text{typedExpressionIR}_{\text{binary}}$ be

- the result of typing $\text{expression}_{\text{binary}}$, under cursor p and typing context TC .

f. Let ($\text{typeIR}_{\text{lvalue}}$) be the note of typedLvalueIR .

g. Let $\text{expressionIR} \# _ \text{ be ! typedExpressionIR}_{\text{binary}}$ implicitly cast to $\text{typeIR}_{\text{lvalue}}$.

h. Check that expressionIR has type $\text{binaryExpressionIR}$.

i. Let $_ \text{ binop'' typedExpressionIR}_{\text{cast}}$ be expressionIR .

j. Check that binop'' is equal to binop' .

k. Result in

- the typing context TC ,
- the abstract control flow f ,
- and the typed statement $\text{typedLvalueIR assignop typedExpressionIR}_{\text{cast}} ;$.

12.2.1. L-values

```
lvalue
: referenceExpression
| lvalue . member
| lvalue `[ expression ]
| lvalue `[ expression : expression ]
| `( lvalue )
;
```

Lvalue_ok:

- Typing lvalue , under cursor cursor and typing context typingContext :
- Results in typedLvalueIR .

Typing $\text{prefixedNonTypeName}$, under cursor p and typing context TC :

1. Let prefixedNameIR be the prefixed name of $\text{prefixedNonTypeName}$.
2. Let $\text{direction typeIR ctk value?}$ be ! the type of variable prefixedNameIR in the p layer of TC .
3. Check that ctk is DYN .

4. Check that $value^?$ is none.
5. Check that $direction = OUT$ or $direction = INOUT$.
6. Result in $prefixedNameIR$ with note ($typeIR$).

Typing $lvalue_{base} . member$, under cursor p and typing context TC :

1. Let $typedLvalueIR_{base}$ be
 - the result of typing $lvalue_{base}$ under cursor p and typing context TC .
2. Let ($typeIR_{base}$) be the note of $typedLvalueIR_{base}$.
3. Let $typeIR$ be $typeIR_{base}$ with typedefs unrolled.
4. If let $typeIR' [_]$ be $typeIR$:
 - a. Let $nameIR$ be the name of member.
 - b. Check that $nameIR = "next"$ or $nameIR = "last"$.
 - c. Check that $p = BLOCK \wedge is_parser_blockKind(TC.BLOCK.KIND)$ or $p = LOCAL \wedge is_parser_state_localKind(TC.LOCAL.KIND)$.
 - d. Let $typedLvalueIR$ be $typedLvalueIR_{base} . nameIR$ with note ($typeIR'$).
 - e. Result in $typedLvalueIR$.
5. Else if let $STRUCT _ < _ ^* > \{ (typeIR_{field} nameIR_{field} ;)^* \}$ be $typeIR$:
 - a. Let $nameIR$ be the name of member.
 - b. Let $typeIR''$ be ! $assoc_<nameIR, typeIR>(nameIR, nameIR_{field}, typeIR_{field}^*)$.
 - c. Let $typedLvalueIR$ be $typedLvalueIR_{base} . nameIR$ with note ($typeIR''$).
 - d. Result in $typedLvalueIR$.
6. Else if let $HEADER _ < _ ^* > \{ (typeIR_{field} nameIR_{field} ;)^* \}$ be $typeIR$:
 - a. Let $nameIR$ be the name of member.
 - b. Let $typeIR''$ be ! $assoc_<nameIR, typeIR>(nameIR, nameIR_{field}, typeIR_{field}^*)$.
 - c. Let $typedLvalueIR$ be $typedLvalueIR_{base} . nameIR$ with note ($typeIR''$).
 - d. Result in $typedLvalueIR$.
7. Else if let $HEADER_UNION _ < _ ^* > \{ (typeIR_{field} nameIR_{field} ;)^* \}$ be $typeIR$:
 - a. Let $nameIR$ be the name of member.
 - b. Let $typeIR''$ be ! $assoc_<nameIR, typeIR>(nameIR, nameIR_{field}, typeIR_{field}^*)$.
 - c. Let $typedLvalueIR$ be $typedLvalueIR_{base} . nameIR$ with note ($typeIR''$).
 - d. Result in $typedLvalueIR$.

Typing $lvalue_{base} [expression_{index}]$, under cursor p and typing context TC :

1. Let $typedLvalueIR_{base}$ be
 - the result of typing $lvalue_{base}$ under cursor p and typing context TC .
2. Let ($typeIR_{base}$) be the note of $typedLvalueIR_{base}$.
3. Let $typeIR$ be $typeIR_{base}$ with typedefs unrolled.

4. Check that typeIR has type headerStackTypeIR .
5. Let $\text{typeIR}' [n_{\text{size}}]$ be typeIR .
6. Let $\text{typedExpressionIR}_{\text{index}}$ be
 - the result of typing $\text{expression}_{\text{index}}$ under cursor p and typing context TC .
7. Let $\text{typeIR}_{\text{index}}$ and compile-time known-ness $\text{ctk}_{\text{index}}$ be the note of $\text{typedExpressionIR}_{\text{index}}$.
8. Let $\text{typedExpressionIR}_{\text{index_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{index}}$ until $\$compat_array_index$ is satisfied.
9. If $\text{ctk}_{\text{index}}$ is LCTK:
 - a. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{index_reduced}}$ under cursor p and typing context TC .
 - b. Check that value has type integerValue .
 - c. Let $\text{integerValue}_{\text{index}}$ be value .
 - d. Let int be the integer representation of $\text{integerValue}_{\text{index}}$.
 - e. Check that int has type nat .
 - f. Let n_{index} be int .
 - g. Check that n_{index} is less than n_{size} .
 - h. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ with note (typeIR').
 - i. Result in typedLvalueIR .
10. Else:
 - a. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ with note (typeIR').
 - b. Result in typedLvalueIR .

Typing $\text{lvalue}_{\text{base}} [\text{expression}_{\text{hi}} : \text{expression}_{\text{lo}}]$, under cursor p and typing context TC :

1. Let $\text{typedLvalueIR}_{\text{base}}$ be
 - the result of typing $\text{lvalue}_{\text{base}}$ under cursor p and typing context TC .
2. Let ($\text{typeIR}_{\text{base}}$) be the note of $\text{typedLvalueIR}_{\text{base}}$.
3. Check that $\text{compat_bitslice_base}(\text{typeIR}_{\text{base}})$.
4. Let $\text{typedExpressionIR}_{\text{hi}}$ be
 - the result of typing $\text{expression}_{\text{hi}}$ under cursor p and typing context TC .
5. Let $\text{typedExpressionIR}_{\text{lo}}$ be
 - the result of typing $\text{expression}_{\text{lo}}$ under cursor p and typing context TC .
6. Let $\text{typedExpressionIR}_{\text{hi_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{hi}}$ until $\$compat_bitslice_index$ is satisfied.
7. Let $\text{typedExpressionIR}_{\text{lo_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{lo}}$ until $\$compat_bitslice_index$ is satisfied.
8. Let $\text{typeIR}_{\text{hi_reduced}}$ and compile-time known-ness $\text{ctk}_{\text{hi_reduced}}$ be the note of $\text{typedExpressionIR}_{\text{hi_reduced}}$.
9. Let $\text{typeIR}_{\text{lo_reduced}}$ and compile-time known-ness $\text{ctk}_{\text{lo_reduced}}$ be the note of $\text{typedExpressionIR}_{\text{lo_reduced}}$.
10. Check that $\text{ctk}_{\text{hi_reduced}}$ is LCTK.

11. Let $value$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{hi_reduced}$, under cursor p and typing context TC .
12. Check that $value$ has type $integerValue$.
13. Let $integerValue_{hi}$ be $value$.
14. Let int be the integer representation of $integerValue_{hi}$.
15. Check that int has type nat .
16. Let n_{hi} be int .
17. Check that $ctk_{lo_reduced}$ is LCTK.
18. Let $value'$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{lo_reduced}$, under cursor p and typing context TC .
19. Check that $value'$ has type $integerValue$.
20. Let $integerValue_{lo}$ be $value'$.
21. Let int' be the integer representation of $integerValue_{lo}$.
22. Check that int' has type nat .
23. Let n_{lo} be int' .
24. Check that $is_valid_bitslice(typeIR_{base}, n_{lo}, n_{hi})$.
25. Let int'' be $n_{hi} - n_{lo} + 1$.
26. Check that int'' has type nat .
27. Let n_{slice} be int'' .
28. Let $typeIR$ be $BIT < n_{slice} >$.
29. Let $typedLvalueIR$ be $typedLvalueIR_{base} [typedExpressionIR_{hi_reduced} : typedExpressionIR_{lo_reduced}]$ with note ($typeIR$).
30. Result in $typedLvalueIR$.

Typing ($lvalue_{base}$), under cursor p and typing context TC :

1. Let $typedLvalueIR_{base}$ be
 - the result of typing $lvalue_{base}$, under cursor p and typing context TC .
2. Let ($typeIR_{base}$) be the note of $typedLvalueIR_{base}$.
3. Let $typedLvalueIR$ be ($typedLvalueIR_{base}$) with note ($typeIR_{base}$).
4. Result in $typedLvalueIR$.

Compile-time evaluation

Compile-time evaluation of $assignmentStatementIR$, under cursor p , instantiation context IC , and store STO :

1. Result in

- the instantiation context *IC*,
- the store *STO*,
- and *assignmentStatementIR*.

12.3. Call statements

```
callStatement
: lvalue `( argumentList ) ;
| lvalue `< typeArgumentList > `( argumentList ) ;
;

callStatementIR
: callableTargetIR `< typeArgumentListIR > `( argumentListIR ) ;
;
```

Type checking

Typing *callStatement*, under cursor *p*, typing context *TC*, abstract control flow *f*, and loop context *l*:

1. If let *lvalue_{callable}* (*argumentList*) ; be *callStatement*:
 - a. Let the typed callable target *callableTargetIR* be
 - the result of typing *lvalue_{callable}*, under cursor *p* and typing context *TC*.
 - b. Let *argument** be the list of arguments in *argumentList*.
 - c. Let *argumentIR** be the list of *argumentIR*, obtained by repeating:
 - Let *argumentIR* be
 - the result of typing *argument*, under cursor *p* and typing context *TC*.
 for each *argument* in *argument**
 - d. Let the callable type *callableTypeIR* with fresh type variables *typeId_{impl}** and default parameter ids *id_{default}** be
 - the result of finding the type of *callableTargetIR* < · > (*argumentIR**), under cursor *p* and typing context *TC*.
 - e. Let the return type *typeIR_{ret}* with inferred type arguments *typeArgumentIR_{inferred}** and the casted argument list *argumentIR_{cast}** be
 - the result of typing the invocation *callableTypeIR* < · > (*argumentIR**) with omitted type arguments *typeId_{impl}** and omitted arguments *id_{default}**, under cursor *p* and typing context *TC*.
 - f. If *is_static_assert_callableTarget(callableTargetIR)*:
 - i. Let *callExpressionIR* be *callableTargetIR* < *typeArgumentIR_{inferred}** > (*argumentIR_{cast}**).
 - ii. Let *expressionNoteIR* be type *typeIR_{ret}* and compile-time known-ness *LCTK*.
 - iii. Let *value* be
 - the result of local compile-time evaluation of *callExpressionIR* annotated with *expressionNoteIR*, under cursor *p* and typing context *TC*.
 - iv. Check that *value* is equal to *`B true*.

- v. Let `emptyStatementIR` be ;.
- vi. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `emptyStatementIR`.
- g. Let `argumentIR*` be
 - the result of `typing argumentList`, under cursor `p` and typing context `TC`.
- h. If `~ is_static_assert_callableTarget(callableTargetIR)`:
 - i. Let `callStatementIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* ) ;`.
 - ii. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `callStatementIR`.
- 2. Else:
 - a. Let `lvaluecallable < typeArgumentList > ( argumentList ) ;` be `callStatement`.
 - b. Let the typed callable target `callableTargetIR` be
 - the result of `typing lvaluecallable`, under cursor `p` and typing context `TC`.
 - c. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of `typing typeArgumentList`, under cursor `p` and typing context `TC`.
 - d. Let `argumentIR*` be
 - the result of `typing argumentList`, under cursor `p` and typing context `TC`.
 - e. Let the callable type `callableTypeIR` with fresh type variables `typeIdinserted*` and default parameter ids `iddefault*` be
 - the result of `finding the type of callableTargetIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - f. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
 - g. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of `typing the invocation callableTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*` and omitted arguments `iddefault*`, under cursor `p` and typing context `TC`.
 - h. Let `callStatementIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* ) ;`.
 - i. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `callStatementIR`.

12.3.1. L-values

`lvalue`
: `referenceExpression`

```

| lvalue . member
| lvalue `[ expression ]
| lvalue `[ expression : expression ]
| `( lvalue )
;

```

CallableTarget_ok:

- Typing `callableTarget`, under cursor `cursor` and typing context `typingContext`:
- Results in the typed callable target `callableTargetIR`.

CallableTarget_lvalue_ok:

- Typing `lvalue`, under cursor `cursor` and typing context `typingContext`:
- Results in the typed callable target `callableTargetIR`.

Typing `lvalue`, under cursor `p` and typing context `TC`:

1. Let `expression` be the expression corresponding to `lvalue`.
2. Let the typed callable target `callableTargetIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
3. Result in the typed callable target `callableTargetIR`.

Typing `callableTarget`, under cursor `p` and typing context `TC`:

1. If let `prefixedNonTypeName` be `callableTarget`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
 - b. Check that `prefixedNameIR = "verify" ∨ prefixedNameIR = . "verify"` implies `p = BLOCK ∧ is_parser_blockKind(TC.BLOCK.KIND) ∨ p = LOCAL ∧ is_parser_state_localKind(TC.LOCAL.KIND)`.
 - c. Result in the typed callable target `prefixedNameIR`.
2. If `callableTarget` is equal to `THIS`:
 - a. Result in the typed callable target `"this"`.

Typing `memberAccessBase . member`, under cursor `p` and typing context `TC`:

1. If let `prefixedTypeName` be `memberAccessBase`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - b. Let `nameIR` be the name of `member`.
 - c. Result in the typed callable target `TYPE prefixedNameIR . nameIR`.
2. Else:
 - a. Let `expressionbase` be `memberAccessBase`.

- b. Let `nameIR` be the name of member.
- c. Let `typedExpressionIRbase` be
 - the result of typing `expressionbase`, under cursor `p` and typing context `TC`.
- d. Result in the typed callable target `typedExpressionIRbase . nameIR`.

Typing (`expression`), under cursor `p` and typing context `TC`:

1. Let the typed callable target `callableTargetIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
2. Result in the typed callable target (`callableTargetIR`).

Compile-time evaluation

Compile-time evaluation of `callStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in
 - the instantiation context `IC`,
 - the store `STO`,
 - and `callStatementIR`.

12.4. Direct application statements

```
directApplicationStatement
  : namedType . APPLY `( argumentList ) ;
;

directApplicationStatementIR
  : prefixedNameIR . APPLY `( argumentListIR ) ;
;
```

Controls and parsers are often instantiated exactly once. As a light syntactic sugar, control and parser declarations with no constructor parameters may be applied directly, as if they were an instance. This has the effect of creating and applying a local instance of that type.

```
control Callee(/* parameters omitted */) { /* body omitted */ }

control Caller(/* parameters omitted */)(/* parameters omitted */) {
  apply {
    Callee.apply(/* arguments omitted */); // Callee is treated as an instance
  }
}
```

The definition of `Caller` is equivalent to the following.

```
control Caller(/* parameters omitted */)(/* parameters omitted */) {
  @name("Callee") Callee() Callee_inst; // local instance of Callee
  apply {
```

```

    Callee_inst.apply(/* arguments omitted */);           // Callee_inst is applied
  }
}

```

This feature is intended to streamline the common case where a type is instantiated exactly once.

The second production in the grammar allows direct calls for generic controls or parsers:

```

control Callee<T>(/* parameters omitted */) { /* body omitted */ }

control Caller(/* parameters omitted */)(/* parameters omitted */) {
  apply {
    // Callee<bit<32>> is treated as an instance
    Callee<bit<32>>.apply(/* arguments omitted */);
  }
}

```

For completeness, the behavior of directly invoking the same type more than once is defined as follows.

- Direct type invocation in different scopes will result in different local instances with different fully-qualified control names.
- In the same scope, direct type invocation will result in a different local instance per invocation--- however, instances of the same type will share the same global name, via the `@name` annotation. If the type contains controllable entities, then invoking it directly more than once in the same scope is illegal, because it will produce multiple controllable entities with the same fully-qualified control name.

See [\[sec-name-annotations\]](#) for details of `@name` annotations.

No direct invocation is possible for controls or parsers that require constructor arguments. These need to be instantiated before they are invoked.

Type checking

Typing `namedType . APPLY ( argumentList ) ;`, **under cursor** `p`, **typing context** `TC`, **abstract control flow** `f`, **and loop context** `l`:

1. Let `expressionIR` annotated with type `typeIRobject` and compile-time known-ness `_` be
 - the result of `typing namedType ( `EMPTY)`, **under cursor** `p` and **typing context** `TC`.
2. Check that `expressionIR` has type `callExpressionIR`.
3. Let `callExpressionIR` be `expressionIR`.
4. Check that `callExpressionIR` matches pattern `% ( % )`.
5. Let `prefixedNameIR < typeArgumentIR* > ( argumentIR* )` be `callExpressionIR`.
6. Check that `typeArgumentIR*` is an empty list.
7. Check that `argumentIR*` is an empty list.
8. Check that `compat_direct_application(typeIRobject)`.
9. Let `nameIRobject` be `"__direct_application"`.
10. Let `TC1` be `TC` **where** `nameIRobject` **to** ``EMPTY typeIRobject CTK` **is added to the** `p` **layer**.
11. Let `lvalue` be ``ID nameIRobject . `ID "apply"`.
12. Let the typing context `_`, the abstract control flow `_`, and the typed statement `statementIR` be

- the result of **typing** `lvalue ( argumentList ) ;`, under cursor `p`, typing context `TC1`, abstract control flow `f`, and loop context `l`.

13. Check that `statementIR` has type `callStatementIR`.
14. Let `callStatementIR` be `statementIR`.
15. Let `callableTargetIR < typeArgumentIR* > ( argumentIRcast* ) ;` be `callStatementIR`.
16. Check that `callableTargetIR` is equal to `nameIRobject # ( typeIRobject CTK ) . "apply"`.
17. Check that `typeArgumentIR*` is an empty list.
18. Let `directApplicationStatementIR` be `prefixedNameIR . APPLY ( argumentIRcast* ) ;`.
19. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `directApplicationStatementIR`.

Compile-time evaluation

Compile-time evaluation of `prefixedNameIR . APPLY ( argumentListIR ) ;`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Let the constructor `constructorDef < typeArgumentIR* >` with default parameters `id*` be
 - the result of **finding constructor for** `prefixedNameIR < · > ( · )` under cursor `p`, and instantiation context `IC`.
2. Check that `typeArgumentIR*` is an empty list.
3. Check that `id*` is an empty list.
4. Let `typeId` be **the string form of** `prefixedNameIR`.
5. Let `ICcallee` be `IC` with `typeId` added to the path.
6. Let the updated store `STO1`, and the constructed object be
 - the result of **loading** `constructorDef < · > ( · )` with default parameters `·`, under cursor `p`, instantiation context `ICcallee`, and store `STO`.
7. Let `typeIdfresh` be a fresh type variable.
8. Let `typeIR` be `typeId`.
9. Let `nameIR` be **the concatenation of** `[typeId, "_", typeIdfresh]`.
10. Let `objectId` be `IC.PATH` concatenated with `[typeId]`.
11. Let `STO2` be `STO1` where `objectId` to object is added.
12. Let `constantDeclarationIR` be ``EMPTY CONST typeIR nameIR = REF objectId ;`.
13. Let `callableTargetIR` be `nameIR # ( typeIR CTK ) . "apply"`.
14. Let `callStatementIR` be `callableTargetIR < · > ( argumentListIR ) ;`.
15. Let `blockStatementIR` be ``EMPTY { [constantDeclarationIR, callStatementIR] }`.
16. Result in
 - the instantiation context `IC`,
 - the store `STO2`,

- and `blockStatementIR`.

12.5. Return statements

```
returnStatement
: RETURN ;
| RETURN expression ;
;

returnStatementIR
: RETURN ;
| RETURN typedExpressionIR ;
;
```

The `return` statement immediately terminates the execution of the `action`, `function` or `control` containing it. `return` statements are not allowed within parsers. `return` statements followed by an expression are only allowed within functions that return values; in this case the type of the expression must match the return type of the function. Any copy-out behavior due to direction `out` or `inout` parameters of the enclosing `action`, `function`, or `control` are still performed after the execution of the `return` statement. See [\[sec-calling-convention\]](#) for details on copy-out behavior.

Type checking

Typing `returnStatement`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If `returnStatement` is `RETURN` ;:
 - a. Check that `VOID` is equal to the expected return type in `TC`.
 - b. Result in
 - the typing context `TC`,
 - the abstract control flow `RET`,
 - and the typed statement `RETURN` ;.
2. Else:
 - a. Let `RETURN expression` ; be `returnStatement`.
 - b. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `LOCAL` and typing context `TC`.
 - c. Let `typeIRret` be ! the expected return type in `TC`.
 - d. Let `typedExpressionIRcast` be ! `typedExpressionIR` implicitly cast to `typeIRret`.
 - e. Result in
 - the typing context `TC`,
 - the abstract control flow `RET`,
 - and the typed statement `RETURN typedExpressionIRcast` ;.

Compile-time evaluation

Compile-time evaluation of `returnStatementIR`, under cursor `LOCAL`, instantiation context `IC`, and store `STO`:

1. Result in
 - the instantiation context `IC`,
 - the store `STO`,
 - and `returnStatementIR`.

12.6. Exit statements

```
exitStatement
: EXIT ;
;

exitStatementIR = exitStatement
```

The `exit` statement immediately terminates the execution of all the blocks currently executing: the current `action` (if invoked within an `action`), the current `control`, and all its callers. `exit` statements are not allowed within parsers or functions.

Any copy-out behavior due to direction `out` or `inout` parameters of the enclosing `action` or `control`, and all of its callers, are still performed after the execution of the `exit` statement. See [\[sec-calling-convention\]](#) for details on copy-out behavior.

There are some expressions whose evaluation might cause an `exit` statement to be executed. Examples include:

- `table.apply().action_run`, which can only appear as the expression of a `switch` statement (see [\[sec-switch-stmt\]](#)), and when it appears there, it must be the entire expression.
- Any expression containing `table.apply().hit` or `table.apply().miss` (see [\[sec-invoke-mau\]](#)), which can be part of arbitrarily complex expressions in many places of a P4 program, such as:
 - the expression in an `if` statement.
 - the expression `e1` in a conditional expression `e1 ? e2 : e3`.
 - in an assignment statement, in the left and/or right hand sides.
 - an argument passed to a function or method call.
 - an expression to calculate a table key (see [\[sec-mau-semantics\]](#)).

This list is not intended to be exhaustive.

If applying the table causes an action to be executed, which in turn causes an `exit` statement to be executed, then evaluation of the expression ends immediately, and the rest of the current expression or statement does not complete its execution. See [\[sec-expr-eval-order\]](#) for the order of evaluation of the parts of an expression. For the examples listed above, it also means the following behavior after the expression evaluation is interrupted.

- For a `switch` statement, if `table.apply()` exits, then none of the blocks in the `switch` statement are executed.
- If `table.apply().hit` or `table.apply().miss` cause an exit during the evaluation of an expression:

- If it is the expression of an `if` statement, then neither the 'then' nor 'else' branches of the `if` statement are executed.
- If it is the expression `e1` in a conditional expression `e1 ? e2 : e3`, then neither expression `e2` nor `e3` are evaluated.
- If the expression is the right hand side of an assignment statement, or part of the calculation of the L-value on the left hand side (e.g. the index expression of a header stack reference), then no assignment occurs.
- If the expression is an argument passed to a function or method call, then the function/method call does not occur.
- If the expression is a table key, then the table is not applied.

Type checking

Typing `EXIT ;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Result in
 - the typing context `TC`,
 - the abstract control flow `f`,
 - and the typed statement `EXIT ;`.

Compile-time evaluation

Compile-time evaluation of `exitStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in
 - the instantiation context `IC`,
 - the store `STO`,
 - and `exitStatementIR`.

12.7. Block statements

```
blockElementStatement
: constantDeclaration
| variableDeclaration
| statement
;

blockStatement
: annotationList `{ blockElementStatementList }
;

blockElementStatementIR
: constantDeclarationIR
| variableDeclarationIR
| statementIR
;

blockStatementIR
: annotationList `{ blockElementStatementListIR }
```

;

A block statement is denoted by curly braces. It contains a sequence of statements and declarations, which are executed sequentially. The declarations (e.g., variables and constants) within a block statement are only visible within the block.

Type checking

Typing `blockStatement`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `TC1` be `enter_t(TC)`.
2. Let the typing context `TC2`, abstract control flow `f1`, and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context `TC1`, abstract control flow `f`, and loop context `l`.
3. Let `TC3` be `exit_t(TC2)`.
4. Result in
 - the typing context `TC3`,
 - the abstract control flow `f1`,
 - and the typed statement `blockStatementIR`.

`BlockElementStmt_ok`:

- Typing `blockElementStatement`, under typing context `typingContext`, abstract control flow `flow`, and loop context `loopctxt`:
- Results in
 - the typing context `typingContext`,
 - the abstract control flow `flow`,
 - and the typed block element statement `blockElementStatementIR`.

Typing `constantDeclaration`, under typing context `TC0`, abstract control flow `f`, and loop context `l`:

1. Let the updated typing context `TC1` and `constantDeclarationIR` be
 - the result of typing `constantDeclaration`, under cursor `LOCAL` and typing context `TC0`.
2. Result in
 - the typing context `TC1`,
 - the abstract control flow `f`,
 - and the typed block element statement `constantDeclarationIR`.

Typing `variableDeclaration`, under typing context `TC0`, abstract control flow `f`, and loop context `l`:

1. Let the updated typing context `TC1` and `variableDeclarationIR` be

- the result of **typing** `variableDeclaration`, under cursor `LOCAL` and typing context `TC0`.

2. Result in

- the typing context `TC1`,
- the abstract control flow `f`,
- and the typed block element statement `variableDeclarationIR`.

Typing `statement`, under typing context `TC0`, abstract control flow `f`, and loop context `l`:

1. Let the typing context `TC1`, the abstract control flow `fpost`, and the typed statement `statementIR` be

- the result of **typing** `statement`, under cursor `LOCAL`, typing context `TC0`, abstract control flow `f`, and loop context `l`.

2. Result in

- the typing context `TC1`,
- the abstract control flow `fpost`,
- and the typed block element statement `statementIR`.

Compile-time evaluation

Compile-time evaluation of `annotationList { blockElementStatementListIR }`, under cursor `LOCAL`, instantiation context `IC`, and store `STO`:

1. Let `IC1` be `enter_i(IC)`.

2. Let the updated instantiation context `IC2`, the store `STO1`, and `_ { blockElementStatementListIRinst }` be

- the result of **compile-time evaluation of** `annotationList { blockElementStatementListIR }`, under instantiation context `IC1`, and store `STO`.

3. Let `IC3` be `exit_i(IC2)`.

4. Result in

- the instantiation context `IC3`,
- the store `STO1`,
- and `annotationList { blockElementStatementListIRinst }`.

BlockElementStmt_inst: `IC0 STO0 |- blockElementStatementIR : % % %`

`IC0 STO0 |- constantDeclarationIR : % % %:`

1. Let the updated instantiation context `IC1` and `constantDeclarationIRinst` be

- the result of **loading** `constantDeclarationIR`, under cursor `LOCAL`, and instantiation context `IC0`.

2. Result in `IC1`, `STO0`, and `constantDeclarationIRinst`.

$IC_0 \text{ } STO_0 \mid - \text{variableDeclarationIR} : \% \% \% :$

1. Result in IC_0 , STO_0 , and $\text{variableDeclarationIR}$.

$IC_0 \text{ } STO_0 \mid - \text{statementIR} : \% \% \% :$

1. Let the instantiation context IC_1 , the store STO_1 , and $\text{statementIR}_{inst}$ be
 - the result of compile-time evaluation of statementIR , under cursor $LOCAL$, instantiation context IC_0 , and store STO_0 .
2. Result in IC_1 , STO_1 , and $\text{statementIR}_{inst}$.

12.8. Conditional statements

```
conditionalStatement
: IF `( expression ) statement
| IF `( expression ) statement ELSE statement
;

conditionalStatementIR
: IF `( typedExpressionIR ) statementIR
| IF `( typedExpressionIR ) statementIR ELSE statementIR
;
```

The conditional statement uses standard syntax and semantics familiar from many programming languages.

However, the condition expression in P4 is required to be a Boolean (and not an integer).

When several **if** statements are nested, the **else** applies to the innermost **if** statement that does not have an **else** statement.

Type checking

Typing $\text{conditionalStatement}$, under cursor p , typing context TC , abstract control flow f , and loop context l :

1. If let $IF (\text{expression}_{cond}) \text{statement}_{then}$ be $\text{conditionalStatement}$:
 - a. Let $\text{typedExpressionIR}_{cond}$ be
 - the result of typing expression_{cond} , under cursor p and typing context TC .
 - b. Let typeIR_{cond} be the type of $\text{typedExpressionIR}_{cond}$.
 - c. Check that $BOOL$ is equal to typeIR_{cond} with **typedefs unrolled**.
 - d. Let the typing context TC_{then} , the abstract control flow f_{then} , and the typed statement $\text{statementIR}_{then}$ be
 - the result of typing statement_{then} , under cursor p , typing context TC , abstract control flow f , and loop context l .
 - e. Result in
 - the typing context TC ,

- the abstract control flow f ,
- and the typed statement $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}}$.

2. Else:

- Let $\text{IF} (\text{expression}_{\text{cond}}) \text{statement}_{\text{then}} \text{ELSE} \text{statement}_{\text{else}}$ be $\text{conditionalStatement}$.
- Let $\text{typedExpressionIR}_{\text{cond}}$ be
 - the result of **typing** $\text{expression}_{\text{cond}}$, **under cursor** p and **typing context** TC .
- Let $\text{typeIR}_{\text{cond}}$ be **the type of** $\text{typedExpressionIR}_{\text{cond}}$.
- Check that BOOL is equal to $\text{typeIR}_{\text{cond}}$ **with typedefs unrolled**.
- Let the typing context TC_{then} , the abstract control flow f_{then} , and the typed statement $\text{statementIR}_{\text{then}}$ be
 - the result of **typing** $\text{statement}_{\text{then}}$, **under cursor** p , **typing context** TC , **abstract control flow** f , **and loop context** l .
- Let the typing context TC_{else} , the abstract control flow f_{else} , and the typed statement $\text{statementIR}_{\text{else}}$ be
 - the result of **typing** $\text{statement}_{\text{else}}$, **under cursor** p , **typing context** TC , **abstract control flow** f , **and loop context** l .
- Let f_{post} be **the merged control flow of** f_{then} **and** f_{else} .
- Result in
 - the typing context TC ,
 - the abstract control flow f_{post} ,
 - and the typed statement $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}} \text{ELSE} \text{statementIR}_{\text{else}}$.

Compile-time evaluation

Compile-time evaluation of $\text{conditionalStatementIR}$, **under cursor** p , **instantiation context** IC , and **store** STO :

- If let $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}}$ be $\text{conditionalStatementIR}$:
 - Let the instantiation context IC_{then} , the store STO_1 , and $\text{statementIR}_{\text{then_inst}}$ be
 - the result of **compile-time evaluation of** $\text{statementIR}_{\text{then}}$, **under cursor** p , **instantiation context** IC , and **store** STO .
 - Result in
 - the instantiation context IC ,
 - the store STO_1 ,
 - and $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then_inst}}$.
- Else:
 - Let $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}} \text{ELSE} \text{statementIR}_{\text{else}}$ be $\text{conditionalStatementIR}$.
 - Let the instantiation context IC_{then} , the store STO_1 , and $\text{statementIR}_{\text{then_inst}}$ be
 - the result of **compile-time evaluation of** $\text{statementIR}_{\text{then}}$, **under cursor** p , **instantiation context** IC , and **store** STO .
 - Let the instantiation context IC_{else} , the store STO_2 , and $\text{statementIR}_{\text{else_inst}}$ be

- the result of **compile-time evaluation of** $\text{statementIR}_{\text{else}}$, **under cursor** p , **instantiation context** IC , and **store** STO_1 .

d. Result in

- the instantiation context IC ,
- the store STO_2 ,
- and **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then_inst}}$ **ELSE** $\text{statementIR}_{\text{else_inst}}$.

Dynamic evaluation

The **compile-time evaluation of** $\text{conditionalStatementIR}$, **under cursor** p , **evaluation context** EC , and **store** STO :

1. If let **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then}}$ **be** $\text{conditionalStatementIR}$:

a. Let the updated evaluation context EC_{cond} , store STO_{cond} and expression result expressionResult be

- the result of **evaluating** $\text{typedExpressionIR}_{\text{cond}}$, **under cursor** p , **evaluation context** EC , and **store** STO .

b. If let abortResult be expressionResult :

i. Result in

- the updated evaluation context EC_{cond} ,
- the store STO_{cond} ,
- and the instantiated statement abortResult .

c. If expressionResult is equal to `B true`:

i. Let the updated evaluation context EC_{then} , the store STO_{then} , and the instantiated statement statementResult be

- the result of **the compile-time evaluation of** $\text{statementIR}_{\text{then}}$, **under cursor** p , **evaluation context** EC_{cond} , and **store** STO_{cond} .

ii. Result in

- the updated evaluation context EC_{then} ,
- the store STO_{then} ,
- and the instantiated statement statementResult .

d. If expressionResult is equal to `B false`:

i. Result in

- the updated evaluation context EC_{cond} ,
- the store STO_{cond} ,
- and the instantiated statement `EMPTY`.

2. Else:

a. Let **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then}}$ **ELSE** $\text{statementIR}_{\text{else}}$ **be** $\text{conditionalStatementIR}$.

b. Let the updated evaluation context EC_{cond} , store STO_{cond} and expression result expressionResult be

- the result of **evaluating** $\text{typedExpressionIR}_{\text{cond}}$, **under cursor** p , **evaluation context** EC , and **store** STO .

c. If let `abortResult` be `expressionResult`:

i. Result in

- the updated evaluation context EC_{cond} ,
- the store STO_{cond} ,
- and the instantiated statement `abortResult`.

d. If `expressionResult` is equal to ``B true`:

i. Let the updated evaluation context EC_{then} , the store STO_{then} , and the instantiated statement `statementResult` be

- the result of the compile-time evaluation of `statementIRthen` under cursor `p`, evaluation context EC_{cond} , and store STO_{cond} .

ii. Result in

- the updated evaluation context EC_{then} ,
- the store STO_{then} ,
- and the instantiated statement `statementResult`.

e. If `expressionResult` is equal to ``B false`:

i. Let the updated evaluation context EC_{else} , the store STO_{else} , and the instantiated statement `statementResult` be

- the result of the compile-time evaluation of `statementIRelse` under cursor `p`, evaluation context EC_{cond} , and store STO_{cond} .

ii. Result in

- the updated evaluation context EC_{else} ,
- the store STO_{else} ,
- and the instantiated statement `statementResult`.

12.9. For statements

```
forInitStatement
: annotationList type name initializerOpt
| forUpdateStatement
;

forUpdateStatement
: lvalue `( argumentList )
| lvalue `< typeArgumentList > `( argumentList )
| lvalue assignop expression
;

forCollectionExpression
: expression
| expression .. expression
;

forStatement
: annotationList FOR `( forInitStatementList ; expression ; forUpdateStatementList )
statement
| annotationList FOR `( type name IN forCollectionExpression ) statement
| annotationList FOR `( annotationListNonEmpty type name IN forCollectionExpression )
statement
;
```

```

forInitStatementIR
  : annotationList typeIR nameIR initializerOptIR
  | forUpdateStatementIR
  ;

forUpdateStatementIR
  : callableTargetIR `< typeArgumentListIR > `( argumentListIR )
  | typedLvalueIR assignop typedExpressionIR
  ;

forCollectionExpressionIR
  : typedExpressionIR
  | typedExpressionIR .. typedExpressionIR
  ;

forStatementIR
  : annotationList FOR `( forInitStatementListIR ; typedExpressionIR ;
forUpdateStatementListIR ) statementIR
  | annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR ) statementIR
  | annotationList FOR `( annotationList typeIR nameIR IN forCollectionExpressionIR )
statementIR
  ;

```

12.10. Break statements

```

breakStatement
  : BREAK ;
  ;

breakStatementIR = breakStatement

```

12.11. Continue statements

```

continueStatement
  : CONTINUE ;
  ;

continueStatementIR = continueStatement

```

12.12. Switch statements

```

switchLabel
  : DEFAULT
  | expressionNonBrace
  ;

switchCase
  : switchLabel : blockStatement
  | switchLabel :
  ;

switchStatement
  : SWITCH `( expression ) `{ switchCaseList }
  ;

switchLabelIR
  : DEFAULT
  | typedExpressionIR
  ;

```

```

switchCaseIR
: switchLabelIR : blockStatementIR
| switchLabelIR :
;

switchStatementIR
: SWITCH `( typedExpressionIR ) `{ switchCaseListIR }
;

```

The `switch` statement can only be used within `control` blocks, `action` bodies, or function bodies.

The `expressionNonBrace` is the same as `expression` as defined in [\[sec-exprs\]](#), except it does not include any cases that can begin with a left brace `{` character, to avoid syntactic ambiguity with a block statement.

There are two kinds of `switch` expressions allowed, described separately in the following two subsections.

12.12.1. Notes common to all switch statements

It is a compile-time error if two labels of a `switch` statement equal each other. The switch label values need not include all possible values of the switch expression. It is optional to have a `switch` case with the `default` label, but if one is present, it must be the last one in the `switch` statement.

If a switch label is not followed by a block statement, it falls through to the next label. However, if a block statement is present, it does not fall through. Note that this is different from C-style `switch` statements, where a `break` is needed to prevent fall-through. If the last switch label is not followed by a block statement, the behavior is the same as if the last switch label were followed by an empty block statement `{ }`.

When a `switch` statement is executed, first the switch expression is evaluated, and any side effects from evaluating this expression are visible to any `switch` case that is executed. Among switch labels that are not `default`, at most one of them can equal the value of the switch expression. If one is equal, that switch case is executed.

If no labels are equal to the `switch` expression, then:

- if there is a `default` label, the case with the `default` label is executed.
- if there is no `default` label, then no switch case is executed, and execution continues after the end of the `switch` statement, with no side effects (except any that were caused by evaluating the `switch` expression).

See "Implementing generalized P4_16 switch statements" for possible techniques that one might use to implement generalized switch statements.^[1]

12.12.2. Switch statement on match-action table result

This type of `switch` statement can only be used within `control apply` blocks. For this variant of `switch` statement, the expression must be of the form `t.apply().action_run`, where `t` is the name of a table (see [\[sec-invoke-mau\]](#)). All switch labels must be names of actions of the table `t`, or `default`.

```

switch (t.apply().action_run) {
  action1:           // fall-through to action2:
  action2: { /* body omitted */ }
  action3: { /* body omitted */ } // no fall-through from action2 to action3 labels
  default: { /* body omitted */ }
}

```

Note that the `default` label of the `switch` statement is used to match on the kind of action executed, no matter whether there was a table hit or miss. The `default` label does not indicate that the table missed and the `default_action` was executed.

Type checking

`SwitchLabel_table_ok`: TC typeId_{table} bool |- switchLabel : %

TC typeId_{table} bool |- switchLabel : %:

1. If `bool` is equal to `true`:
 - a. Check that `switchLabel` is `DEFAULT`.
 - b. Result in `DEFAULT`.
2. If let `prefixedNonTypeName` be `switchLabel`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
 - b. Check that `prefixedNameIR` matches pattern `` %`.
 - c. Let `nameIRlabel` be `prefixedNameIR`.
 - d. Let `typeIdtable_enum` be `"action_list(" concatenated with typeIdtable concatenated with ")".`
 - e. Let `idlabel` be `typeIdtable_enum` concatenated with `."` concatenated with `nameIRlabel`.
 - f. Let `_ typeIRlabel ctklabel value?` be `! the type of variable idlabel in the LOCAL layer of TC`.
 - g. Check that `value?` is defined.
 - h. Let `valuelabel` be `value?`.
 - i. Check that `valuelabel` is equal to `TABLE_ENUM typeIdtable_enum . nameIRlabel`.
 - j. Let `typedExpressionIRlabel` be `nameIRlabel` annotated with type `typeIRlabel` and compile-time knownness `ctklabel`.
 - k. Result in `typedExpressionIRlabel`.

`SwitchCase_table_ok`: TC f l typeId_{table} b_{last} |- switchCase : % % # %

TC f l typeId_{table} b_{last} |- switchCase : % % # %:

1. If let `switchLabel : blockStatement` be `switchCase`:
 - a. Let `TC typeIdtable blast |- switchLabel : switchLabelIR`.
 - b. Let the typing context `TCpost`, abstract control flow `fpost`, and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context `TC`, abstract control flow `f`, and loop context `l`.
 - c. Let `switchCaseIR` be `switchLabelIR : blockStatementIR`.
 - d. Result in `fpost`, `switchCaseIR`, and `switchLabel`.
2. Else:
 - a. Let `switchLabel : be switchCase`.

- b. Let `TC typeIdtable blast |- switchLabel : switchLabelIR`.
- c. Let `switchCaseIR` be `switchLabelIR` .
- d. Result in `f`, `switchCaseIR`, and `switchLabel`.

Typing `SWITCH ( expressionswitch ) { switchCaseList }`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `typedExpressionIRswitch` be
 - the result of typing `expressionswitch` under cursor `LOCAL` and typing context `TC`.
2. Let `typeIRswitch` be the type of `typedExpressionIRswitch`.
3. Let `typeIR` be `typeIRswitch` with typedefs unrolled.
4. Check that `typeIR` has type `tableMetadataEnumTypeIR`.
5. Let `TABLE_ENUM typeIdtable_enum { _ }*` be `typeIR`.
6. Let `text` be the text `typeIdtable_enum` with the suffix `"")` removed.
7. Let `typeIdtable` be the text `text` with the prefix `"action_list("` removed.
8. Let `TC f l typeIdtable |- switchCaseList : fpost switchCaseIR* # switchLabel*`.
9. Check that the elements of `switchLabel*` are distinct.
10. Let `switchStatementIR` be `SWITCH ( typedExpressionIRswitch ) { switchCaseIR* }`.
11. Result in
 - the typing context `TC`,
 - the abstract control flow `fpost`,
 - and the typed statement `switchStatementIR`.

12.12.3. Switch statement on numeric or enumerated type

For this variant of `switch` statement, the expression must evaluate to a result with one of these types:

- `bit<W>`
- `int<W>`
- `enum`, either with or without an underlying representation specified
- `error`

All switch labels must be expressions with compile-time known values, and must have a type that can be implicitly cast to the type of the `switch` expression (see [\[sec-implicit-casts\]](#)). Switch labels must not begin with a left brace character `{`, to avoid ambiguity with a block statement.

```
// Assume the expression hdr.ethernet.etherType has type bit<16>.
switch (hdr.ethernet.etherType) {
  0x86dd: { /* body omitted */ }
  0x0800: { /* fall-through to the next body */ }
  0x0802: { /* body omitted */ }
  0xcafe: { /* body omitted */ }
  default: { /* body omitted */ }
}
```

SwitchLabel_general_ok: TC typeIR bool |- switchLabel : %

TC typeIR bool |- switchLabel : %:

1. If bool is equal to true:
 - a. Check that switchLabel is DEFAULT.
 - b. Result in DEFAULT.
2. If let expressionNonBrace_{label} be switchLabel:
 - a. Let expression_{label} be the expression corresponding to expressionNonBrace_{label}.
 - b. Let typedExpressionIR_{label} be
 - the result of typing expression_{label}, under cursor LOCAL and typing context TC.
 - c. Let typedExpressionIR_{label_cast} be ! typedExpressionIR_{label} implicitly cast to typeIR.
 - d. Let type _ and compile-time known-ness ctk be the note of typedExpressionIR_{label_cast}.
 - e. Check that ctk is LCTK.
 - f. Result in typedExpressionIR_{label_cast}.

SwitchCase_general_ok: TC f I typeIR_{switch} b_{last} |- switchCase : % % # %

TC f I typeIR_{switch} b_{last} |- switchCase : % % # %:

1. If let switchLabel : blockStatement be switchCase:
 - a. Let TC typeIR_{switch} b_{last} |- switchLabel : switchLabelIR.
 - b. Let the typing context TC_{post}, abstract control flow f_{post}, and blockStatementIR be
 - the result of typing blockStatement, under typing context TC, abstract control flow f, and loop context I.
 - c. Let switchCaseIR be switchLabelIR : blockStatementIR.
 - d. Result in f_{post}, switchCaseIR, and switchLabel.
2. Else:
 - a. Let switchLabel : be switchCase.
 - b. Let TC typeIR_{switch} b_{last} |- switchLabel : switchLabelIR.
 - c. Let switchCaseIR be switchLabelIR :.
 - d. Result in f, switchCaseIR, and switchLabel.

Typing SWITCH (expression_{switch}) { switchCaseList }, under cursor LOCAL, typing context TC, abstract control flow f, and loop context I:

1. Let typedExpressionIR_{switch} be
 - the result of typing expression_{switch}, under cursor LOCAL and typing context TC.

2. Let $\text{typeIR}_{\text{switch}}$ be the type of $\text{typedExpressionIR}_{\text{switch}}$.
3. Check that $\text{compat_switch}(\text{typeIR}_{\text{switch}})$.
4. Let $\text{TC} \vdash \text{typeIR}_{\text{switch}} \vdash \text{switchCaseList} : f_{\text{post}} \text{switchCaseIR}^* \# \text{switchLabel}^*$.
5. Check that the elements of switchLabel^* are distinct.
6. Let switchStatementIR be $\text{SWITCH} (\text{typedExpressionIR}_{\text{switch}}) \{ \text{switchCaseIR}^* \}$.
7. Result in
 - the typing context TC ,
 - the abstract control flow f_{post} ,
 - and the typed statement switchStatementIR .

Compile-time evaluation

$\text{SwitchCase_inst} : p \text{ IC store } \vdash \text{switchCaseIR} : \% \%$

$p \text{ IC store } \vdash \text{switchCaseIR} : \% \%$:

1. If let $\text{switchLabelIR} : \text{blockStatementIR}$ be switchCaseIR :
 - a. Let the instantiation context $\text{IC}_{\text{switch}}$, the store STO_i , and statementIR be
 - the result of compile-time evaluation of blockStatementIR , under cursor p , instantiation context IC , and store store .
 - b. Check that statementIR has type blockStatementIR .
 - c. Let $\text{blockStatementIR}_{\text{inst}}$ be statementIR .
 - d. Result in STO_i and $\text{switchLabelIR} : \text{blockStatementIR}_{\text{inst}}$.
2. Else:
 - a. Let $\text{switchLabelIR} : \text{switchCaseIR}$.
 - b. Result in store and $\text{switchLabelIR} :$.

Compile-time evaluation of $\text{SWITCH} (\text{typedExpressionIR}) \{ \text{switchCaseListIR} \}$, under cursor p , instantiation context IC , and store STO :

1. Let $p \text{ IC STO } \vdash \text{switchCaseListIR} : \text{STO}_i \text{switchCaseListIR}_{\text{inst}}$.
2. Result in
 - the instantiation context IC ,
 - the store STO_i ,
 - and $\text{SWITCH} (\text{typedExpressionIR}) \{ \text{switchCaseListIR}_{\text{inst}} \}$.

13. Expressions

```
expression
: literalExpression
| referenceExpression
| defaultExpression
| unaryExpression
| binaryExpression
| ternaryExpression
| castExpression
| dataExpression
| accessExpression
| callExpression
| parenthesizedExpression
;
```

```
expressionIR
: literalExpressionIR
| referenceExpressionIR
| defaultExpressionIR
| unaryExpressionIR
| binaryExpressionIR
| ternaryExpressionIR
| castExpressionIR
| dataExpressionIR
| accessExpressionIR
| callExpressionIR
| parenthesizedExpressionIR
;

expressionNoteIR
: `( typeIR ctk )
;

typedExpressionIR
: expressionIR # expressionNoteIR
;
```

Type checking

Expr_ok:

- Typing **expression**, under cursor **cursor** and typing context **typingContext**:
- Results in **typedExpressionIR**.

The type of <<typedExpressionIR, _ annotated with type **typeIR** and compile-time known-ness **_>>**

1. Return **typeIR**.

Local compile-time evaluation

Expr_eval_lctx:

- Local compile-time evaluation of **typedExpressionIR**, under cursor **cursor** and typing context

typingContext:

- Results in **value**.

Compile-time evaluation

Expr_inst: p IC STO |- expressionIR # expressionNoteIR : % %

Dynamic evaluation

Expr_eval:

- Evaluating **typedExpressionIR**, under cursor **cursor**, evaluation context **evalContext**, and store **store**:
- Results in the updated evaluation context **evalContext**, store **store** and expression result **expressionResult**.

13.1. Literal expressions

```
literalExpression
: booleanLiteral
| integerLiteral
| stringLiteral
;

literalExpressionIR = literalExpression
```

13.1.1. Boolean literal expressions

```
booleanLiteral
: TRUE
| FALSE
;
```

Type checking

▮ *Click to view the specification source*

```
rulegroup Expr_ok/literalExpression-boolean:
  rule Expr_ok/true:
    p TC |- TRUE : TRUE # expressionNoteIR
    -- if expressionNoteIR = `(BOOL LCTK)
    rule Expr_ok/false:
      p TC |- FALSE : FALSE # expressionNoteIR
      -- if expressionNoteIR = `(BOOL LCTK)
```

Typing **booleanLiteral**, under cursor **p** and typing context **TC**:

1. If **booleanLiteral** is **TRUE**:
 - a. Let **expressionNoteIR** be **type BOOL and compile-time known-ness LCTK**.

b. Result in **TRUE** annotated with `expressionNotelR`.

2. Else:

a. Let `expressionNotelR` be **type** **BOOL** and **compile-time known-ness** **LCTK**.

b. Result in **FALSE** annotated with `expressionNotelR`.

Local compile-time evaluation

▮ *Click to view the specification source*

```
rulegroup Expr_eval_lctk/literalExpressionIR-boolean:
  rule Expr_eval_lctk/true:
    p TC |- TRUE # _ ~> `B true
  rule Expr_eval_lctk/false:
    p TC |- FALSE # _ ~> `B false
```

Local compile-time evaluation of `<<typedExpressionIR, booleanLiteral annotated with expressionNotelR`, under cursor `p` and typing context `TC>>`:

1. If `booleanLiteral` is **TRUE**:

a. Result in ``B true`.

2. Else:

a. Result in ``B false`.

Compile-time evaluation

▮ *Click to view the specification source*

```
rulegroup Expr_inst/literalExpressionIR-boolean:
  rule Expr_inst/true:
    p IC STO |- TRUE # _ : STO (`B true)
  rule Expr_inst/false:
    p IC STO |- FALSE # _ : STO (`B false)
```

`p IC STO |- booleanLiteral # expressionNotelR : % %:`

1. If `booleanLiteral` is **TRUE**:

a. Result in **STO** and ``B true`.

2. Else:

a. Result in **STO** and ``B false`.

13.1.2. Numeric literal expressions

```
integerLiteral
: D int
| nat W int
| nat S int
;
```

The types of integer literals are as follows:

- An integer with no type prefix has type `int`.
- A non-negative integer prefixed with an integer width `W` and the character `w` has type `bit<W>`.
- An integer prefixed with an integer width `W` and the character `s` has type `int<W>`.

The table below shows several examples of integer literals and their types. For additional examples of literals see [\[sec-literals\]](#).

Literal	Interpretation
<code>10</code>	Type is <code>int</code> , value is 10
<code>8w10</code>	Type is <code>bit<8></code> , value is 10
<code>8s10</code>	Type is <code>int<8></code> , value is 10
<code>2s3</code>	Type is <code>int<2></code> , value is -1 (last 2 bits), overflow warning
<code>1w10</code>	Type is <code>bit<1></code> , value is 0 (last bit), overflow warning
<code>1s1</code>	Type is <code>int<1></code> , value is -1, overflow warning

Type checking

Typing `integerLiteral`, under cursor `p` and typing context `TC`:

1. If let `D i` be `integerLiteral`:
 - a. Let `expressionNoteIR` be `type INT and compile-time known-ness LCTK`.
 - b. Result in `D i` `annotated with` `expressionNoteIR`.
2. Else if let `n S i` be `integerLiteral`:
 - a. Let `expressionNoteIR` be `type INT < n > and compile-time known-ness LCTK`.
 - b. Result in `n S i` `annotated with` `expressionNoteIR`.
3. Else:
 - a. Let `n W i` be `integerLiteral`.
 - b. Let `expressionNoteIR` be `type BIT < n > and compile-time known-ness LCTK`.
 - c. Result in `n W i` `annotated with` `expressionNoteIR`.

Local compile-time evaluation

Local compile-time evaluation of `<<typedExpressionIR, integerLiteral annotated with expressionNoteIR, under cursor p and typing context TC>>`:

1. If let `D i` be `integerLiteral`:
 - a. Result in `D i`.
2. Else if let `n W i` be `integerLiteral`:
 - a. Result in `n W i`.
3. Else:
 - a. Let `n S i` be `integerLiteral`.

- b. Result in $n\ S\ i$.

Compile-time evaluation

$p\ IC\ STO \mid -\ integerLiteral\ \# \ expressionNoteIR : \% \%$:

1. If let $D\ i$ be $integerLiteral$:
 - a. Result in STO and $D\ i$.
2. Else if let $n\ W\ i$ be $integerLiteral$:
 - a. Result in STO and $n\ W\ i$.
3. Else:
 - a. Let $n\ S\ i$ be $integerLiteral$.
 - b. Result in STO and $n\ S\ i$.

13.1.3. String literal expressions

```
stringLiteral
: " text "
;
```

Type checking

Typing `" text "`, under cursor p and typing context TC :

1. Let $expressionNoteIR$ be **type** $STRING$ and **compile-time known-ness** $LCTK$.
2. Result in `" text "` annotated with $expressionNoteIR$.

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle typedExpressionIR, \text{" text " annotated with } expressionNoteIR, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Result in `" text "`.

Compile-time evaluation

$p\ IC\ STO \mid -\ \text{" text " } \# \ expressionNoteIR : \% \%$:

1. Result in STO and `" text "`.

13.2. Reference expressions

```
referenceExpression
: prefixedNonTypeName
| THIS
;

referenceExpressionIR = prefixedNameIR
```

Type checking

Typing `expression`, under cursor `p` and typing context `TC`:

1. If let `prefixedNonTypeName` be `expression`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
 - b. Let `_typeIR ctk _?` be ! the type of variable `prefixedNameIR` in the `p` layer of `TC`.
 - c. Let `expressionNoteIR` be type `typeIR` and compile-time known-ness `ctk`.
 - d. Result in `prefixedNameIR` annotated with `expressionNoteIR`.
2. If `expression` is equal to `THIS`:
 - a. Let `prefixedNameIR` be "this".
 - b. Let `_typeIR ctk _?` be ! the type of variable `prefixedNameIR` in the `p` layer of `TC`.
 - c. Let `expressionNoteIR` be type `typeIR` and compile-time known-ness `ctk`.
 - d. Result in `prefixedNameIR` annotated with `expressionNoteIR`.

Local compile-time evaluation

Local compile-time evaluation of `<<typedExpressionIR, prefixedNameIR annotated with expressionNoteIR, under cursor p and typing context TC>>`:

1. Let `value` be the value of variable `prefixedNameIR` in the `p` layer of `TC`.
2. Result in `value`.

Compile-time evaluation

`p IC STO | - prefixedNameIR # expressionNoteIR : % %:`

1. Let `value` be `find_var_i(p, IC, prefixedNameIR)`.
2. Result in `STO` and `value`.

13.3. Default expressions

```
defaultExpression
```

```
: ...  
;
```

```
defaultExpressionIR = defaultExpression
```

Type checking

Typing ..., under cursor p and typing context TC :

1. Let $expressionNoteIR$ be type $DEFAULT$ and compile-time known-ness $LCTK$.
2. Result in ... annotated with $expressionNoteIR$.

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle typedExpressionIR, \dots \text{ annotated with } expressionNoteIR, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Result in $DEFAULT$.

Compile-time evaluation

$p \text{ IC STO} \mid - \dots \# expressionNoteIR : \% \%$:

1. Result in STO and $DEFAULT$.

13.4. Unary expressions

```
unop  
: !  
| ~  
| -  
| +  
;  
  
unaryExpression  
: unop expression  
;  
  
unaryExpressionIR  
: unop typedExpressionIR  
;
```

13.4.1. Negation operators

Type checking

Typing $! expression'$, under cursor p and typing context TC :

1. Let typedExpressionIR be
 - the result of typing expression' , under cursor p and typing context TC .
2. Let $\text{typedExpressionIR}_{\text{reduced}}$ be ! the result of reducing serializable enums in typedExpressionIR until $\$compat_lnot$ is satisfied.
3. Let $\text{typeIR}_{\text{reduced}}$ be the type of $\text{typedExpressionIR}_{\text{reduced}}$.
4. Let $\text{ctk}_{\text{reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{reduced}}$.
5. Let expressionNoteIR be type $\text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
6. Result in ! $\text{typedExpressionIR}_{\text{reduced}}$ annotated with expressionNoteIR .

13.4.2. Bitwise complement operators

Type checking

Typing $\sim \text{expression'}$, under cursor p and typing context TC :

1. Let typedExpressionIR be
 - the result of typing expression' , under cursor p and typing context TC .
2. Let $\text{typedExpressionIR}_{\text{reduced}}$ be ! the result of reducing serializable enums in typedExpressionIR until $\$compat_bnot$ is satisfied.
3. Let $\text{typeIR}_{\text{reduced}}$ be the type of $\text{typedExpressionIR}_{\text{reduced}}$.
4. Let $\text{ctk}_{\text{reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{reduced}}$.
5. Let expressionNoteIR be type $\text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
6. Result in $\sim \text{typedExpressionIR}_{\text{reduced}}$ annotated with expressionNoteIR .

13.4.3. Plus and minus operators

Type checking

Typing $\text{unop } \text{expression'}$, under cursor p and typing context TC :

1. Check that $\text{unop} = +$ or $\text{unop} = -$.
2. Let typedExpressionIR be
 - the result of typing expression' , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_{\text{reduced}}$ be ! the result of reducing serializable enums in typedExpressionIR until $\$compat_uplusminus$ is satisfied.
4. Let $\text{typeIR}_{\text{reduced}}$ be the type of $\text{typedExpressionIR}_{\text{reduced}}$.
5. Let $\text{ctk}_{\text{reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{reduced}}$.
6. Let expressionNoteIR be type $\text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
7. Result in $\text{unop } \text{typedExpressionIR}_{\text{reduced}}$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{unop typedExpressionIR} \text{ annotated with expressionNoteIR}, \text{under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
2. Result in unop value .

Compile-time evaluation

$p \text{ IC STO} \mid - \text{unop typedExpressionIR} \# \text{expressionNoteIR} : \% \%$:

1. Let $p \text{ IC STO} \mid - \text{typedExpressionIR} : \text{STO}_1 \text{ value}$.
2. Result in STO_1 and unop value .

13.5. Binary expressions

```
binop
: *
| /
| %
| +
| -
| |+|
| |-|
| <<
| >>
| <=
| >=
| <
| >
| !=
| ==
| &
| ^
| |
| ++
| &&
| ||
;

binaryExpression
: expression binop expression
;

binaryExpressionIR
: typedExpressionIR binop typedExpressionIR
;
```

13.5.1. Plus, minus, and multiplication operators

Type checking

Typing expression_l binop expression_r, under cursor p and typing context TC:

1. Check that binop is in [+ , - , *].
2. Let typedExpressionIR_l be
 - the result of typing expression_l, under cursor p and typing context TC.
3. Let typedExpressionIR_r be
 - the result of typing expression_r, under cursor p and typing context TC.
4. Let (typedExpressionIR_{l_castr} typedExpressionIR_{r_castr}) be ! typedExpressionIR_l and typedExpressionIR_r implicitly cast to equal types.
5. Let (typedExpressionIR_{l_reduced} typedExpressionIR_{r_reduced}) be ! reduce_serenum_binary(typedExpressionIR_{l_castr} typedExpressionIR_{r_castr} compat_plusminmult).
6. Let typeIR_{reduced} be the type of typedExpressionIR_{l_reduced}.
7. Let ctk_{l_reduced} be the compile-time known-ness of typedExpressionIR_{l_reduced}.
8. Let ctk_{r_reduced} be the compile-time known-ness of typedExpressionIR_{r_reduced}.
9. Let ctk_{reduced} be ctk_{l_reduced} and ctk_{r_reduced} joined.
10. Let expressionNoteIR be type typeIR_{reduced} and compile-time known-ness ctk_{reduced}.
11. Result in typedExpressionIR_{l_reduced} binop typedExpressionIR_{r_reduced} annotated with expressionNoteIR.

13.5.2. Saturating plus and minus operators

Type checking

Typing expression_l binop expression_r, under cursor p and typing context TC:

1. Check that binop is in [|+| , |-|].
2. Let typedExpressionIR_l be
 - the result of typing expression_l, under cursor p and typing context TC.
3. Let typedExpressionIR_r be
 - the result of typing expression_r, under cursor p and typing context TC.
4. Let (typedExpressionIR_{l_castr} typedExpressionIR_{r_castr}) be ! typedExpressionIR_l and typedExpressionIR_r implicitly cast to equal types.
5. Let (typedExpressionIR_{l_reduced} typedExpressionIR_{r_reduced}) be ! reduce_serenum_binary(typedExpressionIR_{l_castr} typedExpressionIR_{r_castr} compat_satplusminus).
6. Let typeIR_{reduced} be the type of typedExpressionIR_{l_reduced}.
7. Let ctk_{l_reduced} be the compile-time known-ness of typedExpressionIR_{l_reduced}.
8. Let ctk_{r_reduced} be the compile-time known-ness of typedExpressionIR_{r_reduced}.
9. Let ctk_{reduced} be ctk_{l_reduced} and ctk_{r_reduced} joined.

10. Let expressionNoteIR be $\text{type } \text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
11. Result in $\text{typedExpressionIR}_{\text{l_reduced}}$ binop $\text{typedExpressionIR}_{\text{r_reduced}}$ annotated with expressionNoteIR .

13.5.3. Division and modulo operators

Type checking

Typing $\text{expression}_\text{l}$ binop $\text{expression}_\text{r}$, under cursor p and typing context TC :

1. Check that binop is in $[/, \%]$.
2. Let $\text{typedExpressionIR}_\text{l}$ be
 - the result of typing $\text{expression}_\text{l}$, under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_\text{r}$ be
 - the result of typing $\text{expression}_\text{r}$, under cursor p and typing context TC .
4. Let $(\text{typedExpressionIR}_{\text{l_cast}}, \text{typedExpressionIR}_{\text{r_cast}})$ be ! $\text{typedExpressionIR}_\text{l}$ and $\text{typedExpressionIR}_\text{r}$ implicitly cast to equal types.
5. Let $(\text{typedExpressionIR}_{\text{l_reduced}}, \text{typedExpressionIR}_{\text{r_reduced}})$ be ! $\text{reduce_serenum_binary}(\text{typedExpressionIR}_{\text{l_cast}}, \text{typedExpressionIR}_{\text{r_cast}}, \text{compat_divmod})$.
6. Let $\text{typeIR}_{\text{reduced}}$ be the type of $\text{typedExpressionIR}_{\text{l_reduced}}$.
7. Let $\text{ctk}_{\text{l_reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{l_reduced}}$.
8. Let $\text{ctk}_{\text{r_reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{r_reduced}}$.
9. If $\text{ctk}_{\text{r_reduced}}$ is LCTK :
 - a. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{r_reduced}}$, under cursor p and typing context TC .
 - b. Check that value has type integerValue .
 - c. Let $\text{integerValue}_\text{r}$ be value .
 - d. Let int be the integer representation of $\text{integerValue}_\text{r}$.
 - e. Check that int has type nat .
 - f. Let n_r be int .
 - g. Check that n_r is greater than 0.
 - h. Let $\text{ctk}_{\text{reduced}}$ be $\text{ctk}_{\text{l_reduced}}$ and $\text{ctk}_{\text{r_reduced}}$ joined.
 - i. Let expressionNoteIR be $\text{type } \text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
 - j. Result in $\text{typedExpressionIR}_{\text{l_reduced}}$ binop $\text{typedExpressionIR}_{\text{r_reduced}}$ annotated with expressionNoteIR .
10. Else:
 - a. Let $\text{ctk}_{\text{reduced}}$ be $\text{ctk}_{\text{l_reduced}}$ and $\text{ctk}_{\text{r_reduced}}$ joined.
 - b. Let expressionNoteIR be $\text{type } \text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
 - c. Result in $\text{typedExpressionIR}_{\text{l_reduced}}$ binop $\text{typedExpressionIR}_{\text{r_reduced}}$ annotated with expressionNoteIR .

13.5.4. Shift left and right operators

Type checking

Typing expression_i binop expression_r , under cursor p and typing context TC :

1. Check that binop is in $[<<, >>]$.
2. Let $\text{typedExpressionIR}_i$ be
 - the result of typing expression_i , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be ! $\text{reduce_serenum_binary}(\text{typedExpressionIR}_i, \text{typedExpressionIR}_r, \text{compat_shift})$.
5. Let $\text{typeIR}_{l_reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
6. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{typeIR}_{r_reduced}$ be the type of $\text{typedExpressionIR}_{r_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. If $\text{typeIR}_{r_reduced}$ is fixed-width bit type:
 - a. Check that $\text{is_arbitraryIntTypeIR}(\text{typeIR}_{l_reduced})$ implies $\text{ctk}_{r_reduced} = \text{LCTK}$.
 - b. Let $\text{ctk}_{reduced}$ be $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$ joined.
 - c. Let expressionNoteIR be type $\text{typeIR}_{l_reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
 - d. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .
10. If $\text{ctk}_{r_reduced}$ is LCTK:
 - a. Check that $\text{is_arbitraryIntTypeIR}(\text{typeIR}_{r_reduced})$ or $\text{is_fixedIntTypeIR}(\text{typeIR}_{r_reduced})$.
 - b. Let $\text{ctk}_{reduced}$ be $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$ joined.
 - c. Let expressionNoteIR be type $\text{typeIR}_{l_reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
 - d. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

13.5.5. Equality and inequality operators

Type checking

Typing expression_i binop expression_r , under cursor p and typing context TC :

1. Check that binop is in $[=, !=]$.
2. Let $\text{typedExpressionIR}_i$ be
 - the result of typing expression_i , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let $(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast})$ be ! $\text{typedExpressionIR}_i$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
5. Let typeIR_{cast} be the type of $\text{typedExpressionIR}_{l_cast}$.

6. Let ctk_{l_cast} be the compile-time known-ness of $typedExpressionIR_{l_cast}$.
7. Let ctk_{r_cast} be the compile-time known-ness of $typedExpressionIR_{r_cast}$.
8. Check that $typeIR_{cast}$ supports equality comparison.
9. Let ctk_{cast} be ctk_{l_cast} and ctk_{r_cast} joined.
10. Let $expressionNoteIR$ be type `BOOL` and compile-time known-ness ctk_{cast} .
11. Result in $typedExpressionIR_{l_cast}$ binop $typedExpressionIR_{r_cast}$ annotated with $expressionNoteIR$.

13.5.6. Comparison operators

Type checking

Typing $expression_l$ binop $expression_r$, under cursor p and typing context TC :

1. Check that $binop$ is in $[, >=, <, >]$.
2. Let $typedExpressionIR_l$ be
 - the result of typing $expression_l$, under cursor p and typing context TC .
3. Let $typedExpressionIR_r$ be
 - the result of typing $expression_r$, under cursor p and typing context TC .
4. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be ! $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
5. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be ! $reduce_serenum_binary(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast}, compat_compare)$.
6. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
7. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
8. Let $ctk_{reduced}$ be $ctk_{l_reduced}$ and $ctk_{r_reduced}$ joined.
9. Let $expressionNoteIR$ be type `BOOL` and compile-time known-ness $ctk_{reduced}$.
10. Result in $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with $expressionNoteIR$.

13.5.7. Bitwise and, xor, and or operators

Type checking

Typing $expression_l$ binop $expression_r$, under cursor p and typing context TC :

1. Check that $binop$ is in $[\&, \wedge, |]$.
2. Let $typedExpressionIR_l$ be
 - the result of typing $expression_l$, under cursor p and typing context TC .
3. Let $typedExpressionIR_r$ be
 - the result of typing $expression_r$, under cursor p and typing context TC .
4. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be ! $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.

5. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be ! $\text{reduce_serenum_binary}(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast}, \text{compat_bitwise})$.
6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. Let $\text{ctk}_{reduced}$ be $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$ joined.
10. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
11. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

13.5.8. Concatenation operators

Type checking

Typing $\text{expression}_l ++ \text{expression}_r$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
2. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
3. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be ! $\text{reduce_serenum_binary}(\text{typedExpressionIR}_l, \text{typedExpressionIR}_r, \text{compat_concat})$.
4. Let $\text{typeIR}_{l_reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
5. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
6. Let $\text{typeIR}_{r_reduced}$ be the type of $\text{typedExpressionIR}_{r_reduced}$.
7. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
8. Check that $\text{typeIR}_{l_reduced} = \text{STRING} \wedge \text{typeIR}_{r_reduced} = \text{STRING}$ implies $\text{ctk}_{l_reduced} = \text{LCTK} \wedge \text{ctk}_{r_reduced} = \text{LCTK}$.
9. Let $\text{typeIR}_{reduced}$ be $\text{result_concat}(\text{typeIR}_{l_reduced}, \text{typeIR}_{r_reduced})$.
10. Let $\text{ctk}_{reduced}$ be $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$ joined.
11. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
12. Result in $\text{typedExpressionIR}_{l_reduced} ++ \text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

13.5.9. Logical and, and or operator

Type checking

Typing $\text{expression}_l \text{ binop } \text{expression}_r$, under cursor p and typing context TC :

1. Check that binop is in $[\&\&, ||]$.
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be

- the result of **typing** expression_r , under cursor p and typing context TC .
4. Let $(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast})$ be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ **implicitly cast to equal types**.
 5. Let $(\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced})$ be ! $\text{reduce_serenum_binary}(\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast}, \text{compat_logical})$.
 6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
 7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
 8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
 9. Let $\text{ctk}_{reduced}$ be $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$ joined.
 10. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
 11. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{typedExpressionIR}_l \text{ binop } \text{typedExpressionIR}_r, \text{annotated with } \text{expressionNoteIR}, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. If binop is not in $[\&\&, | |]$:
 - a. Let value_l be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_l$, under cursor p and typing context TC .
 - b. Let value_r be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_r$, under cursor p and typing context TC .
 - c. Result in $\text{value}_l \text{ binop } \text{value}_r$.
2. If binop is $\&\&$:
 - a. Let value be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_l$, under cursor p and typing context TC .
 - b. If value is equal to ``B false`:
 - i. Result in ``B false`.
 - c. If value is equal to ``B true`:
 - i. Let value_r be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_r$, under cursor p and typing context TC .
 - ii. Result in value_r .
3. Else if binop is $| |$:
 - a. Let value be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_l$, under cursor p and typing context TC .
 - b. If value is equal to ``B true`:

- i. Result in ``B true`.
- c. If `value` is equal to ``B false`:
 - i. Let `valuer` be
 - the result of **local compile-time evaluation of** `typedExpressionIRr`, **under cursor** `p` **and** **typing context** `TC`.
 - ii. Result in `valuer`.

Compile-time evaluation

`p IC STO` |- `typedExpressionIRl binop typedExpressionIRr # expressionNotelR : % %:`

1. If `binop` is not in `[&&, ||]`:
 - a. Let `p IC STO` |- `typedExpressionIRl : STO1 valuel`.
 - b. Let `p IC STO1` |- `typedExpressionIRr : STO2 valuer`.
 - c. Result in `STO2` and `valuel binop valuer`.
2. If `binop` is `&&`:
 - a. Let `p IC STO` |- `typedExpressionIRl : STO1 value`.
 - b. If `value` is equal to ``B false`:
 - i. Result in `STO1` and ``B false`.
 - c. If `value` is equal to ``B true`:
 - i. Let `p IC STO1` |- `typedExpressionIRr : STO2 valuer`.
 - ii. Result in `STO2` and `valuer`.
3. Else if `binop` is `||`:
 - a. Let `p IC STO` |- `typedExpressionIRl : STO1 value`.
 - b. If `value` is equal to ``B true`:
 - i. Result in `STO1` and ``B true`.
 - c. If `value` is equal to ``B false`:
 - i. Let `p IC STO1` |- `typedExpressionIRr : STO2 valuer`.
 - ii. Result in `STO2` and `valuer`.

13.6. Ternary expressions

```
ternaryExpression
: expression ? expression : expression
;

ternaryExpressionIR
: typedExpressionIR ? typedExpressionIR : typedExpressionIR
;
```

Type checking

Typing $\text{expression}_{\text{cond}} ? \text{expression}_{\text{true}} : \text{expression}_{\text{false}}$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_{\text{cond}}$ be
 - the result of typing $\text{expression}_{\text{cond}}$, under cursor p and typing context TC .
2. Let typeIR and compile-time known-ness ctk_{cond} be the note of $\text{typedExpressionIR}_{\text{cond}}$.
3. Check that typeIR has type boolTypeIR .
4. Let boolTypeIR be typeIR .
5. Let $\text{typedExpressionIR}_{\text{true}}$ be
 - the result of typing $\text{expression}_{\text{true}}$, under cursor p and typing context TC .
6. Let $\text{typedExpressionIR}_{\text{false}}$ be
 - the result of typing $\text{expression}_{\text{false}}$, under cursor p and typing context TC .
7. Let $(\text{typedExpressionIR}_{\text{true_cast}}, \text{typedExpressionIR}_{\text{false_cast}})$ be ! $\text{typedExpressionIR}_{\text{true}}$ and $\text{typedExpressionIR}_{\text{false}}$ implicitly cast to equal types.
8. Let $\text{typeIR}_{\text{cast}}$ be the type of $\text{typedExpressionIR}_{\text{true_cast}}$.
9. Let $\text{ctk}_{\text{true_cast}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{true_cast}}$.
10. Let $\text{ctk}_{\text{false_cast}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{false_cast}}$.
11. Check that $\text{is_arbitraryIntTypeIR}(\text{typeIR}_{\text{cast}})$ implies $\text{ctk}_{\text{cond}} \neq \text{DYN}$.
12. Let ctk be $[\text{ctk}_{\text{cond}}, \text{ctk}_{\text{true_cast}}, \text{ctk}_{\text{false_cast}}]$ joined.
13. Let expressionNoteIR be $\text{typeIR}_{\text{cast}}$ and compile-time known-ness ctk .
14. Result in $\text{typedExpressionIR}_{\text{cond}} ? \text{typedExpressionIR}_{\text{true_cast}} : \text{typedExpressionIR}_{\text{false_cast}}$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{typedExpressionIR}_{\text{cond}} ? \text{typedExpressionIR}_{\text{true}} : \text{typedExpressionIR}_{\text{false}}$ annotated with expressionNoteIR , under cursor p and typing context $TC \rangle\rangle$:

1. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{cond}}$, under cursor p and typing context TC .
2. If value is equal to `B true`:
 - a. Let $\text{value}_{\text{true}}$ be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{true}}$, under cursor p and typing context TC .
 - b. Result in $\text{value}_{\text{true}}$.
3. If value is equal to `B false`:
 - a. Let $\text{value}_{\text{false}}$ be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{false}}$, under cursor p and typing context TC .

b. Result in $\text{value}_{\text{false}}$.

Compile-time evaluation

$p \text{ IC } \text{STO} \mid - \text{typedExpressionIR}_{\text{cond}} ? \text{typedExpressionIR}_{\text{true}} : \text{typedExpressionIR}_{\text{false}} \# \text{expressionNoteIR} : \% \%$:

1. Let $p \text{ IC } \text{STO} \mid - \text{typedExpressionIR}_{\text{cond}} : \text{STO}_1 \text{ value}$.
2. If value is equal to 'B true' :
 - a. Let $p \text{ IC } \text{STO}_1 \mid - \text{typedExpressionIR}_{\text{true}} : \text{STO}_2 \text{ value}_{\text{true}}$.
 - b. Result in STO_2 and $\text{value}_{\text{true}}$.
3. If value is equal to 'B false' :
 - a. Let $p \text{ IC } \text{STO}_1 \mid - \text{typedExpressionIR}_{\text{false}} : \text{STO}_2 \text{ value}_{\text{false}}$.
 - b. Result in STO_2 and $\text{value}_{\text{false}}$.

13.7. Cast expressions

```
castExpression
: `( type ) expression
;

castExpressionIR
: `( typeIR ) typedExpressionIR
;
```

Type checking

Typing (type_t) expression' , under cursor p and typing context TC :

1. Let typeIR_t and a set of fresh type variables typed^* be
 - the result of **typing** type_t , under cursor p and typing context TC .
2. Check that typed^* is an empty list.
3. Let bound be **bound type variables** from the p layer of TC .
4. Check that typeIR_t is a well-formed type, with bound type variables bound .
5. Let typedExpressionIR be
 - the result of **typing** expression' , under cursor p and typing context TC .
6. Let typeIR be **the type of** typedExpressionIR .
7. Let ctk be **the compile-time known-ness of** typedExpressionIR .
8. Check that typeIR can be explicitly cast to typeIR_t .
9. Let expressionNoteIR be **type** typeIR_t and **compile-time known-ness** ctk .
10. Result in (typeIR_t) typedExpressionIR **annotated with** expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\ll \text{typedExpressionIR}, (\text{typeIR}) \text{typedExpressionIR} \text{ annotated with expressionNoteIR}, \text{ under cursor } p \text{ and typing context } TC \gg$:

1. Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
2. Let $\text{value}_{\text{cast}}$ be cast value to type typeIR .
3. Result in $\text{value}_{\text{cast}}$.

Compile-time evaluation

$p \text{ IC STO } |- (\text{typeIR}) \text{typedExpressionIR} \# \text{expressionNoteIR} : \% \%$:

1. Let $p \text{ IC STO } |- \text{typedExpressionIR} : \text{STO}_1 \text{ value}$.
2. Let $\text{value}_{\text{cast}}$ be cast value to type typeIR .
3. Result in STO_1 and $\text{value}_{\text{cast}}$.

13.8. Data expressions

```
dataElementExpression
: sequenceElementExpression
| recordElementExpression
;

dataExpression
: {#}
| `{ dataElementExpression trailingCommaOpt }
;

dataExpressionIR
: {#}
| SEQ `{ typedExpressionListIR }
| SEQ `{ typedExpressionListIR , ... }
| RECORD `{ namedExpressionListIR }
| RECORD `{ namedExpressionListIR , ... }
;
```

13.8.1. Invalid header expressions

Type checking

Typing $\{ \# \}$, under cursor p and typing context TC :

1. Let expressionNoteIR be type HEADER_INVALID and compile-time known-ness LCTK .
2. Result in $\{ \# \}$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \{\#\} \text{ annotated with type } \text{typeIR} \text{ and compile-time known-ness } \text{ctk}, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Result in $\{\#\}$.

Compile-time evaluation

$p \text{ IC STO } | - \{\} \text{ expressionNoteIR} : \% \% :$

1. Result in STO and $\{\#\}$.

13.8.2. Sequence expressions

$\text{sequenceElementExpression} = \text{expressionList}$

Type checking

Typing $\{ \text{expressionList trailingCommaOpt } \}$, under cursor p and typing context TC :

1. Let expression_e^* be the list of expressions in expressionList .
2. If ... is not in expression_e^* :
 - a. Let $\text{typedExpressionIR}_e^*$ be the list of $\text{typedExpressionIR}_e$, obtained by repeating:
 - Let $\text{typedExpressionIR}_e$ be
 - the result of typing expression_e , under cursor p and typing context TC .for each expression_e in expression_e^*
 - b. Let typeIR_e^* be the list of typeIR_e , obtained by repeating:
 - Let typeIR_e be the type of $\text{typedExpressionIR}_e$.for each $\text{typedExpressionIR}_e$ in $\text{typedExpressionIR}_e^*$
 - c. Let ctk_e^* be the list of ctk_e , obtained by repeating:
 - Let ctk_e be the compile-time known-ness of $\text{typedExpressionIR}_e$.for each $\text{typedExpressionIR}_e$ in $\text{typedExpressionIR}_e^*$
 - d. Let typeIR be $\text{SEQ} < \text{typeIR}_e^* >$.
 - e. Let ctk be ctk_e^* joined.
 - f. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
 - g. Result in $\text{SEQ} \{ \text{typedExpressionIR}_e^* \}$ annotated with expressionNoteIR .
3. Else:

- a. Let expression^* be $\text{rev_<expression>}(\text{expression}_e^*)$.
- b. Check that expression^* is a non-empty list.
- c. Let $\text{expression}'' :: \text{expression}_{e,h,\text{rev}}^*$ be expression^* .
- d. Check that $\text{expression}''$ has type `defaultExpression`.
- e. Let `defaultExpression` be $\text{expression}''$.
- f. Let $\text{expression}_{e,h}^*$ be $\text{rev_<expression>}(\text{expression}_{e,h,\text{rev}}^*)$.
- g. Check that $\dots$ is not in $\text{expression}_{e,h}^*$.
- h. Let $\text{typedExpressionIR}_{e,h}^*$ be the list of `typedExpressionIRe,h`, obtained by repeating:
 - Let `typedExpressionIRe,h` be
 - the result of `typing expressione,h`, under cursor `p` and typing context `TC`.
 - for each $\text{expression}_{e,h}$ in $\text{expression}_{e,h}^*$
- i. Let $\text{typeIR}_{e,h}^*$ be the list of `typeIRe,h`, obtained by repeating:
 - Let `typeIRe,h` be the type of `typedExpressionIRe,h`.
 - for each `typedExpressionIRe,h` in $\text{typedExpressionIR}_{e,h}^*$
- j. Let $\text{ctk}_{e,h}^*$ be the list of `ctke,h`, obtained by repeating:
 - Let `ctke,h` be the compile-time known-ness of `typedExpressionIRe,h`.
 - for each `typedExpressionIRe,h` in $\text{typedExpressionIR}_{e,h}^*$
- k. Let `typeIR` be `SEQ < typeIRe,h , ... >`.
- l. Let `ctk` be $\text{ctk}_{e,h}^*$ joined.
- m. Let `expressionNoteIR` be `type typeIR and compile-time known-ness ctk`.
- n. Result in `SEQ { typedExpressionIRe,h^* , ... }` annotated with `expressionNoteIR`.

Local compile-time evaluation

Local compile-time evaluation of `<<typedExpressionIR, dataExpressionIR annotated with expressionNoteIR, under cursor p and typing context TC>>`:

1. If let `SEQ { typedExpressionIR^* }` be `dataExpressionIR`:
 - a. Let value^* be the list of `value`, obtained by repeating:
 - Let `value` be
 - the result of `local compile-time evaluation of typedExpressionIR`, under cursor `p` and typing context `TC`.
 - for each `typedExpressionIR` in $\text{typedExpressionIR}^*$
 - b. Result in `SEQ ( value^* )`.
2. Else if let `SEQ { typedExpressionIR^* , ... }` be `dataExpressionIR`:
 - a. Let value^* be the list of `value`, obtained by repeating:

- Let **value** be
 - the result of **local compile-time evaluation** of **typedExpressionIR**, under cursor **p** and **typing context TC**.

for each **typedExpressionIR** in **typedExpressionIR^{*}**

b. Result in **SEQ (value^{*} , ...)**.

Compile-time evaluation

p IC STO |- **dataExpressionIR # expressionNotelR : % %**:

1. If let **SEQ { typedExpressionListIR }** be **dataExpressionIR**:
 - a. Let **p IC STO** |- **typedExpressionListIR : STO₁ value^{*}**.
 - b. Result in **STO₁** and **SEQ (value^{*})**.
2. Else if let **SEQ { typedExpressionListIR , ... }** be **dataExpressionIR**:
 - a. Let **p IC STO** |- **typedExpressionListIR : STO₁ value^{*}**.
 - b. Result in **STO₁** and **SEQ (value^{*} , ...)**.

13.8.3. Record expressions

```
recordElementExpression
: name = expression
| name = expression , ...
| name = expression , namedExpressionList
| name = expression , namedExpressionList , ...
;
```

Type checking

Typing { recordElementExpression trailingCommaOpt }, under cursor p and typing context TC:

1. If let **name_f = expression_f** be **recordElementExpression**:
 - a. Let **nameIR_f** be **the name of name_f**.
 - b. Let **typedExpressionIR_f** be
 - the result of **typing expression_f**, under cursor **p** and **typing context TC**.
 - c. Let **typeIR_f** be **the type of typedExpressionIR_f**.
 - d. Let **ctk_f** be **the compile-time known-ness of typedExpressionIR_f**.
 - e. Let **typeIR** be **RECORD { [typeIR_f nameIR_f ;] }**.
 - f. Let **expressionNotelR** be **type typeIR and compile-time known-ness ctk_f**.
 - g. Result in **RECORD { [nameIR_f = typedExpressionIR_f] annotated with expressionNotelR**.
2. Else if let **name_f = expression_f , ...** be **recordElementExpression**:
 - a. Let **nameIR_f** be **the name of name_f**.
 - b. Let **typedExpressionIR_f** be

- the result of **typing** $expression_f$, under cursor p and typing context TC .
- c. Let $typeIR_f$ be the **type of** $typedExpressionIR_f$.
 - d. Let ctk_f be the **compile-time known-ness of** $typedExpressionIR_f$.
 - e. Let $typeIR$ be **RECORD** { $[typeIR_f \text{ name}IR_f;], \dots$ }.
 - f. Let $expressionNoteIR$ be **type** $typeIR$ and **compile-time known-ness** ctk_f .
 - g. Result in **RECORD** { $[nameIR_f = typedExpressionIR_f], \dots$ } **annotated with** $expressionNoteIR$.
3. Else if let $name_{f,h} = expression_{f,h}$, $namedExpressionList_t$ be **recordElementExpression**:
 - a. Let $(name_{f,t} = expression_{f,t})^*$ be the **list of named expressions in** $namedExpressionList_t$.
 - b. Let $name_f^*$ be $name_{f,h} :: name_{f,t}^*$.
 - c. Let $nameIR_f^*$ be the list of $nameIR_f$, obtained by repeating:
 - Let $nameIR_f$ be the **name of** $name_f$.
 for each $name_f$ in $name_f^*$
 - d. Let $expression_f^*$ be $expression_{f,h} :: expression_{f,t}^*$.
 - e. Let $typedExpressionIR_f^*$ be the list of $typedExpressionIR_f$, obtained by repeating:
 - Let $typedExpressionIR_f$ be
 - the result of **typing** $expression_f$, under cursor p and typing context TC .
 for each $expression_f$ in $expression_f^*$
 - f. Let $typeIR_f^*$ be the list of $typeIR_f$, obtained by repeating:
 - Let $typeIR_f$ be the **type of** $typedExpressionIR_f$.
 for each $typedExpressionIR_f$ in $typedExpressionIR_f^*$
 - g. Let ctk_f^* be the list of ctk_f , obtained by repeating:
 - Let ctk_f be the **compile-time known-ness of** $typedExpressionIR_f$.
 for each $typedExpressionIR_f$ in $typedExpressionIR_f^*$
 - h. Let $typeIR$ be **RECORD** { $(typeIR_f \text{ name}IR_f;)^*$ }.
 - i. Let ctk be ctk_f^* **joined**.
 - j. Let $expressionNoteIR$ be **type** $typeIR$ and **compile-time known-ness** ctk .
 - k. Result in **RECORD** { $(nameIR_f = typedExpressionIR_f)^*$ } **annotated with** $expressionNoteIR$.
 4. Else:
 - a. Let $name_{f,h} = expression_{f,h}$, $namedExpressionList_t, \dots$ be **recordElementExpression**.
 - b. Let $(name_{f,t} = expression_{f,t})^*$ be the **list of named expressions in** $namedExpressionList_t$.
 - c. Let $name_f^*$ be $name_{f,h} :: name_{f,t}^*$.
 - d. Let $nameIR_f^*$ be the list of $nameIR_f$, obtained by repeating:
 - Let $nameIR_f$ be the **name of** $name_f$.
 for each $name_f$ in $name_f^*$

- e. Let expression_f^* be $\text{expression}_{f_h} :: \text{expression}_{f_t}^*$.
- f. Let $\text{typedExpressionIR}_f^*$ be the list of $\text{typedExpressionIR}_f$, obtained by repeating:
 - Let $\text{typedExpressionIR}_f$ be
 - the result of **typing** expression_f , under cursor p and typing context TC .
 for each expression_f in expression_f^*
- g. Let typeIR_f^* be the list of typeIR_f , obtained by repeating:
 - Let typeIR_f be **the type of** $\text{typedExpressionIR}_f$.
 for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$
- h. Let ctk_f^* be the list of ctk_f , obtained by repeating:
 - Let ctk_f be **the compile-time known-ness of** $\text{typedExpressionIR}_f$.
 for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$
- i. Let typeIR be $\text{RECORD } \{ (\text{typeIR}_f \text{ nameIR}_f;)^*, \dots \}$.
- j. Let ctk be ctk_f^* **joined**.
- k. Let expressionNoteIR be **type** typeIR and **compile-time known-ness** ctk .
- l. Result in $\text{RECORD } \{ (\text{nameIR}_f = \text{typedExpressionIR}_f)^*, \dots \}$ **annotated with** expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{dataExpressionIR} \text{ annotated with } \text{expressionNoteIR}, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. If let $\text{RECORD } \{ (\text{nameIR} = \text{typedExpressionIR})^* \}$ be dataExpressionIR :
 - a. Let value^* be the list of value , obtained by repeating:
 - Let value be
 - the result of **local compile-time evaluation of** typedExpressionIR , under cursor p and **typing context** TC .
 for each typedExpressionIR in $\text{typedExpressionIR}^*$
 - b. Result in $\text{RECORD } \{ (\text{value nameIR};)^* \}$.
2. Else if let $\text{RECORD } \{ (\text{nameIR} = \text{typedExpressionIR})^*, \dots \}$ be dataExpressionIR :
 - a. Let value^* be the list of value , obtained by repeating:
 - Let value be
 - the result of **local compile-time evaluation of** typedExpressionIR , under cursor p and **typing context** TC .
 for each typedExpressionIR in $\text{typedExpressionIR}^*$
 - b. Result in $\text{RECORD } \{ (\text{value nameIR};)^*, \dots \}$.

Compile-time evaluation

$p \mid \text{IC STO} \mid - \text{dataExpressionIR} \# \text{expressionNotelR} : \% \%$:

1. If let $\text{RECORD} \{ (\text{nameIR} = \text{typedExpressionIR}^*) \}$ be dataExpressionIR :
 - a. Let $p \mid \text{IC STO} \mid - \text{typedExpressionIR}^* : \text{STO}_1 \text{ value}^*$.
 - b. Result in STO_1 and $\text{RECORD} \{ (\text{value nameIR};)^* \}$.
2. Else if let $\text{RECORD} \{ (\text{nameIR} = \text{typedExpressionIR}^*, \dots) \}$ be dataExpressionIR :
 - a. Let $p \mid \text{IC STO} \mid - \text{typedExpressionIR}^* : \text{STO}_1 \text{ value}^*$.
 - b. Result in STO_1 and $\text{RECORD} \{ (\text{value nameIR};)^*, \dots \}$.

13.9. Access expressions

```
accessExpression
: errorAccessExpression
| memberAccessExpression
| indexAccessExpression
| sliceAccessExpression
;

accessExpressionIR
: errorAccessExpressionIR
| memberAccessExpressionIR
| indexAccessExpressionIR
| sliceAccessExpressionIR
;
```

13.9.1. Error access expressions

```
errorAccessExpression
: ERROR . member
;

errorAccessExpressionIR
: ERROR . nameIR
;
```

Type checking

Typing $\text{ERROR} . \text{member}$, under cursor p and typing context TC :

1. Let nameIR be the name of member .
2. Let $\text{nameIR}_{\text{error}}$ be "error." concatenated with nameIR .
3. Check that $\text{ERROR} . \text{nameIR}$ is equal to the value of variable $\text{nameIR}_{\text{error}}$ in the p layer of TC .
4. Let expressionNotelR be type ERROR and compile-time known-ness LCTK .
5. Result in $\text{ERROR} . \text{nameIR}$ annotated with expressionNotelR .

Local compile-time evaluation

Local compile-time evaluation of $\ll \text{typedExpressionIR}, \text{ERROR} . \text{nameIR} \text{ annotated with type } \text{typeIR} \text{ and compile-time known-ness } \text{ctk}, \text{ under cursor } p \text{ and typing context } \text{TC} \gg$:

1. Let $\text{nameIR}_{\text{error}}$ be "error." concatenated with nameIR .
2. Let $\text{value}_{\text{error}}$ be the value of variable $\text{nameIR}_{\text{error}}$ in the p layer of TC .
3. Result in $\text{value}_{\text{error}}$.

Compile-time evaluation

$p \text{ IC STO} \mid - \text{ERROR} . \text{nameIR} \# \text{expressionNoteIR} : \% \%$:

1. Let $\text{nameIR}_{\text{error}}$ be "error." concatenated with nameIR .
2. Let $\text{value}_{\text{error}}$ be $\text{find_var_i}(p, \text{IC}, \text{nameIR}_{\text{error}})$.
3. Result in STO and $\text{value}_{\text{error}}$.

13.9.2. Member access expressions

```
memberAccessBase
  : prefixedTypeName
  | expression
  ;

memberAccessExpression
  : memberAccessBase . member
  ;

memberAccessBaseIR
  : TYPE prefixedNameIR
  | typedExpressionIR
  ;

memberAccessExpressionIR
  : memberAccessBaseIR . nameIR
  ;
```

13.9.2.1. Type accesses

Type checking

Typing $\text{prefixedTypeName}_{\text{base}} . \text{member}$, under cursor p and typing context TC :

1. Let $\text{prefixedNameIR}_{\text{base}}$ be the prefixed name of $\text{prefixedTypeName}_{\text{base}}$.
2. Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of $\text{prefixedNameIR}_{\text{base}}$ in the p layer of TC .
3. Check that $\text{typeDefIR}_{\text{base}}$ is monomorphic.
4. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
5. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
6. Check that typeIR has type enumTypeIR .
7. Let enumTypeIR be typeIR .

8. If let $\text{ENUM_}\{ \text{nameIR}_{\text{field}}^* \}$ be enumTypeIR :
 - a. Let nameIR be the name of member.
 - b. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
 - c. Let expressionNotelR be type $\text{typeIR}_{\text{base}}$ and compile-time known-ness LCTK.
 - d. Result in $\text{TYPE prefixedNameIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNotelR .
9. Else:
 - a. Let $\text{ENUM_} < _ > \{ (\text{nameIR}_{\text{field}} = _ ;)^* \}$ be enumTypeIR .
 - b. Let nameIR be the name of member.
 - c. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
 - d. Let expressionNotelR be type $\text{typeIR}_{\text{base}}$ and compile-time known-ness LCTK.
 - e. Result in $\text{TYPE prefixedNameIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNotelR .

Local compile-time evaluation

Local compile-time evaluation of $\langle \langle \text{typedExpressionIR}, \text{TYPE prefixedNameIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNotelR , under cursor p and typing context $\text{TC} \rangle \rangle$:

1. Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of $\text{prefixedNameIR}_{\text{base}}$ in the p layer of TC .
2. Check that $\text{typeDefIR}_{\text{base}}$ is monomorphic.
3. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
4. Let typeIR' be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
5. Check that typeIR' has type enumTypeIR .
6. Let enumTypeIR be typeIR' .
7. If let $\text{ENUM typeId} \{ \text{nameIR}_{\text{field}}^* \}$ be enumTypeIR :
 - a. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
 - b. Result in $\text{typeId} . \text{nameIR}$.
8. Else:
 - a. Let $\text{ENUM typeId} < \text{typeIR}' > \{ (\text{nameIR}_{\text{field}} = \text{value}_{\text{field}} ;)^* \}$ be enumTypeIR .
 - b. Let value' be ! $\text{assoc_} < \text{nameIR}, \text{value} > (\text{nameIR}, \text{nameIR}_{\text{field}}, \text{value}_{\text{field}}^*)$.
 - c. Result in $\text{typeId} . \text{nameIR} . \text{value}'$.

Compile-time evaluation

$p \text{ IC STO } | - \text{TYPE prefixedNameIR} . \text{nameIR} \# \text{expressionNotelR} : \% \% :$

1. Let $\text{typeDefIR}'$ be ! $\text{find_typeDef_i}(p, \text{IC}, \text{prefixedNameIR})$.
2. Check that $\text{typeDefIR}'$ has type enumTypeIR .
3. Let enumTypeIR be $\text{typeDefIR}'$.
4. If let $\text{ENUM typeId} \{ _ \}$ be enumTypeIR :
 - a. Result in STO and $\text{typeId} . \text{nameIR}$.

5. Else:

- a. Let $\text{ENUM typeId} < _ > \{ (\text{id}_{\text{member}} = \text{value}_{\text{member}};)^* \}$ be enumTypeIR .
- b. Let value' be $! \text{assoc}_{<\text{nameIR}, \text{value}>}(\text{nameIR}, \text{id}_{\text{member}}, \text{value}_{\text{member}}^*)$.
- c. Result in STO and $\text{typeId} . \text{nameIR} . \text{value}'$.

13.9.2.2. Field accesses

Type checking

Typing $\text{expression}_{\text{base}} . \text{member}$, under cursor p and typing context TC :

1. If " size " is equal to the name of member:
 - a. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
 - b. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - c. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - d. Check that typeIR has type headerStackTypeIR .
 - e. Let expressionNoteIR be type $\text{BIT} < 32 >$ and compile-time known-ness LCTK .
 - f. Result in $\text{typedExpressionIR}_{\text{base}} . \text{"size"} \text{ annotated with } \text{expressionNoteIR}$.
2. If " lastIndex " is equal to the name of member:
 - a. Check that $p = \text{BLOCK} \wedge \text{is_parser_blockKind}(\text{TC.BLOCK.KIND})$ or $p = \text{LOCAL} \wedge \text{is_parser_state_localKind}(\text{TC.LOCAL.KIND})$.
 - b. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
 - c. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - d. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - e. Check that typeIR has type headerStackTypeIR .
 - f. Let expressionNoteIR be type $\text{BIT} < 32 >$ and compile-time known-ness DYN .
 - g. Result in $\text{typedExpressionIR}_{\text{base}} . \text{"lastIndex"} \text{ annotated with } \text{expressionNoteIR}$.
3. If " last " is equal to the name of member:
 - a. Check that $p = \text{BLOCK} \wedge \text{is_parser_blockKind}(\text{TC.BLOCK.KIND})$ or $p = \text{LOCAL} \wedge \text{is_parser_state_localKind}(\text{TC.LOCAL.KIND})$.
 - b. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
 - c. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - d. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - e. Check that typeIR has type headerStackTypeIR .
 - f. Let $\text{typeIR}' [n_{\text{size}}]$ be typeIR .
 - g. Let expressionNoteIR be type typeIR' and compile-time known-ness DYN .
 - h. Result in $\text{typedExpressionIR}_{\text{base}} . \text{"last"} \text{ annotated with } \text{expressionNoteIR}$.

4. If "next" is equal to the name of member:
 - a. Check that $p = \text{BLOCK} \wedge \text{is_parser_blockKind}(\text{TC.BLOCK.KIND})$ or $p = \text{LOCAL} \wedge \text{is_parser_state_localKind}(\text{TC.LOCAL.KIND})$.
 - b. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
 - c. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - d. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - e. Check that typeIR has type headerStackTypeIR .
 - f. Let $\text{typeIR}' [n_{\text{size}}]$ be typeIR .
 - g. Let expressionNoteIR be type typeIR' and compile-time known-ness DYN .
 - h. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{"next"}$ annotated with expressionNoteIR .
5. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
6. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
7. Let ctk_{base} be the compile-time known-ness of $\text{typedExpressionIR}_{\text{base}}$.
8. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
9. If let $\text{STRUCT_} _ < _ > \{ (\text{typeIR}_f \text{ nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR'' be $! \text{assoc_} < \text{nameIR}, \text{typeIR} > (\text{nameIR}, \text{nameIR}_f, \text{typeIR}_f^*)$.
 - c. Let expressionNoteIR be type typeIR'' and compile-time known-ness ctk_{base} .
 - d. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .
10. Else if let $\text{HEADER_} _ < _ > \{ (\text{typeIR}_f \text{ nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR'' be $! \text{assoc_} < \text{nameIR}, \text{typeIR} > (\text{nameIR}, \text{nameIR}_f, \text{typeIR}_f^*)$.
 - c. Let expressionNoteIR be type typeIR'' and compile-time known-ness ctk_{base} .
 - d. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .
11. Else if let $\text{HEADER_UNION_} _ < _ > \{ (\text{typeIR}_f \text{ nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR'' be $! \text{assoc_} < \text{nameIR}, \text{typeIR} > (\text{nameIR}, \text{nameIR}_f, \text{typeIR}_f^*)$.
 - c. Let expressionNoteIR be type typeIR'' and compile-time known-ness ctk_{base} .
 - d. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .
12. Else if let $\text{TABLE_STRUCT_} _ \{ \text{HIT boolTypeIR} ; \text{MISS boolTypeIR}' ; \text{ACTION_RUN tableMetadataEnumTypeIR} ; \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. If nameIR is equal to "hit":
 - i. Let expressionNoteIR be type boolTypeIR and compile-time known-ness DYN .
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .
 - c. Else if nameIR is equal to "miss":
 - i. Let expressionNoteIR be type $\text{boolTypeIR}'$ and compile-time known-ness DYN .

- ii. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
- d. Else if nameIR is equal to "action_run":
 - i. Let expressionNoteIR be type $\text{tableMetadataEnumTypeIR}$ and compile-time known-ness DYN.
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p and typing context $TC \rangle\rangle$:

1. If nameIR is equal to "size":
 - a. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - b. Let typeIR' be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - c. Check that typeIR' has type headerStackTypeIR .
 - d. Let $[_{n_{\text{size}}}]$ be typeIR' .
 - e. Result in $D_{n_{\text{size}}}$.
2. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{base}}$, under cursor p and typing context TC .
3. If let structValue be value :
 - a. Let $\text{STRUCT_}\{ (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be structValue .
 - b. Let value'' be $! \text{assoc_}\langle \text{nameIR}, \text{value} \rangle (\text{nameIR}, \text{nameIR}_{\text{field}}, \text{value}_{\text{field}}^*)$.
 - c. Result in value'' .
4. Else if let headerValue be value :
 - a. Let $\text{HEADER_}\{ _ ; (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be headerValue .
 - b. Let value'' be $! \text{assoc_}\langle \text{nameIR}, \text{value} \rangle (\text{nameIR}, \text{nameIR}_{\text{field}}, \text{value}_{\text{field}}^*)$.
 - c. Result in value'' .
5. Else if let headerUnionValue be value :
 - a. Let $\text{HEADER_UNION_}\{ (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be headerUnionValue .
 - b. Let value'' be $! \text{assoc_}\langle \text{nameIR}, \text{value} \rangle (\text{nameIR}, \text{nameIR}_{\text{field}}, \text{value}_{\text{field}}^*)$.
 - c. Result in value'' .

Compile-time evaluation

$p \text{ IC STO} \mid - \text{typedExpressionIR}_{\text{base}} . \text{nameIR} \# \text{expressionNoteIR} : \% \%$:

1. If nameIR is equal to "size":
 - a. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
 - b. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
 - c. Check that typeIR has type headerStackTypeIR .

- d. Let $_ [n_{size}]$ be $typeIR$.
 - e. Result in STO and $D n_{size}$.
2. Let $p \text{ IC } STO \vdash typedExpressionIR_{base} : STO_1 \text{ value}$.
 3. If let $structValue$ be $value$:
 - a. Let $STRUCT_ \{ (value_{field} \ id_{field} ;) ^* \}$ be $structValue$.
 - b. Let $value''$ be $! \text{ assoc_} \langle id, value \rangle (nameIR, id_{field}, value_{field} ^*)$.
 - c. Result in STO_1 and $value''$.
 4. Else if let $headerValue$ be $value$:
 - a. Let $HEADER_ \{ _ ; (value_{field} \ id_{field} ;) ^* \}$ be $headerValue$.
 - b. Let $value''$ be $! \text{ assoc_} \langle id, value \rangle (nameIR, id_{field}, value_{field} ^*)$.
 - c. Result in STO_1 and $value''$.
 5. Else if let $headerUnionValue$ be $value$:
 - a. Let $HEADER_UNION_ \{ (value_{field} \ id_{field} ;) ^* \}$ be $headerUnionValue$.
 - b. Let $value''$ be $! \text{ assoc_} \langle id, value \rangle (nameIR, id_{field}, value_{field} ^*)$.
 - c. Result in STO_1 and $value''$.

13.9.3. Index access expressions

```

indexAccessExpression
: expression `[ expression ]
;

indexAccessExpressionIR
: typedExpressionIR `[ typedExpressionIR ]
;

```

Type checking

Typing $expression_{base} [expression_{index}]$, under cursor p and typing context TC :

1. Let $typedExpressionIR_{base}$ be
 - the result of typing $expression_{base}$, under cursor p and typing context TC .
2. Let $typedExpressionIR_{index}$ be
 - the result of typing $expression_{index}$, under cursor p and typing context TC .
3. Let $typeIR_{base}$ be the type of $typedExpressionIR_{base}$.
4. Let ctk_{base} be the compile-time known-ness of $typedExpressionIR_{base}$.
5. Let ctk_{index} be the compile-time known-ness of $typedExpressionIR_{index}$.
6. Let $typedExpressionIR_{index_reduced}$ be $!$ the result of reducing serializable enums in $typedExpressionIR_{index}$ until $\$compat_array_index$ is satisfied.
7. Let $typeIR$ be $typeIR_{base}$ with typedefs unrolled.
8. If let $TUPLE < typeIR_e ^* >$ be $typeIR$:
 - a. Check that ctk_{index} is LCTK.

- b. Let $value$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{index_reduced}$, under cursor p and typing context TC .
 - c. Check that $value$ has type $integerValue$.
 - d. Let $integerValue_{index}$ be $value$.
 - e. Let int be the integer representation of $integerValue_{index}$.
 - f. Check that int has type nat .
 - g. Let n_{index} be int .
 - h. Check that n_{index} is less than the length of $typeIR_e^*$.
 - i. Let $expressionNoteIR$ be type $typeIR_e^*[n_{index}]$ and compile-time known-ness ctk_{base} .
 - j. Result in $typedExpressionIR_{base} [typedExpressionIR_{index_reduced}]$ annotated with $expressionNoteIR$.
9. Else if let $typeIR' [n_{size}]$ be $typeIR$:
- a. If ctk_{index} is LCTK:
 - i. Let $value$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{index_reduced}$, under cursor p and typing context TC .
 - ii. Check that $value$ has type $integerValue$.
 - iii. Let $integerValue_{index}$ be $value$.
 - iv. Let int be the integer representation of $integerValue_{index}$.
 - v. Check that int has type nat .
 - vi. Let n_{index} be int .
 - vii. Check that n_{index} is less than n_{size} .
 - viii. Let $expressionNoteIR$ be type $typeIR'$ and compile-time known-ness ctk_{base} .
 - ix. Result in $typedExpressionIR_{base} [typedExpressionIR_{index_reduced}]$ annotated with $expressionNoteIR$.
 - b. Else:
 - i. Let ctk be ctk_{base} and ctk_{index} joined.
 - ii. Let $expressionNoteIR$ be type $typeIR'$ and compile-time known-ness ctk .
 - iii. Result in $typedExpressionIR_{base} [typedExpressionIR_{index_reduced}]$ annotated with $expressionNoteIR$.

13.9.4. Bitslice access expressions

```

sliceAccessExpression
  : expression `[ expression : expression ]
  ;

sliceAccessExpressionIR
  : typedExpressionIR `[ typedExpressionIR : typedExpressionIR ]
  ;

```

Type checking

Typing $expression_{base} [expression_{hi} : expression_{lo}]$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
2. Let $\text{typedExpressionIR}_{\text{hi}}$ be
 - the result of typing $\text{expression}_{\text{hi}}$, under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_{\text{lo}}$ be
 - the result of typing $\text{expression}_{\text{lo}}$, under cursor p and typing context TC .
4. Let $\text{typedExpressionIR}_{\text{base_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{base}}$ until $\$compat_bitslice_base$ is satisfied.
5. Let $\text{typeIR}_{\text{base_reduced}}$ be the type of $\text{typedExpressionIR}_{\text{base_reduced}}$.
6. Let $\text{ctk}_{\text{base_reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{base_reduced}}$.
7. Let $\text{typedExpressionIR}_{\text{hi_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{hi}}$ until $\$compat_bitslice_index$ is satisfied.
8. Let $\text{typedExpressionIR}_{\text{lo_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{lo}}$ until $\$compat_bitslice_index$ is satisfied.
9. Let $\text{ctk}_{\text{hi_reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{hi_reduced}}$.
10. Let $\text{ctk}_{\text{lo_reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{lo_reduced}}$.
11. Check that $\text{ctk}_{\text{hi_reduced}}$ is LCTK.
12. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{hi_reduced}}$, under cursor p and typing context TC .
13. Check that value has type integerValue .
14. Let $\text{integerValue}_{\text{hi}}$ be value .
15. Let int be the integer representation of $\text{integerValue}_{\text{hi}}$.
16. Check that int has type nat .
17. Let n_{hi} be int .
18. Check that $\text{ctk}_{\text{lo_reduced}}$ is LCTK.
19. Let value' be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{lo_reduced}}$, under cursor p and typing context TC .
20. Check that value' has type integerValue .
21. Let $\text{integerValue}_{\text{lo}}$ be value' .
22. Let int' be the integer representation of $\text{integerValue}_{\text{lo}}$.
23. Check that int' has type nat .
24. Let n_{lo} be int' .
25. Check that $\text{is_valid_bitslice}(\text{typeIR}_{\text{base_reduced}}, n_{\text{lo}}, n_{\text{hi}})$.
26. Let int'' be $n_{\text{hi}} - n_{\text{lo}} + 1$.
27. Check that int'' has type nat .
28. Let n_{slice} be int'' .
29. Let typeIR be $\text{BIT} < n_{\text{slice}} >$.
30. Let expressionNoteIR be type typeIR and compile-time known-ness $\text{ctk}_{\text{base_reduced}}$.

31. Result in $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{hi_reduced}} : \text{typedExpressionIR}_{\text{lo_reduced}}]$ annotated with expressionNodeIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, \text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{hi}} : \text{typedExpressionIR}_{\text{lo}}] \text{ annotated with } \text{expressionNodeIR}, \text{ under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Let $\text{value}_{\text{base}}$ be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{base}}$, under cursor p and typing context TC .
2. Let value_{hi} be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{hi}}$, under cursor p and typing context TC .
3. Let value_{lo} be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{lo}}$, under cursor p and typing context TC .
4. Result in $\text{bitacc_op}(\text{value}_{\text{base}}, \text{value}_{\text{hi}}, \text{value}_{\text{lo}})$.

Compile-time evaluation

$p \text{ IC STO } |- \text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{hi}} : \text{typedExpressionIR}_{\text{lo}}] \# \text{expressionNodeIR} : \% \%$:

1. Let $p \text{ IC STO } |- \text{typedExpressionIR}_{\text{base}} : \text{STO}_1 \text{ value}_{\text{base}}$.
2. Let $p \text{ IC STO}_1 |- \text{typedExpressionIR}_{\text{hi}} : \text{STO}_2 \text{ value}_{\text{hi}}$.
3. Let $p \text{ IC STO}_2 |- \text{typedExpressionIR}_{\text{lo}} : \text{STO}_3 \text{ value}_{\text{lo}}$.
4. Result in STO_3 and $\text{bitacc_op}(\text{value}_{\text{base}}, \text{value}_{\text{hi}}, \text{value}_{\text{lo}})$.

13.10. Call expressions

```
callableTarget = expression
constructorTarget = namedType

callTarget
  : callableTarget
  | constructorTarget
  ;

callExpression
  : callTarget `( argumentList )
  | callableTarget `< realTypeArgumentList > `( argumentList )
  ;

callableTargetIR
  : referenceExpressionIR
  | typedExpressionIR . nameIR
  | TYPE prefixedNameIR . nameIR
  | `( callableTargetIR )
  ;
```

```

constructorTargetIR
: prefixedNameIR `< typeArgumentListIR >
;

callExpressionIR
: constructorTargetIR `( argumentListIR )
| callableTargetIR `< typeArgumentListIR > `( argumentListIR )
;

```

13.10.1. Callable call expressions

Type checking

Typing `callExpression`, under cursor `p` and typing context `TC`:

1. If let `callTarget ( argumentList )` be `callExpression`:
 - a. Check that `callTarget` has type `callableTarget`.
 - b. Let `callableTarget` be `callTarget`.
 - c. Let the typed callable target `callableTargetIR` be
 - the result of typing `callableTarget`, under cursor `p` and typing context `TC`.
 - d. Let `argument*` be the list of arguments in `argumentList`.
 - e. Let `argumentIR*` be the list of `argumentIR`, obtained by repeating:
 - Let `argumentIR` be
 - the result of typing `argument`, under cursor `p` and typing context `TC`.
 for each `argument` in `argument*`
 - f. Let the callable type `callableTypeIR` with fresh type variables `typeIdinfer*` and default parameter ids `iddefault*` be
 - the result of finding the type of `callableTargetIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - g. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of typing the invocation `callableTypeIR < · > ( argumentIR* )` with omitted type arguments `typeIdinfer*` and omitted arguments `iddefault*`, under cursor `p` and typing context `TC`.
 - h. If `typeIRret` is not equal to `VOID`:
 - i. Let `ctk` be `is_static_callableTarget(callableTargetIR)`.
 - ii. Check that `is_static_assert_callableTarget(callableTargetIR)`.
 - iii. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - iv. Let `expressionNoteIR` be type `typeIRret` and compile-time known-ness `ctk`.
 - v. Let `value` be
 - the result of local compile-time evaluation of `callExpressionIR` annotated with `expressionNoteIR`, under cursor `p` and typing context `TC`.
 - vi. Check that `value` is equal to ``B true`.

- vii. Let `literalExpressionIR` be `TRUE`.
- viii. Result in `literalExpressionIR` annotated with `expressionNoteIR`.
- i. Let `argumentIR*` be
 - the result of `typing` `argumentList`, under cursor `p` and typing context `TC`.
- j. If `typeIRret` is not equal to `VOID`:
 - i. Let `ctk` be `is_static_callableTarget(callableTargetIR)`.
 - ii. Check that `~ is_static_assert_callableTarget(callableTargetIR)`.
 - iii. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - iv. Let `expressionNoteIR` be `type typeIRret` and `compile-time known-ness ctk`.
 - v. Result in `callExpressionIR` annotated with `expressionNoteIR`.
- 2. Else:
 - a. Let `callableTarget < realTypeArgumentList > ( argumentList )` be `callExpression`.
 - b. Let the typed callable target `callableTargetIR` be
 - the result of `typing` `callableTarget`, under cursor `p` and typing context `TC`.
 - c. Let `realTypeArgument*` be the list of real type arguments in `realTypeArgumentList`.
 - d. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of `typing` `realTypeArgument*`, under cursor `p` and typing context `TC`.
 - e. Let `argumentIR*` be
 - the result of `typing` `argumentList`, under cursor `p` and typing context `TC`.
 - f. Let the callable type `callableTypeIR` with fresh type variables `typeIdinserted*` and default parameter ids `iddefault*` be
 - the result of `finding the type of` `callableTargetIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - g. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
 - h. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of `typing the invocation` `callableTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*` and omitted arguments `iddefault*`, under cursor `p` and typing context `TC`.
 - i. Check that `typeIRret` is not equal to `VOID`.
 - j. Let `ctk` be `is_static_callableTarget(callableTargetIR)`.
 - k. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - l. Let `expressionNoteIR` be `type typeIRret` and `compile-time known-ness ctk`.
 - m. Result in `callExpressionIR` annotated with `expressionNoteIR`.

Local compile-time evaluation

Local compile-time evaluation of `<<typedExpressionIR, callableTargetIR < · > ( · ) annotated with expressionNoteIR, under cursor p and typing context TC>>`:

1. If let `typedExpressionIRbase . nameIR` be `callableTargetIR`:

- a. Let $\text{typeIR}_{\text{base}}$ and compile-time known-ness $_$ be the note of $\text{typedExpressionIR}_{\text{base}}$.
 - b. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - c. Result in $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.
2. Else if let TYPE prefixedNameIR . nameIR be callableTargetIR :
 - a. Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of prefixedNameIR in the p layer of TC.
 - b. If $\text{typeDefIR}_{\text{base}}$ is monomorphic:
 - i. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - ii. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - iii. Result in $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.
 - c. Else:
 - i. Check that $(\cdot, \cdot)$ is equal to the type parameters of $\text{typeDefIR}_{\text{base}}$.
 - ii. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - iii. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - iv. Result in $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.

Compile-time evaluation

p IC STO | - $\text{callableTargetIR} < \cdot > (\cdot) \# \text{expressionNoteIR} : \% \%$:

1. If let $\text{typedExpressionIR}_{\text{base}}$. nameIR be callableTargetIR :
 - a. Let $\text{typeIR}_{\text{base}}$ and compile-time known-ness $_$ be the note of $\text{typedExpressionIR}_{\text{base}}$.
 - b. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - c. Result in STO and $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.
2. Else if let TYPE prefixedNameIR . nameIR be callableTargetIR :
 - a. Let $\text{typeDefIR}_{\text{base}}$ be ! $\text{find_typeDef_i}(p, \text{IC}, \text{prefixedNameIR})$.
 - b. If $\text{typeDefIR}_{\text{base}}$ is monomorphic:
 - i. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - ii. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - iii. Result in STO and $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.
 - c. Else:
 - i. Check that $(\cdot, \cdot)$ is equal to the type parameters of $\text{typeDefIR}_{\text{base}}$.
 - ii. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - iii. Check that nameIR is in $["\text{minSizeInBits}", "\text{minSizeInBytes}", "\text{maxSizeInBits}", "\text{maxSizeInBytes}"]$.
 - iv. Result in STO and $\text{sizeof}(\text{typeIR}_{\text{base}}, \text{nameIR})$.

13.10.2. Constructor call expressions

Type checking

Typing `callTarget ( argumentList )`, under cursor `p` and typing context `TC`:

1. If let `prefixedTypeName` be `callTarget`:
 - a. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC`.
 - b. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - c. Let `constructorTypeIR` with fresh type variables `typeIdimpl*` and default parameter ids `iddefault*` be
 - the result of type checking constructor `prefixedNameIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - d. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of typing the instantiation `constructorTypeIR < · > ( argumentIR* )` with omitted type arguments `typeIdimpl*` and omitted arguments `iddefault*`, under cursor `p`, typing context `TC`, and ANON instantiation site context.
 - e. Check that `typeIRobject` is an extern object type without abstract methods.
 - f. Let `callExpressionIR` be `prefixedNameIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - g. Let `expressionNoteIR` be type `typeIRobject` and compile-time known-ness `CTK`.
 - h. Result in `callExpressionIR` annotated with `expressionNoteIR`.
2. Else if let `prefixedTypeName < typeArgumentList >` be `callTarget`:
 - a. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context `TC`.
 - b. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC`.
 - c. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - d. Let `constructorTypeIR` with fresh type variables `typeIdinserted*` and default parameter ids `iddefault*` be
 - the result of type checking constructor `prefixedNameIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - e. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
 - f. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of typing the instantiation `constructorTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*` and omitted arguments `iddefault*`, under cursor `p`, typing context `TC`, and ANON instantiation site context.
 - g. Check that `typeIRobject` is an extern object type without abstract methods.
 - h. Let `callExpressionIR` be `prefixedNameIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - i. Let `expressionNoteIR` be type `typeIRobject` and compile-time known-ness `CTK`.
 - j. Result in `callExpressionIR` annotated with `expressionNoteIR`.

Compile-time evaluation

Typing `callTarget ( argumentList )`, under cursor `p` and typing context `TC`:

1. If let `prefixedTypeName` be `callTarget`:
 - a. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC`.
 - b. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - c. Let `constructorTypeIR` with fresh type variables `typeIdimpl*` and default parameter ids `iddefault*` be
 - the result of type checking constructor `prefixedNameIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - d. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of typing the instantiation `constructorTypeIR < · > ( argumentIR* )` with omitted type arguments `typeIdimpl*` and omitted arguments `iddefault*`, under cursor `p`, typing context `TC`, and ANON instantiation site context.
 - e. Check that `typeIRobject` is an extern object type without abstract methods.
 - f. Let `callExpressionIR` be `prefixedNameIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - g. Let `expressionNoteIR` be type `typeIRobject` and compile-time known-ness `CTK`.
 - h. Result in `callExpressionIR` annotated with `expressionNoteIR`.
2. Else if let `prefixedTypeName < typeArgumentList >` be `callTarget`:
 - a. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context `TC`.
 - b. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC`.
 - c. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - d. Let `constructorTypeIR` with fresh type variables `typeIdinserted*` and default parameter ids `iddefault*` be
 - the result of type checking constructor `prefixedNameIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - e. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
 - f. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of typing the instantiation `constructorTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*` and omitted arguments `iddefault*`, under cursor `p`, typing context `TC`, and ANON instantiation site context.
 - g. Check that `typeIRobject` is an extern object type without abstract methods.
 - h. Let `callExpressionIR` be `prefixedNameIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - i. Let `expressionNoteIR` be type `typeIRobject` and compile-time known-ness `CTK`.
 - j. Result in `callExpressionIR` annotated with `expressionNoteIR`.

13.10.3. Parenthesized calls

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, (\text{callableTargetIR}' < \text{typeArgumentIR}^* > (\text{argumentIR}^*)) \text{ annotated with type } \text{typeIR} \text{ and compile-time known-ness } \text{ctk}, \text{ under cursor } p \text{ and typing context } \text{TC} \rangle\rangle$:

1. Let value be
 - the result of local compile-time evaluation of $\text{callableTargetIR}' < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$ annotated with type typeIR and compile-time known-ness ctk , under cursor p and typing context TC .
2. Result in value .

Compile-time evaluation

$p \text{ IC STO } |- (\text{callableTargetIR}'' < \text{typeArgumentListIR} > (\text{argumentListIR}) \# \text{expressionNoteIR} : \% \% :$

1. Let $p \text{ IC STO } |- \text{callableTargetIR}'' < \text{typeArgumentListIR} > (\text{argumentListIR}) \# \text{expressionNoteIR} : \text{STO}_1 \text{ value}$.
2. Result in STO_1 and value .

13.11. Parenthesized expressions

```
parenthesizedExpression
: `( expression )
;

parenthesizedExpressionIR
: `( typedExpressionIR )
;
```

Type checking

Typing $(\text{expression}')$, under cursor p and typing context TC :

1. Let typedExpressionIR be
 - the result of typing $\text{expression}'$, under cursor p and typing context TC .
2. Let typeIR be the type of typedExpressionIR .
3. Let ctk be the compile-time known-ness of typedExpressionIR .
4. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
5. Result in $(\text{typedExpressionIR})$ annotated with expressionNoteIR .

Local compile-time evaluation

Local compile-time evaluation of $\langle\langle \text{typedExpressionIR}, (\text{typedExpressionIR}) \text{ annotated with expressionNoteIR, under cursor } p \text{ and typing context } TC \rangle\rangle$:

1. Let $value$ be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
2. Result in $value$.

Compile-time evaluation

$p \text{ IC STO } |- (\text{typedExpressionIR}) \# \text{expressionNoteIR} : \% \%$:

1. Let $p \text{ IC STO } |- \text{typedExpressionIR} : \text{STO}_1 \text{ value}$.
2. Result in STO_1 and $value$.

14. Casting

14.1. Explicit casting

14.2. Implicit casting

14.3. Coercion

15. Abstract Parser Machine

15.1. Parser local declarations

```
parserLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| valueSetDeclaration
;

parserLocalDeclarationIR
: constantDeclarationIR
| instantiationIR
| variableDeclarationIR
| valueSetDeclarationIR
;
```

15.1.1. Constant declarations

```
constantDeclaration
  : annotationList CONST type name initializer ;
  ;

constantDeclarationIR
  : annotationList CONST typeIR nameIR constantInitializerIR ;
  ;
```

15.1.2. Instantiations

```
instantiation
  : annotationList type `( argumentList ) name ;
  | annotationList type `( argumentList ) name objectInitializer ;
  ;

instantiationIR
  : annotationList typeIR prefixedNameIR `< typeArgumentListIR > `( argumentListIR )
  nameIR objectInitializerOptIR ;
  ;
```

15.1.3. Variable declarations

```
variableDeclaration
  : annotationList type name initializerOpt ;
  ;

variableDeclarationIR
  : annotationList typeIR nameIR initializerOptIR ;
  ;
```

15.1.4. Parser value set declarations

```
valueSetType
  : baseType
  | tupleType
  | prefixedTypeName
  ;

valueSetDeclaration
  : annotationList VALUE_SET `< valueSetType > `( expression ) name ;
  ;

valueSetDeclarationIR
  : annotationList VALUE_SET `< typeIR > `( typedExpressionIR ) nameIR ;
  ;
```

15.2. Parser states

```
parserState
  : annotationList STATE name `{ parserStatementList transitionStatement }
  ;

parserStateIR
  : annotationList STATE nameIR `{ parserStatementListIR transitionStatementIR }
```

```
;
```

15.3. Parser statements

```
parserStatement
: constantDeclaration
| variableDeclaration
| emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| parserBlockStatement
| parserConditionalStatement
;

parserStatementIR
: constantDeclarationIR
| variableDeclarationIR
| emptyStatementIR
| assignmentStatementIR
| callStatementIR
| directApplicationStatementIR
| parserBlockStatementIR
| parserConditionalStatementIR
;
```

15.3.1. Constant declaration

```
constantDeclaration
: annotationList CONST type name initializer ;
;

constantDeclarationIR
: annotationList CONST typeIR nameIR constantInitializerIR ;
;
```

15.3.2. Variable declaration

```
variableDeclaration
: annotationList type name initializerOpt ;
;

variableDeclarationIR
: annotationList typeIR nameIR initializerOptIR ;
;
```

15.3.3. Empty statement

```
emptyStatement
: ;
;

emptyStatementIR = emptyStatement
```

15.3.4. Assignment statement

```

assignmentStatement
: lvalue assignop expression ;
;

assignmentStatementIR
: typedLvalueIR assignop typedExpressionIR ;
;

```

15.3.5. Call statement

```

callStatement
: lvalue `( argumentList ) ;
| lvalue `< typeArgumentList > `( argumentList ) ;
;

callStatementIR
: callableTargetIR `< typeArgumentListIR > `( argumentListIR ) ;
;

```

15.3.6. Direct application statement

```

directApplicationStatement
: namedType . APPLY `( argumentList ) ;
;

directApplicationStatementIR
: prefixedNameIR . APPLY `( argumentListIR ) ;
;

```

15.3.7. Block statement

```

blockStatement
: annotationList `{ blockElementStatementList }
;

blockStatementIR
: annotationList `{ blockElementStatementListIR }
;

```

15.3.8. Conditional statement

```

conditionalStatement
: IF `( expression ) statement
| IF `( expression ) statement ELSE statement
;

conditionalStatementIR
: IF `( typedExpressionIR ) statementIR
| IF `( typedExpressionIR ) statementIR ELSE statementIR
;

```

15.4. Parser transition statements

```

stateExpression

```

```

: name ;
| selectExpression
;

transitionStatement
: /* empty */
| TRANSITION stateExpression
;

stateExpressionIR
: nameIR ;
| selectExpressionIR
;

transitionStatementIR
: TRANSITION stateExpressionIR
;

```

15.4.1. Parser select expression

```

keysetExpression
: simpleKeysetExpression
| tupleKeysetExpression
;

selectCase
: keysetExpression : name ;
;

selectExpression
: SELECT `( expressionList ) `{ selectCaseList }
;

keysetExpressionIR
: simpleKeysetExpressionIR
| tupleKeysetExpressionIR
;

selectCaseIR
: keysetExpressionIR : nameIR ;
;

selectExpressionIR
: SELECT `( typedExpressionListIR ) `{ selectCaseListIR }
;

```

16. Abstract Control Machine

16.1. Control local declarations

```

controlLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| actionDeclaration
| tableDeclaration
;

controlLocalDeclarationIR
: constantDeclarationIR
| instantiationIR

```

```

| variableDeclarationIR
| actionDeclarationIR
| tableDeclarationIR
;

```

16.1.1. Constant declarations

```

constantDeclaration
: annotationList CONST type name initializer ;
;

constantDeclarationIR
: annotationList CONST typeIR nameIR constantInitializerIR ;
;

```

16.1.2. Instantiations

```

instantiation
: annotationList type `( argumentList ) name ;
| annotationList type `( argumentList ) name objectInitializer ;
;

instantiationIR
: annotationList typeIR prefixedNameIR `< typeArgumentListIR > `( argumentListIR )
nameIR objectInitializerOptIR ;
;

```

16.1.3. Variable declarations

```

variableDeclaration
: annotationList type name initializerOpt ;
;

variableDeclarationIR
: annotationList typeIR nameIR initializerOptIR ;
;

```

16.1.4. Action declarations

```

actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;

actionDeclarationIR
: annotationList ACTION nameIR `( parameterListIR ) blockStatementIR
;

```

16.1.5. Table declarations

```

tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;

tableDeclarationIR
: annotationList TABLE typeIR nameIR `{ tablePropertyListIR }
;

```

```
;
```

16.2. Match-action tables

```
tableProperty
: KEY = `{ tableKeyList }
| ACTIONS = `{ tableActionList }
| annotationList constOpt ENTRIES = `{ tableEntryList }
| annotationList constOpt tableCustomName initializer ;
;

tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;

tablePropertyIR
: tableKeysPropertyIR
| tableActionsPropertyIR
| tableDefaultActionPropertyIR
| tableEntriesPropertyIR
| tableCustomPropertyIR
;

tableDeclarationIR
: annotationList TABLE typeIR nameIR `{ tablePropertyListIR }
;
```

16.2.1. Table keys

```
tableKey
: expression : name annotationList ;
;

tableKeyIR
: typedExpressionIR : nameIR annotationList ;
;
```

16.2.2. Table actions

```
tableActionReference
: prefixedNonTypeName
| prefixedNonTypeName `( argumentList )
;

tableAction
: annotationList tableActionReference ;
;

tableActionReferenceIR
: prefixedNameIR `( argumentListIR )
;

tableActionIR
: annotationList tableActionReferenceIR # `( parameterListIR , parameterListIR ) ;
;
```

16.2.3. Table default actions

16.2.4. Table entries

```
keysetExpression
  : simpleKeysetExpression
  | tupleKeysetExpression
  ;

tableEntryPriority
  : PRIORITY = integerLiteral :
  | PRIORITY = `( expression ) :
  ;

tableEntry
  : constOpt tableEntryPriority keysetExpression : tableActionReference annotationList ;
  | constOpt keysetExpression : tableActionReference annotationList ;
  ;

keysetExpressionIR
  : simpleKeysetExpressionIR
  | tupleKeysetExpressionIR
  ;

tableEntryPriorityIR
  : PRIORITY = integerLiteral :
  | PRIORITY = `( typedExpressionIR ) :
  ;

tableEntryIR
  : constOptIR tableEntryPriorityOptIR keysetExpressionIR : tableActionReferenceIR
  annotationList ;
  ;
```

16.2.5. Table custom properties

17. Call convention

17.1. Arguments and type arguments

17.1.1. Arguments

```
argument
  : expression
  | name = expression
  | name = _
  | _
  ;

argumentIR
  : typedExpressionIR
  | nameIR = typedExpressionIR
  | nameIR = _
  | _
  ;
```

17.1.2. Type arguments

```
typeArgument
```

```

: realTypeArgument
| nonTypeName
;

realTypeArgument
: type
| VOID
| _
;

typeArgumentIR = typeIR

```

17.2. Overload resolution

Functions and methods are the only P4 constructs that support overloading: there can exist multiple methods with the same name in the same scope. When overloading is used, the compiler must be able to disambiguate at compile-time which method or function is being called, either by the number of arguments or by the names of the arguments, when calls are specifying argument names. Argument type information is not used in disambiguating calls.

Notice that overloading of abstract methods, parsers, controls, or packages is not allowed:

```

parser p(packet_in p, out bit<32> value) {
    ...
}

// The following will cause an error about a duplicate declaration
//parser p(packet_in p, out Headers headers) {
//    ...
//}

```

17.3. Static checks for call typing

17.4. Call semantics

18. Annotations

```

annotationToken
: UNEXPECTED_TOKEN
| ABSTRACT
| ACTION
| ACTIONS
| APPLY
| BOOL
| BIT
| BREAK
| CONST
| CONTINUE
| CONTROL
| DEFAULT
| ELSE
| ENTRIES
| ENUM
| ERROR
| EXIT
| EXTERN
| FALSE

```

```

FOR
HEADER
HEADER_UNION
IF
IN
INOUT
INT
KEY
MATCH_KIND
TYPE
OUT
PARSER
PACKAGE
PRAGMA
RETURN
SELECT
STATE
STRING
STRUCT
SWITCH
TABLE
THIS
TRANSITION
TRUE
TUPLE
TYPEDEF
VARBIT
VALUE_SET
LIST
VOID
-
identifier
typeIdentifier
stringLiteral
integerLiteral
&&
..
<<
&&
||
==
!=
>=
<=
++
+
|+|
-
|-|
*
/
%
|
&
^
~
[
]
{
}
<
>
!
:
,
?
.
=
;

```

```

| @
;

annotationBody
: /* empty */
| annotationBody `( annotationBody )
| annotationBody annotationToken
;

structuredAnnotationBody
: dataElementExpression trailingCommaOpt
;

annotation
: @ name
| @ name `( annotationBody )
| @ name `[ structuredAnnotationBody ]
;

```

Appendix A: Revision History

A.1. Summary of changes made in unreleased version

- Clarified that numeric priorities cannot be assigned to entries of a table that has **const entries** ([[sec-entries](#)]).
- Clarified that **switch** statements are allowed in action and function bodies, and that **switch** statements with **action_run** expressions are only allowed in control **apply** blocks ([[sec-stmts](#)] and [[sec-switch-stmt](#)]).
- Added standard branch prediction annotations ([[sec-branch-annotations](#)])
- Added support for compile-time string concatenation using **++** operator ([[sec-string-type](#)] and [[sec-string-ops](#)]).
- Added compound assignment statements ([[sec-assignment](#)])
- Update the specification grammar for **for** loops to match the grammar used in the **p4c** implementation ([[sec-loop-stmt](#)]).

A.2. Summary of changes made in version 1.2.5, released October 11, 2024

- Improved type nesting rules ([[sec-type-nesting](#)]).
- Clarified that directionless extern parameters are passed by reference.
- Introduced distinction between local compile-time known and compile-time known values ([Section 7.5](#)).

A.3. Summary of changes made in version 1.2.4, released May 15, 2023

- Added header stack expressions ([[sec-hs-init](#)]).
- Allow casts from a type to itself ([[sec-casts](#)]).
- Added an invalid header or header union expression **{#}** ([[sec-ops-on-hdrs](#)] and [[sec-expr-hu](#)]).
- Added a concept of numeric values ([[sec-numeric-values](#)]).

- Added a section on operations on extern objects ([[sec-extern-operations](#)]).
- Added note in sections operations on types for types that support compile-time size determination.
- Clarified that header stacks are arrays of headers or header unions.
- Added distinctness of fields for types that have fields including error, match kind, struct, header, and header union.
- Clarified types `bit<W>`, `int<W>`, and `varbit<W>` encompass the case where the width is a compile-time known expression evaluating to an appropriate integer ([[sec-unsigned-integers](#)], [[sec-signed-integers](#)], [[sec-dynamically-sized-integers](#)]).
- Clarified restrictions for parameters with default values ([[sec-calling-convention](#)]).
- Added optional trailing commas ([[sec-trailing-commas](#)]).
- Clarified the scope of parser namespaces ([[sec-parser-decl](#)]).
- Specified that algorithm for generating control-plane names for keys is optional ([[sec-cp-keys](#)]).
- Clarified types of expressions that may appear in `select` ([[sec-select](#)]).
- Added description of semantics of the core.p4 match kinds ([[sec-table-keys](#)]).
- Explicitly disallow overloading of parsers, controls, and packages ([[sec-extern-objects](#)]).
- Clarified implicit casts present in select expressions ([[sec-select](#)]).
- Clarified that slices can be applied to arbitrary-precision integers ([[sec-varint-ops](#)]).
- Clarified that direct invocation is not possible for objects that have constructor arguments ([[sec-direct-invocation](#)]).
- Added comparison for tuples as a legal operation ([[sec-tuple-exprs](#)]).
- Clarified the behavior of `lookahead` on header-typed values ([[sec-packet-lookahead](#)]).
- Added `static_assert` function ([[sec-static-assert](#)]).
- Clarified semantics of ranges where the start is bigger than the end ([[sec-ranges](#)]).
- Allow ranges to be specified by serializable enums ([[sec-ranges](#)]).
- Specified type produced by the `*sizeInB*` methods ([[sec-minsizeinbits](#)]).
- Added section with operations on `match_kind` values ([[sec-match-kind-exprs](#)]).
- Renamed infinite-precision integers to arbitrary-precision integers ([[sec-arbitrary-precision-integers](#)]).
- compiler-inserted `default_action` is not `const` ([[sec-tables](#)]).
- Clarified the restrictions on run time for tables with `const entries` ([[sec-entries](#)]).
- renamed list expressions to tuple expressions
- Added `list` type ([[sec-list-types](#)]).
- Defined `entries` table property without `const`, for entries installed when the P4 program is loaded, but the control plane can later change them or add to them ([[sec-entries](#)]).
- Clarified behavior of table with no `key` property, or if its list of keys is empty ([[sec-table-keys](#)]).

A.4. Summary of changes made in version 1.2.3, released July 11, 2022.

- Extended `minSizeInBits` and `minSizeInBytes` to apply to more expressions ([[sec-minsizeinbits](#)]).
- Added support for `maxSizeInBits` and `maxSizeInBytes` ([[sec-minsizeinbits](#)]).

- Added support for empty lists of const entries in tables ([[sec-entries](#)]).
- Added support for `switch` statements in actions ([[sec-actions](#)]).
- Added support for direct invocation of controls and parsers ([[sec-parameterization](#)]).
- Added parser `value_set` to list of control-plane visible names (Section 7.3.1.1.5).
- Added `match_kind` as a base type ([[sec-match-kind-type](#)]).
- Removed structure initializers as they are subsumed by structure-valued expressions ([[sec-structure-expressions](#)]).
- Specified operations on values typed as type variables ([[sec-type-variable-operations](#)]).
- Clarified semantics of compile-time known values (Section 7.5).
- Clarified semantics of directionless parameters ([[sec-calling-convention](#)]).
- Clarified semantics of arbitrary precision integers ([[sec-arbitrary-precision-integers](#)]).
- Clarified semantics of bit slices, shifts, and concatenation ([[sec-bit-ops](#)]).
- Clarified semantics of optional parameters ([[sec-optional-parameters](#)]).
- Clarified restrictions on extern method and function invocation ([[sec-calling-restrictions](#)]).
- Clarified semantics of implicit casts ([[sec-implicit-casts](#)]).

A.5. Summary of changes made in version 1.2.2, released May 17, 2021

- Added support for accessing tuple fields ([[sec-tuple-exprs](#)]).
- Added support for generic structures ([[sec-type-spec](#)]).
- Added support for integers, `enums`, and `errors` in `switch` statements ([[sec-switch-stmt](#)]).
- Added support for additional enumeration types ([[sec-enum-types](#)]).
- Added support for abstract methods ([[sec-abstract-methods](#)]).
- Added support for conditional statements and empty statements in parsers ([[sec-parser-state-stmt](#)]).
- Added support for casts from `int` to `bool` ([[sec-casts](#)]).
- Added support for 0-width bitstrings and varbits ([[sec-uninitialized-values-and-writing-invalid-headers](#)]).
- Clarified that `default_action` is `NoAction` if otherwise unspecified ([[sec-tables](#)]).
- Clarified the types of expressions that may be used as indexes for header stacks ([[sec-expr-hs](#)]).
- Clarified representation of Booleans in headers ([[sec-header-types](#)]).
- Clarified representation of empty types ([[sec-uninitialized-values-and-writing-invalid-headers](#)]).
- Clarified that action data can be specified by the control plane, `default_action` table property, or `const entries` table property ([[sec-actions](#)]).
- Fixed several typos and inconsistencies in grammar ([[sec-grammar](#)]).
- Eliminated annotations on `const` entries in grammar ([[sec-grammar](#)]).

A.6. Summary of changes made in version 1.2.1, released June 11, 2020

- Added structure-value expressions ([[sec-structure-expressions](#)]).

- Added support for default values ([sec-default-values]).
- Added support for concatenating signed strings ([sec-concatenation]).
- Added key-value and list-structured annotations ([sec-annotations]).
- Added `@pure` and `@noSideEffects` annotations ([sec-extern-annotations]).
- Added `@noWarn` annotation ([sec-nowarn-anno]).
- Generalized typing for masks to allow serializable `enums` ([sec-cubes]).
- Restricted the right operands of bit shifts involving arbitrary-precision integers to be constant and positive ([sec-varint-ops]).
- Clarified copy-out behavior for `return` ([sec-return-stmt]) and `exit` ([sec-exit-stmt]) statements.
- Clarified semantics of invalid header stacks ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified initialization semantics ([sec-lvalues] and [sec-calling-convention]), especially for headers and local variables.
- Clarified evaluation order for table keys ([sec-mau-semantic]).
- Fixed grammar to clarify parsing of right shift operator (`>>`), allow empty statements in parser ([sec-parser-state-stmt]), and eliminate annotations on const entries ([sec-entries]).

A.7. Summary of changes made in version 1.2.0, released October 14, 2019

- Added `table.apply().miss` ([sec-invoke-mau]).
- Added `string` type ([sec-string-type]).
- Added implicit casts from enum values ([sec-enum-exprs]).
- Allow 1-bit signed values
- Define the type of bit slices from signed and unsigned values to be unsigned.
- Constrain `default` label position for `switch` statements.
- Allow empty tuples.
- Added `@deprecated` annotation.
- Relaxed the structure of annotation bodies.
- Removed the `@pkginfo` annotation, which is now defined by the P4Runtime specification.
- Added `int` type ([sec-arbitrary-precision-integers]).
- Added error `ParserInvalidArgument` ([sec-packet-extract-two], [sec-skip-bits]).
- Clarified the significance of order of entries in `const entries` ([sec-entries]).
- Added methods to calculate header size ([sec-ops-on-hdrs]).

A.8. Summary of changes made in version 1.1.0, released November 26, 2017.

- Top-level functions ([sec-functions])
 - Functions may be declared at the top-level of a P4 program.
- Optional and named parameters ([sec-calling-convention])
 - Parameters may be specified by name, with a default value, or designated as optional.

- `enum` representations ([sec-enum-exprs])
 - `enum` values to be specified with a concrete representation.
- Parser values sets ([sec-value-set])
 - `value_set` objects for control-plane programmable `select` labels.
- Type definitions (Section 8.5.2)
 - New types may be introduced in programs.
- Saturating arithmetic ([sec-bit-ops])
 - Saturating arithmetic is supported on some targets.
- Structured annotations ([sec-annotations])
 - Annotations may be specified as lists of key-value pairs
- Globalname ([sec-name-annotations])
 - The reserved `globalname` annotation has been removed.
- Table `size` property ([sec-size-table-property])
 - Meaning of optional `size` property for tables has been defined.
- Invalid headers ([sec-ops-on-hdrs])
 - Clarified semantics of operations on invalid headers.
- Calling restrictions ([sec-calling-restrictions])
 - Added restrictions on kinds of values that may be passed as arguments to calls.
- Bitwise operator precedence ([sec-grammar])
 - Modified precedence conventions so that bitwise operators `&` `|` and `^` have higher precedence than relation operators `<` `>` `<=` `>=`.
- Computed bitwidths ([sec-base-types])
 - Added support for specifying widths using expressions in `bit` and `varbit` types.

A.9. Initial version 1.0.0, released May 17, 2017

Appendix B: P4 reserved keywords

The following table shows all P4 reserved keywords. Some identifiers are treated as keywords only in specific contexts (e.g., the keyword `actions`).

<code>abstract</code>	<code>action</code>	<code>apply</code>	<code>bit</code>
<code>bool</code>	<code>const</code>	<code>control</code>	<code>default</code>
<code>else</code>	<code>enum</code>	<code>error</code>	<code>extern</code>
<code>exit</code>	<code>false</code>	<code>header</code>	<code>header_union</code>
<code>if</code>	<code>in</code>	<code>inout</code>	<code>int</code>
<code>list</code>	<code>match_kind</code>	<code>package</code>	<code>parser</code>
<code>out</code>	<code>return</code>	<code>select</code>	<code>state</code>
<code>string</code>	<code>struct</code>	<code>switch</code>	<code>table</code>
<code>this</code>	<code>transition</code>	<code>true</code>	<code>tuple</code>

type	typedef	value_set	varbit
verify	void		

Appendix C: P4 reserved annotations

The following table shows all P4 reserved annotations.

Annotation	Purpose	See Section
atomic	specify atomic execution	[sec-concurrency]
defaultonly	action can only appear in the default action	[sec-annotations]
hidden	hides a controllable entity from the control plane	[sec-name-annotations]
match	specify <code>match_kind</code> of a field in a <code>value_set</code>	[sec-value-set]
name	assign local control-plane name	[sec-name-annotations]
optional	parameter is optional	[sec-optional-parameters]
tableonly	action cannot be a default_action	[sec-annotations]
deprecated	Construct has been deprecated	[sec-deprecated-anno]
pure	pure function	[sec-extern-annotations]
noSideEffects	function with no side effects	[sec-extern-annotations]
noWarn	Has a string argument; inhibits compiler warnings	[sec-nowarn-anno]

Appendix D: P4 core library

The P4 core library contains declarations that are useful to most programs.

For example, the core library includes the declarations of the predefined `packet_in` and `packet_out` extern objects, used in parsers and deparsers to access packet data.

```

/// Standard error codes. New error codes can be declared by users.
error {
    NoError,           /// No error.
    PacketTooShort,    /// Not enough bits in packet for 'extract'.
    NoMatch,           /// 'select' expression has no matches.
    StackOutOfBounds,  /// Reference to invalid element of a header stack.
    HeaderTooShort,    /// Extracting too many bits into a varbit field.
    ParserTimeout,     /// Parser execution time limit exceeded.
    ParserInvalidArgument /// Parser operation was called with a value
                        /// not supported by the implementation.
}
extern packet_in {
    /// Read a header from the packet into a fixed-sized header @hdr
    /// and advance the cursor.
    /// May trigger error PacketTooShort or StackOutOfBounds.
    /// @T must be a fixed-size header type
    void extract<T>(out T hdr);

```

```

    /// Read bits from the packet into a variable-sized header @variableSizeHeader
    /// and advance the cursor.
    /// @T must be a header containing exactly 1 varbit field.
    /// May trigger errors PacketTooShort, StackOutOfBounds, or HeaderTooShort.
    void extract<T>(out T variableSizeHeader,
                   in bit<32> variableFieldSizeInBits);
    /// Read bits from the packet without advancing the cursor.
    /// @returns: the bits read from the packet.
    /// T may be an arbitrary fixed-size type.
    T lookahead<T>();
    /// Advance the packet cursor by the specified number of bits.
    void advance(in bit<32> sizeInBits);
    /// @return packet length in bytes. This method may be unavailable on
    /// some target architectures.
    bit<32> length();
}

extern packet_out {
    /// Write @data into the output packet, skipping invalid headers
    /// and advancing the cursor
    /// @T can be a header type, a header stack, a header_union, or a struct
    /// containing fields with such types.
    void emit<T>(in T data);
}

action NoAction() {}
/// Standard match kinds for table key fields.
/// Some architectures may not support all these match kinds.
/// Architectures can declare additional match kinds.
match_kind {
    /// Match bits exactly.
    exact,
    /// Ternary match, using a mask.
    ternary,
    /// Longest-prefix match.
    lpm
}

/// Static assert evaluates a boolean expression
/// at compilation time. If the expression evaluates to
/// false, compilation is stopped and the corresponding message is printed.
/// The function returns a boolean, so that it can be used
/// as a global constant value in a program, e.g.:
/// const version = static_assert(V1MODEL_VERSION > 20180000, "Expected a v1 model
version >= 20180000");
extern bool static_assert(bool check, string message);

/// Like the above but using a default message.
extern bool static_assert(bool check);

```